```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py                启动并自动打开浏览器
    python run.py --no-browser   只启动服务，不打开浏览器
    python run.py --dev          开发模式（开启代码热重载与调试器）
    python run.py --port 5001    指定端口
默认以 127.0.0.1:5000 启动，就绪后自动打开浏览器。
"""
import os
import sys
import threading
import time
import webbrowser
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")    # 控制台中文正常显示
from app import create_app
from app.models import db
PORT = 5000
for i, a in enumerate(sys.argv):
    if a == "--port" and i + 1 < len(sys.argv):
        PORT = int(sys.argv[i + 1])
URL = "http://127.0.0.1:%d" % PORT
NO_BROWSER = "--no-browser" in sys.argv
DEV = "--dev" in sys.argv
app = create_app()
def open_browser_later(delay=1.2):
    """等服务工作起来再打开浏览器，避免打开时页面还没就绪。
    放在后台线程里，不用 sleep 阻塞服务启动；线程设为守护线程，
    用户 Ctrl+C 停止服务时不会残留。
    """
    def run():
        time.sleep(delay)
        try:
            webbrowser.open(URL)
        except Exception:
            pass        # 打开失败不影响服务本身
    t = threading.Thread(target=run, daemon=True)
    t.start()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()   # 首次运行时自动建表
        from app.utils.schema import ensure_schema
        added = ensure_schema(db)   # 旧库补齐新增字段
        if added:
            print("已补充字段：%s" % "、".join(added))
    print("=" * 58)
    print("  历史遗留废弃矿山自然恢复适宜性评价系统 V1.0")
    print("=" * 58)
    print("  访问地址：%s" % URL)
    print("  演示账号：admin    密码：admin123")
    print("  停止运行：按 Ctrl+C，或直接关闭本窗口")
    print("=" * 58)
    if not NO_BROWSER and not DEV:
        print("  正在打开浏览器……")
    print()
    # 开发模式才启用热重载（重载会二次启动进程，故此时不自动开浏览器）
    if not NO_BROWSER and not DEV:
        open_browser_later()
    app.run(host="127.0.0.1", port=PORT, debug=DEV,
    use_reloader=DEV)
# ==== app/__init__.py ====
# -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from datetime import date
from flask import Flask
from flask_login import LoginManager
from app.config import Config
from app.models import db, User
from app.utils.constants import ROLE_LABELS
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    # 注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report, search
    app.register_blueprint(auth.bp)
    app.register_blueprint(project.bp)
    app.register_blueprint(unit.bp)
    app.register_blueprint(standard.bp)
    app.register_blueprint(constraint.bp)
    app.register_blueprint(ahp.bp)
    app.register_blueprint(indicator.bp)
    app.register_blueprint(evaluate.bp)
    app.register_blueprint(report.bp)
    app.register_blueprint(search.bp)
    # 全局登录保护：除登录页与静态资源外，均需登录
    from flask import request, redirect, url_for,
    send_from_directory
    from flask_login import current_user, login_user
```

```
PUBLIC_ENDPOINTS = {"auth.login", "auth.register", "static", "favicon"}
# 浏览器默认会请求根路径下的 /favicon.ico。
# 页面里虽已用 <link rel="icon"> 指向 static 目录，但出错页等场景仍会
# 回退到这个路径，补一条路由免得日志里一直刷 404。
@app.route("/favicon.ico")
def favicon():
    return send_from_directory(app.static_folder, "favicon.ico",
                               mimetype="image/vnd.microsoft.icon")
@app.before_request
def require_login():
    # 截图模式（仅用于生成说明书配图）：自动以管理员身份登录
    if app.config.get("SCREENSHOT_MODE"):
        if (request.endpoint not in ("auth.login", "auth.register", "static")
                and not current_user.is_authenticated):
            admin = User.query.filter_by(role="admin").first()
            if admin:
                login_user(admin)
        return None
    if request.endpoint in PUBLIC_ENDPOINTS or request.endpoint is None:
        return None
    if not current_user.is_authenticated:
        return redirect(url_for("auth.login"))
    return None
# 侧边栏导航：按论文评价流程的顺序编排，分为「评价流程」与「系统管理」两组。
# 不使用模块编号，避免出现与实际流程不一致的代号。
# 每项带图标名（内联 SVG，见 templates/_macros.html 的 ICONS）
# 与可选的待办角标（该步骤还没做完的图斑数）。
FLOW_LINKS = [
    ("项目与矿山档案", "/project/", "folder"),
    ("政策与标准知识库", "/standard/", "book"),
    ("限制性指标判定", "/constraint/", "shield"),
    ("微地质单元", "/unit/", "layers"),
    ("指标体系与权重", "/ahp/", "sliders"),
    ("指标数据采集", "/indicator/", "database"),
    ("适宜性定量评价", "/evaluate/", "target"),
    ("报告与结果输出", "/report/", "file"),
]
# 「用户与操作日志」已并入「个人设置」，不再是独立菜单，
# 因此系统管理组只留一个入口。
ADMIN_LINKS = [
    ("个人设置", "/profile", "user"),
]
@app.context_processor
def inject_nav():
    path = request.path or "/"
    def hit(prefix):
        """当前路径是否属于某个入口（/unit/ 也匹配 /unit/3）。"""
        return path.startswith(prefix.rstrip("/"))
    badges = {}
    if current_user.is_authenticated:
        from app.services import dashboard as dash
        badges = dash.nav_badges()
    def build(links):
        return [{"label": label, "href": href, "icon":
icon,
                 "active": hit(href), "badge":
badges.get(href, 0)}
                for label, href, icon in links]
    groups = [
        {"title": "评价流程", "links": build(FLOW_LINKS)},
        {"title": "系统管理", "links": build(ADMIN_LINKS)},
    ]
    # 面包屑用的当前页名：按路径命中的入口取，未命中时给个通用名
    if path == "/":
        page_title = "工作台"
    elif path.startswith("/profile"):
        page_title = "个人设置"
    elif path.startswith(("/users", "/logs")):
        page_title = "账号与日志"
    elif path.startswith("/search"):
        page_title = "搜索结果"
    else:
        page_title = "工作台"
        for label, href, _icon in FLOW_LINKS:
            if hit(href):
                page_title = label
                break
    # 层级引导用：项目是归档单位，图斑才是评价对象。
    # 下游各页面按图斑组织数据，因此需要提示「哪些项目下还没有图斑」。
    from app.models import Project, MinePatch
    projects = Project.query.order_by(Project.id).all()
    return {
        "nav_groups": groups,
        "home_active": path == "/",
        "page_title": page_title,
        "today": date.today().strftime("%Y-%m-%d"),
        "search_q": request.args.get("q", ""),
        "role_labels": ROLE_LABELS,
        "project_count": len(projects),
        "patch_count": MinePatch.query.count(),
        "projects_without_patch": [p for p in projects
                                  if p.patches.count()
== 0],
    }
    # 操作审计：统一在响应阶段记录成功的写操作（操作日志）
    from app.services import audit
    @app.after_request
    def audit_writes(response):
        if request.method in ("POST", "PUT", "PATCH",
    "DELETE"):
            audit.record(request.endpoint,
    response.status_code)
        return response
    # 错误页：给出可读提示，而不是默认的英文错误页
    from flask import render_template
    @app.errorhandler(403)
    def forbidden(e):
        return render_template("error.html", code=403,
                               message="没有权限执行该操作"), 403
    @app.errorhandler(404)
    def not_found(e):
        return render_template("error.html", code=404,
                               message="请求的页面不存在"), 404
    return app
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
from datetime import timedelta
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY",
    "mine-restoration-dev-key")
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
    # 「记住我」：登录时勾选则下发持久 cookie，关闭浏览器后仍可自动登录
    REMEMBER_COOKIE_DURATION = timedelta(days=30)
    REMEMBER_COOKIE_HTTPONLY = True
    REMEMBER_COOKIE_SAMESITE = "Lax"
    # 影像与数据文件上传（遥感影像挂接、批量导入）
    UPLOAD_FOLDER = os.path.join(BASE_DIR, "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024      # 单次上传上限
    32 MB
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
    ALLOWED_DATA_EXT = {"csv", "xlsx", "xls"}
    # 头像只用浏览器能直接显示的格式（不含 bmp / tif），并单独限制体积
    ALLOWED_AVATAR_EXT = {"png", "jpg", "jpeg", "gif",
    "webp"}
    MAX_AVATAR_LENGTH = 4 * 1024 * 1024          # 4 MB
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from datetime import datetime
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（账户与权限）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 职务只有两种：管理员 admin（可管理账号与他人的职务）/ 操作员 operator
    role = db.Column(db.String(20), nullable=False,
    default="operator")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    # ---- 个人资料（用户可在「个人设置」里自行维护）----
    display_name = db.Column(db.String(64), nullable=True)
    # 昵称
    email = db.Column(db.String(128), nullable=True)
    phone = db.Column(db.String(32), nullable=True)
    gender = db.Column(db.String(8), nullable=True)
    # 男 / 女 / 保密
    avatar = db.Column(db.String(256), nullable=True)
    # 头像，相对 static 的路径
    @property
    def is_admin(self):
        return self.role == "admin"
    @property
    def shown_name(self):
        """界面上显示的名字：优先用昵称，没填就用账号名。"""
        return self.display_name or self.username
    @property
    def initial(self):
        """没有头像时用首字占位。"""
        return (self.shown_name or "?")[0].upper()
    def to_dict(self):
        return {"id": self.id, "username": self.username,
                "display_name": self.display_name,
    "gender": self.gender,
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志。"""
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, index=True)
class Project(db.Model):
    """评价项目（项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
    created_by = db.Column(db.Integer,
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    patches = db.relationship("MinePatch",
    backref="project", lazy="dynamic",
                               cascade="all, delete-orphan")
class MinePatch(db.Model):
    """矿山图斑。"""
    __tablename__ = "mine_patches"
    id = db.Column(db.Integer, primary_key=True)
    project_id = db.Column(db.Integer,
    db.ForeignKey("projects.id"), nullable=False)
    name = db.Column(db.String(128), nullable=False)
    admin_region = db.Column(db.String(128), nullable=True)
    # 所在行政区
    center_x = db.Column(db.Float, nullable=True)
    # 中心经度/横坐标
    center_y = db.Column(db.Float, nullable=True)
    # 中心纬度/纵坐标
    area_ha = db.Column(db.Float, nullable=True)
    # 图斑面积（公顷）
    mineral_type = db.Column(db.String(64), nullable=True)
    # 开采矿种
    mining_method = db.Column(db.String(64), nullable=True)
    # 开采方式
    stop_years = db.Column(db.Float, nullable=True)
    # 停产年限
    responsible_entity_missing = db.Column(db.Boolean,
    default=True)  # 责任主体是否缺失
    # 遥感底图 / 无人机正射影像挂接
    image_path = db.Column(db.String(256), nullable=True)
    # 相对 static 的路径
    image_note = db.Column(db.String(256), nullable=True)
    # 影像说明（如来源、时相）
    image_uploaded_at = db.Column(db.DateTime,
    nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    units = db.relationship("MicroUnit", backref="patch",
    lazy="dynamic",
                            cascade="all, delete-orphan")
class ReferenceEcosystem(db.Model):
    """参照生态系统：标准化基准值，与图斑绑定。"""
    __tablename__ = "reference_ecosystems"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    coverage_benchmark = db.Column(db.Float, default=75.0)
    # 植被覆盖度基准 %
    shannon_benchmark = db.Column(db.Float, default=0.7)
    # Shannon-Wiener 基准
    dominant_species = db.Column(db.String(256),
    nullable=True)  # 建群种
    soil_benchmark = db.Column(db.String(256),
    nullable=True)    # 土壤基准
class Standard(db.Model):
    """地方标准/政策文件（政策与标准知识库）。"""
    __tablename__ = "standards"
    id = db.Column(db.Integer, primary_key=True)
    province = db.Column(db.String(32), nullable=False)
    # 省份
    name = db.Column(db.String(256), nullable=False)
    # 名称
    doc_number = db.Column(db.String(128), nullable=True)
    # 标准编号/文号
    publish_date = db.Column(db.String(32), nullable=True)
    implement_date = db.Column(db.String(32), nullable=True)
    # 类别：地方标准 / 政策文件
    category = db.Column(db.String(32), nullable=False,
    default="地方标准")
    summary = db.Column(db.Text, nullable=True)
    # 特点概述
    rules = db.relationship("StandardRule",
    backref="standard",
                                    cascade="all, delete-orphan")
class StandardRule(db.Model):
    """结构化规则：将标准条文拆解为可执行规则。"""
    __tablename__ = "standard_rules"
    id = db.Column(db.Integer, primary_key=True)
    standard_id = db.Column(db.Integer,
    db.ForeignKey("standards.id"), nullable=False)
    item = db.Column(db.String(128), nullable=False)
    # 判定项
    operator = db.Column(db.String(8), nullable=False)
    # 比较符：> >= < <= ==
    threshold = db.Column(db.String(64), nullable=True)
    # 阈值
    condition_type = db.Column(db.String(32),
    default="必要条件")  # 必要条件/充分条件
    region = db.Column(db.String(64), nullable=True)
    # 适用分区（宁夏分区差异化）
    note = db.Column(db.Text, nullable=True)
class ConstraintCheck(db.Model):
    """限制性判定记录。"""
    __tablename__ = "constraint_checks"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    policy_pass = db.Column(db.Boolean, nullable=True)
    # 政策约束
    location_pass = db.Column(db.Boolean, nullable=True)
    # 区位约束
    pollution_pass = db.Column(db.Boolean, nullable=True)
    # 土壤污染
    geo_pass = db.Column(db.Boolean, nullable=True)
    # 地质灾害
    farmland_pass = db.Column(db.Boolean, nullable=True)
    # 耕地属性
    passed = db.Column(db.Boolean, nullable=True)
    # 总体是否通过
    reason = db.Column(db.Text, nullable=True)
    # 不通过原因
    checked_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class MicroUnit(db.Model):
    """微地质单元：岩质边坡 / 碎石边坡 / 平台。"""
    __tablename__ = "micro_units"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    unit_type = db.Column(db.String(32), nullable=False)
    # 单元类型
    area = db.Column(db.Float, nullable=True)
    # 面积
    ratio = db.Column(db.Float, nullable=True)
    # 面积占比（自动计算）
    description = db.Column(db.Text, nullable=True)
    # 现场描述
class Indicator(db.Model):
    """指标定义：指标树，含角色与启用单元类型。"""
    __tablename__ = "indicators"
    id = db.Column(db.Integer, primary_key=True)
    code = db.Column(db.String(32), unique=True,
    nullable=False)  # 指标编码
    name = db.Column(db.String(64), nullable=False)
    # 指标名称
    level = db.Column(db.Integer, nullable=False)
    # 1/2/3 级
    parent_code = db.Column(db.String(32), nullable=True)
    # 父指标编码
    # 角色：土壤污染指标同时作为限制性指标与评价指标
    is_constraint = db.Column(db.Boolean, default=False)
    is_evaluative = db.Column(db.Boolean, default=True)
    unit = db.Column(db.String(32), nullable=True)
    # 单位
    ideal_value = db.Column(db.String(64), nullable=True)
    # 理想值（可配置）
    ideal_type = db.Column(db.String(32), nullable=True)
    # benefit/cost/mid/discrete
    # 启用该指标的微地质单元（逗号分隔）。岩质边坡不启用土壤指标、平台不启用地形指标
    enabled_units = db.Column(db.String(128), nullable=True)
    description = db.Column(db.Text, nullable=True)
    # 取值依据说明
    def units_list(self):
        return [s for s in (self.enabled_units or
    "").split(",") if s]
class WeightScheme(db.Model):
    """权重方案：按微地质单元类型分别保存。"""
    __tablename__ = "weight_schemes"
    id = db.Column(db.Integer, primary_key=True)
    unit_type = db.Column(db.String(32), nullable=False)
    # 所属单元类型
    name = db.Column(db.String(64), nullable=True)
    # 方案名称
    level = db.Column(db.Integer, nullable=False,
    default=1)        # 1=一级指标 2=二级指标
    is_default = db.Column(db.Boolean, default=False)
    # 是否默认（论文表 3）
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class WeightItem(db.Model):
    """权重明细：方案内的各项权重。"""
    __tablename__ = "weight_items"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
    indicator_code = db.Column(db.String(32),
    nullable=False)       # 指标编码
    weight = db.Column(db.Float, nullable=False)
    # 权重值
class ExpertMatrix(db.Model):
    """专家判断矩阵：多专家打分的原始矩阵。"""
    __tablename__ = "expert_matrices"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=True)
    # 直接标注适用对象，便于无需先建权重方案即可录入专家矩阵
    unit_type = db.Column(db.String(32), nullable=True)
    # 微地质单元类型
    l1_name = db.Column(db.String(64), nullable=True)
    # 所属一级指标（二级矩阵用）
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
    level = db.Column(db.Integer, nullable=False,
    default=1)      # 矩阵对应指标层级
    matrix_json = db.Column(db.Text, nullable=False)
    # 判断矩阵（JSON 二维数组）
    cr = db.Column(db.Float, nullable=True)
    # 该专家矩阵的一致性比例
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class IndicatorData(db.Model):
    """指标实测数据：按微地质单元组织。"""
    __tablename__ = "indicator_data"
    id = db.Column(db.Integer, primary_key=True)
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_uni
    ts.id"), nullable=False, unique=True)
    soil_ratio = db.Column(db.Float, nullable=True)
    # 土石比
    thickness = db.Column(db.Float, nullable=True)
    # 土层厚度 cm
    ph = db.Column(db.Float, nullable=True)
    organic_matter = db.Column(db.Float, nullable=True)
    # 有机质 g/kg
    # 重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null
    heavy_metals = db.Column(db.Text, nullable=True)
    coverage = db.Column(db.Float, nullable=True)
    # 植被覆盖度 %
    shannon = db.Column(db.Float, nullable=True)
    # Shannon-Wiener 多样性指数
    slope = db.Column(db.Float, nullable=True)
    # 坡度 °
    aspect = db.Column(db.String(16), nullable=True)
    # 坡向：阳坡/阴阳坡/阴坡
    def heavy_metals_dict(self):
        return json.loads(self.heavy_metals) if
    self.heavy_metals else {}
class EvaluationResult(db.Model):
    """评价结果。"""
    __tablename__ = "evaluation_results"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    final_score = db.Column(db.Float, nullable=True)
    # 最终评价因子 F
    grade = db.Column(db.String(32), nullable=True)
    # 适宜性等级
    measures = db.Column(db.Text, nullable=True)
    # 推荐措施
    unit_scores = db.Column(db.Text, nullable=True)
    # 各单元得分 JSON
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class ScoreDetail(db.Model):
    """得分明细：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""
    __tablename__ = "score_details"
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
import re
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
```

```
from wtforms.validators import (DataRequired, Length,
    Optional, EqualTo,
                                ValidationError)
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES,
    GENDERS
# 邮箱格式：够用的轻量校验。
# WTForms 自带的 Email() 需要额外安装 email_validator,
# 为一个字段引入依赖不划算，这里自己用正则判断。
EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[A-Za-z]{2,}$")
def email_format(form, field):
    v = (field.data or "").strip()
    if v and not EMAIL_RE.match(v):
        raise ValidationError("邮箱格式不正确")
def email_ok(value):
    """邮箱格式判断，供不用 WTForms 的表单复用（如管理员编辑用户）。"""
    v = (value or "").strip()
    return (not v) or bool(EMAIL_RE.match(v))
class AHPSelectForm(FlaskForm):
    """选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """登录表单。支持「记住我」以在关闭浏览器后保持登录状态。"""
    username = StringField("账户名称", validators=[DataRequired(
    ), Length(max=64)])
    password = PasswordField("密码", validators=[DataRequired(
    )])
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
GENDER_CHOICES = [("", "不填")] + [(g, g) for g in GENDERS]
class ProfileForm(FlaskForm):
    """个人资料：昵称、性别与联系方式。字段都可留空。
    职务不在这里 —— 只能由管理员在「个人设置 · 账号」里改。
    """
    display_name = StringField("昵称", validators=[Optional(),
    Length(max=64)])
    gender = SelectField("性别", choices=GENDER_CHOICES,
    validators=[Optional()])
    email = StringField("邮箱", validators=[Optional(),
    email_format,
                                            Length(max=128)])
    phone = StringField("手机号", validators=[Optional(),
    Length(max=32)])
    submit_info = SubmitField("保存资料")
class RegisterForm(FlaskForm):
    """注册：账号与密码必填，其余选填。注册后默认是操作员。"""
    username = StringField("账号名称",
                           validators=[DataRequired("请填写账号名称
    "),
                                       Length(min=2, max=64,
                                              message="账号名称
    2-64 个字符")])
    password = PasswordField("密码",
                             validators=[DataRequired("请设置密码
    "),
                                         Length(min=6,
    max=64,

    message="密码至少 6 位")])
    confirm = PasswordField("确认密码",
                            validators=[DataRequired(),
                                        EqualTo("password",

    message="两次输入的密码不一致")])
    display_name = StringField("昵称", validators=[Optional(),
    Length(max=64)])
    gender = SelectField("性别", choices=GENDER_CHOICES,
    validators=[Optional()])
    email = StringField("邮箱", validators=[Optional(),
    email_format,
                                            Length(max=128)])
    phone = StringField("手机号", validators=[Optional(),
    Length(max=32)])
    submit_reg = SubmitField("注册并登录")
class PasswordForm(FlaskForm):
    """修改密码：需要先验证当前密码。"""
    old_password = PasswordField("当前密码",
    validators=[DataRequired()])
    new_password = PasswordField("新密码",
                                 validators=[DataRequired(),
    Length(min=6, max=64,

    message="新密码至少 6 位")])
    confirm = PasswordField("确认新密码",
                            validators=[DataRequired(),

        EqualTo("new_password",

        message="两次输入的新密码不一致")])
        submit_pwd = SubmitField("修改密码")
# ==== app/utils/constants.py ====
# -*- coding: utf-8 -*-
"""评价指标常量表。
全部数值取自论文，作为系统计算的唯一基准：
- 论文 4.2.2 节：指标标准化理想极值与取值依据
- 论文 4.2.3 节 / 表 4：适宜性等级划分
- 论文表 3：按微地质单元分别确定的一级、二级指标权重
- 论文 4.2.1 节：平均随机一致性指标 RI
"""
# ---------------------------------------------------------
        ------------------
# 一、微地质单元类型
# ---------------------------------------------------------
        ------------------
# 论文 4.1 节："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡
# 三种地质微单元类型"
UNIT_TYPES = ("岩质边坡", "碎石边坡", "平台")
# 每个单元类型启用的指标类别（论文表 3 隐含：岩质边坡无土壤指标，平台无地形指标）
# 供数据录入界面与评价引擎判断"该单元是否需要某类指标"
UNIT_ENABLED_INDICATORS = {
        "岩质边坡": ("植被指标", "地形指标"),          # 不设土壤指标
        "碎石边坡": ("土壤指标", "植被指标", "地形指标"),
        "平台": ("土壤指标", "植被指标"),                # 不设地形指标
}
# ---------------------------------------------------------
        ------------------
# 二、指标标准化理想极值（论文 4.2.2 节）
# ---------------------------------------------------------
        ------------------
IDEAL_SOIL_RATIO = 0.8          # 土石比理想值
IDEAL_THICKNESS = 20.0          # 土层厚度理想值 (cm)
IDEAL_PH_RANGE = (5.0, 9.0)     # pH 理想区间（适中型指标）
IDEAL_ORGANIC = 30.0            # 有机质理想值
        (g/kg)，全国第二次土壤普查养分等级划分标准二等最低值
IDEAL_COVERAGE = 75.0           # 植被覆盖度参照生态系统 (%)
IDEAL_SHANNON = 0.7             # Shannon-Wiener 多样性指数参照生态系统
# 重金属极值 (mg/kg)，《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值
# 论文原文：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、530、400、45、220、600、800
HEAVY_METALS = ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu",
        "Zn")
HEAVY_METAL_LIMIT = {
        "Cd": 2.0, "Hg": 2.0, "Pb": 530.0, "Cr": 400.0,
        "As": 45.0, "Ni": 220.0, "Cu": 600.0, "Zn": 800.0,
}
# 污染指标下 8 种重金属"重要性相当，权重平均分配"（论文 4.2.1 节）
HEAVY_METAL_EQUAL_WEIGHT = 1.0 / 8.0
```

```
# 坡度分界，论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号
SLOPE_BREAK = 25.0
# 坡向离散赋值，论文 4.2.2 节："阳坡、阴阳坡、阴坡分别赋值 0.2、0.6、1"
ASPECT_SCORES = {"阳坡": 0.2, "阴阳坡": 0.6, "阴坡": 1.0}
ASPECT_TYPES = ("阳坡", "阴阳坡", "阴坡")
# pH 适中型标准化的容差边界（区间外线性衰减，避免得分突变）
PH_TOL_LOW = 4.0
PH_TOL_HIGH = 10.0
# ----------------------------------------------------------
    -----------------
# 三、适宜性等级划分（论文表 4）
# ----------------------------------------------------------
    -----------------
GRADE_LEVELS = (
    {"grade": "不适宜", "min": None, "max": 0.3,
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",
     "measure": "生态重建、人工辅助"},
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},
    {"grade": "适宜", "min": 0.7, "max": None,
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",
     "measure": "自然恢复"},
)
def grade_of(score):
    """按论文表 4 判定适宜性等级。"""
    for g in GRADE_LEVELS:
        lo, hi = g["min"], g["max"]
        if lo is None and score < hi:
            return g
        if hi is None and score >= lo:
            return g
        if lo is not None and hi is not None and lo <=
    score < hi:
            return g
    # score 恰好等于 0.7 时落入"适宜"
    return GRADE_LEVELS[-1]
# ----------------------------------------------------------
    -----------------
# 四、平均随机一致性指标 RI（论文 4.2.1 节给出 n=2,3,4,5，其余为常用扩展值）
# ----------------------------------------------------------
    -----------------
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:
    1.24,
            7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}
# 一致性比例阈值（论文：当 CR<0.1 时，判断矩阵满足一致性要求）
CR_THRESHOLD = 0.1
# ----------------------------------------------------------
    -----------------
# 五、默认权重方案（论文表 3）
# -----------------------------------------------------------
    -----------------
# 论文 4.2.2 节："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，
# 分别计算评价指标权重"（论文摘要、图 2 流程与表 5 均作"碎石边坡"，
# 系统统一采用"碎石边坡"）。权重由 10 位专家打分后经 AHP 方法确定。
# 下表为论文表 3 所载的归一化权重，作为系统默认方案。
DEFAULT_WEIGHTS = {
    "岩质边坡": {
        "l1": {"植被指标": 0.45, "地形指标": 0.55},
        "l2": {
            "植被指标": {"植被覆盖度": 0.67, "生物多样性": 0.33},
            "地形指标": {"坡度": 0.64, "坡向": 0.36},
        },
    },
    "碎石边坡": {
        "l1": {"土壤指标": 0.44, "植被指标": 0.31, "地形指标": 0.25},
        "l2": {
            "土壤指标": {"土石比": 0.25, "土层厚度": 0.25, "pH": 0.17,
                         "有机质": 0.17, "污染指标": 0.17},
            "植被指标": {"植被覆盖度": 0.64, "生物多样性": 0.36},
            "地形指标": {"坡度": 0.67, "坡向": 0.33},
        },
    },
    "平台": {
        "l1": {"土壤指标": 0.54, "植被指标": 0.46},
        "l2": {
            "土壤指标": {"土石比": 0.25, "土层厚度": 0.25, "pH": 0.17,
                         "有机质": 0.17, "污染指标": 0.17},
            "植被指标": {"植被覆盖度": 0.61, "生物多样性": 0.39},
        },
    },
}
# -----------------------------------------------------------
    -----------------
# 六、限制性指标（论文表 2 + 4.1 节流程）
# -----------------------------------------------------------
    -----------------
# 论文摘要与 4.1 节：四项限制性指标；流程段另明确"是否为耕地"为约束条件
CONSTRAINT_ITEMS = ("政策", "区位约束", "土壤污染", "地质灾害", "耕地属性")
# -----------------------------------------------------------
    -----------------
# 七、账号职务
# -----------------------------------------------------------
    -----------------
# 只分两种：管理员可管理账号、维护基础数据并查看操作日志，操作员用业务功能。
ROLE_LABELS = {"admin": "管理员", "operator": "操作员"}
ROLE_CODES = ("admin", "operator")
DEFAULT_ROLE = "operator"          # 新注册账号的默认职务
ROLE_ORDER = ("admin", "operator")  # 列表排序用：管理员在前
# 性别（注册与个人资料中选填）
```

```
GENDERS = ("男", "女", "保密")
# ==== app/utils/schema.py ====
# -*- coding: utf-8 -*-
"""轻量结构迁移。
SQLite 下 `db.create_all()` 只建缺失的表，不会给已有表补列。
本模块在启动时检查目标列是否存在，缺失则 ALTER TABLE 补上，
使旧数据库无需重建即可用上新字段。
"""
from sqlalchemy import text
# (表名, 列名, 列定义)
REQUIRED_COLUMNS = [
    # 遥感影像挂接
    ("mine_patches", "image_path", "VARCHAR(256)"),
    ("mine_patches", "image_note", "VARCHAR(256)"),
    ("mine_patches", "image_uploaded_at", "DATETIME"),
    # 指标维护
    ("indicators", "enabled_units", "VARCHAR(128)"),
    ("indicators", "description", "TEXT"),
    # 多专家矩阵
    ("expert_matrices", "unit_type", "VARCHAR(32)"),
    ("expert_matrices", "l1_name", "VARCHAR(64)"),
    ("expert_matrices", "cr", "FLOAT"),
    # 用户个人资料（账户页）
    ("users", "display_name", "VARCHAR(64)"),
    ("users", "email", "VARCHAR(128)"),
    ("users", "phone", "VARCHAR(32)"),
    ("users", "gender", "VARCHAR(8)"),
    ("users", "avatar", "VARCHAR(256)"),
]
# 职务体系由「管理员 / 评价员 / 查看者」三种收敛为「管理员 / 操作员」两种，
# 旧数据里的后两种统一归为操作员。
ROLE_MIGRATION = {"evaluator": "operator", "viewer":
    "operator"}
def _add_columns(db):
    added = []
    with db.engine.connect() as conn:
        for table, column, ddl in REQUIRED_COLUMNS:
            rows = conn.execute(text("PRAGMA
    table_info(%s)" % table)).fetchall()
            if not rows:
                continue                        #
    表还不存在，create_all 会建
            existing = {r[1] for r in rows}
            if column in existing:
                continue
            conn.execute(text(
                "ALTER TABLE %s ADD COLUMN %s %s" % (table,
    column, ddl)))
            conn.commit()
            added.append("%s.%s" % (table, column))
    return added
def _migrate_roles(db):
    """把旧角色名统一为新的两种，返回改动的行数。"""
    changed = 0
    with db.engine.connect() as conn:
        rows = conn.execute(text("PRAGMA
    table_info(users)")).fetchall()
        if not rows:
            return 0
        for old, new in ROLE_MIGRATION.items():
            r = conn.execute(text("UPDATE users SET role =
    :new "
                                  "WHERE role = :old"),
                             {"new": new, "old": old})
            changed += r.rowcount or 0
        conn.commit()
    return changed
def ensure_schema(db):
    """为已有表补齐缺失列，并做必要的数据迁移。返回变更清单。"""
    try:
        changed = _add_columns(db)
        n = _migrate_roles(db)
        if n:
            changed.append("用户角色归一为管理员/操作员 %d 条" % n)
        return changed
    except Exception as e:                        #
    迁移失败不应阻断启动
        print("[schema] 结构检查跳过：%s" % e)
        return []
# ==== app/utils/uploads.py ====
# -*- coding: utf-8 -*-
"""上传文件处理（遥感影像、批量导入）。"""
import os
import re
import uuid
from datetime import datetime
from werkzeug.utils import secure_filename
def _safe_ext(filename):
    return os.path.splitext(filename)[1].lower().lstrip(".")
def ext_allowed(filename, allowed):
    return _safe_ext(filename) in allowed
def save_upload(file_storage, subdir, allowed_ext,
    upload_root):
    """保存上传文件，返回相对 static 的路径（如 uploads/patches/xxx.png）。
    文件名重命名为「时间戳_随机串.扩展名」，避免中文名与重名问题。
    """
    if file_storage is None or not file_storage.filename:
        return None, "未选择文件"
    ext = _safe_ext(file_storage.filename)
    if ext not in allowed_ext:
        return None, "不支持的文件类型：.%s" % ext
    target_dir = os.path.join(upload_root, subdir)
    os.makedirs(target_dir, exist_ok=True)
    # 仅保留安全字符，并加时间戳与随机串防重名
    base = secure_filename(os.path.splitext(file_storage.fil
    ename)[0]) or "file"
    base = re.sub(r"[^A-Za-z0-9_\-]", "", base)[:40] or
    "file"
    name = "%s_%s.%s" % (datetime.now().strftime("%Y%m%d%H%M
    %S"),
                         uuid.uuid4().hex[:6], ext)
    abs_path = os.path.join(target_dir, name)
    file_storage.save(abs_path)
    rel = os.path.join("uploads", subdir,
    name).replace("\\", "/")
    return rel, None
def remove_upload(rel_path, upload_root):
    """删除已保存的上传文件（相对 static 的路径）。"""
    if not rel_path:
        return
    abs_path = os.path.join(upload_root,
    rel_path.replace("uploads/", "", 1))
    try:
        if os.path.isfile(abs_path):
            os.remove(abs_path)
    except OSError:
        pass
# ==== app/utils/charts.py ====
# -*- coding: utf-8 -*-
"""轻量图表。
用纯 SVG 输出，不依赖任何绘图库或前端脚本：
- 服务端渲染，页面与打印（PDF）中都能稳定显示
- 每个图元都带数值标签，黑白打印时仍可读（不单靠颜色区分）
- 单色方案，符合界面「一个视图一个强调色」的约定
"""
from html import escape
ACCENT = "#1f3a5f"
ACCENT_SOFT = "#7f93ad"
INK = "#23262b"
MUTED = "#6b7280"
LINE = "#dfe3e8"
# 灰度梯度：黑白打印时相邻扇区/条柱仍可区分
GRAYS = ["#1f3a5f", "#456188", "#6d86a8", "#96a9c4",
    "#bcc9da", "#dbe3ec"]
def _fmt(v, digits=4):
    try:
        return ("%%.%df" % digits) % float(v)
    except (TypeError, ValueError):
        return str(v)
def hbars(items, width=620, bar_h=22, gap=12, label_w=150,
    value_w=76):
    """水平条形图。items: [(标签, 数值, 显示文本)]"""
    if not items:
        return '<p class="muted">暂无数据</p>'
    n = len(items)
    height = n * (bar_h + gap) + 10
    max_v = max((abs(float(v)) for _, v, _ in items),
    default=1) or 1
    body_w = width - label_w - value_w
    parts = ['<svg viewBox="0 0 %d %d" width="100%%"
    height="%d" '
             'role="img" aria-label="条形图" '
             'style="max-width:%dpx;font-family:inherit;">'
    % (width, height, height, width)]
    for i, (label, value, disp) in enumerate(items):
        y = i * (bar_h + gap) + 5
        frac = (abs(float(value)) / max_v) if max_v else 0
        w = max(2.0, body_w * frac)
        fill = GRAYS[i % len(GRAYS)]
        parts.append(
            '<text x="%d" y="%d" text-anchor="end"
    font-size="12" fill="%s">%s</text>'
            % (label_w - 10, y + bar_h * 0.72, INK,
    escape(str(label))))
        parts.append(
            '<rect x="%d" y="%d" width="%.1f" height="%d"
    rx="3" fill="%s"/>'
            % (label_w, y, w, bar_h, fill))
        parts.append(
            '<text x="%d" y="%d" font-size="12" fill="%s" '
            'style="font-variant-numeric:tabular-nums;">%s</
    text>'
            % (label_w + w + 8, y + bar_h * 0.72, MUTED,
    escape(str(disp))))
    parts.append('</svg>')
    return "".join(parts)
def donut(items, size=210, thickness=34):
    """环形图（权重分布）。items: [(标签, 数值, 显示文本)]
    图例中给出数值与占比，黑白打印时靠图例与灰度深浅区分。
    """
    vals = [(l, float(v), d) for l, v, d in items if
    float(v) > 0]
    if not vals:
        return '<p class="muted">暂无数据</p>'
    total = sum(v for _, v, _ in vals) or 1
    r = (size - thickness) / 2
    cx = cy = size / 2
    import math
    parts = ['<svg viewBox="0 0 %d %d" width="%d"
    height="%d" role="img" '
```

```
             'aria-label="权重分布环形图">' % (size, size, size,
    size)]
    angle = -math.pi / 2
    for i, (label, v, disp) in enumerate(vals):
        sweep = 2 * math.pi * (v / total)
        a0, a1 = angle, angle + sweep
        x0, y0 = cx + r * math.cos(a0), cy + r *
    math.sin(a0)
        x1, y1 = cx + r * math.cos(a1), cy + r *
    math.sin(a1)
        large = 1 if sweep > math.pi else 0
        parts.append(
            '<path d="M %.2f %.2f A %.2f %.2f 0 %d 1 %.2f
    %.2f" '
            'fill="none" stroke="%s" stroke-width="%d"/>'
            % (x0, y0, r, r, large, x1, y1, GRAYS[i %
    len(GRAYS)], thickness))
        angle = a1
    parts.append('<text x="%d" y="%d" text-anchor="middle"
    font-size="12" '
                 'fill="%s">合计</text>' % (cx, cy - 2,
    MUTED))
    parts.append('<text x="%d" y="%d" text-anchor="middle"
    font-size="15" '
                 'fill="%s" style="font-variant-numeric:tabu
    lar-nums;">%s</text>'
                 % (cx, cy + 18, INK, _fmt(total, 4)))
    parts.append('</svg>')
    legend = ['<table style="width:auto;margin-top:10px;"><t
    head><tr>'
              '<th scope="col">项目</th><th
    scope="col">数值</th>'
              '<th scope="col">占比</th></tr></thead><tbody>']
    for i, (label, v, disp) in enumerate(vals):
        legend.append(
            '<tr><td style="text-align:left;">'
            '<span style="display:inline-block;width:10px;he
    ight:10px;'
            'border-radius:2px;background:%s;margin-right:6p
    x;"></span>%s</td>'
            '<td class="mono">%s</td><td
    class="mono">%.1f%%</td></tr>'
            % (GRAYS[i % len(GRAYS)], escape(str(label)),
    escape(str(disp)),
               100.0 * v / total))
    legend.append('</tbody></table>')
    return ('<div style="display:flex;flex-wrap:wrap;gap:20p
    x;align-items:center;">'
            + "".join(parts) + "".join(legend) + '</div>')
# ==== app/utils/browser.py ====
# -*- coding: utf-8 -*-
"""HTML 转 PDF（报告导出）。
复用本机已安装的 Chrome/Edge 无头模式打印，避免引入 reportlab、
weasyprint 等额外依赖。生成过程使用系统临时目录中的 ASCII 文件名，
避免中文路径在 file:// URL 下出问题。
"""
import os
import shutil
import subprocess
import tempfile
import uuid
CANDIDATES = [
    r"C:\Program Files\Google\Chrome\Application\chrome.exe"
    ,
    r"C:\Program Files (x86)\Google\Chrome\Application\chrom
    e.exe",
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msed
    ge.exe",
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe
    ",
]
# 允许通过环境变量覆盖，便于在其他机器上指定浏览器
ENV_KEY = "DSH_BROWSER_PATH"
def browser_path():
    """定位可用的浏览器可执行文件，找不到返回 None。"""
    override = os.environ.get(ENV_KEY)
    if override and os.path.isfile(override):
        return override
    for p in CANDIDATES:
        if os.path.isfile(p):
            return p
    for name in ("chrome", "msedge", "chromium"):
        found = shutil.which(name)
        if found:
            return found
    return None
def html_to_pdf(html, timeout=90):
    """把 HTML 字符串渲染为 PDF，返回字节；失败返回 (None, 原因)。"""
    exe = browser_path()
    if exe is None:
        return None, ("未找到可用的浏览器（Chrome/Edge），无法生成 PDF。"
                      "可设置环境变量 %s 指向浏览器可执行文件。" % ENV_KEY)
    tmpdir = tempfile.mkdtemp(prefix="dshpdf_")
    tag = uuid.uuid4().hex[:8]
    html_path = os.path.join(tmpdir, "r_%s.html" % tag)
    pdf_path = os.path.join(tmpdir, "r_%s.pdf" % tag)
    try:
        with open(html_path, "w", encoding="utf-8") as f:
            f.write(html)
        cmd = [
            exe, "--headless=new", "--disable-gpu",
    "--no-sandbox",
            "--no-first-run", "--no-default-browser-check",
            "--no-pdf-header-footer",
            "--virtual-time-budget=4000",
            "--print-to-pdf=%s" % pdf_path,
            "file:///" + html_path.replace("\\", "/"),
        ]
        p = subprocess.run(cmd, capture_output=True,
    text=True,
                           timeout=timeout,
    encoding="utf-8", errors="replace")
        if not os.path.isfile(pdf_path) or
    os.path.getsize(pdf_path) < 800:
            return None, "浏览器未能生成 PDF（%s）" % (p.stderr or
    "").strip()[:200]
        with open(pdf_path, "rb") as f:
            return f.read(), None
    except subprocess.TimeoutExpired:
        return None, "生成 PDF 超时，请稍后重试"
    except Exception as e:
        return None, "生成 PDF 失败：%s" % e
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True)
# ==== app/services/ahp_service.py ====
# -*- coding: utf-8 -*-
"""AHP 权重引擎。
严格按论文 4.2.1 节实现：
- 式(2)(3)(4)：特征根法求权重（行元素乘积 → n 次方根 → 归一化）
- 式(5)(6)(7)：一致性检验（λmax、CI、CR）
- 论文要求 CR < 0.1；RI 按阶数查表
"""
import numpy as np
from app.utils.constants import RI_TABLE, CR_THRESHOLD
def matrix_from_upper(upper):
    """由判断矩阵上三角构造完整互反矩阵。
    参数
    ----
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
            A[j, i] = 1.0 / upper[idx]
            idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict：{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
    返回
    ----
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)
    及对应取值。
    """
    A = np.asarray(A, dtype=float)
    r = weights_and_consistency(A)
    w = r["weights"]
    n = A.shape[0]
    best = None
    for i in range(n):
        for j in range(i + 1, n):
            expected = w[i] / w[j]
            if expected == 0:
                continue
            deviation = abs(A[i, j] - expected) / expected
            if best is None or deviation >
    best["deviation"]:
                best = {"i": i, "j": j, "value": A[i, j],
                        "expected": expected, "deviation":
    deviation}
    return best
def aggregate_matrices(matrices):
    """多专家判断矩阵聚合：几何平均（保持互反性）。
    参数
    ----
    matrices : list[np.ndarray]，多个同阶判断矩阵。
    """
    stack = np.stack([np.asarray(m, dtype=float) for m in
    matrices])
    return np.prod(stack, axis=0) ** (1.0 / len(stack))
def judge_from_weights(weights):
    """由权重向量构造完全一致判断矩阵并检验。
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。
    """
    w = np.asarray(weights, dtype=float)
    n = w.shape[0]
    A = np.outer(w, 1.0 / w)
    r = weights_and_consistency(A)
    r["matrix"] = A
    return r
# ==== app/services/standardization.py ====
# -*- coding: utf-8 -*-
"""指标标准化（的一部分）。
按论文 4.2.2 节式(8) 实现：
    x' = (x - x_min) / (x_max - x_min)
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。
所有理想极值取自 app.utils.constants，可配置。
"""
from app.utils.constants import (
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,
    PH_TOL_LOW, PH_TOL_HIGH,
)
def std_benefit(value, ideal):
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, value / ideal))
```

```
def std_cost(value, extreme):
    """越小越优指标标准化：以极值为参考，线性折减至 [0, 1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, 1.0 - value / extreme))
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,
    tol_hi=PH_TOL_HIGH):
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""
    if value is None:
        return None
    if lo <= value <= hi:
        return 1.0
    if value > hi:
        return max(0.0, (tol_hi - value) / (tol_hi - hi))
    return max(0.0, (value - tol_lo) / (lo - tol_lo))
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):
    """坡度标准化。
    论文 4.2.2 节：以 25° 为陡坡重要分界。25° 及以下记满分 1，
    超过 25° 线性折减，90° 记 0。
    """
    if value is None:
        return None
    if value <= break_point:
        return 1.0
    return max(0.0, (upper - value) / (upper - break_point))
def aspect_score(aspect):
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（论文 4.2.2 节）。"""
    if aspect is None:
        return None
    return ASPECT_SCORES.get(aspect)
def pollution_score(heavy_metals):
    """污染指标：8 种重金属等权（各 0.125）合成。
    论文注：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染(=1)处理。
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。
    """
    total = 0.0
    for metal, extreme in HEAVY_METAL_LIMIT.items():
        value = heavy_metals.get(metal)
        total += std_cost(value, extreme) if value is not
    None else 1.0
    return total / len(HEAVY_METAL_LIMIT)
def standardize_unit(unit_type, data, weights=None):
    """对单个微地质单元的全部指标做标准化。
    参数
    ----
    unit_type : str，岩质边坡 / 碎石边坡 / 平台
    data : dict，实测数据，字段见接口契约：
        soil_ratio, thickness, ph, organic_matter,
    heavy_metals(dict),
        coverage, shannon, slope, aspect
```

```
weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIGHTS 结构），
          未提供时用默认值。
返回
----
dict：{一级指标: {二级指标: (实测值, 标准化值, 权重, 贡献)}}
标准化值可能为 None，表示该指标不适用或方法待定。
"""
from app.utils.constants import DEFAULT_WEIGHTS
if weights is None:
    weights = DEFAULT_WEIGHTS[unit_type]
l1_enabled = weights["l1"].keys()
l2w = weights["l2"]
result = {}
# ---- 土壤指标 ----
if "土壤指标" in l1_enabled:
    result["土壤指标"] = {
        "土石比": (data.get("soil_ratio"),
                   std_benefit(data.get("soil_ratio"),
IDEAL_SOIL_RATIO),
                   l2w["土壤指标"]["土石比"]),
        "土层厚度": (data.get("thickness"),
                    std_benefit(data.get("thickness"),
IDEAL_THICKNESS),
                    l2w["土壤指标"]["土层厚度"]),
        "pH": (data.get("ph"),
               std_mid_ph(data.get("ph"),
*IDEAL_PH_RANGE),
               l2w["土壤指标"]["pH"]),
        "有机质": (data.get("organic_matter"),
                  std_benefit(data.get("organic_matter"), IDEAL_ORGANIC),
                  l2w["土壤指标"]["有机质"]),
        "污染指标": (data.get("heavy_metals") or {},
                    pollution_score(data.get("heavy_metals") or {}),
                    l2w["土壤指标"]["污染指标"]),
    }
# ---- 植被指标 ----
if "植被指标" in l1_enabled:
    result["植被指标"] = {
        "植被覆盖度": (data.get("coverage"),
                      std_benefit(data.get("coverage"),
IDEAL_COVERAGE),
                      l2w["植被指标"]["植被覆盖度"]),
        "生物多样性": (data.get("shannon"),
                      std_benefit(data.get("shannon"),
IDEAL_SHANNON),
                      l2w["植被指标"]["生物多样性"]),
    }
    # ---- 地形指标 ----
    if "地形指标" in l1_enabled:
        result["地形指标"] = {
            "坡度": (data.get("slope"),
                     std_slope(data.get("slope")),
                     l2w["地形指标"]["坡度"]),
            "坡向": (data.get("aspect"),
                     aspect_score(data.get("aspect")),
                     l2w["地形指标"]["坡向"]),
        }
    return result
# ==== app/services/weight_service.py ====
# -*- coding: utf-8 -*-
"""权重方案的存取与生效权重解析。
系统有两套权重来源：
1. 论文表 3 给出的默认权重（app.utils.constants.DEFAULT_WEIGHTS），
   是**权威取值来源**，案例复算与报告结论以此为准；
2. 通过层次分析法在本系统中实际算出的权重，保存在
   weight_schemes / weight_items 两张表中，可标记为「生效」。
本模块负责把这两者合成为计算引擎直接可用的嵌套权重结构：
以论文表 3 为底，用生效方案中已保存的条目覆盖。
没有任何生效方案时，即完全使用论文表 3 的权重。
这样既保证默认行为与论文一致，又使 AHP 计算结果真正参与评价，
而不是算完即弃。
"""
import copy
from app.models import db, WeightScheme, WeightItem
from app.utils.constants import DEFAULT_WEIGHTS
from app.services.indicator_service import TREE
# 方案名称：每个微地质单元类型固定一个可生效的方案
ACTIVE_NAME = "自定义权重方案（AHP 计算结果）"
def _code_maps():
    """由指标树导出 编码 → 名称 的映射，避免名称在两处各写一遍。"""
    l1, l2 = {}, {}
    for row in TREE:
        code, name, level, parent = row[0], row[1], row[2],
    row[3]
        if level == 1:
            l1[code] = name
        elif level == 2:
            l2[code] = (l1.get(parent, parent), name)
    return l1, l2
def effective_weights(unit_type):
    """返回该单元类型当前生效的权重。
    返回 (weights, source_label, scheme)
        weights      嵌套权重结构，可直接交给标准化与评价引擎
        source_label 权重来源说明，用于界面与报告标注
        scheme       生效的权重方案对象，无自定义方案时为 None
    """
    base = DEFAULT_WEIGHTS.get(unit_type)
    if base is None:
        return None, "未定义该微地质单元类型的权重", None
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ
    e,

    name=ACTIVE_NAME).first()
    if scheme is None:
        return copy.deepcopy(base), "论文表 3 默认权重", None
    merged = copy.deepcopy(base)
    l1_map, l2_map = _code_maps()
    applied = 0
    for item in WeightItem.query.filter_by(scheme_id=scheme.
    id).all():
        code, w = item.indicator_code, item.weight
        if w is None or w <= 0:
            continue
        if code in l1_map and l1_map[code] in merged["l1"]:
            merged["l1"][l1_map[code]] = w
            applied += 1
        elif code in l2_map:
            l1_name, l2_name = l2_map[code]
            if l1_name in merged["l2"] and l2_name in
    merged["l2"][l1_name]:
                merged["l2"][l1_name][l2_name] = w
                applied += 1
    if applied == 0:
        return copy.deepcopy(base), "论文表 3 默认权重", None
    return merged, "自定义方案（%d 项覆盖论文表 3）" % applied, scheme
def save_weights(unit_type, code_weight_pairs, note=None):
    """把一组「指标编码 → 权重」写入生效方案（已存在则更新对应条目）。
    仅覆盖本次传入的条目，其余条目保留，便于分层次（一级 / 二级）
    分次录入而无需一次填完。
    """
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ
    e,

    name=ACTIVE_NAME).first()
    if scheme is None:
        scheme = WeightScheme(unit_type=unit_type,
    name=ACTIVE_NAME,
                              level=1, is_default=True)
        db.session.add(scheme)
        db.session.flush()
    if note:
        scheme.level = scheme.level or 1
    if not scheme.is_default:
        scheme.is_default = True
    written = 0
    for code, w in code_weight_pairs:
        if w is None or w <= 0:
            continue
        item = WeightItem.query.filter_by(scheme_id=scheme.i
    d,

    indicator_code=code).first()
        if item is None:
            item = WeightItem(scheme_id=scheme.id,
    indicator_code=code)
            db.session.add(item)
        item.weight = float(w)
        written += 1
    db.session.commit()
    return scheme, written
def reset_weights(unit_type):
    """删除该单元类型的自定义方案，恢复为论文表3的默认权重。"""
    schemes = WeightScheme.query.filter_by(unit_type=unit_ty
    pe,

    name=ACTIVE_NAME).all()
    n = 0
    for s in schemes:
        WeightItem.query.filter_by(scheme_id=s.id).delete(
            synchronize_session=False)
        db.session.delete(s)
        n += 1
    db.session.commit()
    return n
def scheme_items(unit_type):
    """返回该单元类型已保存的自定义权重条目 {指标编码: 权重}。"""
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ
    e,

    name=ACTIVE_NAME).first()
    if scheme is None:
        return {}, None
    items = {i.indicator_code: i.weight
             for i in WeightItem.query.filter_by(scheme_id=s
    cheme.id).all()}
    return items, scheme
def summary():
    """各单元类型的权重来源概览，供权重首页展示。"""
    rows = []
    for unit_type in DEFAULT_WEIGHTS:
        weights, label, scheme = effective_weights(unit_type
    )
        rows.append({
            "unit_type": unit_type,
            "label": label,
            "custom": scheme is not None,
        })
    return rows
# ==== app/services/evaluation_service.py ====
# -*- coding: utf-8 -*-
"""适宜性评价引擎。
按论文式(9) 实现：
- 单元级：F' = Σ（二级指标权重 × 该二级指标下三级指标标准化值）
- 矿山级：F = Σ（Si × F'i），Si 为各微地质单元面积占比
- 等级判定按论文表 4。
"""
from app.utils.constants import DEFAULT_WEIGHTS, grade_of
from app.services import standardization as std
def score_unit(unit_type, data, weights=None):
    """计算单个微地质单元的评价因子 F'。
    返回
    ----
    dict：{ l1_scores: {一级指标: 得分}, unit_score: F', details:
    [...] }
    若某一级指标下存在"不适用"（标准化值为 None）的二级指标，
    则按其余项权重归一化后计算，保证单元得分始终有定义。
    """
    if weights is None:
        weights = DEFAULT_WEIGHTS[unit_type]
    std_result = std.standardize_unit(unit_type, data,
    weights)
    l1_weights = weights["l1"]
    l1_scores = {}
    details = []
    for l1_name, items in std_result.items():
        wsum = 0.0
        acc = 0.0
        for l2_name, (raw, std_value, w2) in items.items():
            if std_value is None:
                # 不适用项：不计入
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw":
    raw,
                    "std_value": None, "weight": w2,
    "contribution": None,
                })
                continue
            contribution = std_value * w2
            acc += contribution
            wsum += w2
            details.append({
                "l1": l1_name, "l2": l2_name, "raw": raw,
                "std_value": std_value, "weight": w2,
                "contribution": contribution,
            })
        l1_scores[l1_name] = acc / wsum if wsum > 0 else 0.0
    # 一级指标权重按实际参与计算的指标归一化。
    # 论文表 3 的权重本身已归一（和为 1），此处为恒等变换；
    # 若某层指标缺失或自定义权重不完整，可避免单元得分被整体压低。
    lw_sum = sum(l1_weights[k] for k in l1_scores if k in
l1_weights)
    if lw_sum > 0:
        unit_score = sum(l1_weights[k] * l1_scores[k]
                         for k in l1_scores if k in
l1_weights) / lw_sum
    else:
        unit_score = 0.0
    return {"l1_scores": l1_scores, "unit_score":
unit_score, "details": details}
def score_mine(units, weights_map=None):
    """计算整个矿山的评价因子 F 与适宜性等级。
    参数
    ----
    units : list[dict]，每个元素：
        {unit_type, ratio, data}
        ratio 为该单元面积占比（0~1），data 为实测数据 dict。
    weights_map : dict，{单元类型: 权重结构}。未提供的单元类型
        使用论文表 3 的默认权重。权重可来自 AHP 计算并经
        weight_service 保存生效，从而真正参与评价。
    返回
    ----
    dict：{ final_score, grade, grade_info, unit_results }
    """
    weights_map = weights_map or {}
    total = 0.0
    unit_results = []
    for u in units:
        r = score_unit(u["unit_type"], u["data"],
                       weights_map.get(u["unit_type"]))
        weighted = u["ratio"] * r["unit_score"]
        total += weighted
        unit_results.append({
            "unit_type": u["unit_type"],
            "ratio": u["ratio"],
            "unit_score": r["unit_score"],
            "weighted": weighted,
            "l1_scores": r["l1_scores"],
            "details": r["details"],
        })
    grade_info = grade_of(total)
    return {
        "final_score": total,
        "grade": grade_info["grade"],
        "grade_info": grade_info,
        "unit_results": unit_results,
    }
# ==== app/services/report_service.py ====
# -*- coding: utf-8 -*-
"""报告生成：生成 Word 评价报告。
依赖 python-docx。报告结构：
标题 → 图斑信息 → 评价结论 → 各微地质单元得分 → 完整得分明细 → 生成时间。
"""
import io
from datetime import datetime
from docx import Document
from docx.shared import Pt, Cm
from docx.oxml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
def _set_run(run, text=None, size=12, bold=False):
    if text is not None:
        run.text = text
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    # 中文字体
    run._element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")
    return run
def _para(doc, text, size=12, bold=False, align=None,
    space_after=6):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    _set_run(p.add_run(), text, size, bold)
    return p
def _heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for run in h.runs:
        run.font.name = "Times New Roman"
        run._element.rPr.rFonts.set(qn("w:eastAsia"), "黑体")
    return h
def _table(doc, headers, rows, widths=None):
    t = doc.add_table(rows=1, cols=len(headers))
    t.style = "Table Grid"
    hdr = t.rows[0].cells
    for i, htext in enumerate(headers):
        _set_run(hdr[i].paragraphs[0].add_run(), htext,
    10.5, bold=True)
    for row in rows:
        cells = t.add_row().cells
        for i, val in enumerate(row):
            _set_run(cells[i].paragraphs[0].add_run(),
    str(val), 10.5)
    return t
def build_report(patch, mine_result):
    """生成评价报告 Word 文档，返回 BytesIO。
    参数
```

```
----
patch : MinePatch，图斑对象
mine_result : evaluation_service.score_mine 的返回 dict
"""
doc = Document()
# 页边距
for section in doc.sections:
    section.top_margin = Cm(2.5)
    section.bottom_margin = Cm(2.5)
    section.left_margin = Cm(2.5)
    section.right_margin = Cm(2.5)
_heading(doc, "历史遗留废弃矿山自然恢复适宜性评价报告", 0)
_para(doc, "软件：历史遗留废弃矿山自然恢复适宜性评价系统 V1.0",
      size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER)
_para(doc, "生成时间：%s" % datetime.now().strftime("%Y-%m-%d
%H:%M:%S"),
      size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER,
space_after=12)
# 图斑信息
_heading(doc, "一、图斑基本信息", 1)
info = [
    ("图斑名称", patch.name),
    ("所在行政区", patch.admin_region or "—"),
    ("图斑面积（公顷）", str(patch.area_ha) if patch.area_ha is
not None else "—"),
    ("开采矿种", patch.mineral_type or "—"),
    ("开采方式", patch.mining_method or "—"),
]
_table(doc, ["项目", "内容"], info)
# 评价结论
_heading(doc, "二、评价结论", 1)
gi = mine_result["grade_info"]
concl = [
    ("最终评价因子 F", "%.4f" % mine_result["final_score"]),
    ("适宜性等级", mine_result["grade"]),
    ("判定依据", gi["meaning"]),
    ("推荐措施", gi["measure"]),
]
_table(doc, ["项目", "结果"], concl)
# 各单元得分
_heading(doc, "三、各微地质单元得分", 1)
rows = []
for u in mine_result["unit_results"]:
    rows.append((u["unit_type"], "%.2f%%" % (u["ratio"]
* 100),
                         "%.4f" % u["unit_score"], "%.4f" %
u["weighted"]))
_table(doc, ["微地质单元", "面积占比", "单元评价因子 F'", "加权贡献"],
rows)
# 完整得分明细
```

```
    _heading(doc, "四、得分明细", 1)
    for u in mine_result["unit_results"]:
        _para(doc, "%s（占比 %.2f%%）" % (u["unit_type"],
    u["ratio"] * 100),
              size=11, bold=True, space_after=4)
        detail_rows = []
        for d in u["details"]:
            raw = "—" if d["raw"] is None else d["raw"]
            std = "—" if d["std_value"] is None else "%.4f"
    % d["std_value"]
            w = "%.4f" % d["weight"]
            c = "—" if d["contribution"] is None else
    "%.4f" % d["contribution"]
            detail_rows.append((d["l1"], d["l2"], raw, std,
    w, c))
        _table(doc, ["一级指标", "二级指标", "实测值", "标准化值", "权重",
    "贡献"],
               detail_rows)
        # 一级指标得分
        l1_row = []
        for k, v in u["l1_scores"].items():
            l1_row.append("%s=%.4f" % (k, v))
        _para(doc, "一级指标得分：" + "，".join(l1_row), size=10.5,
    space_after=10)
    buf = io.BytesIO()
    doc.save(buf)
    buf.seek(0)
    return buf
# ==== app/services/indicator_service.py ====
# -*- coding: utf-8 -*-
"""指标定义维护。
把论文表 2/表 3 的指标体系写入 indicators 表，使指标名称、单位、
理想值与取值依据可在界面上查看维护，而不再只存在于代码常量中。
代码常量（app.utils.constants）仍是计算时的**权威取值来源**；
本表用于展示、核对与记录取值依据。二者由 seed_indicators() 保持同步。
"""
from app.models import db, Indicator
from app.utils.constants import (
    UNIT_TYPES, UNIT_ENABLED_INDICATORS, IDEAL_SOIL_RATIO,
    IDEAL_THICKNESS,
    IDEAL_PH_RANGE, IDEAL_ORGANIC, IDEAL_COVERAGE,
    IDEAL_SHANNON,
    HEAVY_METAL_LIMIT, SLOPE_BREAK, ASPECT_SCORES,
)
ALL_UNITS = ",".join(UNIT_TYPES)
# (code, name, level, parent, unit, ideal_value,
    ideal_type, enabled_units, desc, is_constraint)
TREE = [
    # ---- 一级指标 ----
    ("soil", "土壤指标", 1, None, None, None, None, "碎石边坡,平台",
    "论文表 2 评价指标之一级指标", False),
    ("veg", "植被指标", 1, None, None, None, None, ALL_UNITS,
    "论文表 2 评价指标之一级指标", False),
    ("terrain", "地形指标", 1, None, None, None, None,
    "岩质边坡,碎石边坡", "论文表 2 评价指标之一级指标", False),
    # ---- 二级指标：土壤 ----
    ("soil_ratio", "土石比", 2, "soil", "—",
    str(IDEAL_SOIL_RATIO), "benefit", "碎石边坡,平台",
     "论文 4.2.2 节：理想值 0.8，根据矿山修复实际情况确定", False),
    ("thickness", "土层厚度", 2, "soil", "cm",
    str(IDEAL_THICKNESS), "benefit", "碎石边坡,平台",
     "论文 4.2.2 节：理想值 20 cm", False),
    ("ph", "pH", 2, "soil", "—", "%g~%g" % IDEAL_PH_RANGE,
    "mid", "碎石边坡,平台",
     "论文 4.2.2 节：pH 用适中指标的标准化方法，理想区间 %g~%g" %
    IDEAL_PH_RANGE, False),
    ("organic_matter", "有机质", 2, "soil", "g/kg",
    str(IDEAL_ORGANIC), "benefit", "碎石边坡,平台",
     "《全国第二次土壤普查养分等级划分标准》二等（较丰富）最低值 30 g/kg", False),
    ("pollution", "污染指标", 2, "soil", "mg/kg", "见三级指标",
    "cost", "碎石边坡,平台",
     "同时作为限制性指标与评价指标；8 种重金属等权合成（各 0.125）", True),
    # ---- 二级指标：植被 ----
    ("coverage", "植被覆盖度", 2, "veg", "%",
    str(IDEAL_COVERAGE), "benefit", ALL_UNITS,
     "论文 4.2.2 节：参照生态系统取值 75%", False),
    ("shannon", "生物多样性", 2, "veg", "—", str(IDEAL_SHANNON),
    "benefit", ALL_UNITS,
     "Shannon-Wiener 多样性指数，参照生态系统取值 0.7", False),
    # ---- 二级指标：地形 ----
    ("slope", "坡度", 2, "terrain", "°", "%g° 分界" %
    SLOPE_BREAK, "cost", "岩质边坡,碎石边坡",
     "论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号：25°
    为陡坡重要分界", False),
    ("aspect", "坡向", 2, "terrain", "—",
    "阳坡0.2/阴阳坡0.6/阴坡1.0", "discrete", "岩质边坡,碎石边坡",
     "论文 4.2.2 节：阴坡及半阴坡水分条件更优，故赋值高于阳坡", False),
]
# 污染指标下的三级指标（8 种重金属）
for _m, _limit in HEAVY_METAL_LIMIT.items():
    TREE.append((
        "hm_" + _m, _m, 3, "pollution", "mg/kg",
    str(_limit), "cost",
        "碎石边坡,平台",
        "《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值 %g mg/kg" %
    _limit,
        False,
    ))
def seed_indicators():
    """写入/更新指标定义（幂等）。返回写入条数。"""
    code_map = {
        "benefit": "越大越优", "cost": "越小越优",
        "mid": "适中型", "discrete": "离散赋值",
    }
    n = 0
    for (code, name, level, parent, unit, ideal, itype,
    units, desc, is_con) in TREE:
        ind = Indicator.query.filter_by(code=code).first()
        if ind is None:
            ind = Indicator(code=code)
            db.session.add(ind)
        ind.name = name
        ind.level = level
        ind.parent_code = parent
        ind.unit = unit
        ind.ideal_value = ideal
        ind.ideal_type = itype
        ind.enabled_units = units
        ind.description = desc
        ind.is_constraint = is_con
        ind.is_evaluative = (level in (2, 3))
        n += 1
    db.session.commit()
    return n
def ideal_type_label(code):
    return {"benefit": "越大越优", "cost": "越小越优",
            "mid": "适中型", "discrete": "离散赋值"}.get(code,
    code or "—")
# ==== app/services/import_service.py ====
# -*- coding: utf-8 -*-
"""指标数据批量导入。
支持 CSV 与 Excel(.xlsx/.xls)。一行对应一个微地质单元，
按「单元类型」匹配图斑内已有的微地质单元后写入指标数据。
表头约定（缺失列按不适用处理）：
    单元类型, 土石比, 土层厚度, pH, 有机质,
    Hg, As, Ni, Cu, Cd, Pb, Cr, Zn,
    植被覆盖度, Shannon, 坡度, 坡向
"""
import csv
import io
import json
from app.utils.constants import HEAVY_METALS, ASPECT_TYPES,
    UNIT_TYPES
# 表头（顺序即导出模板顺序）
HEADERS = (["单元类型", "土石比", "土层厚度", "pH", "有机质"]
           + list(HEAVY_METALS)
           + ["植被覆盖度", "Shannon", "坡度", "坡向"])
FIELD_ALIASES = {
    "单元类型": "unit_type",
    "土石比": "soil_ratio",
    "土层厚度": "thickness",
    "pH": "ph", "PH": "ph", "ph": "ph",
    "有机质": "organic_matter",
    "植被覆盖度": "coverage",
    "Shannon": "shannon", "shannon": "shannon", "生物多样性":
    "shannon",
    "坡度": "slope",
    "坡向": "aspect",
}
for _m in HEAVY_METALS:
    FIELD_ALIASES[_m] = "hm_" + _m
def build_template_csv():
    """生成导入模板（含表头与示例行）。"""
    buf = io.StringIO()
    w = csv.writer(buf)
    w.writerow(HEADERS)
    # 示例：碎石边坡一行，说明每列含义
    row = {h: "" for h in HEADERS}
    row.update({"单元类型": "碎石边坡", "土石比": "0.34", "土层厚度": "10",
                "pH": "9.24", "有机质": "4.89", "Hg": "0.05",
    "As": "5.17",
                "Ni": "12.60", "Cu": "11.70", "Cd": "0.04",
    "Pb": "6.50",
                "植被覆盖度": "87.92", "Shannon": "0.60",
                "坡度": "28.75", "坡向": "阳坡"})
    w.writerow([row[h] for h in HEADERS])
    return buf.getvalue()
def _read_rows(filename, stream):
    """读取上传文件为「表头 -> 值」的字典列表。"""
    ext = filename.rsplit(".", 1)[-1].lower()
    if ext == "csv":
        text = stream.read().decode("utf-8-sig",
    errors="replace")
        reader = csv.DictReader(io.StringIO(text))
        return [dict(r) for r in reader]
    if ext in ("xlsx", "xls"):
        from openpyxl import load_workbook
        wb = load_workbook(io.BytesIO(stream.read()),
    data_only=True)
        ws = wb.active
        rows = list(ws.iter_rows(values_only=True))
        if not rows:
            return []
        header = [str(c).strip() if c is not None else ""
    for c in rows[0]]
        out = []
        for r in rows[1:]:
            if all(c is None or str(c).strip() == "" for c
    in r):
                continue
            out.append({header[i]: (r[i] if i < len(r) else
    None)
                        for i in range(len(header)) if
    header[i]})
        return out
    raise ValueError("不支持的文件类型：.%s" % ext)
def _num(value):
    if value is None:
        return None, None
    s = str(value).strip()
    if s == "":
        return None, None
    try:
        return float(s), None
    except ValueError:
        return None, "「%s」不是有效数值" % s
def parse_upload(filename, stream, unit_type_map):
    """解析上传文件。
    参数
    ----
    unit_type_map : dict，{单元类型: MicroUnit 对象}
    返回
    ----
    (parsed, errors)
      parsed : list[dict]，每项 {unit_type, fields{...},
    heavy_metals{}}
      errors : list[str]，逐行问题说明
    """
    rows = _read_rows(filename, stream)
    parsed, errors = [], []
    if not rows:
        return [], ["文件为空或没有数据行"]
    # 表头校验
    unknown = [h for h in (rows[0].keys() if rows else [])
    if h not in FIELD_ALIASES]
    if unknown:
        errors.append("存在无法识别的列，将被忽略：%s" %
    "、".join(map(str, unknown)))
    for idx, raw in enumerate(rows, start=2):      # 第 1 行是表头
        rec = {}
        for k, v in raw.items():
            key = FIELD_ALIASES.get(str(k).strip())
            if key:
                rec[key] = v
        unit_type = str(rec.get("unit_type") or "").strip()
        if not unit_type:
            errors.append("第 %d 行：缺少「单元类型」" % idx)
            continue
        if unit_type not in unit_type_map:
            errors.append("第 %d 行：图斑内不存在单元类型「%s」（现有：%s）"
                          % (idx, unit_type,
    "、".join(unit_type_map) or "无"))
            continue
        if unit_type not in UNIT_TYPES:
            errors.append("第 %d 行：单元类型「%s」不是系统支持的三种之一"
                          % (idx, unit_type))
            continue
        fields, heavy = {}, {}
        ok = True
        for key, val in rec.items():
            if key in ("unit_type",):
                continue
            if key.startswith("hm_"):
                n, err = _num(val)
                if err:
                    errors.append("第 %d 行：%s %s" % (idx,
    key[3:], err))
                    ok = False
                elif n is not None:
                    heavy[key[3:]] = n
                continue
            if key == "aspect":
                s = str(val).strip() if val is not None
    else ""
                if s == "":
                    fields["aspect"] = None
                elif s in ASPECT_TYPES:
                    fields["aspect"] = s
                else:
                    errors.append("第 %d 行：坡向「%s」无效（应为 %s）"
                                  % (idx, s,
    "、".join(ASPECT_TYPES)))
                    ok = False
                continue
            n, err = _num(val)
            if err:
                errors.append("第 %d 行：%s %s" % (idx, key,
    err))
                ok = False
            else:
                fields[key] = n
        if ok:
            parsed.append({"unit_type": unit_type,
    "fields": fields,
                           "heavy_metals": heavy})
    return parsed, errors
# ==== app/services/workflow.py ====
# -*- coding: utf-8 -*-
"""评价流程前置条件（定量评价与报告输出的流程门禁）。
论文 4.1 节确定的评价流程存在严格的先后关系：
```

```
    建立图斑 → 限制性条件判定（通过） → 划分微地质单元并计算面积占比
    → 逐单元录入指标数据 → 计算评价因子并判定等级 → 输出报告
论文原文：「先进行政策、位置约束地质灾害判断……对于满足限制性条件的
废弃矿山，按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡三种
地质微单元类型，并计算各种地形面积占比。」限制性条件判定（一票否决）
在前，微地质单元划分在后，顺序不可颠倒。
本模块把该顺序固化为可检查的状态。后续模块在入口处调用它，
前置步骤未完成时直接拦截并给出「缺什么、去哪补」的提示，
避免在条件不具备时产出看似合理的评价结论。
"""
from app.models import (db, MicroUnit, IndicatorData,
    ConstraintCheck,
                        EvaluationResult)
# 指标数据非空字段（用于判断某单元是否已录入数据）
DATA_FIELDS = ("soil_ratio", "thickness", "ph",
    "organic_matter",
               "coverage", "shannon", "slope", "aspect")
def _has_data(ind):
    if ind is None:
        return False
    if any(getattr(ind, f, None) is not None for f in
    DATA_FIELDS):
        return True
    return bool(ind.heavy_metals_dict())
def patch_progress(patch_id):
    """检查某图斑的评价流程完成情况。
    返回 dict：
        units       {done, count, ratio_sum, detail}
        constraint  {done, passed, detail}
        data        {done, ready, missing, detail}
        blocking    未满足的前置步骤列表（按流程顺序）
        ready       是否可进入定量评价
    """
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    ratio_sum = sum(u.ratio or 0.0 for u in units)
    # ---- 步骤一：微地质单元 ----
    units_step = {
        "done": bool(units),
        "count": len(units),
        "ratio_sum": ratio_sum,
        "detail": ("已划分 %d 个微地质单元，面积占比合计 %.2f%%"
                   % (len(units), ratio_sum * 100))
                  if units else "尚未划分微地质单元",
    }
    # ---- 步骤二：限制性条件判定 ----
    cc = ConstraintCheck.query.filter_by(patch_id=patch_id)
    \
        .order_by(ConstraintCheck.id.desc()).first()
    if cc is None:
        constraint_step = {"done": False, "passed": False,
                            "detail": "尚未执行限制性条件判定"}
    elif cc.passed:
        constraint_step = {"done": True, "passed": True,
                            "detail": "限制性条件判定已通过"}
    else:
        constraint_step = {"done": True, "passed": False,
                            "detail": "限制性条件判定未通过（%s）" %
    (cc.reason or "")}
    # ---- 步骤三：指标数据 ----
    missing = []
    for u in units:
        ind = IndicatorData.query.filter_by(unit_id=u.id).fi
    rst()
        if not _has_data(ind):
            missing.append(u.unit_type)
    if not units:
        data_step = {"done": False, "ready": False,
    "missing": [],
                      "detail": "尚无可录入数据的微地质单元"}
    elif missing:
        data_step = {"done": False, "ready": False,
    "missing": missing,
                      "detail": "以下微地质单元尚未录入指标数据：%s"
                                % "、".join(missing)}
    else:
        data_step = {"done": True, "ready": True,
    "missing": [],
                      "detail": "全部 %d 个微地质单元的指标数据已录入" %
    len(units)}
    ratio_ok = ratio_sum > 0.999 if units else False
    if units and not ratio_ok:
        units_step["detail"] += "（占比合计不等于 100%，请检查面积录入）"
    # ---- 汇总未满足的前置条件（按论文 4.1 节的流程顺序：限制性判定优先） ----
    # reason 只写「缺什么」这一个事实，不写为什么重要的说明 ——
    # 缺少什么由 label 指明，怎么补由 fix_label 给出，页面不必再解释一遍。
    blocking = []
    if not constraint_step["done"]:
        blocking.append({
            "step": "constraint", "label": "限制性条件判定未执行",
            "reason": "五项限制性条件尚未逐项判定。",
            "fix_url": "/constraint/%d" % patch_id,
    "fix_label": "去执行判定",
        })
    elif not constraint_step["passed"]:
        blocking.append({
            "step": "constraint", "label": "限制性条件判定未通过",
            "reason": cc.reason or "",
            "fix_url": "/constraint/%d" % patch_id,
    "fix_label": "去复核判定结果",
        })
    if not units_step["done"]:
        blocking.append({
            "step": "units", "label": "未划分微地质单元",
            "reason": "该图斑下没有任何微地质单元。",
            "fix_url": "/unit/%d" % patch_id, "fix_label": "去划分",
        })
    elif not ratio_ok:
        blocking.append({
            "step": "units", "label": "面积占比合计不为 100%",
            "reason": "当前合计 %.2f%%。" % (ratio_sum * 100),
            "fix_url": "/unit/%d" % patch_id, "fix_label": "去核对面积",
        })
    if units_step["done"] and not data_step["ready"]:
        blocking.append({
            "step": "data", "label": "指标数据未录入完整",
            "reason": data_step["detail"],
            "fix_url": "/indicator/", "fix_label": "去录入",
        })
    return {
        "units": units_step,
        "constraint": constraint_step,
        "data": data_step,
        "blocking": blocking,
        "ready": not blocking,
    }
def latest_result(patch_id):
    """取该图斑最近一次的评价记录。"""
    return EvaluationResult.query.filter_by(patch_id=patch_id) \
        .order_by(EvaluationResult.id.desc()).first()
def report_progress(patch_id):
    """报告输出的前置条件：评价流程齐备，且已有评价记录。
    报告本质上是评价结论的呈现，因此必须先真实执行过定量评价，
    导出时读取已保存的评价结果，而不是在导出环节重新计算。
    否则评价完成后再修改指标数据，报告会与系统记录的评价结论不一致，
    同一份评价出现两个数字，结论无法追溯。
    返回 dict：{ready, blocking, result, flow}
        result 为最近一次评价记录（EvaluationResult），无记录时为空。
    """
    flow = patch_progress(patch_id)
    blocking = list(flow["blocking"])
    record = latest_result(patch_id)
    if not blocking and record is None:
        blocking.append({
            "step": "evaluate", "label": "尚未执行适宜性评价",
            "reason": "该图斑还没有评价记录。",
            "fix_url": "/evaluate/%d" % patch_id,
            "fix_label": "去执行评价",
        })
    return {
        "ready": not blocking,
        "blocking": blocking,
        "result": record,
        "flow": flow,
    }
# ==== app/services/dashboard.py ====
# -*- coding: utf-8 -*-
"""工作台数据：统计数字、评价流程步骤状态、侧边栏待办角标。
首页（工作台）需要把「现在有哪些数据」「流程走到哪一步」「哪一步还差活」
一次算清楚，供模板直接渲染。集中在此，避免在视图与模板里各算一遍。
"""
from flask import g
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult,
    Standard)
from app.services import workflow as wf
# 评价流程的八个步骤（顺序即论文 4.1 节的作业顺序）
#   key        步骤标识
#   no         步骤编号（01~08）
#   title      名称
#   icon       图标名（见 templates/_macros.html 的 ICONS）
#   desc       一句话说明
#   href       入口地址
STEPS = [
    dict(key="project", no="01", title="项目与矿山档案",
    icon="folder",
         href="/project/",
         desc="建立评价项目、录入矿山图斑、挂接遥感影像、设置参照生态系统"),
    dict(key="standard", no="02", title="政策与标准知识库",
    icon="book",
         href="/standard/",
         desc="收录地方标准与政策文件，结构化判定规则，支持省际口径比对"),
    dict(key="constraint", no="03", title="限制性指标判定",
    icon="shield",
         href="/constraint/",
         desc="政策、区位约束、土壤污染、地质灾害、耕地属性五项一票否决"),
    dict(key="unit", no="04", title="微地质单元", icon="layers",
         href="/unit/",
         desc="划分岩质边坡、碎石边坡、平台三类单元，自动计算面积占比"),
    dict(key="ahp", no="05", title="指标体系与权重",
    icon="sliders",
         href="/ahp/",
         desc="指标定义维护，判断矩阵录入、权重计算与一致性检验、多专家汇总"),
    dict(key="indicator", no="06", title="指标数据采集",
    icon="database",
         href="/indicator/",
         desc="按微地质单元录入土壤、植被、地形指标，支持 CSV / Excel 批量导入"),
    dict(key="evaluate", no="07", title="适宜性定量评价",
    icon="target",
         href="/evaluate/",
         desc="指标标准化、分单元赋分、面积加权合成，判定适宜性等级"),
    dict(key="report", no="08", title="报告与结果输出",
    icon="file",
         href="/report/",
         desc="结果图表与逐层得分明细，导出 PDF 与 Word 评价报告"),
]
def _has_data(ind):
    return wf._has_data(ind)
# 系统的九个功能模块（登录页介绍用，含不在评价流程内的账号与权限管理）
MODULES = [
    ("01", "账号与权限管理", "自助注册与登录注销、管理员与操作员两种职务、写操作自动留痕"),
    ("02", "项目与矿山档案", "建立项目、录入图斑、挂接遥感影像、设置参照生态系统"),
    ("03", "政策与标准知识库", "标准判定条件结构化为规则，支持省际口径比对"),
    ("04", "限制性指标判定", "政策、区位、污染、地质灾害、耕地五项一票否决"),
    ("05", "微地质单元管理", "划分岩质边坡、碎石边坡、平台，自动算面积占比"),
    ("06", "指标体系与权重", "指标定义维护、AHP 判断矩阵与一致性检验、多专家汇总"),
    ("07", "指标数据采集", "按单元类型录入土壤、植被、地形指标，支持批量导入"),
    ("08", "适宜性定量评价", "四类标准化、分单元赋分、面积加权、等级判定"),
    ("09", "报告与结果输出", "结果图表与逐层得分明细，导出 PDF 与 Word 报告"),
]
def snapshot():
    """一次算出工作台需要的全部数据（同一请求内只算一次）。
    返回 dict：
        stats     {projects, patches, unit_types, standards}
        steps     八个步骤，附 available / pending / note
        badges    {key: 待办数量}，仅含有待办的项
        progress  {patches, units, data_units, ready,
    evaluated}
    """
    cached = getattr(g, "_dash_snapshot", None)
    if cached is None:
        cached = _compute()
        g._dash_snapshot = cached
    return cached
def _compute():
    projects = Project.query.count()
    patches = MinePatch.query.all()
    patch_ids = [p.id for p in patches]
    units = MicroUnit.query.filter(
        MicroUnit.patch_id.in_(patch_ids)).all() if
    patch_ids else []
    unit_by_patch = {}
    for u in units:
        unit_by_patch.setdefault(u.patch_id, []).append(u)
    unit_ids = [u.id for u in units]
    data_unit_ids = set()
```

```
if unit_ids:
    for ind in IndicatorData.query.filter(
            IndicatorData.unit_id.in_(unit_ids)).all():
        if _has_data(ind):
            data_unit_ids.add(ind.unit_id)
passed_constraint = set()
if patch_ids:
    for c in ConstraintCheck.query.filter(
            ConstraintCheck.patch_id.in_(patch_ids)).all():
        if c.passed:
            passed_constraint.add(c.patch_id)
evaluated = set()
if patch_ids:
    for r in EvaluationResult.query.filter(
            EvaluationResult.patch_id.in_(patch_ids)).all():
        evaluated.add(r.patch_id)
# ---- 各步骤的待办数量 ----
pending = {
    "constraint": sum(1 for p in patches if p.id not in passed_constraint),
    "unit": sum(1 for p in patches if not unit_by_patch.get(p.id)),
    "indicator": 0,
    "evaluate": 0,
}
ready_patches = 0
for p in patches:
    us = unit_by_patch.get(p.id, [])
    if us and any(u.id not in data_unit_ids for u in us):
        pending["indicator"] += 1
    pr = wf.patch_progress(p.id)
    if pr["ready"]:
        ready_patches += 1
        if p.id not in evaluated:
            pending["evaluate"] += 1
# ---- 步骤可用性：前置数据不具备时置灰 ----
avail = {
    "project": True,
    "standard": True,
    "constraint": bool(patches),
    "unit": bool(patches),
    "ahp": True,
    "indicator": bool(units),
    "evaluate": ready_patches > 0,
    "report": bool(evaluated),
}
notes = {
        "constraint": "需先建立矿山图斑",
        "unit": "需先建立矿山图斑",
        "indicator": "需先划分微地质单元",
        "evaluate": "需前置步骤齐备的图斑",
        "report": "需先执行适宜性评价",
    }
    steps = []
    for s in STEPS:
        steps.append(dict(
            s,
            available=avail[s["key"]],
            pending=pending.get(s["key"], 0),
            note=None if avail[s["key"]] else
    notes.get(s["key"], ""),
        ))
    unit_types = len({u.unit_type for u in units})
    stats = {
        "projects": projects,
        "patches": len(patches),
        "unit_types": unit_types,
        "standards": Standard.query.count(),
    }
    return {
        "stats": stats,
        "steps": steps,
        "badges": {k: v for k, v in pending.items() if v},
        "progress": {
            "patches": len(patches),
            "units": len(units),
            "data_units": len(data_unit_ids),
            "ready": ready_patches,
            "evaluated": len(evaluated),
        },
    }
def nav_badges():
    """侧边栏待办角标：{入口地址前缀: 数量}，只含有待办的项。"""
    snap = snapshot()
    return {
        "/constraint/": snap["badges"].get("constraint", 0),
        "/unit/": snap["badges"].get("unit", 0),
        "/indicator/": snap["badges"].get("indicator", 0),
        "/evaluate/": snap["badges"].get("evaluate", 0),
    }
# ==== app/services/audit.py ====
# -*- coding: utf-8 -*-
"""操作审计。
在应用工厂中以 after_request 钩子统一埋点：凡是成功的写操作（POST/PUT/DELETE）
且已登录，即写入 operation_logs 表，无需在每个视图里手工调用。
动作名称由「端点 → 中文动作」映射表给出，未收录的端点回退为方法 + 路径。
"""
```

```
from flask import request
from flask_login import current_user
from app.models import db, OperationLog
# 端点 → 可读动作名
ACTION_NAMES = {
    "auth.login": "登录系统",
    "auth.register": "注册账号",
    "auth.logout": "退出登录",
    "auth.add_user": "新增账号",
    "auth.edit_user": "编辑账号",
    "auth.delete_user": "删除账号",
    "auth.reset_password": "重置用户密码",
    "auth.profile": "修改个人资料",
    "auth.upload_avatar": "更新头像",
    "auth.delete_avatar": "移除头像",
    "project.add": "新建项目",
    "project.edit": "编辑项目",
    "project.delete": "删除项目",
    "project.patch_add": "新建矿山图斑",
    "project.patch_edit": "编辑矿山图斑",
    "project.patch_delete": "删除矿山图斑",
    "project.upload_image": "上传遥感影像",
    "unit.add": "添加微地质单元",
    "unit.edit": "编辑微地质单元",
    "unit.delete": "删除微地质单元",
    "standard.add": "添加标准/政策文件",
    "standard.edit": "编辑标准/政策文件",
    "standard.delete": "删除标准/政策文件",
    "standard.rules": "维护标准规则",
    "standard.rule_delete": "删除标准规则",
    "constraint.check": "执行限制性条件判定",
    "ahp.save_expert": "录入专家判断矩阵",
    "ahp.delete_expert": "删除专家判断矩阵",
    "ahp.aggregate": "汇总多专家矩阵",
    "indicator.edit": "保存指标数据",
    "indicator.import_data": "批量导入指标数据",
    "indicator.update_meta": "维护指标定义",
    "evaluate.run": "执行适宜性评价",
}
# 无需记录的操作（查看类或纯导航）
SKIP_ENDPOINTS = {"static", "auth.index"}
def action_name(endpoint):
    # 个人设置页的两个表单共用一个端点，按提交的动作区分
    if endpoint == "auth.profile":
        return "修改密码" if request.form.get("action") ==
    "password" \
            else "修改个人资料"
    if endpoint in ACTION_NAMES:
        return ACTION_NAMES[endpoint]
    return "写操作 %s %s" % (request.method, request.path)
```

```
def extract_target_id():
    """从视图参数中取出被操作对象的主键，取不到则返回 None。"""
    args = request.view_args or {}
    for key in ("pid", "patch_id", "unit_id", "sid", "rid",
    "result_id", "user_id"):
        if key in args:
            try:
                return int(args[key])
            except (TypeError, ValueError):
                return None
    return None
def record(endpoint, status_code):
    """记录一次写操作。失败（4xx/5xx）也记录，便于追溯。"""
    if endpoint in SKIP_ENDPOINTS:
        return
    try:
        detail = "路径 %s" % request.path
        if status_code >= 400:
            detail += ";结果 HTTP %d" % status_code
        db.session.add(OperationLog(
            user_id=current_user.id if
    current_user.is_authenticated else None,
            action=action_name(endpoint),
            target_type=(endpoint or "").split(".")[0] or
    None,
            target_id=extract_target_id(),
            detail=detail,
        ))
        db.session.commit()
    except Exception:
        # 审计失败不应影响业务响应
        db.session.rollback()
# ==== app/views/auth.py ====
# -*- coding: utf-8 -*-
"""账号与权限 —— 登录、注册、个人资料、用户维护、操作日志。
页面组织：
  /login      登录页（含「点我注册」入口）
  /register   自助注册（默认「操作员」）
  /profile    个人设置，分类切换：info / security / users / logs
              后两个分类仅管理员可见，由本模块统一管控。
"""
from flask import (Blueprint, render_template, redirect,
    url_for, request,
                   flash, abort, current_app)
from flask_login import login_user, logout_user,
    login_required, current_user
from werkzeug.security import check_password_hash,
    generate_password_hash
from app.forms import (LoginForm, EmptyForm, ProfileForm,
    PasswordForm,
                       RegisterForm, email_ok)
from app.models import db, User, OperationLog
from app.utils import uploads as up
from app.utils.constants import ROLE_LABELS, DEFAULT_ROLE,
    GENDERS
bp = Blueprint("auth", __name__)
# 「个人设置」的四个分类；后两个是管理员专属
PROFILE_TABS = ("info", "security", "users", "logs")
ADMIN_TABS = ("users", "logs")
def require_admin():
    """非管理员直接 403，避免误操作基础数据。"""
    if current_user.role != "admin":
        abort(403)
def _intro_data():
    """登录页下方的系统介绍所用的静态内容。"""
    from app.services import dashboard as dash
    return {"modules": dash.MODULES}
@bp.route("/")
def index():
    if current_user.is_authenticated:
        from app.services import dashboard as dash
        return render_template("auth/home.html",
    dash=dash.snapshot())
    return redirect(url_for("auth.login"))
@bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        return redirect(url_for("auth.index"))
    form = LoginForm()
    # 登录失败的原因直接交给模板，渲染在登录框内部。
    # 不用 flash：flash 的提示条在正文顶部，会被登录框的渐变遮罩盖住，用户看不到。
    error = None
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.d
    ata).first()
        if user and check_password_hash(user.password_hash,
    form.password.data):
            if not user.active:
                error = "该账号已停用，请联系管理员"
            else:
                login_user(user, remember=form.remember.data
    )
                # 用 toast 类别：渲染为页面正中的浮层，淡入停留后淡出。
                # 登录成功只是告知性反馈，不需要用户停下来阅读。
                flash("登录成功", "toast")
                return redirect(url_for("auth.index"))
        else:
            error = "账户名称或密码错误"
    return render_template("auth/login.html", form=form,
    error=error,
                           **_intro_data())
@bp.route("/register", methods=["GET", "POST"])
def register():
    """自助注册。新账号一律为「操作员」，权限只能由管理员事后调整。"""
    if current_user.is_authenticated:
        return redirect(url_for("auth.index"))
    form = RegisterForm()
    if form.validate_on_submit():
        username = form.username.data.strip()
        if User.query.filter_by(username=username).first():
            form.username.errors.append("该账户名称已被使用")
        else:
            user = User(username=username,
                        role=DEFAULT_ROLE,
                        password_hash=generate_password_hash
    (form.password.data),
                        display_name=(form.display_name.data
    or "").strip() or None,
                        gender=(form.gender.data or
    "").strip() or None,
                        email=(form.email.data or
    "").strip() or None,
                        phone=(form.phone.data or
    "").strip() or None)
            rel, err = _save_avatar(request.files.get("avata
    r"))
            if err:
                flash(err, "error")
            else:
                user.avatar = rel
                db.session.add(user)
                db.session.commit()
                login_user(user)
                flash("注册成功，欢迎使用", "toast")
                return redirect(url_for("auth.index"))
    return render_template("auth/register.html", form=form)
@bp.route("/logout")
@login_required
def logout():
    logout_user()
    flash("已退出登录", "ok")
    return redirect(url_for("auth.login"))
# ---------------------------- 个人设置 ------------------------
    ----
# 任何登录用户在这里维护自己；管理员额外能管理账号与查看日志。
# 「用户与操作日志」不再是独立菜单，而是本页的管理员分类。
@bp.route("/profile", methods=["GET", "POST"])
@login_required
def profile():
    is_admin = current_user.role == "admin"
```

```
tab = request.values.get("tab") or "info"
if tab not in PROFILE_TABS or (tab in ADMIN_TABS and
not is_admin):
    tab = "info"
# 表单放在同一页，用字段前缀区分，避免 submit / csrf_token 重名
info_form = ProfileForm(prefix="info", obj=current_user)
pwd_form = PasswordForm(prefix="pwd")
if request.method == "POST":
    action = request.form.get("action")
    # 校验失败要重新渲染本页，此时必须回到提交表单所在的那个分类，
    # 否则错误提示会出现在另一个分类里，用户根本看不到。
    if action == "password":
        tab = "security"
    elif action == "info":
        tab = "info"
    if action == "info" and info_form.validate_on_submit
():
        current_user.display_name =
(info_form.display_name.data or "").strip() or None
        current_user.gender = (info_form.gender.data or
"").strip() or None
        current_user.email = (info_form.email.data or
"").strip() or None
        current_user.phone = (info_form.phone.data or
"").strip() or None
        db.session.commit()
        flash("个人资料已保存", "ok")
        return redirect(url_for("auth.profile",
tab="info"))
    if action == "password" and pwd_form.validate_on_sub
mit():
        if not check_password_hash(current_user.password
_hash,

pwd_form.old_password.data):
            pwd_form.old_password.errors.append("当前密码不正确
")
        else:
            current_user.password_hash =
generate_password_hash(
                pwd_form.new_password.data)
            db.session.commit()
            flash("密码已修改", "ok")
            return redirect(url_for("auth.profile",
tab="security"))
ctx = dict(info_form=info_form, pwd_form=pwd_form,
avatar_form=EmptyForm(),
           tab=tab, role_labels=ROLE_LABELS,
genders=GENDERS)
if tab == "users":
        ctx.update(_users_context())
    elif tab == "logs":
        ctx.update(_logs_context())
    return render_template("auth/profile.html", **ctx)
def _users_context():
    """管理员分类「账号」所需的数据。"""
    return {"users": User.query.order_by(User.id).all(),
            "form": EmptyForm()}
def _logs_context():
    """管理员分类「操作日志」所需的数据。"""
    page = request.args.get("page", 1, type=int)
    per_page = 50
    q = OperationLog.query.order_by(OperationLog.id.desc())
    total = q.count()
    records = q.offset((page - 1) * per_page).limit(per_page).all()
    users_map = {u.id: u.username for u in User.query.all()}
    pages = max(1, (total + per_page - 1) // per_page)
    return {"records": records, "page": page, "pages": pages,
            "total": total, "users_map": users_map}
# --------------------------- 头像 -----------------------------
def _avatar_length_ok(file_storage):
    """按字节检查头像大小。超限时 werkzeug 读到内存里的内容也能量出来。"""
    limit = current_app.config.get("MAX_AVATAR_LENGTH", 4 * 1024 * 1024)
    stream = file_storage.stream
    pos = stream.tell()
    stream.seek(0, 2)
    size = stream.tell()
    stream.seek(pos)
    return size <= limit, size
def _save_avatar(file):
    """保存头像文件，返回 (相对路径, 错误信息)。没有文件时返回 (None, None)。"""
    if file is None or not file.filename:
        return None, None
    ok, size = _avatar_length_ok(file)
    if not ok:
        return None, ("头像不能超过 %.0f MB（当前 %.1f MB）"
                      % (current_app.config["MAX_AVATAR_LENGTH"] / 1024 / 1024,
                         size / 1024 / 1024))
    return up.save_upload(file, "avatars",
                          current_app.config["ALLOWED_AVATAR_EXT"],
                          current_app.config["UPLOAD_FOLDER"])
@bp.route("/profile/avatar", methods=["POST"])
@login_required
```

```
def upload_avatar():
    form = EmptyForm()
    back = url_for("auth.profile", tab=request.form.get("tab
") or "info")
    if not form.validate_on_submit():
        flash("表单校验失败，请重试", "error")
        return redirect(back)
    rel, err = _save_avatar(request.files.get("avatar"))
    if err:
        flash(err, "error")
        return redirect(back)
    if rel is None:
        flash("请选择头像图片", "error")
        return redirect(back)
    # 换新头像时删掉旧文件，避免残留
    up.remove_upload(current_user.avatar,
    current_app.config["UPLOAD_FOLDER"])
    current_user.avatar = rel
    db.session.commit()
    flash("头像已更新", "ok")
    return redirect(back)
@bp.route("/profile/avatar/delete", methods=["POST"])
@login_required
def delete_avatar():
    form = EmptyForm()
    back = url_for("auth.profile", tab=request.form.get("tab
") or "info")
    if form.validate_on_submit():
        up.remove_upload(current_user.avatar,
    current_app.config["UPLOAD_FOLDER"])
        current_user.avatar = None
        db.session.commit()
        flash("头像已移除", "ok")
    return redirect(back)
# ---------------------------- 账号维护（仅管理员）
    ----------------------------
@bp.route("/users")
@login_required
def users():
    """旧地址保留，统一并入「个人设置 · 账号」。"""
    return redirect(url_for("auth.profile", tab="users"))
@bp.route("/users/add", methods=["GET", "POST"])
@login_required
def add_user():
    require_admin()
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        role = request.form.get("role", DEFAULT_ROLE)
        if role not in ROLE_LABELS:
            role = DEFAULT_ROLE
        if not username or not password:
            flash("账户名称和密码不能为空", "error")
            return redirect(url_for("auth.add_user"))
        if User.query.filter_by(username=username).first():
            flash("账户名称已存在", "error")
            return redirect(url_for("auth.add_user"))
        db.session.add(User(
            username=username, role=role,
            password_hash=generate_password_hash(password),
            display_name=request.form.get("display_name",
    "").strip() or None,
            gender=request.form.get("gender", "").strip()
    or None,
            email=request.form.get("email", "").strip() or
    None,
            phone=request.form.get("phone", "").strip() or
    None))
        db.session.commit()
        flash("账号已添加", "ok")
        return redirect(url_for("auth.profile",
    tab="users"))
    return render_template("auth/add_user.html",
    role_labels=ROLE_LABELS,
                           genders=GENDERS,
    default_role=DEFAULT_ROLE,
                           form=EmptyForm())
@bp.route("/users/<int:user_id>/edit", methods=["GET",
    "POST"])
@login_required
def edit_user(user_id):
    require_admin()
    user = db.get_or_404(User, user_id)
    if request.method == "POST":
        role = request.form.get("role", user.role)
        if role not in ROLE_LABELS:
            role = user.role
        active = request.form.get("active") == "1"
        # 保护：不能把最后一个管理员降级或停用，否则无人能管理系统
        if user.role == "admin" and (role != "admin" or not
    active):
            others = User.query.filter(User.role == "admin",
                                       User.id != user.id,

    User.active.is_(True)).count()
            if others == 0:
                flash("系统至少需保留一名启用状态的管理员", "error")
                return redirect(url_for("auth.edit_user",
    user_id=user.id))
        # 个人资料字段（用户自己也能在「个人设置」里改，管理员在此代改）
        email = request.form.get("email", "").strip()
        if not email_ok(email):
            flash("邮箱格式不正确", "error")
            return redirect(url_for("auth.edit_user",
    user_id=user.id))
        user.role = role
        user.active = active
        user.display_name = request.form.get("display_name",
    "").strip() or None
        user.gender = request.form.get("gender",
    "").strip() or None
        user.email = email or None
        user.phone = request.form.get("phone", "").strip()
    or None
        new_password = request.form.get("password",
    "").strip()
        if new_password:
            user.password_hash = generate_password_hash(new_
    password)
        db.session.commit()
        flash("账号信息已保存", "ok")
        return redirect(url_for("auth.profile",
    tab="users"))
    return render_template("auth/edit_user.html", user=user,
                           role_labels=ROLE_LABELS,
    genders=GENDERS,
                           form=EmptyForm())
@bp.route("/users/<int:user_id>/delete", methods=["POST"])
@login_required
def delete_user(user_id):
    require_admin()
    user = db.get_or_404(User, user_id)
    if user.id == current_user.id:
        flash("不能删除当前登录的账号", "error")
        return redirect(url_for("auth.profile",
    tab="users"))
    if user.role == "admin":
        others = User.query.filter(User.role == "admin",
                                   User.id != user.id,
                                   User.active.is_(True)).co
    unt()
        if others == 0:
            flash("系统至少需保留一名启用状态的管理员", "error")
            return redirect(url_for("auth.profile",
    tab="users"))
    db.session.delete(user)
    db.session.commit()
    flash("账号「%s」已删除" % user.username, "ok")
    return redirect(url_for("auth.profile", tab="users"))
# ------------------------------ 操作日志（仅管理员）
```

```
    ----------------------------
@bp.route("/logs")
@login_required
def logs():
    """旧地址保留，统一并入「个人设置 · 操作日志」。"""
    return redirect(url_for("auth.profile", tab="logs"))
# ==== app/views/project.py ====
# -*- coding: utf-8 -*-
"""项目与矿山档案 —— 项目、图班、参照生态系统、遥感影像挂接。"""
from datetime import datetime
from flask import (Blueprint, render_template, request,
    redirect, url_for,
                   flash, current_app)
from flask_login import login_required, current_user
from app.forms import EmptyForm
from app.models import db, Project, MinePatch,
    ReferenceEcosystem
bp = Blueprint("project", __name__, url_prefix="/project")
@bp.route("/")
@login_required
def index():
    projects = Project.query.order_by(Project.id.desc()).all
    ()
    return render_template("project/index.html",
    projects=projects,
                           form=EmptyForm())
@bp.route("/add", methods=["GET", "POST"])
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        p = Project(
            name=request.form.get("name", "").strip(),
            code=request.form.get("code", "").strip() or
    None,
            description=request.form.get("description",
    "").strip() or None,
            created_by=current_user.id,
        )
        if not p.name:
            flash("项目名称不能为空", "error")
        else:
            db.session.add(p)
            db.session.commit()
            flash("项目已创建", "ok")
            return redirect(url_for("project.index"))
    return render_template("project/edit.html", form=form,
    project=None)
@bp.route("/<int:pid>/edit", methods=["GET", "POST"])
```

```
@login_required
def edit(pid):
    p = db.get_or_404(Project, pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        p.name = request.form.get("name", "").strip()
        p.code = request.form.get("code", "").strip() or
    None
        p.description = request.form.get("description",
    "").strip() or None
        p.status = request.form.get("status", "进行中")
        db.session.commit()
        flash("项目已保存", "ok")
        return redirect(url_for("project.index"))
    return render_template("project/edit.html", form=form,
    project=p)
@bp.route("/<int:pid>/delete", methods=["POST"])
@login_required
def delete(pid):
    p = db.get_or_404(Project, pid)
    db.session.delete(p)
    db.session.commit()
    flash("项目已删除", "ok")
    return redirect(url_for("project.index"))
# ---------------------------- 矿山图斑 ------------------------
    ----
@bp.route("/<int:pid>/patches")
@login_required
def patches(pid):
    p = db.get_or_404(Project, pid)
    return render_template("project/patches.html",
    project=p,
                               form=EmptyForm())
@bp.route("/patch/<int:patch_id>/delete", methods=["POST"])
@login_required
def patch_delete(patch_id):
    """删除矿山图斑，及其微地质单元与指标数据。"""
    from app.models import MicroUnit, IndicatorData,
    EvaluationResult, \
        ScoreDetail, ConstraintCheck, ReferenceEcosystem
    patch = db.get_or_404(MinePatch, patch_id)
    pid = patch.project_id
    unit_ids = [u.id for u in MicroUnit.query.filter_by(patc
    h_id=patch.id).all()]
    if unit_ids:
        IndicatorData.query.filter(IndicatorData.unit_id.in_
    (unit_ids)).delete(
            synchronize_session=False)
    for r in EvaluationResult.query.filter_by(patch_id=patch
    .id).all():
        ScoreDetail.query.filter_by(result_id=r.id).delete(
            synchronize_session=False)
    EvaluationResult.query.filter_by(patch_id=patch.id).dele
    te(
        synchronize_session=False)
    ConstraintCheck.query.filter_by(patch_id=patch.id).delet
    e(
        synchronize_session=False)
    MicroUnit.query.filter_by(patch_id=patch.id).delete(sync
    hronize_session=False)
    ReferenceEcosystem.query.filter_by(patch_id=patch.id).de
    lete(
        synchronize_session=False)
    db.session.delete(patch)
    db.session.commit()
    flash("图斑「%s」及其数据已删除" % patch.name, "ok")
    return redirect(url_for("project.patches", pid=pid))
@bp.route("/<int:pid>/patch/add", methods=["GET", "POST"])
@login_required
def patch_add(pid):
    p = db.get_or_404(Project, pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch = MinePatch(
            project_id=p.id,
            name=request.form.get("name", "").strip(),
            admin_region=fv("admin_region"),
            area_ha=fv("area_ha", float),
            mineral_type=fv("mineral_type"),
            mining_method=fv("mining_method"),
            stop_years=fv("stop_years", float),
            responsible_entity_missing=bool(request.form.get
    ("responsible_entity_missing")),
        )
        if not patch.name:
            flash("图斑名称不能为空", "error")
        else:
            db.session.add(patch)
            db.session.flush()
            # 参照生态系统默认取论文值
            db.session.add(ReferenceEcosystem(
                patch_id=patch.id, coverage_benchmark=75.0,
    shannon_benchmark=0.7))
            db.session.commit()
            flash("图斑已创建", "ok")
            return redirect(url_for("project.patches",
    pid=p.id))
    return render_template("project/patch_edit.html",
    form=form, project=p, patch=None)
@bp.route("/patch/<int:patch_id>/edit", methods=["GET",
    "POST"])
@login_required
def patch_edit(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch.name = request.form.get("name", "").strip()
        patch.admin_region = fv("admin_region")
        patch.area_ha = fv("area_ha", float)
        patch.mineral_type = fv("mineral_type")
        patch.mining_method = fv("mining_method")
        patch.stop_years = fv("stop_years", float)
        patch.responsible_entity_missing =
    bool(request.form.get("responsible_entity_missing"))
        # 参照生态系统
        ref = ReferenceEcosystem.query.filter_by(patch_id=pa
    tch.id).first()
        if ref is None:
            ref = ReferenceEcosystem(patch_id=patch.id)
            db.session.add(ref)
        ref.coverage_benchmark = fv("coverage_benchmark",
    float) or 75.0
        ref.shannon_benchmark = fv("shannon_benchmark",
    float) or 0.7
        ref.dominant_species = fv("dominant_species")
        patch.image_note = fv("image_note")
        db.session.commit()
        flash("图斑已保存", "ok")
        return redirect(url_for("project.patches",
    pid=patch.project_id))
    return render_template("project/patch_edit.html",
    form=form,
                           project=patch.project,
    patch=patch,
                           reference=ReferenceEcosystem.quer
    y.filter_by(
                               patch_id=patch.id).first())
# ---------------------------- 遥感影像挂接
    ----------------------------
@bp.route("/patch/<int:patch_id>/image", methods=["POST"])
@login_required
```

```
def upload_image(patch_id):
    """上传遥感底图 / 无人机正射影像并挂接到图斑。"""
    from app.utils import uploads as up
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if not form.validate_on_submit():
        flash("表单校验失败，请重试", "error")
        return redirect(url_for("project.patch_edit",
    patch_id=patch.id))
    rel, err = up.save_upload(request.files.get("image"),
    "patches",
                              current_app.config["ALLOWED_IM
    AGE_EXT"],
                              current_app.config["UPLOAD_FOL
    DER"])
    if err:
        flash(err, "error")
        return redirect(url_for("project.patch_edit",
    patch_id=patch.id))
    # 替换旧影像时清理原文件，避免残留
    if patch.image_path:
        up.remove_upload(patch.image_path,
    current_app.config["UPLOAD_FOLDER"])
    patch.image_path = rel
    patch.image_uploaded_at = datetime.utcnow()
    note = request.form.get("image_note", "").strip()
    if note:
        patch.image_note = note
    db.session.commit()
    flash("遥感影像已挂接", "ok")
    return redirect(url_for("project.patch_edit",
    patch_id=patch.id))
@bp.route("/patch/<int:patch_id>/image/delete",
    methods=["POST"])
@login_required
def delete_image(patch_id):
    from app.utils import uploads as up
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if form.validate_on_submit():
        up.remove_upload(patch.image_path,
    current_app.config["UPLOAD_FOLDER"])
        patch.image_path = None
        patch.image_uploaded_at = None
        db.session.commit()
        flash("遥感影像已移除", "ok")
    return redirect(url_for("project.patch_edit",
    patch_id=patch.id))
# ==== app/views/unit.py ====
# -*- coding: utf-8 -*-
```

```
"""微地质单元管理 —— 岩质边坡 / 碎石边坡 / 平台的划分与面积占比。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
    IndicatorData
from app.utils.constants import UNIT_TYPES
bp = Blueprint("unit", __name__, url_prefix="/unit")
def recalc_ratio(patch_id):
    """按面积重算该图斑各单元面积占比。"""
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    total = sum(u.area or 0.0 for u in units)
    if total > 0:
        for u in units:
            u.ratio = round((u.area or 0.0) / total, 6)
    return total
@bp.route("/")
@login_required
def list_patches():
    """微地质单元入口：列出各图斑及其单元划分情况。"""
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    rows = []
    for p in patches:
        units = MicroUnit.query.filter_by(patch_id=p.id).all
    ()
        rows.append({
            "patch": p,
            "count": len(units),
            "ratio_sum": sum(u.ratio or 0.0 for u in units),
        })
    return render_template("unit/list.html", rows=rows)
@bp.route("/<int:patch_id>")
@login_required
def index(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    total = sum(u.area or 0.0 for u in units)
    ratio_sum = sum(u.ratio or 0.0 for u in units)
    return render_template("unit/index.html", patch=patch,
    units=units,
                           total_area=total,
    ratio_sum=ratio_sum,
                           unit_types=UNIT_TYPES,
    form=EmptyForm())
@bp.route("/<int:patch_id>/add", methods=["POST"])
@login_required
```

```
def add(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if form.validate_on_submit():
        unit_type = request.form.get("unit_type")
        if unit_type not in UNIT_TYPES:
            flash("微地质单元类型不正确", "error")
            return redirect(url_for("unit.index",
    patch_id=patch_id))
        # 论文 4.1 节将图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。
        # 允许重复会让面积占比与按单元类型组织的指标数据对不上号。
        if MicroUnit.query.filter_by(patch_id=patch.id,
                                     unit_type=unit_type).fi
    rst():
            flash("该图斑已存在「%s」，同一类型只能划分一个" % unit_type,
    "error")
            return redirect(url_for("unit.index",
    patch_id=patch_id))
        area = request.form.get("area", "").strip()
        u = MicroUnit(patch_id=patch.id,
    unit_type=unit_type,
                      area=float(area) if area else None,
                      description=request.form.get("descript
    ion", "").strip() or None)
        db.session.add(u)
        db.session.flush()
        recalc_ratio(patch.id)
        db.session.commit()
        flash("微地质单元已添加", "ok")
    return redirect(url_for("unit.index",
    patch_id=patch_id))
@bp.route("/edit/<int:unit_id>", methods=["GET", "POST"])
@login_required
def edit(unit_id):
    """单个微地质单元的编辑页。"""
    u = db.get_or_404(MicroUnit, unit_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        new_type = request.form.get("unit_type",
    u.unit_type)
        if new_type not in UNIT_TYPES:
            flash("微地质单元类型不正确", "error")
            return redirect(url_for("unit.edit",
    unit_id=u.id))
        # 改类型时同样不允许与本图斑已有的其他单元重复
        dup = MicroUnit.query.filter_by(patch_id=u.patch_id,

    unit_type=new_type).first()
        if dup is not None and dup.id != u.id:
            flash("该图斑已存在「%s」，同一类型只能划分一个" % new_type,
    "error")
            return redirect(url_for("unit.edit",
    unit_id=u.id))
        u.unit_type = new_type
        area = request.form.get("area", "").strip()
        u.area = float(area) if area else None
        u.description = request.form.get("description",
    "").strip() or None
        recalc_ratio(u.patch_id)
        db.session.commit()
        flash("微地质单元已保存", "ok")
        return redirect(url_for("unit.index",
    patch_id=u.patch_id))
    return render_template("unit/edit.html", unit=u,
    patch=u.patch,
                           unit_types=UNIT_TYPES, form=form)
@bp.route("/delete/<int:unit_id>", methods=["POST"])
@login_required
def delete(unit_id):
    u = db.get_or_404(MicroUnit, unit_id)
    pid = u.patch_id
    name = u.unit_type
    # 一并删除该单元的指标数据，避免残留
    IndicatorData.query.filter_by(unit_id=u.id).delete(synch
    ronize_session=False)
    db.session.delete(u)
    db.session.flush()
    recalc_ratio(pid)
    db.session.commit()
    flash("微地质单元「%s」已删除" % name, "ok")
    return redirect(url_for("unit.index", patch_id=pid))
# ==== app/views/standard.py ====
# -*- coding: utf-8 -*-
"""政策与标准知识库 —— 标准/政策文件的结构化规则与省际口径比对。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, Standard, StandardRule
bp = Blueprint("standard", __name__, url_prefix="/standard")
@bp.route("/")
@login_required
def index():
    standards = Standard.query.order_by(Standard.category,
    Standard.province).all()
    local = [s for s in standards if s.category == "地方标准"]
    policy = [s for s in standards if s.category != "地方标准"]
    return render_template("standard/index.html",
    local=local, policy=policy,
                           form=EmptyForm())
@bp.route("/add", methods=["GET", "POST"])
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s = Standard(
            province=request.form.get("province",
    "").strip(),
            name=request.form.get("name", "").strip(),
            doc_number=request.form.get("doc_number",
    "").strip() or None,
            publish_date=request.form.get("publish_date",
    "").strip() or None,
            implement_date=request.form.get("implement_date"
    , "").strip() or None,
            category=request.form.get("category", "地方标准"),
            summary=request.form.get("summary", "").strip()
    or None,
        )
        if not s.province or not s.name:
            flash("省份与名称不能为空", "error")
        else:
            db.session.add(s)
            db.session.commit()
            flash("标准已添加", "ok")
            return redirect(url_for("standard.index"))
    return render_template("standard/edit.html", form=form,
    standard=None)
@bp.route("/<int:sid>/edit", methods=["GET", "POST"])
@login_required
def edit(sid):
    s = db.get_or_404(Standard, sid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s.province = request.form.get("province",
    "").strip()
        s.name = request.form.get("name", "").strip()
        s.doc_number = request.form.get("doc_number",
    "").strip() or None
        s.publish_date = request.form.get("publish_date",
    "").strip() or None
        s.implement_date = request.form.get("implement_date"
    , "").strip() or None
        s.category = request.form.get("category", "地方标准")
        s.summary = request.form.get("summary", "").strip()
    or None
        db.session.commit()
        flash("已保存", "ok")
        return redirect(url_for("standard.rules", sid=s.id))
    return render_template("standard/edit.html", form=form,
    standard=s)
@bp.route("/<int:sid>/delete", methods=["POST"])
@login_required
def delete(sid):
    s = db.get_or_404(Standard, sid)
    StandardRule.query.filter_by(standard_id=sid).delete()
    db.session.delete(s)
    db.session.commit()
    flash("已删除", "ok")
    return redirect(url_for("standard.index"))
@bp.route("/<int:sid>/rules", methods=["GET", "POST"])
@login_required
def rules(sid):
    s = db.get_or_404(Standard, sid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        r = StandardRule(
            standard_id=s.id,
            item=request.form.get("item", "").strip(),
            operator=request.form.get("operator", ">="),
            threshold=request.form.get("threshold",
    "").strip() or None,
            condition_type=request.form.get("condition_type"
    , "必要条件"),
            region=request.form.get("region", "").strip()
    or None,
            note=request.form.get("note", "").strip() or
    None,
        )
        if not r.item:
            flash("判定项不能为空", "error")
        else:
            db.session.add(r)
            db.session.commit()
            flash("规则已添加", "ok")
        return redirect(url_for("standard.rules", sid=s.id))
    rs = StandardRule.query.filter_by(standard_id=s.id).all(
    )
    return render_template("standard/rules.html",
    form=form, standard=s, rules=rs)
@bp.route("/rule/<int:rid>/delete", methods=["POST"])
@login_required
def rule_delete(rid):
    r = db.get_or_404(StandardRule, rid)
    sid = r.standard_id
    db.session.delete(r)
    db.session.commit()
    flash("规则已删除", "ok")
    return redirect(url_for("standard.rules", sid=sid))
@bp.route("/compare")
@login_required
def compare():
    """省际口径比对：同一指标在不同省份的阈值横向对比。"""
    rules = StandardRule.query.join(Standard).all()
    groups = {}
    for r in rules:
        groups.setdefault(r.item, []).append(r)
    return render_template("standard/compare.html",
    groups=groups)
# ==== app/views/constraint.py ====
# -*- coding: utf-8 -*-
"""限制性指标判定 —— 一票否决判定。
按论文表 2 与 4.1 节流程，共 5 项：政策、区位约束、土壤污染、地质灾害、耕地属性。
任一项不通过即判定为"人工辅助或生态重建"，不再进入定量评价。
"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, ConstraintCheck
from app.utils.constants import CONSTRAINT_ITEMS
bp = Blueprint("constraint", __name__,
    url_prefix="/constraint")
# 字段名 -> 中文名
FIELDS = [
    ("policy_pass", "政策约束"),
    ("location_pass", "区位约束"),
    ("pollution_pass", "土壤污染"),
    ("geo_pass", "地质灾害"),
    ("farmland_pass", "耕地属性"),
]
@bp.route("/")
@login_required
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    latest = {}
    for p in patches:
        latest[p.id] = ConstraintCheck.query.filter_by(patch
    _id=p.id) \
            .order_by(ConstraintCheck.id.desc()).first()
    return render_template("constraint/index.html",
    patches=patches, latest=latest)
@bp.route("/<int:patch_id>", methods=["GET", "POST"])
@login_required
def check(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    record = ConstraintCheck.query.filter_by(patch_id=patch_
    id) \
        .order_by(ConstraintCheck.id.desc()).first()
    if request.method == "POST" and form.validate_on_submit(
    ):
        if record is None:
            record = ConstraintCheck(patch_id=patch.id)
            db.session.add(record)
        for field, _ in FIELDS:
            setattr(record, field, request.form.get(field)
    == "1")
        record.passed = all(getattr(record, f) for f, _ in
    FIELDS)
        failed = [name for f, name in FIELDS if not
    getattr(record, f)]
        record.reason = ("全部通过" if record.passed
                         else "未通过：" + "、".join(failed))
        db.session.commit()
        flash("判定结果已保存", "ok")
        return redirect(url_for("constraint.check",
    patch_id=patch.id))
    # 预处理各条件的当前取值，供模板直接使用（Jinja 无 getattr）
    field_states = [(f, name, None if record is None else
    getattr(record, f))
                    for f, name in FIELDS]
    return render_template("constraint/check.html",
    patch=patch, form=form,
                           fields=FIELDS,
    field_states=field_states,
                           record=record)
# ==== app/views/ahp.py ====
# -*- coding: utf-8 -*-
"""指标体系与 AHP 权重 —— 路由与页面逻辑。"""
import json
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from app.forms import AHPSelectForm, EmptyForm
from app.models import db, Indicator, ExpertMatrix,
    WeightScheme
from app.utils.constants import DEFAULT_WEIGHTS, UNIT_TYPES
from app.services import ahp_service as ahp
from app.services import indicator_service as ind_svc
from app.services import weight_service as ws
bp = Blueprint("ahp", __name__, url_prefix="/ahp")
# 一次计算的权重结果对应的指标编码：名称 → 编码
_L1_CODE = {"土壤指标": "soil", "植被指标": "veg", "地形指标":
    "terrain"}
_L2_CODE = {
    ("土壤指标", "土石比"): "soil_ratio",
    ("土壤指标", "土层厚度"): "thickness",
    ("土壤指标", "pH"): "ph",
    ("土壤指标", "有机质"): "organic_matter",
    ("土壤指标", "污染指标"): "pollution",
    ("植被指标", "植被覆盖度"): "coverage",
    ("植被指标", "生物多样性"): "shannon",
    ("地形指标", "坡度"): "slope",
    ("地形指标", "坡向"): "aspect",
}
def to_code_pairs(unit_type, level, l1, names, weights):
    """把一次 AHP 计算的权重结果转成「指标编码 → 权重」，便于落库。"""
    pairs = []
    for name, w in zip(names, weights):
        if level == 1:
            code = _L1_CODE.get(name)
        else:
            code = _L2_CODE.get((l1, name))
        if code:
            pairs.append((code, w))
    return pairs
def normalize_l1(unit_type, level, l1):
    """把 l1 参数规整为该单元类型下确实存在的一级指标名。
    level=2 的页面（判断矩阵、多专家矩阵）都按 l1 取二级指标。
    直接带 ?level=2 访问而没给 l1 时，原先会 KeyError: None 直接 500；
    这里退回该单元类型的第一个一级指标，页面照常可用。
    """
    if level != 2:
        return None
    valid = list(DEFAULT_WEIGHTS.get(unit_type,
    {}).get("l2", {}))
    if not valid:
        return None
    return l1 if l1 in valid else valid[0]
def indicator_list(unit_type, level, l1=None):
    """返回 （指标名称列表， 默认权重列表）。
    level=1 时返回一级指标；level=2 时返回指定一级指标下的二级指标。
    l1 缺失或不属于该单元类型时，退回第一个一级指标，不抛异常。
    """
    weights = DEFAULT_WEIGHTS[unit_type]
    if level == 1:
        names = list(weights["l1"].keys())
        vec = [weights["l1"][n] for n in names]
        return names, vec
    l2_all = weights["l2"]
    if l1 not in l2_all:
        l1 = next(iter(l2_all))
    l2 = l2_all[l1]
    names = list(l2.keys())
    vec = [l2[n] for n in names]
    return names, vec
def l1_choices(unit_type):
    """某单元类型的一级指标名称列表（用于二级指标录入时选择父指标）。"""
    return list(DEFAULT_WEIGHTS[unit_type]["l1"].keys())
@bp.route("/", methods=["GET", "POST"])
def index():
    form = AHPSelectForm()
    # 先根据请求中的单元类型填充 l1 选项，再进行表单校验
    unit = request.form.get("unit_type") or
    form.unit_type.choices[0][0]
    form.l1.choices = [(n, n) for n in l1_choices(unit)]
    if form.validate_on_submit():
        unit_type = form.unit_type.data
        level = int(form.level.data)
        if level == 1:
            return redirect(url_for("ahp.matrix",
    unit_type=unit_type, level=1))
        if not form.l1.data:
            flash("选择「二级指标」时，请同时选择所属的一级指标", "error")
            return render_template("ahp/index.html",
    form=form,
                                   weight_rows=ws.summary())
        return redirect(url_for("ahp.matrix",
    unit_type=unit_type,
                                level=2, l1=form.l1.data))
    return render_template("ahp/index.html", form=form,
                           weight_rows=ws.summary())
@bp.route("/matrix", methods=["GET", "POST"])
def matrix():
    unit_type = request.values.get("unit_type")
    level = int(request.values.get("level", 1))
    l1 = normalize_l1(unit_type, level,
    request.values.get("l1"))
    if unit_type not in DEFAULT_WEIGHTS:
        flash("无效的微地质单元类型", "error")
        return redirect(url_for("ahp.index"))
    names, vec = indicator_list(unit_type, level, l1)
    n = len(names)
    form = EmptyForm()
    # 预填默认值：论文表 3 权重对应的完全一致矩阵 a_ij = w_i / w_j
    default = [[round(vec[i] / vec[j], 4) for j in
    range(n)] for i in range(n)]
    def compute(upper):
        A = ahp.matrix_from_upper(upper)
        r = ahp.weights_and_consistency(A)
        bad = ahp.find_most_inconsistent(A)
        return {
            "weights": [round(v, 4) for v in r["weights"]],
            "lambda_max": round(r["lambda_max"], 4),
            "ci": r["ci"],
        "cr": r["cr"],
        "consistent": r["consistent"],
        "matrix": [[round(A[i, j], 4) for j in
range(n)] for i in range(n)],
        "most_inconsistent": bad,
    }
result = None
if request.method == "POST" and form.validate_on_submit(
):
    action = request.form.get("action", "calc")
    try:
        upper = [float(request.form.get("m_%d_%d" % (i,
j), "1"))
                 for i in range(n) for j in range(i +
1, n)]
        result = compute(upper)
    except (ValueError, ZeroDivisionError):
        flash("矩阵输入有误：请填入正数（对角线自动为 1，下三角自动取倒数）",
"error")
        return redirect(url_for("ahp.matrix",
unit_type=unit_type,
                                level=level, l1=l1))
    if action == "save":
        if not result["consistent"]:
            flash("一致性比例 CR=%.4f ≥ 0.1，判断矩阵不满足一致性要求，"
                  "请调整后再保存" % result["cr"], "error")
        else:
            pairs = to_code_pairs(unit_type, level, l1,
names,
                                  result["weights"])
            scheme, cnt = ws.save_weights(unit_type,
pairs)
            flash("已保存并为「%s」生效 %d 项权重（CR=%.4f）。"
                  "此后该单元类型的评价将以此权重计算。"
                  % (unit_type, cnt, result["cr"]),
"ok")
            return redirect(url_for("ahp.matrix",
unit_type=unit_type,
                                    level=level, l1=l1))
elif request.values.get("calc") == "1":
    # 按默认（论文表 3）矩阵直接试算，便于一键查看权重与一致性
    result = compute([default[i][j] for i in range(n)
for j in range(i + 1, n)])
_, source_label, custom_scheme = ws.effective_weights(un
it_type)
return render_template(
    "ahp/matrix.html",
    form=form, unit_type=unit_type, level=level, l1=l1,
    names=names, vec=vec, default=default,
result=result,
        source_label=source_label, custom_scheme=custom_sche
    me,
    )
@bp.route("/weights/reset", methods=["POST"])
def weights_reset():
    """删除自定义权重方案，恢复为该单元类型的论文表 3 默认权重。"""
    unit_type = request.form.get("unit_type")
    form = EmptyForm()
    if form.validate_on_submit():
        if unit_type not in DEFAULT_WEIGHTS:
            flash("无效的微地质单元类型", "error")
        else:
            n = ws.reset_weights(unit_type)
            if n:
                flash("已恢复「%s」为论文表 3 默认权重" % unit_type,
    "ok")
            else:
                flash("「%s」本就是论文表 3 默认权重，无需恢复" % unit_type,
    "ok")
    return redirect(url_for("ahp.index"))
# --------------------------- 指标维护 -----------------------
    ----
@bp.route("/indicators")
def indicators():
    """指标定义维护：查看指标树、单位、理想值与取值依据。"""
    if Indicator.query.count() == 0:
        ind_svc.seed_indicators()
    all_ind = Indicator.query.order_by(Indicator.level,
    Indicator.id).all()
    by_parent = {}
    for i in all_ind:
        by_parent.setdefault(i.parent_code, []).append(i)
    return render_template("ahp/indicators.html",
    by_parent=by_parent,
                           all_ind=all_ind,
    unit_types=UNIT_TYPES,
                           type_label=ind_svc.ideal_type_lab
    el,
                           form=EmptyForm())
@bp.route("/indicators/<code>/edit", methods=["GET",
    "POST"])
def indicator_edit(code):
    ind = Indicator.query.filter_by(code=code).first_or_404(
    )
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        name = request.form.get("name", "").strip()
        if not name:
            flash("指标名称不能为空", "error")
            return redirect(url_for("ahp.indicator_edit",
    code=code))
        ind.name = name
        ind.unit = request.form.get("unit", "").strip() or
    None
        ind.ideal_value = request.form.get("ideal_value",
    "").strip() or None
        ind.ideal_type = request.form.get("ideal_type") or
    ind.ideal_type
        units = request.form.getlist("enabled_units")
        ind.enabled_units = ",".join(units) if units else
    None
        ind.is_constraint = request.form.get("is_constraint"
    ) == "1"
        ind.description = request.form.get("description",
    "").strip() or None
        db.session.commit()
        flash("指标「%s」已保存" % ind.name, "ok")
        return redirect(url_for("ahp.indicators"))
    return render_template("ahp/indicator_edit.html",
    ind=ind,
                                   unit_types=UNIT_TYPES,
                                   type_label=ind_svc.ideal_type_lab
    el, form=form)
@bp.route("/indicators/reset", methods=["POST"])
def indicators_reset():
    """按论文取值重置指标定义。"""
    form = EmptyForm()
    if form.validate_on_submit():
        n = ind_svc.seed_indicators()
        flash("已按论文取值重置 %d 项指标定义" % n, "ok")
    return redirect(url_for("ahp.indicators"))
# ---------------------------- 多专家矩阵 ----------------------
    -----
def expert_scope():
    """从请求中取（并规整）单元类型、层级与所属一级指标。"""
    unit_type = request.values.get("unit_type") or
    UNIT_TYPES[0]
    if unit_type not in DEFAULT_WEIGHTS:
        unit_type = UNIT_TYPES[0]
    level = int(request.values.get("level", 1))
    l1 = normalize_l1(unit_type, level,
    request.values.get("l1"))
    return unit_type, level, l1
def expert_query(unit_type, level, l1):
    q = ExpertMatrix.query.filter_by(unit_type=unit_type,
    level=level)
    if level == 2:
        q = q.filter_by(l1_name=l1)
    else:
        q = q.filter(ExpertMatrix.l1_name.is_(None))
    return q.order_by(ExpertMatrix.id)
def default_scheme(unit_type, level, l1=None):
    """取（或建）该范围的权重方案，专家矩阵统一挂到它下面。"""
    name = "%s · %s%s" % (unit_type,
                          "一级指标" if level == 1 else "二级指标",
                          "（%s）" % l1 if l1 else "")
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ
    e,
                                          level=level,
    name=name).first()
    if scheme is None:
        scheme = WeightScheme(unit_type=unit_type,
    level=level, name=name,
                              is_default=False)
        db.session.add(scheme)
        db.session.flush()
    return scheme
@bp.route("/experts", methods=["GET", "POST"])
def experts():
    """多专家判断矩阵录入与汇总。"""
    unit_type, level, l1 = expert_scope()
    if unit_type not in DEFAULT_WEIGHTS:
        flash("无效的微地质单元类型", "error")
        return redirect(url_for("ahp.index"))
    names, vec = indicator_list(unit_type, level, l1)
    n = len(names)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        action = request.form.get("action")
        if action == "add":
            expert_name = request.form.get("expert_name",
    "").strip() or "未具名专家"
            try:
                upper = [float(request.form.get("m_%d_%d" %
    (i, j), "1"))
                         for i in range(n) for j in range(i
    + 1, n)]
            except ValueError:
                flash("矩阵输入有误，请填入正数", "error")
                return redirect(url_for("ahp.experts",
    unit_type=unit_type,
                                        level=level, l1=l1))
            A = ahp.matrix_from_upper(upper)
            r = ahp.weights_and_consistency(A)
            scheme = default_scheme(unit_type, level, l1)
            db.session.add(ExpertMatrix(
                scheme_id=scheme.id,
                unit_type=unit_type, l1_name=(l1 if level
    == 2 else None),
                expert_name=expert_name, level=level,
                matrix_json=json.dumps([list(map(float,
    row)) for row in A]),
                cr=round(r["cr"], 6)))
            db.session.commit()
            flash("已录入专家「%s」的判断矩阵（CR=%.4f）" % (expert_name,
    r["cr"]), "ok")
        elif action == "save_agg":
            # 把汇总权重落库并生效，使多专家结果真正参与评价计算
            weights = _aggregate(unit_type, level, l1)
            if weights is None:
                flash("暂无可汇总的专家矩阵", "error")
            elif not weights["consistent"]:
                flash("汇总矩阵一致性比例 CR=%.4f ≥ 0.1，不满足要求，"
                      "请先检查或调整专家矩阵" % weights["cr"],
    "error")
            else:
                names_, _ = indicator_list(unit_type,
    level, l1)
                pairs = to_code_pairs(unit_type, level, l1,
    names_,
                                      weights["weights"])
                _, cnt = ws.save_weights(unit_type, pairs)
                flash("已保存并为「%s」生效 %d 项汇总权重（CR=%.4f）"
                      % (unit_type, cnt, weights["cr"]),
    "ok")
        return redirect(url_for("ahp.experts",
    unit_type=unit_type,
                                level=level, l1=l1))
    rows = expert_query(unit_type, level, l1).all()
    default = [[round(vec[i] / vec[j], 4) for j in
    range(n)] for i in range(n)]
    return render_template("ahp/experts.html",
    unit_type=unit_type, level=level,
                           l1=l1, names=names, rows=rows,
    default=default,
                           n=n, form=form)
@bp.route("/experts/<int:mid>/delete", methods=["POST"])
def expert_delete(mid):
    row = db.get_or_404(ExpertMatrix, mid)
    unit_type, level, l1 = row.unit_type, row.level,
    row.l1_name
    form = EmptyForm()
    if form.validate_on_submit():
        db.session.delete(row)
        db.session.commit()
        flash("已删除专家矩阵", "ok")
    return redirect(url_for("ahp.experts",
    unit_type=unit_type,
                            level=level, l1=l1))
def _aggregate(unit_type, level, l1):
    """按几何平均汇总多专家矩阵并做一致性检验。无矩阵时返回 None。"""
    rows = expert_query(unit_type, level, l1).all()
    mats = []
    for r in rows:
        try:
            mats.append([list(map(float, line))
                         for line in json.loads(r.matrix_jso
    n)])
        except Exception:
            continue
    if not mats:
        return None
    agg = ahp.aggregate_matrices(mats)
    res = ahp.weights_and_consistency(agg)
    return {
        "count": len(mats),
        "weights": res["weights"],
        "lambda_max": res["lambda_max"],
        "ci": res["ci"], "cr": res["cr"],
        "consistent": res["consistent"],
        "most_inconsistent": ahp.find_most_inconsistent(agg)
    ,
        "expert_crs": [r.cr for r in rows if r.cr is not
    None],
    }
@bp.route("/experts/aggregate")
def experts_aggregate():
    """按几何平均汇总多专家矩阵，并做一致性检验。"""
    unit_type, level, l1 = expert_scope()
    if unit_type not in DEFAULT_WEIGHTS:
        flash("无效的微地质单元类型", "error")
        return redirect(url_for("ahp.index"))
    names, vec = indicator_list(unit_type, level, l1)
    rows = expert_query(unit_type, level, l1).all()
    result = _aggregate(unit_type, level, l1)
    if result is not None:
        result["weights"] = [round(v, 4) for v in
    result["weights"]]
        result["lambda_max"] = round(result["lambda_max"],
    4)
    return render_template("ahp/experts_aggregate.html",
    unit_type=unit_type,
                           level=level, l1=l1, names=names,
    rows=rows,
                           result=result, form=EmptyForm())
# ==== app/views/indicator.py ====
# -*- coding: utf-8 -*-
"""指标数据采集与管理 —— 路由与页面逻辑。
```

```
按微地质单元组织录入；系统按单元类型自动决定应录入哪些指标
（岩质边坡无土壤指标、平台无地形指标）。
"""
import io
import json
from flask import (Blueprint, render_template, request,
    redirect, url_for,
                        flash, send_file)
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
    IndicatorData
from app.utils.constants import HEAVY_METALS, ASPECT_TYPES,
    UNIT_ENABLED_INDICATORS
from app.services import import_service as imp
bp = Blueprint("indicator", __name__,
    url_prefix="/indicator")
# 指标字段定义：字段名 -> (中文名, 单位, 是否可选)
SOIL_FIELDS = [
    ("soil_ratio", "土石比", ""),
    ("thickness", "土层厚度", "cm"),
    ("ph", "pH", ""),
    ("organic_matter", "有机质", "g/kg"),
]
VEG_FIELDS = [
    ("coverage", "植被覆盖度", "%"),
    ("shannon", "Shannon-Wiener 多样性指数", ""),
]
TERRAIN_FIELDS = [
    ("slope", "坡度", "°"),
    ("aspect", "坡向", ""),
]
def unit_field_config(unit_type):
    """根据单元类型返回需录入的字段分组。"""
    enabled = UNIT_ENABLED_INDICATORS.get(unit_type, ())
    groups = []
    if "土壤指标" in enabled:
        groups.append(("土壤指标", SOIL_FIELDS, True))  # True
    = 含重金属
    if "植被指标" in enabled:
        groups.append(("植被指标", VEG_FIELDS, False))
    if "地形指标" in enabled:
        groups.append(("地形指标", TERRAIN_FIELDS, False))
    return groups
@bp.route("/")
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    return render_template("indicator/index.html",
    patches=patches)
@bp.route("/<int:unit_id>", methods=["GET", "POST"])
```

```python
def edit(unit_id):
    unit = db.get_or_404(MicroUnit, unit_id)
    ind = IndicatorData.query.filter_by(unit_id=unit.id).first()
    form = EmptyForm()
    groups = unit_field_config(unit.unit_type)
    if request.method == "POST" and form.validate_on_submit():
        def f(name, cast=float):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        heavy = {}
        for m in HEAVY_METALS:
            v = f("hm_" + m)
            if v is not None:
                heavy[m] = v
        if ind is None:
            ind = IndicatorData(unit_id=unit.id)
            db.session.add(ind)
        ind.soil_ratio = f("soil_ratio")
        ind.thickness = f("thickness")
        ind.ph = f("ph")
        ind.organic_matter = f("organic_matter")
        ind.heavy_metals = json.dumps(heavy, ensure_ascii=False) if heavy else None
        ind.coverage = f("coverage")
        ind.shannon = f("shannon")
        ind.slope = f("slope")
        ind.aspect = request.form.get("aspect", "").strip() or None
        db.session.commit()
        flash("指标数据已保存", "ok")
        return redirect(url_for("indicator.index"))
    # 组装当前值供模板回显
    current = {}
    if ind:
        current = {
            "soil_ratio": ind.soil_ratio, "thickness": ind.thickness,
            "ph": ind.ph, "organic_matter": ind.organic_matter,
            "coverage": ind.coverage, "shannon": ind.shannon,
            "slope": ind.slope, "aspect": ind.aspect,
        }
        current["heavy"] = ind.heavy_metals_dict()
    return render_template("indicator/edit.html",
                           unit=unit, form=form, groups=groups,
                           heavy_metals=HEAVY_METALS,
                           aspect_types=ASPECT_TYPES,
    current=current)
# ---------------------------- 批量导入 ------------------------
    ----
@bp.route("/import", methods=["GET", "POST"])
def import_data():
    """批量导入指标数据（CSV / Excel）。"""
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    form = EmptyForm()
    report = None
    if request.method == "POST" and form.validate_on_submit(
    ):
        patch_id = request.form.get("patch_id", type=int)
        patch = db.session.get(MinePatch, patch_id) if
    patch_id else None
        if patch is None:
            flash("请选择矿山图斑", "error")
            return redirect(url_for("indicator.import_data")
    )
        file = request.files.get("datafile")
        if file is None or not file.filename:
            flash("请选择要导入的文件", "error")
            return redirect(url_for("indicator.import_data")
    )
        ext = file.filename.rsplit(".", 1)[-1].lower()
        if ext not in ("csv", "xlsx", "xls"):
            flash("仅支持 CSV 或 Excel（.xlsx/.xls）文件", "error")
            return redirect(url_for("indicator.import_data")
    )
        units = MicroUnit.query.filter_by(patch_id=patch.id)
    .all()
        unit_map = {}
        for u in units:
            unit_map.setdefault(u.unit_type, u)
        if not unit_map:
            flash("该图斑尚未划分微地质单元，请先划分后再导入", "error")
            return redirect(url_for("indicator.import_data")
    )
        try:
            parsed, errors = imp.parse_upload(file.filename,
    file.stream, unit_map)
        except Exception as e:
            flash("文件解析失败：%s" % e, "error")
            return redirect(url_for("indicator.import_data")
    )
        saved = 0
        for rec in parsed:
            unit = unit_map[rec["unit_type"]]
            ind = IndicatorData.query.filter_by(unit_id=unit
    .id).first()
            if ind is None:
                ind = IndicatorData(unit_id=unit.id)
                db.session.add(ind)
            f = rec["fields"]
            for key in ("soil_ratio", "thickness", "ph",
    "organic_matter",
                        "coverage", "shannon", "slope",
    "aspect"):
                if key in f:
                    setattr(ind, key, f[key])
            if rec["heavy_metals"]:
                ind.heavy_metals = json.dumps(rec["heavy_met
    als"],

    ensure_ascii=False)
            saved += 1
        db.session.commit()
        report = {"patch": patch.name, "saved": saved,
    "errors": errors,
                  "total": len(parsed) + len(
                      [e for e in errors if e.startswith("第
    ")])}
        if saved:
            flash("已导入 %d 个微地质单元的指标数据" % saved, "ok")
        else:
            flash("没有可导入的数据行，请检查文件内容", "error")
    return render_template("indicator/import.html",
    patches=patches,
                           form=form, report=report,
    headers=imp.HEADERS)
@bp.route("/import/template")
def import_template():
    """下载导入模板（CSV，含一行示例）。"""
    csv_text = imp.build_template_csv()
    data = csv_text.encode("utf-8-sig")      # 带 BOM，便于
    Excel 正确识别中文
    return send_file(io.BytesIO(data), as_attachment=True,
                     download_name="指标数据导入模板.csv",
                     mimetype="text/csv")
# ==== app/views/evaluate.py ====
# -*- coding: utf-8 -*-
"""适宜性定量评价引擎 —— 路由与页面逻辑。
从数据库加载某图斑的微地质单元与指标数据，调用评价引擎计算，
展示各单元得分、最终评价因子 F、适宜性等级与推荐措施。
入口处按论文 4.1 节的流程顺序检查前置条件（见 app/services/workflow.py），
前置步骤未完成时不予计算，避免在条件不具备时产出评价结论。
"""
import json
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from app.forms import EmptyForm
from app.models import (db, MinePatch, MicroUnit,
    IndicatorData,
                        EvaluationResult, ScoreDetail)
from app.services import evaluation_service as ev
from app.services import workflow as wf
bp = Blueprint("evaluate", __name__, url_prefix="/evaluate")
def load_patch_input(patch_id):
    """从数据库组装评价引擎输入。"""
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    result = []
    for u in units:
        ind = IndicatorData.query.filter_by(unit_id=u.id).fi
    rst()
        data = {
            "soil_ratio": ind.soil_ratio if ind else None,
            "thickness": ind.thickness if ind else None,
            "ph": ind.ph if ind else None,
            "organic_matter": ind.organic_matter if ind
    else None,
            "heavy_metals": ind.heavy_metals_dict() if ind
    else {},
            "coverage": ind.coverage if ind else None,
            "shannon": ind.shannon if ind else None,
            "slope": ind.slope if ind else None,
            "aspect": ind.aspect if ind else None,
        }
        result.append({
            "unit_id": u.id,
            "unit_type": u.unit_type,
            "ratio": u.ratio if u.ratio is not None else
    0.0,
            "data": data,
        })
    return result
def _round(v, n=6):
    return None if v is None else round(v, n)
def load_weights(units):
    """解析各单元类型当前生效的权重与来源说明。
    权重可来自 AHP 计算并保存的生效方案；未配置时用论文表 3 的默认值。
    来源一并记录，便于报告标注与事后复核。
    """
    from app.services import weight_service as ws
    weights, sources = {}, {}
    for u in units:
        ut = u["unit_type"]
        if ut not in weights:
            w, label, _ = ws.effective_weights(ut)
            weights[ut] = w
            sources[ut] = label
    return weights, sources
def pack_units(unit_results, sources=None):
    """把评价结果整理为可持久化的 JSON。
    完整保留各单元得分、一级指标得分与逐条二级指标明细，
    使结果页与报告能够复现「已记录的那一次评价」，
    而不是每次打开时重新推算。
    """
    sources = sources or {}
    out = []
    for u in unit_results:
        out.append({
            "unit_type": u["unit_type"],
            "ratio": _round(u["ratio"]),
            "unit_score": _round(u["unit_score"]),
            "weighted": _round(u["weighted"]),
            "weight_source": sources.get(u["unit_type"]),
            "l1_scores": {k: _round(v) for k, v in
    u["l1_scores"].items()},
            "details": [{
                "l1": d["l1"], "l2": d["l2"],
                "raw": None if d["raw"] is None else
    str(d["raw"]),
                "std_value": _round(d["std_value"]),
                "weight": _round(d["weight"]),
                "contribution": _round(d["contribution"]),
            } for d in u["details"]],
        })
    return out
def save_evaluation(patch_id, units):
    """执行评价并把结果与逐层明细落库，返回评价记录。
    权重取各单元类型当前生效的方案（AHP 计算结果），未配置时用
    论文表 3 的默认权重；来源一并记录。结果 JSON 保留各单元得分、
    一级指标得分与逐条二级指标明细，结果页与报告据此复现
    记录下来的那次评价，不再重新推算。
    """
    weights, sources = load_weights(units)
    res = ev.score_mine(units, weights)
    saved = EvaluationResult(
        patch_id=patch_id,
        final_score=round(res["final_score"], 4),
        grade=res["grade"],
        measures=res["grade_info"]["measure"],
        unit_scores=json.dumps(pack_units(res["unit_results"
    ], sources),
                               ensure_ascii=False),
    )
    db.session.add(saved)
    db.session.flush()      # 先取得 result.id，再写明细
    unit_ids = {u["unit_type"]: u["unit_id"] for u in units}
    for u in res["unit_results"]:
        for d in u["details"]:
            db.session.add(ScoreDetail(
                result_id=saved.id,
                unit_id=unit_ids.get(u["unit_type"]),
                l1_name=d["l1"], l2_name=d["l2"],
                raw_value=None if d["raw"] is None else
    str(d["raw"]),
                std_value=d["std_value"],
    weight=d["weight"],
                contribution=d["contribution"]))
    db.session.commit()
    return saved
def block_reasons(progress):
    """把未满足的前置条件转成可读的拦截原因。"""
    return ["%s:%s" % (b["label"], b["reason"]) for b in
    progress["blocking"]]
@bp.route("/")
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    latest = {}
    progress = {}
    for p in patches:
        latest[p.id] = EvaluationResult.query.filter_by(patc
    h_id=p.id) \
            .order_by(EvaluationResult.id.desc()).first()
        progress[p.id] = wf.patch_progress(p.id)
    return render_template("evaluate/index.html",
    patches=patches,
                           latest=latest, progress=progress)
@bp.route("/<int:patch_id>", methods=["GET", "POST"])
def run(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    progress = wf.patch_progress(patch_id)
    if request.method == "POST" and form.validate_on_submit(
    ):
        if not progress["ready"]:
            for reason in block_reasons(progress):
                flash("无法执行评价 —— " + reason, "error")
            return redirect(url_for("evaluate.run",
    patch_id=patch_id))
        units = load_patch_input(patch_id)
        saved = save_evaluation(patch.id, units)
        return redirect(url_for("evaluate.result",
    result_id=saved.id))
    # 展示将采用的权重来源，便于在执行前确认
    units_now = load_patch_input(patch_id)
    _, sources = load_weights(units_now) if units_now else
    ({}, {})
    return render_template("evaluate/run.html",
    patch=patch, form=form,
                           progress=progress,
    weight_sources=sources)
@bp.route("/result/<int:result_id>")
def result(result_id):
    r = db.get_or_404(EvaluationResult, result_id)
    patch = db.session.get(MinePatch, r.patch_id)
    unit_scores = json.loads(r.unit_scores) if
    r.unit_scores else []
    return render_template("evaluate/result.html", r=r,
    patch=patch,
                           unit_scores=unit_scores)
# ==== app/views/report.py ====
# -*- coding: utf-8 -*-
"""报告与结果输出 —— 得分明细、图表、Word / PDF 导出。"""
import io
import json
from datetime import datetime
from flask import Blueprint, render_template, send_file,
    flash, redirect, url_for
from app.models import db, MinePatch, ReferenceEcosystem
from app.services import report_service as rs
from app.services import workflow as wf
from app.utils import charts
from app.utils import browser
from app.utils.constants import grade_of
bp = Blueprint("report", __name__, url_prefix="/report")
def _prepare(patch_id):
    """加载图斑并取出已记录的评价结果，供明细页与导出复用。
    报告是评价结论的呈现，因此这里读取最近一次保存的评价记录，
    而不是重新计算：评价完成后再改动指标数据，也不会让报告与
    系统记录的评价结论对不上。没有任何评价记录时，返回流程检查
    结果，由调用方拦截并提示先去执行评价。
    """
    patch = db.get_or_404(MinePatch, patch_id)
    prog = wf.report_progress(patch_id)
    if not prog["ready"]:
        return patch, None, None, None, prog
    rec = prog["result"]
    result = {
        "final_score": rec.final_score,
        "grade": rec.grade,
        "grade_info": grade_of(rec.final_score),
        "unit_results": json.loads(rec.unit_scores) if
    rec.unit_scores else [],
    }
    reference = ReferenceEcosystem.query.filter_by(patch_id=
    patch_id).first()
    return patch, reference, result, rec, prog
def _deny(progress, patch_id, endpoint="report.detail"):
    """前置条件未满足时的统一处理：逐条提示并退回报告列表。"""
    for b in progress["blocking"]:
        flash("暂不能生成报告 —— %s：%s" % (b["label"],
    b["reason"]), "error")
    if progress["blocking"]:
        flash("请先完成上述步骤后再生成报告。", "error")
    return redirect(url_for("report.index"))
def _charts(result):
    """生成两张图表：单元得分对比 + 一级指标平均得分分布。"""
    units = [(u["unit_type"], u["unit_score"],
              "%.4f" % u["unit_score"]) for u in
    result["unit_results"]]
    chart_units = charts.hbars(units)
    # 各一级指标在各单元上的平均得分，反映其对评价结果的实际影响
    acc, cnt = {}, {}
    for u in result["unit_results"]:
        for k, v in (u.get("l1_scores") or {}).items():
            acc[k] = acc.get(k, 0.0) + v
            cnt[k] = cnt.get(k, 0) + 1
    items = [(k, acc[k] / cnt[k], "%.4f" % (acc[k] /
    cnt[k])) for k in acc]
    items.sort(key=lambda x: -x[1])
    chart_weight = charts.donut(items)
    return chart_units, chart_weight
@bp.route("/")
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    progress = {p.id: wf.report_progress(p.id) for p in
    patches}
    return render_template("report/index.html",
    patches=patches,
                           progress=progress)
@bp.route("/<int:patch_id>")
def detail(patch_id):
    patch, reference, result, record, prog =
    _prepare(patch_id)
    if result is None:
        return _deny(prog, patch_id)
    chart_units, chart_weight = _charts(result)
    return render_template("report/detail.html",
    patch=patch, result=result,
                           record=record,
    reference=reference,
                           chart_units=chart_units,
    chart_weight=chart_weight)
@bp.route("/<int:patch_id>/word")
```

```
def word(patch_id):
    patch, reference, result, record, prog =
    _prepare(patch_id)
    if result is None:
        return _deny(prog, patch_id)
    buf = rs.build_report(patch, result)
    filename = "自然恢复适宜性评价报告_%s.docx" % patch.name
    return send_file(buf, as_attachment=True,
    download_name=filename,
                     mimetype="application/vnd.openxmlformat
    s-officedocument."
                              "wordprocessingml.document")
@bp.route("/<int:patch_id>/pdf")
def pdf(patch_id):
    """导出 PDF 报告：服务端渲染打印模板，再交浏览器无头打印。"""
    patch, reference, result, record, prog =
    _prepare(patch_id)
    if result is None:
        return _deny(prog, patch_id)
    chart_units, chart_weight = _charts(result)
    html = render_template(
        "report/print.html", patch=patch,
    reference=reference, result=result,
        chart_units=chart_units, chart_weight=chart_weight,
        generated_at=datetime.now().strftime("%Y-%m-%d
    %H:%M"))
    data, err = browser.html_to_pdf(html)
    if err:
        flash("PDF 导出失败：%s" % err, "error")
        return redirect(url_for("report.detail",
    patch_id=patch_id))
    return send_file(io.BytesIO(data), as_attachment=True,
                     download_name="自然恢复适宜性评价报告_%s.pdf" %
    patch.name,
                     mimetype="application/pdf")
# ==== app/views/search.py ====
# -*- coding: utf-8 -*-
"""全局搜索：在评价项目、矿山图斑、标准与政策文件中查找关键词。
顶栏的搜索框指向本模块。只做关键词包含匹配，不做分词，
但覆盖名称、编号、行政区、矿种、文号、摘要等常用检索字段。
"""
from sqlalchemy import or_
from flask import Blueprint, render_template, request
from app.models import db, Project, MinePatch, Standard
bp = Blueprint("search", __name__)
LIMIT = 50          # 每类结果上限，避免一次渲染过多
@bp.route("/search")
def index():
    q = (request.args.get("q") or "").strip()
    projects, patches, standards = [], [], []
    if q:
        like = "%" + q + "%"
        projects = Project.query.filter(or_(
            Project.name.like(like),
            Project.code.like(like),
            Project.description.like(like),
        )).order_by(Project.id.desc()).limit(LIMIT).all()
        patches = MinePatch.query.filter(or_(
            MinePatch.name.like(like),
            MinePatch.admin_region.like(like),
            MinePatch.mineral_type.like(like),
        )).order_by(MinePatch.id.desc()).limit(LIMIT).all()
        standards = Standard.query.filter(or_(
            Standard.name.like(like),
            Standard.doc_number.like(like),
            Standard.province.like(like),
            Standard.summary.like(like),
        )).order_by(Standard.id).limit(LIMIT).all()
    total = len(projects) + len(patches) + len(standards)
    return render_template("search.html", q=q,
    projects=projects,
                           patches=patches,
    standards=standards, total=total)
# ==== app/templates/base.html ====
{% import "_macros.html" as ui %}<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
    initial-scale=1.0">
  <link rel="icon" href="{{ url_for('static',
    filename='favicon.ico') }}">
  <title>{% block title %}历史遗留废弃矿山自然恢复适宜性评价系统{% endblock
    %}</title>
  <style>
    /* ---------- 设计变量：绿色主色 + 中性灰阶 ---------- */
    :root {
      --accent: #1f6b45;          /* 主色：深绿，呼应自然恢复主题 */
      --accent-hover: #185738;
      --accent-soft: #e9f4ee;
      --side-1: #17503a;          /* 侧边栏渐变起止色 */
      --side-2: #113c2b;
      --danger: #b3261e;
      --danger-soft: #fdecea;
      --ok: #1e6b3a;
      --ok-soft: #eaf6ef;
      --todo: #d98324;            /* 待办角标 */
      --ink: #23262b;
      --ink-soft: #6b7280;
      --line: #e3e7ea;
  --surface: #ffffff;
  --canvas: #f4f7f5;
  --radius: 10px;
  --shadow: 0 1px 2px rgba(0, 0, 0, .05);
  --header-h: 60px;
  --nav-w: 226px;
}
* { box-sizing: border-box; }
html, body { margin: 0; padding: 0; }
body {
  font-family: "Microsoft YaHei", "PingFang SC",
"SimSun", sans-serif;
  font-size: 14px;
  line-height: 1.6;
  color: var(--ink);
  background: var(--canvas);
}
/* ---------- 跳到主要内容（键盘可达性） ---------- */
.skip-link {
  position: absolute;
  left: 12px;
  top: -60px;
  z-index: 30;
  padding: 8px 14px;
  background: var(--accent);
  color: #fff;
  border-radius: var(--radius);
  text-decoration: none;
  transition: top .15s ease-out;
}
.skip-link:focus { top: 12px; }
/* ---------- 整体骨架：左侧深绿导航 + 右侧内容 ----------
   固定为一屏高，页面滚动交给右侧内容区，
   这样左侧导航栏（含底部用户区）不会随长页面一起被拉到下面去。 */
.app-shell {
  display: flex;
  align-items: stretch;
  height: 100vh;
  height: 100dvh;
  overflow: hidden;
}
/* ---------- 侧边栏 ---------- */
.app-side {
  flex: 0 0 var(--nav-w);
  width: var(--nav-w);
  display: flex;
  flex-direction: column;
  background: linear-gradient(180deg, var(--side-1) 0%,
var(--side-2) 100%);
  color: #fff;
```

```
}
.app-brand {
  display: flex;
  align-items: center;
  gap: 11px;
  padding: 17px 16px 15px;
  border-bottom: 1px solid rgba(255, 255, 255, .10);
}
.app-brand__mark {
  flex: none;
  display: grid;
  place-items: center;
  width: 36px;
  height: 36px;
  border-radius: 10px;
  background: rgba(255, 255, 255, .15);
}
.app-brand__name {
  font-size: 14.5px;
  font-weight: 600;
  line-height: 1.35;
  letter-spacing: .2px;
}
.app-brand__sub {
  font-size: 11.5px;
  line-height: 1.35;
  color: rgba(255, 255, 255, .58);
}
.app-nav { flex: 1; padding: 6px 12px 14px; overflow-y:
auto; }
.app-nav__group {
  margin: 12px 0 5px;
  padding: 0 10px;
  font-size: 11.5px;
  letter-spacing: .05em;
  color: rgba(255, 255, 255, .48);
}
.app-nav ul { margin: 0; padding: 0; list-style: none; }
.app-nav a {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 9px 11px;
  margin-bottom: 2px;
  border-radius: 8px;
  color: rgba(255, 255, 255, .80);
  text-decoration: none;
  transition: background-color .16s ease, color .16s
ease;
}
```

```
.app-nav a:hover { background: rgba(255, 255, 255,
.10); color: #fff; }
.app-nav a:focus-visible {
  outline: 2px solid rgba(255, 255, 255, .60);
  outline-offset: 1px;
}
.app-nav a[aria-current="page"] {
  background: rgba(255, 255, 255, .17);
  color: #fff;
  font-weight: 600;
}
.app-nav .ico { flex: none; opacity: .92; }
.app-nav__badge {
  margin-left: auto;
  min-width: 20px;
  padding: 0 6px;
  font-size: 11px;
  font-weight: 600;
  line-height: 18px;
  text-align: center;
  border-radius: 9px;
  background: var(--todo);
  color: #fff;
  font-variant-numeric: tabular-nums;
}
.app-user {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 10px 14px;
  border-top: 1px solid rgba(255, 255, 255, .10);
}
.app-user__me {
  display: flex;
  flex: 1;
  min-width: 0;
  align-items: center;
  gap: 10px;
  padding: 4px 6px;
  border-radius: 8px;
  color: inherit;
  text-decoration: none;
  transition: background-color .16s ease;
}
.app-user__me:hover { background: rgba(255, 255, 255,
.10); }
.app-user__me:focus-visible {
  outline: 2px solid rgba(255, 255, 255, .60);
  outline-offset: 1px;
}
```

```
.app-user__avatar {
  flex: none;
  display: grid;
  place-items: center;
  width: 32px;
  height: 32px;
  border-radius: 50%;
  background: rgba(255, 255, 255, .16);
  font-size: 13px;
  font-weight: 600;
  object-fit: cover;
}
.app-user__name {
  display: block;
  font-size: 13px;
  line-height: 1.3;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.app-user__state {
  display: flex;
  align-items: center;
  gap: 5px;
  font-size: 11.5px;
  line-height: 1.3;
  color: rgba(255, 255, 255, .60);
}
.app-user__dot {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background: #6ed49a;
}
.app-user__out {
  flex: none;
  padding: 6px;
  border-radius: 6px;
  color: rgba(255, 255, 255, .68);
  text-decoration: none;
}
.app-user__out:hover { color: #fff; background:
rgba(255, 255, 255, .12); }
/* ---------- 右侧内容区 ----------
   整个骨架固定一屏高，滚动只发生在内容区内部：
   左侧导航栏（含底部用户区）始终留在原位，右侧页面自己滚。 */
.app-content {
  flex: 1;
  min-width: 0;
  display: flex;
  flex-direction: column;
  overflow-y: auto;
  overflow-x: hidden;
  /* 细滚动条：默认半透明，悬停时加深，不抢视线 */
  scrollbar-width: thin;
  scrollbar-color: rgba(31, 107, 69, .28) transparent;
}
.app-content::-webkit-scrollbar { width: 12px; }
.app-content::-webkit-scrollbar-track { background:
transparent; }
.app-content::-webkit-scrollbar-thumb {
  background: rgba(31, 107, 69, .26);
  border: 3px solid transparent;
  background-clip: content-box;
  border-radius: 999px;
}
.app-content::-webkit-scrollbar-thumb:hover {
  background: rgba(31, 107, 69, .46);
  background-clip: content-box;
}
/* ---------- 顶栏：面包屑 + 搜索 ----------
   粘在内容区顶部，滚动时面包屑与搜索框始终可用 */
.app-header {
  position: sticky;
  top: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  gap: 14px;
  height: var(--header-h);
  padding: 0 22px;
  background: var(--surface);
  border-bottom: 1px solid var(--line);
}
.crumb {
  display: flex;
  align-items: center;
  gap: 7px;
  font-size: 13px;
  color: var(--ink-soft);
  white-space: nowrap;
}
.crumb a { color: var(--ink-soft); text-decoration:
none; }
.crumb a:hover { color: var(--accent); }
.crumb strong { color: var(--ink); font-weight: 600; }
.crumb .ico { color: #c3c9d0; }
.app-header__spacer { flex: 1; }
.search-box {
  display: flex;
  align-items: center;
  gap: 8px;
  width: 268px;
  padding: 7px 12px;
  background: var(--canvas);
  border: 1px solid var(--line);
  border-radius: 9px;
  color: var(--ink-soft);
  transition: border-color .16s ease, background-color
.16s ease;
}
.search-box:focus-within { border-color: var(--accent);
background: #fff; }
.search-box input {
  flex: 1;
  min-width: 0;
  border: none;
  outline: none;
  background: transparent;
  font-family: inherit;
  font-size: 13px;
  color: var(--ink);
}
.search-box button {
  display: grid;
  place-items: center;
  padding: 0;
  border: none;
  background: transparent;
  color: inherit;
  cursor: pointer;
}
.icon-btn {
  display: grid;
  place-items: center;
  width: 34px;
  height: 34px;
  border-radius: 9px;
  border: 1px solid transparent;
  color: var(--ink-soft);
  text-decoration: none;
  transition: background-color .16s ease, color .16s
ease,
              border-color .16s ease;
}
.icon-btn:hover {
  color: var(--accent);
  background: var(--accent-soft);
  border-color: rgba(31, 107, 69, .18);
}
```

```
.app-main {
  flex: 1;
  min-width: 0;
  padding: 22px 26px 26px;
}
.container { max-width: 1120px; }
/* ---------- 页脚 ---------- */
.app-foot {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 14px;
  padding: 0 26px 18px;
  font-size: 12px;
  color: var(--ink-soft);
}
/* ---------- 未登录时的裸页面（登录页） ---------- */
.login-shell {
  display: flex;
  flex-direction: column;
  min-height: 100dvh;
}
/* ---------- 卡片 ---------- */
.card {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 22px 24px;
  margin-bottom: 18px;
}
.card > h2 {
  margin: 0 0 14px;
  font-size: 16px;
  font-weight: 600;
  color: var(--accent);
  text-wrap: balance;
}
.card > h3 {
  margin: 20px 0 10px;
  font-size: 14px;
  font-weight: 600;
  color: var(--accent);
}
/* ---------- 表格 ---------- */
table {
  width: 100%;
  border-collapse: collapse;
  margin: 10px 0 16px;
  font-variant-numeric: tabular-nums;
```

```
}
caption {
  text-align: left;
  color: var(--ink-soft);
  font-size: 12px;
  padding-bottom: 6px;
}
th, td {
  border: 1px solid var(--line);
  padding: 8px 12px;
  text-align: center;
  vertical-align: middle;
}
th {
  background: var(--accent-soft);
  font-weight: 600;
  white-space: nowrap;
}
tbody tr:hover { background: #fafbfc; }
/* ---------- 表单控件 ---------- */
label { display: inline-block; }
.field { margin-bottom: 12px; }
.field__label {
  display: block;
  margin-bottom: 4px;
  color: var(--ink-soft);
  font-size: 13px;
}
input[type="text"], input[type="number"],
input[type="password"],
select, textarea {
  padding: 6px 9px;
  font-family: inherit;
  font-size: 14px;
  color: var(--ink);
  background: #fff;
  border: 1px solid #c6ccd4;
  border-radius: 4px;
  max-width: 100%;
}
input[type="number"] {
  width: 104px;
  text-align: center;
  font-variant-numeric: tabular-nums;
}
textarea { width: 100%; min-height: 64px; resize:
vertical; }
input:focus-visible, select:focus-visible,
textarea:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 1px;
  border-color: var(--accent);
}
.checkbox {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  cursor: pointer;
  user-select: none;
}
.checkbox input { width: 16px; height: 16px; margin: 0;
cursor: pointer; }
/* ---------- 按钮 ----------
   统一为圆角矩形：默认半透明底色，鼠标悬停时填充为实色。 */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  padding: 8px 18px;
  font-family: inherit;
  font-size: 14px;
  line-height: 1.4;
  color: #fff;
  background: rgba(31, 107, 69, .80);
  border: 1px solid rgba(31, 107, 69, .30);
  border-radius: 8px;
  cursor: pointer;
  text-decoration: none;
  transition: background-color .16s ease, border-color
.16s ease,
              color .16s ease, box-shadow .16s ease,
transform .16s ease;
}
.btn:hover {
  background: var(--accent);
  border-color: var(--accent);
  box-shadow: 0 3px 12px rgba(31, 107, 69, .22);
}
.btn:active { transform: translateY(1px); box-shadow:
none; }
.btn:focus-visible { outline: 2px solid var(--accent);
outline-offset: 2px; }
.btn--ghost {
  color: var(--accent);
  background: rgba(31, 107, 69, .06);
  border-color: rgba(31, 107, 69, .22);
}
.btn--ghost:hover {
  color: #fff;
  background: var(--accent);
  border-color: var(--accent);
}
.btn--danger {
  color: var(--danger);
  background: rgba(179, 38, 30, .07);
  border-color: rgba(179, 38, 30, .26);
}
.btn--danger:hover {
  color: #fff;
  background: var(--danger);
  border-color: var(--danger);
  box-shadow: 0 3px 12px rgba(179, 38, 30, .20);
}
.btn--sm { padding: 5px 13px; font-size: 13px;
border-radius: 7px; }
.btn:disabled { opacity: .45; cursor: not-allowed; }
.btn:disabled:hover {
  background: rgba(31, 107, 69, .80);
  border-color: rgba(31, 107, 69, .30);
  box-shadow: none;
}
.btn--ghost:disabled:hover { background: rgba(31, 107,
69, .06); color: var(--accent); }
/* ---------- 表格操作列的行内小按钮 ----------
   取代原来「文字 | 文字」的写法：
   圆角矩形 + 半透明底色，悬停时变实色。 */
.row-actions {
  display: inline-flex;
  flex-wrap: nowrap;          /* 按钮不折行；宁可让旁边长名称换行 */
  align-items: center;
  gap: 6px;
  white-space: nowrap;
}
.row-actions form { display: inline-flex; margin: 0; }
.link-btn {
  display: inline-flex;
  align-items: center;
  padding: 4px 11px;
  font-family: inherit;
  font-size: 13px;
  line-height: 1.5;
  color: var(--accent);
  background: rgba(31, 107, 69, .07);
  border: 1px solid rgba(31, 107, 69, .20);
  border-radius: 7px;
  cursor: pointer;
  text-decoration: none;
  white-space: nowrap;
  transition: background-color .16s ease, border-color
```

```
.16s ease,
              color .16s ease, box-shadow .16s ease,
transform .16s ease;
}
.link-btn:hover {
  color: #fff;
  background: var(--accent);
  border-color: var(--accent);
  box-shadow: 0 2px 8px rgba(31, 107, 69, .20);
}
.link-btn:active { transform: translateY(1px);
box-shadow: none; }
.link-btn:focus-visible { outline: 2px solid
var(--accent); outline-offset: 2px; }
.link-btn--danger {
  color: var(--danger);
  background: rgba(179, 38, 30, .07);
  border-color: rgba(179, 38, 30, .24);
}
.link-btn--danger:hover {
  color: #fff;
  background: var(--danger);
  border-color: var(--danger);
  box-shadow: 0 2px 8px rgba(179, 38, 30, .22);
}
.actions { display: flex; flex-wrap: wrap; align-items:
center; gap: 8px; }
.actions form { display: inline; }
/* ---------- 提示条 ---------- */
.flash {
  padding: 10px 14px;
  margin-bottom: 14px;
  border-radius: var(--radius);
  border-left: 4px solid;
}
.flash.error {
  color: var(--danger);
  background: var(--danger-soft);
  border-left-color: var(--danger);
}
.flash.ok {
  color: var(--ok);
  background: var(--ok-soft);
  border-left-color: var(--ok);
}
/* ---------- 居中浮层提示（登录成功、各类操作完成） ----------
   外层 toast-stack 固定在页面正中心并纵向排列，
   内层每条各自淡入 → 停留 → 淡出，多条提示会依次排开而不重叠。
   淡灰半透明底 + 背景模糊做出毛玻璃质感，不做成实心块，也不用高饱和色：
   提示只是告知性反馈，不该比页面内容更抢眼。
   pointer-events: none 让它不拦截鼠标，出现期间也能正常操作。 */
.toast-stack {
  position: fixed;
  top: 50%;
  left: 50%;
  z-index: 100;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 10px;
  transform: translate(-50%, -50%);
  pointer-events: none;
}
.toast-center {
  max-width: min(560px, 78vw);
  padding: 13px 28px;
  border-radius: 999px;
  background: rgba(244, 245, 246, .82);         /*
淡灰、半透明，非实体 */
  color: #4a4f52;
  font-size: 16px;
  font-weight: 500;
  line-height: 1.65;
  letter-spacing: .03em;
  text-align: center;
  text-wrap: pretty;
  border: 1px solid rgba(0, 0, 0, .06);
  backdrop-filter: blur(9px) saturate(1.1);
  -webkit-backdrop-filter: blur(9px) saturate(1.1);
  box-shadow: 0 8px 26px rgba(23, 32, 27, .10);
  animation: toast-fade 2.6s ease-in-out forwards;
}
@keyframes toast-fade {
  0%       { opacity: 0; transform: scale(.90); }
  14%, 74% { opacity: 1; transform: scale(1); }
  100%     { opacity: 0; transform: scale(1); }
}
/* 系统设置里要求「减少动态效果」时，去掉缩放，只做淡入淡出 */
@media (prefers-reduced-motion: reduce) {
  .toast-center { animation: toast-fade-plain 2.6s
linear forwards; }
  @keyframes toast-fade-plain {
    0%       { opacity: 0; }
    14%, 74% { opacity: 1; }
    100%     { opacity: 0; }
  }
}
/* 不支持背景模糊时，把底色加实一些保证文字仍然清楚 */
@supports not (backdrop-filter: blur(1px)) {
  .toast-center { background: rgba(240, 241, 242, .96);
}
}
/* ---------- 文字工具类 ---------- */
.muted { color: var(--ink-soft); font-size: 12.5px; }
.mono { font-family: Consolas, "Courier New",
monospace; font-variant-numeric: tabular-nums; }
.num { font-variant-numeric: tabular-nums; }
.warn { color: var(--danger); font-weight: 600; }
.ok { color: var(--ok); font-weight: 600; }
.nowrap { white-space: nowrap; }
/* 表格操作列里常把删除按钮放在 <form> 中。form 默认是块级元素，
   会把「删除」挤到下一行，导致操作列参差不齐，这里让它留在行内。 */
.nowrap form { display: inline; margin: 0; }
.stack { display: flex; flex-direction: column; gap:
14px; }
/* 状态标签：不依赖颜色单独表意 */
.tag {
  display: inline-block;
  padding: 1px 9px;
  font-size: 12.5px;
  border-radius: 10px;
  border: 1px solid;
  white-space: nowrap;
}
.tag--ok { color: var(--ok); background:
var(--ok-soft); border-color: #b8ddc6; }
.tag--warn { color: var(--danger); background:
var(--danger-soft); border-color: #eec4c0; }
.tag--muted { color: var(--ink-soft); background:
#f1f3f6; border-color: #dde1e6; }
/* ---------- 登录页 ---------- */
/* 仿 Apple 账户页：接近纯白的底，仅在顶部有一层极淡的绿色晕染，
   让彩色徽标成为整页唯一的视觉重心。排版见 auth/login.html。 */
.body--login {
  background:
    radial-gradient(1100px 640px at 50% 8%,
                    #edf6f1 0%, rgba(237, 246, 241, 0)
72%),
    linear-gradient(180deg, #ffffff 0%, #fbfdfc 100%);
}
.body--login .app-main { padding: 0; }
.body--login .container { max-width: none; width: 100%;
}
/* 仅供读屏软件的隐藏文本 */
.visually-hidden {
  position: absolute !important;
  width: 1px;
  height: 1px;
  margin: -1px;
  padding: 0;
```

```
    overflow: hidden;
    clip: rect(0 0 0 0);
    white-space: nowrap;
    border: 0;
  }
  /* ---------- 工作台：页首横幅 ---------- */
  .hero {
    position: relative;
    overflow: hidden;
    min-height: 176px;
    padding: 26px 28px;
    margin-bottom: 16px;
    border-radius: 12px;
    /* 左侧压深保证文字可读，右侧透出远山插画；
       背景按底部对齐，避免远山被居中裁掉 */
    background:
      linear-gradient(100deg, rgba(9, 40, 27, .92) 0%,
                              rgba(11, 48, 33, .68) 50%,
                              rgba(11, 48, 33, .06) 100%),
      url("{{ url_for('static', filename='hero.svg') }}")
      center bottom / cover no-repeat;
    color: #fff;
  }
  .hero__body { max-width: 720px; }
  .hero__title {
    margin: 0 0 10px;
    font-size: 21px;
    font-weight: 700;
    line-height: 1.4;
    letter-spacing: .3px;
  }
  .hero__desc {
    margin: 0;
    max-width: 640px;
    font-size: 12.5px;
    line-height: 1.85;
    color: rgba(255, 255, 255, .86);
  }
  .hero__actions {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
    margin-top: 18px;
  }
  .hero-btn {
    display: inline-flex;
    align-items: center;
    gap: 6px;
    padding: 7px 15px;
    font-size: 13px;
  border-radius: 8px;
  text-decoration: none;
  transition: background-color .16s ease, border-color
.16s ease,
              color .16s ease;
}
.hero-btn--solid {
  color: #14432f;
  background: rgba(255, 255, 255, .92);
  border: 1px solid rgba(255, 255, 255, .92);
}
.hero-btn--solid:hover { background: #fff; }
.hero-btn--ghost {
  color: #fff;
  background: rgba(255, 255, 255, .14);
  border: 1px solid rgba(255, 255, 255, .45);
}
.hero-btn--ghost:hover { background: rgba(255, 255,
255, .26); }
.hero-btn:focus-visible { outline: 2px solid #fff;
outline-offset: 2px; }
/* ---------- 工作台：数据概览 ---------- */
.stat-cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(190px,
1fr));
  gap: 12px;
  margin-bottom: 22px;
}
.stat-card {
  display: block;
  padding: 15px 17px;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  color: var(--ink);
  text-decoration: none;
  transition: border-color .16s ease, box-shadow .16s
ease;
}
.stat-card:hover {
  border-color: rgba(31, 107, 69, .45);
  box-shadow: 0 3px 12px rgba(19, 60, 43, .08);
}
.stat-card__label {
  display: block;
  font-size: 12.5px;
  color: var(--ink-soft);
}
```

```
.stat-card__value {
  display: block;
  margin: 3px 0 2px;
  font-size: 26px;
  font-weight: 700;
  line-height: 1.25;
  color: var(--accent);
  font-variant-numeric: tabular-nums;
}
.stat-card__unit {
  margin-left: 3px;
  font-size: 13px;
  font-weight: 500;
  color: var(--ink-soft);
}
.stat-card__note {
  display: block;
  font-size: 12px;
  color: var(--ink-soft);
}
/* ---------- 工作台：分区标题 ---------- */
.section { margin-bottom: 22px; }
.section__head {
  display: flex;
  align-items: baseline;
  flex-wrap: wrap;
  gap: 6px 14px;
  margin-bottom: 12px;
}
.section__title { margin: 0; font-size: 16px; }
.section__desc { margin: 0; font-size: 12.5px; color:
var(--ink-soft); }
/* ---------- 工作台：流程步骤卡 ---------- */
.step-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill,
minmax(238px, 1fr));
  gap: 12px;
}
.step-card {
  position: relative;
  display: block;
  padding: 16px 16px 15px;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  color: var(--ink);
  text-decoration: none;
  transition: border-color .16s ease, box-shadow .16s
```

```
ease,
              transform .16s ease;
}
.step-card:hover {
  border-color: rgba(31, 107, 69, .55);
  box-shadow: 0 5px 16px rgba(19, 60, 43, .10);
  transform: translateY(-1px);
}
.step-card:focus-visible { outline: 2px solid
var(--accent); outline-offset: 2px; }
.step-card__no {
  position: absolute;
  top: 13px;
  right: 15px;
  font-size: 17px;
  font-weight: 700;
  color: #d7e3db;
  font-variant-numeric: tabular-nums;
}
.step-card__icon {
  display: grid;
  place-items: center;
  width: 32px;
  height: 32px;
  margin-bottom: 10px;
  border-radius: 9px;
  background: var(--accent-soft);
  color: var(--accent);
}
.step-card__title {
  display: block;
  margin-bottom: 5px;
  font-size: 14px;
  font-weight: 600;
  color: var(--accent);
}
.step-card__desc {
  display: block;
  font-size: 12.5px;
  line-height: 1.65;
  color: var(--ink-soft);
  text-wrap: pretty;
}
.step-card__badge {
  display: inline-block;
  margin-top: 10px;
  padding: 1px 9px;
  font-size: 11.5px;
  border-radius: 9px;
  background: #fdf1e2;
```

```
      color: #9a5a12;
      border: 1px solid #f0d3ab;
    }
    /* 前置数据不具备时置灰，并说明缺什么 */
    .step-card--locked {
      background: #fafbfa;
      border-color: #eceff0;
      box-shadow: none;
      cursor: default;
    }
    .step-card--locked .step-card__no { color: #e6e9e8; }
    .step-card--locked .step-card__icon {
      background: #f1f3f2;
      color: #a9b2ae;
    }
    .step-card--locked .step-card__title { color: #9aa3a0; }
    .step-card--locked .step-card__desc { color: #b3bab7; }
    .step-card__lock {
      display: inline-block;
      margin-top: 10px;
      padding: 1px 9px;
      font-size: 11.5px;
      border-radius: 9px;
      background: #f1f3f2;
      color: #8b948f;
    }
    /* ---------- 工作台：整行入口 ---------- */
    .row-card {
      display: flex;
      align-items: center;
      gap: 14px;
      padding: 15px 18px;
      background: var(--surface);
      border: 1px solid var(--line);
      border-radius: var(--radius);
      box-shadow: var(--shadow);
      color: var(--ink);
      text-decoration: none;
      transition: border-color .16s ease, box-shadow .16s
    ease;
    }
    .row-card:hover {
      border-color: rgba(31, 107, 69, .45);
      box-shadow: 0 3px 12px rgba(19, 60, 43, .08);
    }
    .row-card__icon {
      display: grid;
      place-items: center;
      width: 36px;
      height: 36px;
```

```
  flex: none;
  border-radius: 10px;
  background: var(--accent-soft);
  color: var(--accent);
}
.row-card__main { flex: 1; min-width: 0; }
.row-card__title {
  display: block;
  font-size: 14px;
  font-weight: 600;
  color: var(--accent);
}
.row-card__desc {
  display: block;
  font-size: 12.5px;
  color: var(--ink-soft);
}
.row-card__arrow { color: #c3c9d0; flex: none; }
/* ---------- 结果摘要 ---------- */
.stat-row {
  display: flex;
  flex-wrap: wrap;
  gap: 28px;
  margin: 4px 0 6px;
}
.stat__label { font-size: 12.5px; color:
var(--ink-soft); }
.stat__value { font-size: 20px; font-weight: 700;
font-variant-numeric: tabular-nums; }
/* ---------- 流程步骤条 ---------- */
.steps { display: flex; flex-wrap: wrap; gap: 8px;
margin: 4px 0 18px; }
.step {
  display: flex; align-items: center; gap: 7px;
  padding: 8px 12px; font-size: 13px;
  background: #fff; border: 1px solid var(--line);
  border-radius: var(--radius);
}
.step__no {
  display: inline-flex; align-items: center;
justify-content: center;
  width: 20px; height: 20px; border-radius: 50%;
  background: #eef1f5; color: var(--ink-soft);
  font-size: 12px; font-variant-numeric: tabular-nums;
}
.step__name { font-weight: 600; }
.step__state { font-size: 12.5px; color:
var(--ink-soft); }
.step--done { border-color: #b8ddc6; background:
var(--ok-soft); }
```

```
.step--done .step__no { background: var(--ok); color:
#fff; }
.step--done .step__state { color: var(--ok); }
.step--fail { border-color: #eec4c0; background:
var(--danger-soft); }
.step--fail .step__no { background: var(--danger);
color: #fff; }
.step--fail .step__state { color: var(--danger);
font-weight: 600; }
.step--next { border-color: var(--accent); }
.step--next .step__no { background: var(--accent);
color: #fff; }
/* ---------- 提示块 ---------- */
.notice {
  padding: 14px 16px; margin: 0 0 16px;
  border-radius: var(--radius); border-left: 4px solid;
}
.notice--warn {
  background: var(--danger-soft);
  border-left-color: var(--danger);
  color: #7a1c17;
}
.notice__title { margin: 0 0 8px; font-weight: 600; }
.notice__list { margin: 0; padding-left: 20px;
line-height: 1.9; }
.notice__list a { color: var(--accent); }
.notice__hint { margin: 8px 0 0; font-size: 12.5px;
color: var(--ink-soft); }
/* ---------- 图斑面板（按图斑分块展示，如指标数据采集） ---------- */
.patch-panel {
  border: 1px solid var(--line);
  border-radius: var(--radius);
  background: var(--surface);
  padding: 14px 16px;
  margin: 0 0 14px;
}
.patch-panel:last-child { margin-bottom: 0; }
.patch-panel__head {
  display: flex;
  align-items: baseline;
  flex-wrap: wrap;
  gap: 6px 14px;
  margin: 0 0 12px;
  padding-bottom: 9px;
  border-bottom: 1px solid var(--line);
}
.patch-panel__name {
  margin: 0;
  font-size: 15px;
  font-weight: 600;
```

```
    }
    .patch-panel__meta {
      font-size: 12.5px;
      color: var(--ink-soft);
    }
    .patch-panel__meta a { color: var(--accent); }
    .patch-panel__tail { margin-left: auto; }
    .patch-panel .notice { margin: 0; }
    .patch-panel table { margin: 0; }
    /* ---------- 窄屏适配 ---------- */
    @media (max-width: 960px) {
      /* 上下布局时取消「一屏高 + 内容区内部滚动」，
         否则导航会占掉半屏、内容被挤在剩下的小格子里滚动 */
      .app-shell {
        flex-direction: column;
        height: auto;
        min-height: 100vh;
        min-height: 100dvh;
        overflow: visible;
      }
      .app-content { overflow: visible; }
      .app-header { position: static; }
      .app-side { width: auto; flex: none; }
      .app-brand { padding: 12px 14px; }
      .app-nav { padding: 4px 10px 10px; }
      .app-nav ul { display: flex; flex-wrap: wrap; gap:
    4px; }
      .app-nav a { margin-bottom: 0; }
      .app-nav__group { margin: 8px 0 4px; }
      .app-user { padding: 10px 14px; }
      .app-header { padding: 0 14px; gap: 10px; }
      .search-box { width: 180px; }
      .app-main { padding: 16px 14px 20px; }
      .app-foot { padding: 0 14px 14px; }
    }
    @media (max-width: 640px) {
      .crumb { display: none; }
      .search-box { width: auto; flex: 1; }
    }
  </style>
</head>
<body class="{% if not current_user.is_authenticated
    %}body--login{% endif %}">
  <a class="skip-link" href="#main">跳到主要内容</a>
  {# 取一次即可（同一请求内重复调用返回相同内容）。
     类别为 toast 的渲染成居中浮层，其余仍是页面顶部的提示条 ——
     错误提示需要停下来细读，不适合做成会消失的浮层。 #}
  {% set flash_msgs = get_flashed_messages(with_categories=t
    rue) %}
  {# 类别为 toast / ok 的渲染成页面正中的浮层（操作完成的即时反馈）；
```

```
   其余（尤其 error）仍是页面顶部的提示条 ——
   错误提示需要停下来细读，不适合做成会消失的浮层。
   没有浮层时这个容器是空的，不占任何位置。 #}
<div class="toast-stack">
  {% for category, message in flash_msgs if category in
  ("toast", "ok") %}
  <div class="toast-center" role="status"
  aria-live="polite">{{ message }}</div>
  {% endfor %}
</div>
{% if current_user.is_authenticated %}
<div class="app-shell">
  <aside class="app-side">
    <div class="app-brand">
      <span class="app-brand__mark">{{
  ui.icon("mountain", 21) }}</span>
      <span>
        <span class="app-brand__name">矿山自然恢复评价</span><br>
        <span class="app-brand__sub">适宜性评价系统 V1.0</span>
      </span>
    </div>
    <nav class="app-nav" aria-label="功能导航">
      <ul>
        <li>
          <a href="/"{% if home_active %}
  aria-current="page"{% endif %}>
            {{ ui.icon("home") }}<span>工作台</span>
          </a>
        </li>
      </ul>
      {% for g in nav_groups %}
      <p class="app-nav__group">{{ g.title }}</p>
      <ul>
        {% for item in g.links %}
        <li>
          <a href="{{ item.href }}"{% if item.active %}
  aria-current="page"{% endif %}>
            {{ ui.icon(item.icon) }}<span>{{ item.label
  }}</span>
            {% if item.badge %}<span
  class="app-nav__badge">{{ item.badge }}</span>{% endif
  %}
          </a>
        </li>
        {% endfor %}
      </ul>
      {% endfor %}
    </nav>
    <div class="app-user">
      <a class="app-user__me" href="/profile"
```

```
title="个人设置">
      {% if current_user.avatar %}
      <img class="app-user__avatar"
           src="{{ url_for('static',
filename=current_user.avatar) }}" alt="">
      {% else %}
      <span class="app-user__avatar"
aria-hidden="true">{{ current_user.initial }}</span>
      {% endif %}
      <span style="min-width:0;">
        <span class="app-user__name">{{
current_user.shown_name }}</span>
        <span class="app-user__state">
          <span class="app-user__dot"
aria-hidden="true"></span>
          {{ role_labels.get(current_user.role,
current_user.role) }}
        </span>
      </span>
    </a>
    <a class="app-user__out" href="/logout" title="退出登录"
       aria-label="退出登录">{{ ui.icon("logout", 17) }}</a>
  </div>
</aside>
<div class="app-content">
  <header class="app-header">
    <nav class="crumb" aria-label="当前位置">
      <a href="/">首页</a>
      {{ ui.icon("chevron", 13) }}
      <strong>{{ page_title }}</strong>
    </nav>
    <span class="app-header__spacer"></span>
    <form class="search-box" method="get"
action="/search" role="search">
      {{ ui.icon("search", 16) }}
      <label for="q" class="visually-hidden">搜索项目、图斑、标准<
/label>
      <input id="q" type="search" name="q"
autocomplete="off"
             placeholder="搜索项目、图斑、标准" value="{{
search_q }}">
      <button type="submit" aria-label="搜索">{{
ui.icon("chevron", 15) }}</button>
    </form>
    {% if current_user.is_admin %}
    <a class="icon-btn" href="/profile?tab=logs"
title="操作日志"
       aria-label="操作日志">{{ ui.icon("bell", 18) }}</a>
    {% endif %}
  </header>
  {% else %}
  <div class="login-shell">
  {% endif %}
    <main id="main" class="app-main">
      <div class="container">
        {% for category, message in flash_msgs if category
    not in ("toast", "ok") %}
        <p class="flash {{ category }}" role="status">{{
    message }}</p>
        {% endfor %}
        {% block content %}{% endblock %}
      </div>
    </main>
  {% if current_user.is_authenticated %}
      <footer class="app-foot">
        <span>历史遗留废弃矿山自然恢复适宜性评价系统 V1.0</span>
        <span>数据截止 {{ today }}</span>
      </footer>
    </div>
  </div>
  {% else %}
  </div>
  {% endif %}
  {% block scripts %}{% endblock %}
</body>
</html>
# ==== app/templates/_macros.html ====
{# 共用宏：图标、流程状态面板 #}
{# ==========================================================
    ==================
   图标：内联 SVG 线性图标（24×24 视图框，随文字颜色走）
   不引图标字体或外部图片，避免额外请求与资源依赖；
   stroke 用 currentColor，在深色侧边栏与浅色内容区都能自适应。
   ==========================================================
    ================== #}
{% set ICONS = {
  "dot": '<circle cx="12" cy="12" r="3"/>',
  "home": '<path d="M3 9.6 12 3l9 6.6V20a2 2 0 0 1-2 2H5a2
    2 0 0 1-2-2z"/><path d="M9.5 22v-7h5v7"/>',
  "folder": '<path d="M3 7.5A2 2 0 0 1 5 5.5h3.6l2 2.2H19a2
    2 0 0 1 2 2v7.8a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2z"/>',
  "book": '<path d="M4 4.8A2.8 2.8 0 0 1 6.8
    2H20v17.5H6.8A2.8 2.8 0 0 0 4 22z"/><path d="M4
    19.5A2.8 2.8 0 0 1 6.8 16.7H20"/>',
  "shield": '<path d="M12 22s8-4 8-10V5.2L12 2 4 5.2V12c0 6
    8 10 8 10z"/><path d="m9 12 2.2 2.2L15.5 10"/>',
  "layers": '<path d="m12 2.8 9 4.7-9 4.7-9-4.7z"/><path
    d="m3 12.5 9 4.7 9-4.7"/><path d="m3 17 9 4.7 9-4.7"/>',
  "sliders": '<path d="M4 21v-6M4 11V3M12 21v-9M12 8V3M20
    21v-4M20 13V3"/><path d="M1.5 15h5M9.5 12h5M17.5
    17h5"/>',
  "database": '<ellipse cx="12" cy="5.5" rx="8"
    ry="3.2"/><path d="M4 5.5v13c0 1.8 3.6 3.2 8 3.2s8-1.4
    8-3.2v-13"/><path d="M4 12c0 1.8 3.6 3.2 8 3.2s8-1.4
    8-3.2"/>',
  "target": '<circle cx="12" cy="12" r="9"/><circle cx="12"
    cy="12" r="4.6"/><circle cx="12" cy="12" r="1"/>',
  "file": '<path d="M14 2.8H7a2 2 0 0 0-2 2v14.4a2 2 0 0 0
    2 2h10a2 2 0 0 0 2-2V7.8z"/><path d="M14
    2.8V7.8h5"/><path d="M8.8 13h6.4M8.8 17h6.4"/>',
  "users": '<circle cx="9.5" cy="8" r="3.6"/><path d="M3.5
    20.5v-1.4A3.6 3.6 0 0 1 7.1 15.5h4.8a3.6 3.6 0 0 1 3.6
    3.6v1.4"/><path d="M17 4.9a3.6 3.6 0 0 1 0 6.9"/><path
    d="M19 15.8a3.6 3.6 0 0 1 2.4 3.4v1.3"/>',
  "user": '<circle cx="12" cy="8" r="4"/><path d="M4.5
    21v-1.6A4.4 4.4 0 0 1 8.9 15h6.2a4.4 4.4 0 0 1 4.4
    4.4V21"/>',
  "mail": '<rect x="3" y="5" width="18" height="14"
    rx="2.5"/><path d="m3.8 7 8.2 5.8L20.2 7"/>',
  "phone": '<path d="M6.4 3h2.8l1.5 4.4-2 1.4a12.4 12.4 0 0
    0 6.5 6.5l1.4-2L21 14.8v2.8a2.4 2.4 0 0 1-2.6 2.4A16.6
    16.6 0 0 1 3.5 5.6 2.4 2.4 0 0 1 5.9 3z"/>',
  "lock": '<rect x="4" y="10.4" width="16" height="10.6"
    rx="2.6"/><path d="M8 10.4V7.6a4 4 0 0 1 8 0v2.8"/>',
  "briefcase": '<rect x="3" y="7.4" width="18"
    height="13.1" rx="2.6"/><path d="M8.4 7.4V6A2 2 0 0 1
    10.4 4h3.2a2 2 0 0 1 2 2v1.4"/><path d="M3 12.6h18"/>',
  "search": '<circle cx="11" cy="11" r="7"/><path d="m20.5
    20.5-4-4"/>',
  "bell": '<path d="M18 9.5a6 6 0 1 0-12 0c0 5-2.2 6.2-2.2
    6.2h16.4S18 14.5 18 9.5"/><path d="M13.8 19.5a2 2 0 0
    1-3.6 0"/>',
  "plus": '<path d="M12 5v14M5 12h14"/>',
  "upload": '<path d="M12 16V4"/><path d="m7.5 8.5 4.5-4.5
    4.5 4.5"/><path d="M4 16.5v2A2.5 2.5 0 0 0 6.5
    21h11a2.5 2.5 0 0 0 2.5-2.5v-2"/>',
  "logout": '<path d="M15 4h3.5A2.5 2.5 0 0 1 21 6.5v11A2.5
    2.5 0 0 1 18.5 20H15"/><path d="M10 16.5 5.5 12 10
    7.5"/><path d="M5.5 12H15"/>',
  "chevron": '<path d="m9.5 5.5 6.5 6.5-6.5 6.5"/>',
  "warn": '<path d="M12 3.5 21.5 20H2.5z"/><path d="M12
    10v4.2"/><path d="M12 17.4h.01"/>',
  "mountain": '<path d="m3 20 6.5-11L14 17l2.5-4L21
    20z"/><circle cx="17" cy="6" r="2.2"/>',
} %}
{% macro icon(name, size=18) %}
<svg class="ico" width="{{ size }}" height="{{ size }}"
    viewBox="0 0 24 24"
     fill="none" stroke="currentColor" stroke-width="1.7"
     stroke-linecap="round" stroke-linejoin="round"
     aria-hidden="true" focusable="false">{{
    ICONS.get(name, ICONS.dot)|safe }}</svg>
{% endmacro %}
{# 评价流程步骤条：按论文 4.1 节顺序显示各前置步骤的完成情况
   顺序为：限制性条件判定 → 划分微地质单元 → 录入指标数据 → 执行评价。
   论文原文：「先进行政策、位置约束地质灾害判断……对于满足限制性条件的
   废弃矿山，按照地形地貌划分……三种地质微单元类型」，故判定在前。 #}
{% macro workflow_steps(progress) %}
<div class="steps">
  <div class="step {% if progress.constraint.passed
    %}step--done{% elif progress.constraint.done
    %}step--fail{% else %}step--todo{% endif %}">
    <span class="step__no">1</span>
    <span class="step__name">限制性条件判定</span>
    <span class="step__state">
      {% if progress.constraint.passed %}已通过
      {% elif progress.constraint.done %}未通过
      {% else %}待完成{% endif %}
    </span>
  </div>
  <div class="step {% if progress.units.done %}step--done{%
    else %}step--todo{% endif %}">
    <span class="step__no">2</span>
    <span class="step__name">划分微地质单元</span>
    <span class="step__state">
      {% if progress.units.done %}已完成{% else %}待完成{% endif
    %}
    </span>
  </div>
  <div class="step {% if progress.data.ready %}step--done{%
    else %}step--todo{% endif %}">
    <span class="step__no">3</span>
    <span class="step__name">录入指标数据</span>
    <span class="step__state">
      {% if progress.data.ready %}已完成{% else %}待完成{% endif
    %}
    </span>
  </div>
  <div class="step step--next">
    <span class="step__no">4</span>
    <span class="step__name">执行适宜性评价</span>
    <span class="step__state">当前步骤</span>
  </div>
</div>
{% endmacro %}
{# 前置条件未满足时的提示块：只列缺什么，并给出处理入口 #}
{% macro workflow_blocking(progress) %}
{% if progress.blocking %}
<div class="notice notice--warn">
  <p class="notice__title">尚不能执行本步骤，需要先完成：</p>
  <ol class="notice__list">
    {% for b in progress.blocking %}
    <li>
      <strong>{{ b.label }}</strong>{% if b.reason %}：{{
    b.reason }}{% endif %}
      <a class="link-btn" href="{{ b.fix_url }}">{{
    b.fix_label }}</a>
    </li>
    {% endfor %}
  </ol>
</div>
{% endif %}
{% endmacro %}
{# 一行式状态摘要，用于列表页 #}
{% macro workflow_badge(progress) %}
{% if progress.ready %}
<span class="tag tag--ok">可执行评价</span>
{% elif progress.blocking %}
<span class="tag tag--warn">{{ progress.blocking[0].label
    }}</span>
<span class="muted">{{ progress.blocking[0].reason }}</span>
{% endif %}
{% endmacro %}
{# 层级引导：本系统是「项目 → 图斑」两级结构，评价针对的是图斑而非项目。
   下游各页按图斑组织数据，故只要存在「还没建图斑的项目」，就直接列出并给出入口。 #}
{% macro patch_guide(projects_without_patch) %}
{% if projects_without_patch %}
<div class="notice notice--warn">
  <p class="notice__title">
    以下 {{ projects_without_patch|length }}
    个项目下还没有图斑，本页不会出现它们：
  </p>
  <ul class="notice__list">
    {% for p in projects_without_patch %}
    <li>
      {{ p.name }}
      <a class="link-btn" href="/project/{{ p.id
    }}/patch/add">新建图斑</a>
    </li>
    {% endfor %}
  </ul>
</div>
{% endif %}
{% endmacro %}
# ==== app/templates/_login_badge.svg ====
<svg class="lp-badge__svg" xmlns="http://www.w3.org/2000/svg
    " viewBox="0 0 240 240" role="img"
    aria-label="历史遗留废弃矿山自然恢复适宜性评价系统徽标">
  <title>系统徽标</title>
  <g class="lp-ring" style="--k: 2.7">
```

```
    <circle cx="162.00" cy="120.00" r="2.60"
    fill="#c4f5ab"/>
    <circle cx="157.84" cy="138.22" r="2.97"
    fill="#b8f5ab"/>
    <circle cx="146.19" cy="152.84" r="3.06"
    fill="#acf5ab"/>
    <circle cx="129.35" cy="160.95" r="2.80"
    fill="#abf5b7"/>
    <circle cx="110.65" cy="160.95" r="2.40"
    fill="#abf5c2"/>
    <circle cx="93.81" cy="152.84" r="2.14" fill="#abf5ce"/>
    <circle cx="82.16" cy="138.22" r="2.23" fill="#abf5da"/>
    <circle cx="78.00" cy="120.00" r="2.60" fill="#abf5e6"/>
    <circle cx="82.16" cy="101.78" r="2.97" fill="#abf5da"/>
    <circle cx="93.81" cy="87.16" r="3.06" fill="#abf5ce"/>
    <circle cx="110.65" cy="79.05" r="2.80" fill="#abf5c2"/>
    <circle cx="129.35" cy="79.05" r="2.40" fill="#abf5b7"/>
    <circle cx="146.19" cy="87.16" r="2.14" fill="#acf5ab"/>
    <circle cx="157.84" cy="101.78" r="2.23"
    fill="#b8f5ab"/>
  </g>
  <g class="lp-ring" style="--k: 2.15">
    <circle cx="177.29" cy="129.07" r="3.92"
    fill="#a4e28d"/>
    <circle cx="171.68" cy="146.33" r="4.01"
    fill="#9be28d"/>
    <circle cx="161.01" cy="161.01" r="3.87"
    fill="#91e28d"/>
    <circle cx="146.33" cy="171.68" r="3.55"
    fill="#8de292"/>
    <circle cx="129.07" cy="177.29" r="3.17"
    fill="#8de29c"/>
    <circle cx="110.93" cy="177.29" r="2.88"
    fill="#8de2a6"/>
    <circle cx="93.67" cy="171.68" r="2.79" fill="#8de2af"/>
    <circle cx="78.99" cy="161.01" r="2.93" fill="#8de2b9"/>
    <circle cx="68.32" cy="146.33" r="3.25" fill="#8de2c2"/>
    <circle cx="62.71" cy="129.07" r="3.63" fill="#8de2cc"/>
    <circle cx="62.71" cy="110.93" r="3.92" fill="#8de2cc"/>
    <circle cx="68.32" cy="93.67" r="4.01" fill="#8de2c2"/>
    <circle cx="78.99" cy="78.99" r="3.87" fill="#8de2b9"/>
    <circle cx="93.67" cy="68.32" r="3.55" fill="#8de2af"/>
    <circle cx="110.93" cy="62.71" r="3.17" fill="#8de2a6"/>
    <circle cx="129.07" cy="62.71" r="2.88" fill="#8de29c"/>
    <circle cx="146.33" cy="68.32" r="2.79" fill="#8de292"/>
    <circle cx="161.01" cy="78.99" r="2.93" fill="#91e28d"/>
    <circle cx="171.68" cy="93.67" r="3.25" fill="#9be28d"/>
    <circle cx="177.29" cy="110.93" r="3.63"
    fill="#a4e28d"/>
  </g>
  <g class="lp-ring" style="--k: 1.85">
    <circle cx="194.00" cy="120.00" r="4.94"
    fill="#91cf72"/>
    <circle cx="191.85" cy="137.71" r="4.92"
    fill="#88cf72"/>
    <circle cx="185.52" cy="154.39" r="4.73"
    fill="#80cf72"/>
    <circle cx="175.39" cy="169.07" r="4.42"
    fill="#78cf72"/>
    <circle cx="162.04" cy="180.90" r="4.06"
    fill="#72cf73"/>
    <circle cx="146.24" cy="189.19" r="3.73"
    fill="#72cf7b"/>
    <circle cx="128.92" cy="193.46" r="3.51"
    fill="#72cf83"/>
    <circle cx="111.08" cy="193.46" r="3.45"
    fill="#72cf8b"/>
    <circle cx="93.76" cy="189.19" r="3.56" fill="#72cf93"/>
    <circle cx="77.96" cy="180.90" r="3.81" fill="#72cf9c"/>
    <circle cx="64.61" cy="169.07" r="4.16" fill="#72cfa4"/>
    <circle cx="54.48" cy="154.39" r="4.51" fill="#72cfac"/>
    <circle cx="48.15" cy="137.71" r="4.80" fill="#72cfb4"/>
    <circle cx="46.00" cy="120.00" r="4.94" fill="#72cfbc"/>
    <circle cx="48.15" cy="102.29" r="4.92" fill="#72cfb4"/>
    <circle cx="54.48" cy="85.61" r="4.73" fill="#72cfac"/>
    <circle cx="64.61" cy="70.93" r="4.42" fill="#72cfa4"/>
    <circle cx="77.96" cy="59.10" r="4.06" fill="#72cf9c"/>
    <circle cx="93.76" cy="50.81" r="3.73" fill="#72cf93"/>
    <circle cx="111.08" cy="46.54" r="3.51" fill="#72cf8b"/>
    <circle cx="128.92" cy="46.54" r="3.45" fill="#72cf83"/>
    <circle cx="146.24" cy="50.81" r="3.56" fill="#72cf7b"/>
    <circle cx="162.04" cy="59.10" r="3.81" fill="#72cf73"/>
    <circle cx="175.39" cy="70.93" r="4.16" fill="#78cf72"/>
    <circle cx="185.52" cy="85.61" r="4.51" fill="#80cf72"/>
    <circle cx="191.85" cy="102.29" r="4.80"
    fill="#88cf72"/>
  </g>
  <g class="lp-ring" style="--k: 1.62">
    <circle cx="209.57" cy="128.82" r="5.68"
    fill="#76bb59"/>
    <circle cx="206.12" cy="146.13" r="5.40"
    fill="#6fbb59"/>
    <circle cx="199.37" cy="162.43" r="5.06"
    fill="#68bb59"/>
    <circle cx="189.57" cy="177.10" r="4.71"
    fill="#61bb59"/>
    <circle cx="177.10" cy="189.57" r="4.41"
    fill="#5abb59"/>
    <circle cx="162.43" cy="199.37" r="4.19"
    fill="#59bb5f"/>
    <circle cx="146.13" cy="206.12" r="4.10"
    fill="#59bb66"/>
    <circle cx="128.82" cy="209.57" r="4.15"
    fill="#59bb6d"/>
    <circle cx="111.18" cy="209.57" r="4.32"
    fill="#59bb73"/>
    <circle cx="93.87" cy="206.12" r="4.60" fill="#59bb7a"/>
    <circle cx="77.57" cy="199.37" r="4.94" fill="#59bb81"/>
    <circle cx="62.90" cy="189.57" r="5.29" fill="#59bb88"/>
    <circle cx="50.43" cy="177.10" r="5.59" fill="#59bb8f"/>
    <circle cx="40.63" cy="162.43" r="5.81" fill="#59bb96"/>
    <circle cx="33.88" cy="146.13" r="5.90" fill="#59bb9d"/>
    <circle cx="30.43" cy="128.82" r="5.85" fill="#59bba4"/>
    <circle cx="30.43" cy="111.18" r="5.68" fill="#59bba4"/>
    <circle cx="33.88" cy="93.87" r="5.40" fill="#59bb9d"/>
    <circle cx="40.63" cy="77.57" r="5.06" fill="#59bb96"/>
    <circle cx="50.43" cy="62.90" r="4.71" fill="#59bb8f"/>
    <circle cx="62.90" cy="50.43" r="4.41" fill="#59bb88"/>
    <circle cx="77.57" cy="40.63" r="4.19" fill="#59bb81"/>
    <circle cx="93.87" cy="33.88" r="4.10" fill="#59bb7a"/>
    <circle cx="111.18" cy="30.43" r="4.15" fill="#59bb73"/>
    <circle cx="128.82" cy="30.43" r="4.32" fill="#59bb6d"/>
    <circle cx="146.13" cy="33.88" r="4.60" fill="#59bb66"/>
    <circle cx="162.43" cy="40.63" r="4.94" fill="#59bb5f"/>
    <circle cx="177.10" cy="50.43" r="5.29" fill="#5abb59"/>
    <circle cx="189.57" cy="62.90" r="5.59" fill="#61bb59"/>
    <circle cx="199.37" cy="77.57" r="5.81" fill="#68bb59"/>
    <circle cx="206.12" cy="93.87" r="5.90" fill="#6fbb59"/>
    <circle cx="209.57" cy="111.18" r="5.85"
    fill="#76bb59"/>
  </g>
  <g class="lp-ring" style="--k: 1.46">
    <circle cx="226.00" cy="120.00" r="5.94"
    fill="#65a843"/>
    <circle cx="224.55" cy="137.45" r="5.62"
    fill="#5fa843"/>
    <circle cx="220.26" cy="154.42" r="5.29"
    fill="#59a843"/>
    <circle cx="213.22" cy="170.45" r="5.00"
    fill="#53a843"/>
    <circle cx="203.65" cy="185.11" r="4.77"
    fill="#4da843"/>
    <circle cx="191.79" cy="197.99" r="4.63"
    fill="#47a843"/>
    <circle cx="177.98" cy="208.74" r="4.59"
    fill="#43a846"/>
    <circle cx="162.58" cy="217.07" r="4.67"
    fill="#43a84c"/>
    <circle cx="146.02" cy="222.76" r="4.85"
    fill="#43a852"/>
    <circle cx="128.75" cy="225.64" r="5.10"
    fill="#43a858"/>
    <circle cx="111.25" cy="225.64" r="5.42"
    fill="#43a85e"/>
    <circle cx="93.98" cy="222.76" r="5.75" fill="#43a864"/>
    <circle cx="77.42" cy="217.07" r="6.06" fill="#43a86a"/>
    <circle cx="62.02" cy="208.74" r="6.33" fill="#43a870"/>
    <circle cx="48.21" cy="197.99" r="6.52" fill="#43a876"/>
    <circle cx="36.35" cy="185.11" r="6.60" fill="#43a87c"/>
    <circle cx="26.78" cy="170.45" r="6.58" fill="#43a882"/>
    <circle cx="19.74" cy="154.42" r="6.45" fill="#43a888"/>
    <circle cx="15.45" cy="137.45" r="6.23" fill="#43a88e"/>
    <circle cx="14.00" cy="120.00" r="5.94" fill="#43a894"/>
    <circle cx="15.45" cy="102.55" r="5.62" fill="#43a88e"/>
    <circle cx="19.74" cy="85.58" r="5.29" fill="#43a888"/>
    <circle cx="26.78" cy="69.55" r="5.00" fill="#43a882"/>
    <circle cx="36.35" cy="54.89" r="4.77" fill="#43a87c"/>
    <circle cx="48.21" cy="42.01" r="4.63" fill="#43a876"/>
    <circle cx="62.02" cy="31.26" r="4.59" fill="#43a870"/>
    <circle cx="77.42" cy="22.93" r="4.67" fill="#43a86a"/>
    <circle cx="93.98" cy="17.24" r="4.85" fill="#43a864"/>
    <circle cx="111.25" cy="14.36" r="5.10" fill="#43a85e"/>
    <circle cx="128.75" cy="14.36" r="5.42" fill="#43a858"/>
    <circle cx="146.02" cy="17.24" r="5.75" fill="#43a852"/>
    <circle cx="162.58" cy="22.93" r="6.06" fill="#43a84c"/>
    <circle cx="177.98" cy="31.26" r="6.33" fill="#43a846"/>
    <circle cx="191.79" cy="42.01" r="6.52" fill="#47a843"/>
    <circle cx="203.65" cy="54.89" r="6.60" fill="#4da843"/>
    <circle cx="213.22" cy="69.55" r="6.58" fill="#53a843"/>
    <circle cx="220.26" cy="85.58" r="6.45" fill="#59a843"/>
    <circle cx="224.55" cy="102.55" r="6.23"
    fill="#5fa843"/>
  </g>
  <g class="lp-badge__mark">
    <circle cx="120" cy="120" r="27" fill="#14432f"/>
    <g transform="translate(120 120) scale(1.5)
    translate(-12 -12)" fill="none" stroke="#ffffff"
    stroke-width="1.9" stroke-linecap="round"
    stroke-linejoin="round">
      <path d="m3 20 6.5-11L14 17l2.5-4L21 20z"/>
      <circle cx="17" cy="6" r="2.2"/>
    </g>
  </g>
</svg>
# ==== app/templates/error.html ====
{% extends "base.html" %}
{% block title %}{{ code }} · {{ message }}{% endblock %}
{% block content %}
<div class="card" style="max-width:560px;">
  <h2>{{ code }} · {{ message }}</h2>
  <p class="muted">
    请返回上一页重试；若确认该操作应当可用，请检查当前账号的角色权限。
  </p>
  <p class="actions" style="margin-top:16px;">
    <a class="btn" href="/">返回首页</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/auth/login.html ====
{% extends "base.html" %}
{% from "_macros.html" import icon %}
{% block title %}登录 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<style>
  /* ================== 登录页（仿 Apple 账户页） ==================
     纯白底、居中、大量留白；色彩只用在徽标与按钮上。
     首屏放徽标、标题、一句话说明与登录按钮；系统介绍在下方，
     随滚动逐条弹入、往回滚时弹回。 */
  .lp { color: #1d1d1f; }
  /* ---------- 首屏 ---------- */
  .lp-hero {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    text-align: center;
    min-height: 100dvh;
    padding: 56px 20px 72px;
  }
  /* 徽标：滚动时各圈向四周散开并淡出。
     --p 为 0~1 的滚动进度，由页面脚本写入；
     每圈自带 --k（散开倍数，由内向外递减），使各圈逐渐拉开距离 */
  .lp-badge {
    width: 212px;
    height: 212px;
    margin-bottom: 36px;
    --p: 0;
    animation: lp-in .7s ease both;
  }
  .lp-badge__svg {
    display: block;
    width: 100%;
    height: 100%;
    overflow: visible;             /* 散开时圆点不被画布裁掉 */
    opacity: calc(1 - var(--p) * .94);
    filter: blur(calc(var(--p) * 2.6px));
  }
  .lp-ring {
    transform-box: fill-box;
    transform-origin: center;
    transform: scale(calc(1 + (var(--k) - 1) * var(--p)));
  }
  .lp-badge__mark {
    transform-box: fill-box;
    transform-origin: center;
    transform: scale(calc(1 + .18 * var(--p)));
  }
  .lp-title {
    margin: 0 0 18px;
    font-size: clamp(28px, 4.2vw, 42px);
    font-weight: 700;
    line-height: 1.3;
    letter-spacing: -.4px;
    animation: lp-in .7s ease .06s both;
  }
  .lp-cta {
    padding: 13px 52px;
    font-family: inherit;
    font-size: 17px;
    font-weight: 500;
    color: #fff;
    background: var(--accent);
    border: none;
    border-radius: 980px;
    cursor: pointer;
    animation: lp-in .7s ease .18s both;
    transition: background-color .2s ease, box-shadow .2s
    ease,
                transform .2s ease;
  }
  .lp-cta:hover {
    background: var(--accent-hover);
    box-shadow: 0 10px 26px rgba(31, 107, 69, .26);
  }
  .lp-cta:active { transform: translateY(1px); box-shadow:
    none; }
  .lp-cta:focus-visible { outline: 3px solid rgba(31, 107,
    69, .40); outline-offset: 3px; }
  .lp-more {
    margin-top: 38px;
    font-size: 13px;
    color: #86868b;
    text-decoration: none;
    animation: lp-in .7s ease .24s both;
  }
  .lp-more:hover { color: var(--accent); }
  @keyframes lp-in {
    from { opacity: 0; transform: translateY(12px); }
    to   { opacity: 1; transform: none; }
  }
```

```
/* ---------- 滚动进场 ----------
   进入视口时按方向弹入，离开视口时弹回，可反复。
   交错延迟只加在「进入」的规则上，退出时不延迟，否则会显得拖沓。 */
.lp-reveal {
  opacity: 0;
  transition: opacity .62s ease,
              transform .62s cubic-bezier(.16, .84, .34,
  1),
              border-color .62s ease, background-color
  .62s ease;
  transition-delay: 0s;
  will-change: opacity, transform;
}
.lp-reveal[data-dir="left"]  { transform:
  translate3d(-48px, 0, 0); }
.lp-reveal[data-dir="right"] { transform:
  translate3d(48px, 0, 0); }
.lp-reveal[data-dir="up"]    { transform: translate3d(0,
  36px, 0); }
.lp-reveal.is-in {
  opacity: 1;
  transform: none;
  transition-delay: calc(var(--i, 0) * 85ms);
}
/* 标题颜色由淡到深，强化「浮现」的层次感 */
.lp-reveal h2,
.lp-reveal h3,
.lp-reveal .lp-mod__name { color: #c2d2c8; transition:
  color .7s ease; }
.lp-reveal.is-in h3,
.lp-reveal.is-in .lp-mod__name { color: var(--accent); }
.lp-reveal.is-in h2 { color: #1d1d1f; }
/* ---------- 系统介绍 ---------- */
.lp-intro {
  max-width: 980px;
  margin: 0 auto;
  padding: 20px 24px 72px;
}
.lp-intro__title {
  margin: 0 0 26px;
  font-size: clamp(22px, 2.6vw, 30px);
  text-align: center;
  letter-spacing: -.3px;
}
.lp-points {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px,
  1fr));
  gap: 16px;
  margin-bottom: 46px;
```

```
}
.lp-point {
  padding: 24px 22px;
  background: #f5f5f7;
  border-radius: 18px;
}
.lp-point__icon {
  display: grid;
  place-items: center;
  width: 38px;
  height: 38px;
  margin-bottom: 14px;
  border-radius: 11px;
  background: #fff;
  color: var(--accent);
}
.lp-point h3 { margin: 0 0 8px; font-size: 16px; }
.lp-point p {
  margin: 0;
  font-size: 13.5px;
  line-height: 1.85;
  color: #6e6e73;
  text-wrap: pretty;
}
.lp-sub-title { margin: 0 0 20px; font-size: 17px; }
.lp-mods {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(258px,
  1fr));
  gap: 12px;
  margin-bottom: 46px;
  align-items: start;
}
/* 模块卡：默认只显示编号与名称，鼠标悬停或键盘聚焦时才展开职责说明 */
.lp-mod {
  position: relative;
  padding: 17px 18px 17px 48px;
  background: #fff;
  border: 1px solid #e6e6e8;
  border-radius: 14px;
  outline: none;
  transition: border-color .3s ease, box-shadow .3s ease;
}
.lp-mod:hover,
.lp-mod:focus-within,
.lp-mod:focus {
  border-color: rgba(31, 107, 69, .42);
  box-shadow: 0 8px 22px rgba(19, 60, 43, .08);
}
.lp-mod__no {
    position: absolute;
    left: 17px;
    top: 17px;
    font-size: 13px;
    font-weight: 700;
    color: #c8d6cd;
    font-variant-numeric: tabular-nums;
  }
  .lp-mod__name {
    display: block;
    font-size: 14px;
    font-weight: 600;
  }
  /* 用 grid-template-rows 0fr→1fr 展开，高度过渡自然 */
  .lp-mod__body {
    display: grid;
    grid-template-rows: 0fr;
    transition: grid-template-rows .34s ease, margin-top
    .34s ease;
  }
  .lp-mod:hover .lp-mod__body,
  .lp-mod:focus-within .lp-mod__body {
    grid-template-rows: 1fr;
    margin-top: 7px;
  }
  .lp-mod__desc {
    overflow: hidden;
    margin: 0;
    font-size: 12.5px;
    line-height: 1.7;
    color: #6e6e73;
    opacity: 0;
    transition: opacity .3s ease;
    text-wrap: pretty;
  }
  .lp-mod:hover .lp-mod__desc,
  .lp-mod:focus-within .lp-mod__desc { opacity: 1; }
  /* 触摸设备没有悬停，直接把说明摊开 */
  @media (hover: none) {
    .lp-mod__body { grid-template-rows: 1fr; margin-top:
    7px; }
    .lp-mod__desc { opacity: 1; }
  }
  .lp-foot {
    margin: 0;
    padding-top: 26px;
    border-top: 1px solid #e6e6e8;
    text-align: center;
    font-size: 12.5px;
    color: #86868b;
```

```
}
/* ---------- 登录面板：点击后渐变浮现 ---------- */
.lp-auth {
  position: fixed;
  inset: 0;
  z-index: 60;
  display: grid;
  place-items: center;
  padding: 24px;
  opacity: 0;
  visibility: hidden;
  transition: opacity .42s ease, visibility 0s linear
  .42s;
}
.lp-auth.is-open {
  opacity: 1;
  visibility: visible;
  transition: opacity .42s ease, visibility 0s;
}
/* 遮罩用绿色径向渐变，形成「渐变渗出来」的观感 */
.lp-auth__scrim {
  position: absolute;
  inset: 0;
  background:
    radial-gradient(1000px 640px at 50% 0%,
                    rgba(31, 107, 69, .22), rgba(31, 107,
  69, 0) 70%),
    linear-gradient(180deg, rgba(255, 255, 255, .84),
                            rgba(246, 250, 247, .95));
  backdrop-filter: blur(16px) saturate(1.25);
  -webkit-backdrop-filter: blur(16px) saturate(1.25);
}
.lp-auth__panel {
  position: relative;
  width: 100%;
  max-width: 396px;
  padding: 36px 36px 26px;
  background: #fff;
  border-radius: 20px;
  box-shadow: 0 28px 80px rgba(14, 46, 32, .18);
  opacity: 0;
  transform: translateY(18px) scale(.975);
  transition: transform .55s cubic-bezier(.16, .84, .34,
  1) .05s,
              opacity .55s ease .05s;
}
.lp-auth.is-open .lp-auth__panel { opacity: 1; transform:
  none; }
.lp-auth__close {
  position: absolute;
```

```
    top: 14px;
    right: 14px;
    display: grid;
    place-items: center;
    width: 30px;
    height: 30px;
    color: #86868b;
    background: transparent;
    border: none;
    border-radius: 50%;
    cursor: pointer;
    transition: background-color .18s ease, color .18s ease;
  }
  .lp-auth__close:hover { background: #f5f5f7; color:
    #1d1d1f; }
  .lp-auth__title {
    margin: 0 0 6px;
    text-align: center;
    font-size: 24px;
    font-weight: 700;
    letter-spacing: -.2px;
  }
  .lp-auth__sub {
    margin: 0 0 26px;
    text-align: center;
    font-size: 13px;
    color: #86868b;
  }
  /* 登录失败原因：淡红底的小条，落在登录框里面 */
  .lp-auth__err {
    margin: -14px 0 18px;
    padding: 10px 14px;
    text-align: center;
    font-size: 13px;
    line-height: 1.6;
    color: #9c2b23;
    background: #fdeeec;
    border-radius: 11px;
    animation: lp-err-in .28s ease both;
  }
  @keyframes lp-err-in {
    from { opacity: 0; transform: translateY(-5px); }
    to   { opacity: 1; transform: none; }
  }
  .lp-field { margin-bottom: 15px; }
  .lp-field label {
    display: block;
    margin-bottom: 7px;
    font-size: 13px;
    color: #6e6e73;
  }
  .lp-field input {
    width: 100%;
    padding: 13px 16px;
    font-family: inherit;
    font-size: 16px;            /* 不小于 16px，避免手机端聚焦时自动放大 */
    color: #1d1d1f;
    background: #f5f5f7;
    border: 1px solid transparent;
    border-radius: 12px;
    outline: none;
    transition: background-color .18s ease, border-color
    .18s ease,
                box-shadow .18s ease;
  }
  .lp-field input::placeholder { color: #a1a1a6; }
  .lp-field input:focus {
    background: #fff;
    border-color: rgba(31, 107, 69, .55);
    box-shadow: 0 0 0 3px rgba(31, 107, 69, .14);
  }
  .lp-remember {
    display: flex;
    align-items: center;
    gap: 8px;
    margin: 4px 0 20px;
    font-size: 13px;
    color: #6e6e73;
    cursor: pointer;
  }
  .lp-remember input {
    width: 16px;
    height: 16px;
    margin: 0;
    accent-color: var(--accent);
    cursor: pointer;
  }
  .lp-submit {
    width: 100%;
    padding: 13px 20px;
    font-family: inherit;
    font-size: 16px;
    font-weight: 500;
    color: #fff;
    background: var(--accent);
    border: none;
    border-radius: 980px;
    cursor: pointer;
    transition: background-color .2s ease, box-shadow .2s
    ease,
                transform .2s ease;
  }
  .lp-submit:hover {
    background: var(--accent-hover);
    box-shadow: 0 10px 24px rgba(31, 107, 69, .26);
  }
  .lp-submit:active { transform: translateY(1px);
    box-shadow: none; }
  .lp-submit:focus-visible { outline: 3px solid rgba(31,
    107, 69, .40); outline-offset: 3px; }
  .lp-auth__hint {
    margin: 18px 0 0;
    text-align: center;
    font-size: 12.5px;
    color: #a1a1a6;
  }
  .lp-auth__reg {
    color: var(--accent);
    font-weight: 500;
    text-decoration: none;
    border-bottom: 1px solid rgba(31, 107, 69, .35);
    transition: border-color .18s ease;
  }
  .lp-auth__reg:hover { border-bottom-color: var(--accent);
    }
  /* 系统设置里要求减少动态效果时，去掉位移、缩放与滚动特效 */
  @media (prefers-reduced-motion: reduce) {
    .lp-badge, .lp-title, .lp-cta, .lp-more { animation:
    none; }
    .lp-badge__svg { opacity: 1; filter: none; }
    .lp-ring, .lp-badge__mark { transform: none; }
    .lp-reveal { opacity: 1; transform: none !important; }
    .lp-reveal h2, .lp-reveal h3, .lp-reveal .lp-mod__name
    { color: inherit; }
    .lp-reveal.is-in h2 { color: #1d1d1f; }
    .lp-reveal.is-in h3, .lp-reveal.is-in .lp-mod__name {
    color: var(--accent); }
    .lp-auth, .lp-auth__panel { transition: opacity .2s
    linear; }
    .lp-auth__panel { transform: none; }
  }
  @media (max-width: 560px) {
    .lp-hero { padding: 36px 18px 48px; min-height: auto; }
    .lp-badge { width: 160px; height: 160px; margin-bottom:
    26px; }
    .lp-auth__panel { padding: 30px 24px 22px; }
  }
</style>
<div class="lp">
  {# ---------------- 首屏：徽标 + 标题 + 说明 + 登录入口
```

```
    ---------------- #}
  <section class="lp-hero">
    <div class="lp-badge" id="lp-badge" aria-hidden="true">
      {% include "_login_badge.svg" %}
    </div>
    <h1 class="lp-title">历史遗留废弃矿山<br>自然恢复适宜性评价系统</h1>
    <button type="button" class="lp-cta" id="lp-open"
            aria-expanded="false" aria-controls="lp-auth">登录
    </button>
    <a class="lp-more" href="#intro">
      向下滑动，了解系统 <span aria-hidden="true">↓</span>
    </a>
  </section>
  {# ---------------- 系统介绍：随滚动逐条弹入 ---------------- #}
  <section class="lp-intro" id="intro">
    <h2 class="lp-intro__title lp-reveal"
    data-dir="up">一套把评价流程固定下来的工具</h2>
    <div class="lp-points">
      {% set pdirs = ['left', 'up', 'right'] %}
      {% set points = [
        ('sliders', '方法可溯',
          '权重由层次分析法算出并做一致性检验，指标标准化、分单元赋分与面积加权合成的每一步都留有记录，可逐层回
    看。'),
        ('shield', '流程可控',
          '政策、区位约束、土壤污染、地质灾害、耕地属性五项限制性条件一票否决；未通过即终止评价，直接给出生态重
    建或人工辅助的结论。'),
        ('file', '结果可复算',
          '评价结果与逐层得分明细一并留存，报告取自已保存的评价记录，事后修改数据不会让报告与结论对不上。')
      ] %}
      {% for ic, title, text in points %}
      <div class="lp-point lp-reveal" data-dir="{{
    pdirs[loop.index0 % 3] }}"
           style="--i: {{ loop.index0 }}">
        <span class="lp-point__icon">{{ icon(ic, 20)
    }}</span>
        <h3>{{ title }}</h3>
        <p>{{ text }}</p>
      </div>
      {% endfor %}
    </div>
    <h3 class="lp-sub-title lp-reveal"
    data-dir="up">九个功能模块</h3>
    <div class="lp-mods">
      {% set mdirs = ['left', 'up', 'right'] %}
      {% for no, name, desc in modules %}
      <div class="lp-mod lp-reveal" data-dir="{{
    mdirs[loop.index0 % 3] }}"
           style="--i: {{ loop.index0 % 3 }}" tabindex="0">
        <span class="lp-mod__no">{{ no }}</span>
        <span class="lp-mod__name">{{ name }}</span>
      <div class="lp-mod__body"><p class="lp-mod__desc">{{
  desc }}</p></div>
    </div>
    {% endfor %}
  </div>
  <p class="lp-foot lp-reveal" data-dir="up">
    历史遗留废弃矿山自然恢复适宜性评价系统 V1.0 ·
    运行于浏览器，数据保存在本机 SQLite 数据库
  </p>
</section>
{# ---------------- 登录面板：点击「登录」后渐变浮现 ---------------- #}
<div class="lp-auth" id="lp-auth" aria-hidden="true">
  <div class="lp-auth__scrim" data-lp-close></div>
  <div class="lp-auth__panel" role="dialog"
  aria-modal="true"
       aria-labelledby="lp-auth-title">
    <button type="button" class="lp-auth__close"
  data-lp-close
            aria-label="关闭登录框">
      <svg width="16" height="16" viewBox="0 0 24 24"
  fill="none"
           stroke="currentColor" stroke-width="2"
  stroke-linecap="round"
           aria-hidden="true"><path d="M6 6l12 12M18 6L6
  18"/></svg>
    </button>
    <h2 class="lp-auth__title" id="lp-auth-title">登录</h2>
    <p class="lp-auth__sub">使用系统账号登录</p>
    {# 失败原因渲染在登录框内部：放在正文顶部会被遮罩盖住，等于没提示 #}
    {% if error %}
    <p class="lp-auth__err" role="alert">{{ error }}</p>
    {% endif %}
    <form method="post" novalidate>
      {{ form.hidden_tag() }}
      <div class="lp-field">
        <label for="{{ form.username.id }}">账户名称</label>
        {{ form.username(id=form.username.id,
  autocomplete="username",
                         placeholder="请输入账户名称") }}
      </div>
      <div class="lp-field">
        <label for="{{ form.password.id }}">密码</label>
        {{ form.password(id=form.password.id,
  autocomplete="current-password",
                         placeholder="请输入密码") }}
      </div>
      <label class="lp-remember" for="{{ form.remember.id
  }}">
        {{ form.remember(id=form.remember.id) }}
        <span>记住我（下次自动登录，一个月内有效）</span>
        </label>
        {{ form.submit(class_="lp-submit") }}
      </form>
      <p class="lp-auth__hint">
        还没有账号？<a class="lp-auth__reg"
                      href="{{ url_for('auth.register')
    }}">点我注册</a>
      </p>
      <p class="lp-auth__hint" style="margin-top:10px;">
        演示账号：admin 密码：admin123
      </p>
    </div>
  </div>
</div>
{% endblock %}
{% block scripts %}
<script>
(function () {
  var reduce = window.matchMedia &&
    window.matchMedia('(prefers-reduced-motion:
    reduce)').matches;
  /* ---------- 登录面板 ---------- */
  var btn = document.getElementById('lp-open');
  var auth = document.getElementById('lp-auth');
  var lastFocus = null;
  function openPanel() {
    lastFocus = document.activeElement;
    auth.classList.add('is-open');
    auth.setAttribute('aria-hidden', 'false');
    btn.setAttribute('aria-expanded', 'true');
    document.body.style.overflow = 'hidden';
    // 记到地址栏，直接访问 /login#login 也能展开登录框
    if (location.hash !== '#login') {
      history.replaceState(null, '', location.pathname +
    '#login');
    }
    var first = auth.querySelector('input');
    if (first) window.setTimeout(function () {
    first.focus(); }, 300);
  }
  function closePanel() {
    auth.classList.remove('is-open');
    auth.setAttribute('aria-hidden', 'true');
    btn.setAttribute('aria-expanded', 'false');
    document.body.style.overflow = '';
    if (location.hash === '#login') {
      history.replaceState(null, '', location.pathname);
    }
    if (lastFocus && lastFocus.focus) lastFocus.focus();
  }
```

```
if (btn && auth) {
  btn.addEventListener('click', openPanel);
  Array.prototype.forEach.call(
    auth.querySelectorAll('[data-lp-close]'),
    function (el) { el.addEventListener('click',
  closePanel); }
  );
  document.addEventListener('keydown', function (e) {
    if (e.key === 'Escape' && auth.classList.contains('is-
  open')) closePanel();
  });
  // 登录失败回到本页、或直接带 #login 访问时，直接展开登录框
  if (location.hash === '#login' ||
  document.querySelector('.lp-auth__err')) {
    openPanel();
  }
}
/* ---------- 内容进场 + 徽标散开：同一套滚动计算 ----------
   原先用 IntersectionObserver，实测在滚动到位置后回调并未生效，
   内容始终停在 opacity:0。改为直接按元素与视口的相对位置判断，
   与徽标用同一次滚动回调，行为可预期、也便于排查。 */
var items = document.querySelectorAll('.lp-reveal');
var badge = document.getElementById('lp-badge');
var ticking = false;
function applyScroll() {
  ticking = false;
  var vh = window.innerHeight || 800;
  /* 徽标：滚动前约三分之一屏内完成散开淡出 */
  if (badge) {
    var range = vh * 0.34;
    var p = range > 0 ? window.scrollY / range : 0;
    badge.style.setProperty('--p', Math.min(1,
  Math.max(0, p)).toFixed(3));
  }
  /* 内容：先批量测量，再改类名，避免读写交替触发反复重排 */
  if (reduce || !items.length) return;
  var box = [];
  for (var i = 0; i < items.length; i++) {
    box.push(items[i].getBoundingClientRect());
  }
  for (var j = 0; j < items.length; j++) {
    var r = box[j];
    // 上沿进入视口下部、下沿尚未越出视口上部 → 视为在视野内
    var shown = r.top < vh * 0.88 && r.bottom > vh * 0.06;
    if (shown) items[j].classList.add('is-in');
    else items[j].classList.remove('is-in');
  }
}
function onScroll() {
  if (!ticking) {
      ticking = true;
      window.requestAnimationFrame(applyScroll);
    }
  }
  // 减少动态效果时不做滚动特效，内容直接显示
  if (reduce) {
    Array.prototype.forEach.call(items, function (el) {
      el.classList.add('is-in');
    });
    if (badge) badge.style.setProperty('--p', '0');
  } else {
    window.addEventListener('scroll', onScroll, { passive: true });
    window.addEventListener('resize', onScroll);
    applyScroll();
  }
})();
</script>
{% endblock %}
# ==== app/templates/auth/register.html ====
{% extends "base.html" %}
{% block title %}注册账号 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<style>
  /* ================= 注册页 =================
     与登录页同一套观感：浅绿渐变底 + 白色卡片。
     必填只有账户名称与密码，其余收在「选填」分隔线以下，
     这样首次注册不会被一堆输入框挡住。 */
  .rg {
    min-height: 100dvh;
    display: grid;
    place-items: center;
    padding: 40px 20px 56px;
    background:
      radial-gradient(900px 520px at 50% 0%,
                      rgba(31, 107, 69, .14), rgba(31, 107, 69, 0) 72%),
      #f6faf7;
  }
  .rg-card {
    width: 100%;
    max-width: 470px;
    padding: 38px 40px 30px;
    background: #fff;
    border-radius: 20px;
    box-shadow: 0 26px 70px rgba(14, 46, 32, .13);
    animation: rg-in .5s cubic-bezier(.16, .84, .34, 1) both;
  }
  @keyframes rg-in {
    from { opacity: 0; transform: translateY(16px)
    scale(.98); }
    to   { opacity: 1; transform: none; }
  }
  .rg-title {
    margin: 0 0 6px;
    text-align: center;
    font-size: 25px;
    font-weight: 700;
    letter-spacing: -.2px;
  }
  .rg-sub {
    margin: 0 0 28px;
    text-align: center;
    font-size: 13px;
    color: #86868b;
  }
  .rg-field { margin-bottom: 15px; }
  .rg-field > label {
    display: block;
    margin-bottom: 7px;
    font-size: 13px;
    color: #6e6e73;
  }
  .rg-field > label .rg-opt {
    margin-left: 5px;
    font-size: 12px;
    color: #a1a1a6;
  }
  .rg-field input[type="text"],
  .rg-field input[type="password"] {
    width: 100%;
    padding: 12px 15px;
    font-family: inherit;
    font-size: 16px;          /* 不小于 16px，避免手机端聚焦时自动放大 */
    color: #1d1d1f;
    background: #f5f5f7;
    border: 1px solid transparent;
    border-radius: 12px;
    outline: none;
    transition: background-color .18s ease, border-color
    .18s ease,
                box-shadow .18s ease;
  }
  .rg-field input::placeholder { color: #a1a1a6; }
  .rg-field input:focus {
    background: #fff;
    border-color: rgba(31, 107, 69, .55);
    box-shadow: 0 0 0 3px rgba(31, 107, 69, .14);
  }
```

```
.rg-field input[aria-invalid="true"] {
  border-color: rgba(180, 60, 50, .55);
  background: #fff;
}
.rg-field select {
  width: 100%;
  padding: 12px 15px;
  font-family: inherit;
  font-size: 15px;
  color: #1d1d1f;
  background: #f5f5f7;
  border: 1px solid transparent;
  border-radius: 12px;
  outline: none;
}
.rg-field select:focus {
  background: #fff;
  border-color: rgba(31, 107, 69, .55);
  box-shadow: 0 0 0 3px rgba(31, 107, 69, .14);
}
.rg-err {
  margin: 6px 0 0;
  font-size: 12.5px;
  color: var(--danger);
}
/* 「选填」分隔线 */
.rg-split {
  display: flex;
  align-items: center;
  gap: 12px;
  margin: 24px 0 18px;
  font-size: 12.5px;
  color: #a1a1a6;
}
.rg-split::before,
.rg-split::after {
  content: "";
  flex: 1;
  height: 1px;
  background: #ececee;
}
.rg-pair {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 0 14px;
}
@media (max-width: 480px) { .rg-pair {
  grid-template-columns: 1fr; } }
/* 头像：圆形预览 + 选择文件 */
.rg-avatar {
    display: flex;
    align-items: center;
    gap: 14px;
    margin-bottom: 15px;
  }
  .rg-avatar__face {
    display: grid;
    place-items: center;
    flex: none;
    width: 54px;
    height: 54px;
    border-radius: 50%;
    background: #eef2f0;
    color: #9aa8a1;
    font-size: 12px;
    overflow: hidden;
    background-size: cover;
    background-position: center;
  }
  .rg-avatar__pick { min-width: 0; }
  .rg-avatar__pick input[type="file"] {
    max-width: 100%;
    font-family: inherit;
    font-size: 12.5px;
    color: #6e6e73;
  }
  .rg-avatar__tip {
    margin: 4px 0 0;
    font-size: 12px;
    color: #a1a1a6;
  }
  .rg-submit {
    width: 100%;
    margin-top: 24px;
    padding: 13px 20px;
    font-family: inherit;
    font-size: 16px;
    font-weight: 500;
    color: #fff;
    background: var(--accent);
    border: none;
    border-radius: 980px;
    cursor: pointer;
    transition: background-color .2s ease, box-shadow .2s
    ease,
                transform .2s ease;
  }
  .rg-submit:hover {
    background: var(--accent-hover);
    box-shadow: 0 10px 24px rgba(31, 107, 69, .26);
  }
  .rg-submit:active { transform: translateY(1px);
    box-shadow: none; }
  .rg-submit:focus-visible { outline: 3px solid rgba(31,
    107, 69, .40); outline-offset: 3px; }
  .rg-foot {
    margin: 20px 0 0;
    text-align: center;
    font-size: 13px;
    color: #86868b;
  }
  .rg-foot a { color: var(--accent); text-decoration: none;
    }
  .rg-foot a:hover { text-decoration: underline; }
  @media (prefers-reduced-motion: reduce) {
    .rg-card { animation: none; }
  }
</style>
<div class="rg">
  <div class="rg-card">
    <h1 class="rg-title">创建账号</h1>
    <p class="rg-sub">新账号为「操作员」，职务由管理员调整</p>
    <form method="post" enctype="multipart/form-data"
    novalidate>
      {{ form.hidden_tag() }}
      <div class="rg-field">
        <label for="{{ form.username.id }}">账户名称</label>
        {{ form.username(id=form.username.id,
    autocomplete="username",
                         placeholder="2–64 个字符",
                         **({'aria-invalid': 'true'} if
    form.username.errors else {})) }}
        {% for e in form.username.errors %}<p
    class="rg-err">{{ e }}</p>{% endfor %}
      </div>
      <div class="rg-field">
        <label for="{{ form.password.id }}">密码</label>
        {{ form.password(id=form.password.id,
    autocomplete="new-password",
                         placeholder="至少 6 位",
                         **({'aria-invalid': 'true'} if
    form.password.errors else {})) }}
        {% for e in form.password.errors %}<p
    class="rg-err">{{ e }}</p>{% endfor %}
      </div>
      <div class="rg-field">
        <label for="{{ form.confirm.id }}">确认密码</label>
        {{ form.confirm(id=form.confirm.id,
    autocomplete="new-password",
                        placeholder="再次输入密码",
                        **({'aria-invalid': 'true'} if
    form.confirm.errors else {})) }}
        {% for e in form.confirm.errors %}<p
    class="rg-err">{{ e }}</p>{% endfor %}
      </div>
      <p class="rg-split">以下均为选填</p>
      <div class="rg-avatar">
        <span class="rg-avatar__face" id="rg-face"
    aria-hidden="true">头像</span>
        <div class="rg-avatar__pick">
          <label class="visually-hidden"
    for="rg-avatar">选择头像图片</label>
          <input id="rg-avatar" type="file" name="avatar"
                 accept=".png,.jpg,.jpeg,.gif,.webp">
          <p class="rg-avatar__tip">PNG / JPG / GIF /
    WebP，不超过 4 MB</p>
        </div>
      </div>
      <div class="rg-pair">
        <div class="rg-field">
          <label for="{{ form.display_name.id }}">昵称</label>
          {{ form.display_name(id=form.display_name.id,
                               placeholder="留空则显示账户名称") }}
        </div>
        <div class="rg-field">
          <label for="{{ form.gender.id }}">性别</label>
          {{ form.gender(id=form.gender.id) }}
        </div>
      </div>
      <div class="rg-field">
        <label for="{{ form.email.id }}">邮箱</label>
        {{ form.email(id=form.email.id,
    autocomplete="email",
                      placeholder="用于联系，可留空",
                      **({'aria-invalid': 'true'} if
    form.email.errors else {})) }}
        {% for e in form.email.errors %}<p
    class="rg-err">{{ e }}</p>{% endfor %}
      </div>
      <div class="rg-field" style="margin-bottom:0;">
        <label for="{{ form.phone.id }}">手机号</label>
        {{ form.phone(id=form.phone.id, autocomplete="tel",
    placeholder="可留空") }}
      </div>
      {{ form.submit_reg(class_="rg-submit") }}
    </form>
    <p class="rg-foot">已有账号？<a href="{{
    url_for('auth.login') }}">返回登录</a></p>
  </div>
</div>
```

```
{% endblock %}
{% block scripts %}
<script>
// 选好头像后立刻在左侧圆框里预览，省得传上去才知道选错了
(function () {
  var input = document.getElementById('rg-avatar');
  var face = document.getElementById('rg-face');
  if (!input || !face || !window.FileReader) return;
  input.addEventListener('change', function () {
    var file = input.files && input.files[0];
    if (!file) return;
    var reader = new FileReader();
    reader.onload = function (e) {
      face.textContent = '';
      face.style.backgroundImage = 'url("' +
    e.target.result + '")';
    };
    reader.readAsDataURL(file);
  });
})();
</script>
{% endblock %}
# ==== app/templates/auth/home.html ====
{% extends "base.html" %}
{% from "_macros.html" import icon %}
{% block title %}工作台 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
{# ---------- 页首横幅 ---------- #}
<section class="hero">
  <div class="hero__body">
    <h1 class="hero__title">历史遗留废弃矿山自然恢复适宜性评价系统</h1>
    <p class="hero__desc">
      以论文《历史遗留废弃矿山自然恢复适宜性评价指标体系建设及典型案例》
      建立的评价方法为算法内核。下方按论文 4.1 节的评价流程顺序排列，
      系统会在每一步检查前置条件，前置步骤未完成时不会放行后续计算。
    </p>
    <div class="hero__actions">
      <a class="hero-btn hero-btn--ghost"
    href="/indicator/import">
        {{ icon("upload", 16) }}批量导入
      </a>
      <a class="hero-btn hero-btn--solid"
    href="/project/add">
        {{ icon("plus", 16) }}新建评价项目
      </a>
    </div>
  </div>
</section>
{# ---------- 数据概览 ---------- #}
<section class="stat-cards">
```

```
  <a class="stat-card" href="/project/">
    <span class="stat-card__label">评价项目</span>
    <span class="stat-card__value">{{ dash.stats.projects
    }}<span class="stat-card__unit">个</span></span>
    <span class="stat-card__note">
      {%- if projects_without_patch -%}
        其中 {{ projects_without_patch|length }} 个项目尚未建立图斑
      {%- elif dash.progress.patches -%}
        已建立 {{ dash.progress.patches }} 个矿山图斑
      {%- else -%}
        尚未建立矿山图斑
      {%- endif -%}
    </span>
  </a>
  <a class="stat-card" href="/project/">
    <span class="stat-card__label">矿山图斑</span>
    <span class="stat-card__value">{{ dash.stats.patches
    }}<span class="stat-card__unit">个</span></span>
    <span class="stat-card__note">
      {% if dash.progress.ready %}其中 {{ dash.progress.ready
    }} 个具备评价条件{% else %}暂无具备评价条件的图斑{% endif %}
    </span>
  </a>
  <a class="stat-card" href="/unit/">
    <span class="stat-card__label">微地质单元</span>
    <span class="stat-card__value">{{ dash.stats.unit_types
    }}<span class="stat-card__unit">类</span></span>
    <span class="stat-card__note">
      {% if dash.progress.units %}共划分 {{
    dash.progress.units }} 个单元{% else %}尚未划分单元{% endif %}
    </span>
  </a>
  <a class="stat-card" href="/standard/">
    <span class="stat-card__label">标准与政策</span>
    <span class="stat-card__value">{{ dash.stats.standards
    }}<span class="stat-card__unit">项</span></span>
    <span class="stat-card__note">地方标准与政策文件</span>
  </a>
</section>
{# ---------- 评价流程 ---------- #}
<section class="section">
  <div class="section__head">
    <h2 class="section__title">评价流程</h2>
    <p class="section__desc">
      按论文 4.1 节顺序排列，前置步骤未完成时不会执行后续计算
    </p>
  </div>
  <div class="step-grid">
    {% for s in dash.steps %}
    {% if s.available %}
    <a class="step-card" href="{{ s.href }}">
      <span class="step-card__no">{{ s.no }}</span>
      <span class="step-card__icon">{{ icon(s.icon, 18)
    }}</span>
      <span class="step-card__title">{{ s.title }}</span>
      <span class="step-card__desc">{{ s.desc }}</span>
      {% if s.pending %}
      <span class="step-card__badge">待处理 {{ s.pending
    }}</span>
      {% endif %}
    </a>
    {% else %}
    <div class="step-card step-card--locked"
    aria-disabled="true">
      <span class="step-card__no">{{ s.no }}</span>
      <span class="step-card__icon">{{ icon(s.icon, 18)
    }}</span>
      <span class="step-card__title">{{ s.title }}</span>
      <span class="step-card__desc">{{ s.desc }}</span>
      <span class="step-card__lock">{{ s.note }}</span>
    </div>
    {% endif %}
    {% endfor %}
  </div>
</section>
{# ---------- 系统管理 ---------- #}
<section class="section">
  <div class="section__head">
    <h2 class="section__title">系统管理</h2>
  </div>
  <a class="row-card" href="/profile">
    <span class="row-card__icon">{{ icon("user", 19)
    }}</span>
    <span class="row-card__main">
      <span class="row-card__title">个人设置</span>
      <span class="row-card__desc">维护昵称、性别、邮箱、手机号与登录密码</span
    >
    </span>
    <span class="row-card__arrow">{{ icon("chevron", 17)
    }}</span>
  </a>
  {% if current_user.is_admin %}
  <a class="row-card" href="/profile?tab=users">
    <span class="row-card__icon">{{ icon("users", 19)
    }}</span>
    <span class="row-card__main">
      <span class="row-card__title">账号与操作日志</span>
      <span class="row-card__desc">新增账号并设置职务；全部写操作自动留痕，可查询追溯
    </span>
    </span>
    <span class="row-card__arrow">{{ icon("chevron", 17)
    }}</span>
  </a>
  {% endif %}
</section>
{% endblock %}
# ==== app/templates/auth/add_user.html ====
{% extends "base.html" %}
{% block title %}新增账号{% endblock %}
{% block content %}
<div class="card" style="max-width:560px;">
  <h2>新增账号</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="username">账户名称</label
    >
      <input id="username" type="text" name="username"
    required
             maxlength="64" autocomplete="off"
    style="width:280px;">
    </div>
    <div class="field">
      <label class="field__label" for="password">密码</label>
      <input id="password" type="password" name="password"
    required
             autocomplete="new-password"
    style="width:280px;">
    </div>
    <div class="field">
      <label class="field__label" for="role">职务</label>
      <select id="role" name="role">
        {% for code, label in role_labels.items() %}
        <option value="{{ code }}" {% if code ==
    default_role %}selected{% endif %}>{{ label }}</option>
        {% endfor %}
      </select>
      <p class="muted" style="margin:6px 0 0;">
        管理员可管理系统、维护基础数据并查看操作日志；操作员可使用全部业务功能。
      </p>
    </div>
    <hr style="margin:22px 0;border:none;border-top:1px
    solid var(--line);">
    <h3 style="margin:0 0 12px;font-size:15px;">个人资料（选填）</h3
    >
    <div class="field">
      <label class="field__label" for="display_name">昵称</lab
    el>
      <input id="display_name" type="text"
    name="display_name" maxlength="64"
```

```
             placeholder="留空则显示账户名称">
    </div>
    <div class="field">
      <label class="field__label" for="gender">性别</label>
      <select id="gender" name="gender">
        <option value="">不填</option>
        {% for g in genders %}<option value="{{ g }}">{{ g
    }}</option>{% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="email">邮箱</label>
      <input id="email" type="text" name="email"
    maxlength="128">
    </div>
    <div class="field">
      <label class="field__label" for="phone">手机号</label>
      <input id="phone" type="text" name="phone"
    maxlength="32">
    </div>
    <p class="actions" style="margin-top:18px;">
      <button type="submit" class="btn">保存</button>
      <a class="btn btn--ghost" href="{{
    url_for('auth.profile', tab='users') }}">返回</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/auth/edit_user.html ====
{% extends "base.html" %}
{% block title %}编辑账号 · {{ user.username }}{% endblock %}
{% block content %}
<div class="card" style="max-width:560px;">
  <h2>编辑账号 · {{ user.username }}</h2>
  <h3 style="margin:0 0 12px;font-size:15px;">个人资料</h3>
  <p class="muted" style="margin:0 0 14px;font-size:12.5px;"
    >
    以下内容用户也可在「个人设置」中自行修改，此处为管理员代改。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="display_name">昵称</lab
    el>
      <input id="display_name" type="text"
    name="display_name" maxlength="64"
             value="{{ user.display_name or '' }}"
             placeholder="留空则显示账户名称">
    </div>
    <div class="field">
  <label class="field__label" for="gender">性别</label>
  <select id="gender" name="gender">
    <option value="">不填</option>
    {% for g in genders %}
    <option value="{{ g }}" {% if user.gender == g
%}selected{% endif %}>{{ g }}</option>
    {% endfor %}
  </select>
</div>
<div class="field">
  <label class="field__label" for="email">邮箱</label>
  <input id="email" type="text" name="email"
maxlength="128"
         value="{{ user.email or '' }}">
</div>
<div class="field">
  <label class="field__label" for="phone">手机号</label>
  <input id="phone" type="text" name="phone"
maxlength="32"
         value="{{ user.phone or '' }}">
</div>
<hr style="margin:22px 0;border:none;border-top:1px
solid var(--line);">
<h3 style="margin:0 0 12px;font-size:15px;">职务与权限</h3>
<p class="muted" style="margin:0 0
14px;font-size:12.5px;">
  职务只能由管理员设置：管理员可管理系统、维护基础数据并查看操作日志；
  操作员可使用全部业务功能。
</p>
<div class="field">
  <label class="field__label" for="role">职务</label>
  <select id="role" name="role">
    {% for code, label in role_labels.items() %}
    <option value="{{ code }}" {% if user.role == code
%}selected{% endif %}>{{ label }}</option>
    {% endfor %}
  </select>
</div>
<div class="field">
  <label class="checkbox" for="active">
    <input id="active" type="checkbox" name="active"
value="1"
           {% if user.active %}checked{% endif %}>
    <span>启用该账号</span>
  </label>
  <p class="muted" style="margin:6px 0 0;">
    停用后该账号无法登录，但其历史操作记录会保留。
    系统至少需保留一名启用状态的管理员。
  </p>
</div>
```

```
    <div class="field">
      <label class="field__label" for="password">重置密码（留空则不修改
    )</label>
      <input id="password" type="password" name="password"
             autocomplete="new-password"
    style="width:280px;"
             placeholder="留空表示保持原密码">
    </div>
    <p class="actions" style="margin-top:18px;">
      <button type="submit" class="btn">保存</button>
      <a class="btn btn--ghost" href="{{
    url_for('auth.profile', tab='users') }}">返回</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/auth/profile.html ====
{% extends "base.html" %}
{% from "_macros.html" import icon %}
{% block title %}个人设置 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<style>
  /* ================= 个人设置（仿 Apple 账户页） =================
     左栏是身份与分类，右栏是该分类下的功能卡。
     每张卡默认只显示当前值，点开才展开对应的修改表单 ——
     不点就不会误改，页面也不会一上来堆满输入框。 */
  .acct {
    display: grid;
    grid-template-columns: 244px 1fr;
    gap: 40px;
    align-items: start;
  }
  @media (max-width: 900px) {
    .acct { grid-template-columns: 1fr; gap: 24px; }
  }
  /* ---------- 左栏 ---------- */
  .acct-rail__avatar {
    display: grid;
    place-items: center;
    width: 88px;
    height: 88px;
    border-radius: 50%;
    background: #e9edeb;
    color: #97a09b;
    font-size: 32px;
    font-weight: 500;
    object-fit: cover;
  }
  .acct-rail__name {
    margin: 16px 0 2px;
    font-size: 16px;
    font-weight: 600;
  }
  .acct-rail__sub {
    margin: 0 0 24px;
    font-size: 13px;
    color: #86868b;
    word-break: break-all;
  }
  .acct-tabs {
    display: flex;
    flex-direction: column;
    gap: 2px;
    margin-bottom: 24px;
  }
  .acct-tabs a {
    padding: 7px 12px;
    border: 1px solid transparent;
    border-radius: 7px;
    font-size: 15px;
    color: #1d1d1f;
    text-decoration: none;
    transition: background-color .16s ease, border-color
    .16s ease;
  }
  .acct-tabs a:hover { background: #f5f5f7; }
  .acct-tabs a.is-active {
    font-weight: 600;
    border-color: #bcc9c1;
    background: #fff;
  }
  /* 更换头像：折叠在左栏，点开才出现文件选择 */
  .acct-avatar-edit > summary {
    display: inline-block;
    padding: 5px 12px;
    font-size: 13px;
    color: var(--accent);
    background: var(--accent-soft);
    border-radius: 7px;
    cursor: pointer;
    list-style: none;
  }
  .acct-avatar-edit > summary::-webkit-details-marker {
    display: none; }
  .acct-avatar-edit > summary:hover { background: #dcebe2; }
  .acct-avatar-edit__body { margin-top: 12px; }
  .acct-avatar-edit__body .btn { width: 100%;
    margin-bottom: 8px; }
  .acct-avatar-edit__file {
    margin: 0 0 10px;
    font-size: 12px;
    color: var(--ink-soft);
    word-break: break-all;
  }
  /* ---------- 右栏 ---------- */
  .acct-title {
    margin: 0 0 8px;
    font-size: 26px;
    font-weight: 700;
    letter-spacing: -.3px;
  }
  .acct-lead {
    margin: 0 0 26px;
    font-size: 14px;
    color: #6e6e73;
  }
  /* 固定两列，与 Apple 账户页的 2×2 一致；
     卡片也宽一些，邮箱之类的长值不至于被拆成两行 */
  .acct-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 16px;
    align-items: start;
  }
  @media (max-width: 760px) {
    .acct-grid { grid-template-columns: 1fr; }
  }
  /* 功能卡：details/summary 实现，点开才展开表单 */
  .acct-card {
    background: #fff;
    border: 1px solid #e6e6e8;
    border-radius: 14px;
    transition: border-color .2s ease, box-shadow .2s ease;
  }
  .acct-card[open] {
    border-color: rgba(31, 107, 69, .45);
    box-shadow: 0 8px 22px rgba(19, 60, 43, .08);
  }
  .acct-card > summary {
    position: relative;
    display: block;
    padding: 18px 56px 18px 20px;
    border-radius: 14px;
    cursor: pointer;
    list-style: none;
  }
  .acct-card > summary::-webkit-details-marker { display:
    none; }
  .acct-card > summary:hover { background: #fafbfa; }
  .acct-card > summary:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
  .acct-card__title {
    display: block;
    font-size: 15px;
    font-weight: 600;
  }
  .acct-card__value {
    display: block;
    margin-top: 3px;
    font-size: 14px;
    color: #6e6e73;
    overflow-wrap: anywhere;
  }
  .acct-card__icon {
    position: absolute;
    top: 17px;
    right: 17px;
    display: grid;
    place-items: center;
    width: 27px;
    height: 27px;
    border-radius: 8px;
    background: var(--accent-soft);
    color: var(--accent);
  }
  .acct-card__body {
    padding: 4px 20px 20px;
    border-top: 1px solid #f0f1f2;
    margin-top: 2px;
  }
  .acct-card__body .field:first-child { margin-top: 16px; }
  .acct-card__body .btn { width: 100%; }
  /* 只读信息列表 */
  .acct-kv {
    display: grid;
    grid-template-columns: 88px 1fr;
    gap: 7px 12px;
    margin: 16px 0 0;
    font-size: 13.5px;
  }
  .acct-kv dt { color: var(--ink-soft); }
  .acct-kv dd { margin: 0; }
  .field__error {
    margin: 5px 0 0;
    font-size: 12.5px;
    color: var(--danger);
  }
  /* 管理员分类（账号 / 操作日志）里的表格面板 */
  .acct-panel {
    background: #fff;
    border: 1px solid #e6e6e8;
    border-radius: 14px;
    overflow: hidden;
  }
  .acct-panel__head {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
    padding: 13px 20px;
    border-bottom: 1px solid #f0f1f2;
  }
  .acct-panel__count {
    margin: 0;
    font-size: 13.5px;
    color: #6e6e73;
  }
  .acct-panel table { margin: 0; }
  .acct-panel th,
  .acct-panel td { border: 0; border-bottom: 1px solid
    #f0f1f2; }
  .acct-panel thead th { background: #f7f9f8; }
  .acct-panel tbody tr:last-child td { border-bottom: 0; }
  .acct-panel tbody tr:hover { background: #fafbfa; }
</style>
<div class="acct">
  {# ---------------- 左栏：身份 + 分类 ---------------- #}
  <aside class="acct-rail">
    {% if current_user.avatar %}
    <img class="acct-rail__avatar"
         src="{{ url_for('static', filename=current_user.ava
    tar) }}"
         alt="{{ current_user.shown_name }} 的头像">
    {% else %}
    <span class="acct-rail__avatar" aria-hidden="true">{{
    current_user.initial }}</span>
    {% endif %}
    <p class="acct-rail__name">{{ current_user.shown_name
    }}</p>
    <p class="acct-rail__sub">
      {{ role_labels.get(current_user.role,
    current_user.role) }}
      {% if current_user.email %} · {{ current_user.email
    }}{% endif %}
    </p>
    <nav class="acct-tabs" aria-label="个人设置分类">
      <a href="{{ url_for('auth.profile', tab='info') }}"
         {% if tab == 'info' %}class="is-active"
  aria-current="page"{% endif %}>个人信息</a>
    <a href="{{ url_for('auth.profile', tab='security')
  }}"
       {% if tab == 'security' %}class="is-active"
  aria-current="page"{% endif %}>登录与安全性</a>
    {% if current_user.is_admin %}
    <a href="{{ url_for('auth.profile', tab='users') }}"
       {% if tab == 'users' %}class="is-active"
  aria-current="page"{% endif %}>账号</a>
    <a href="{{ url_for('auth.profile', tab='logs') }}"
       {% if tab == 'logs' %}class="is-active"
  aria-current="page"{% endif %}>操作日志</a>
    {% endif %}
  </nav>
  <details class="acct-avatar-edit">
    <summary>更换头像</summary>
    <div class="acct-avatar-edit__body">
      <form method="post" action="/profile/avatar"
            enctype="multipart/form-data">
        {{ avatar_form.hidden_tag() }}
        <input type="hidden" name="tab" value="{{ tab }}">
        <label class="visually-hidden"
  for="avatar-file">选择头像图片</label>
        <input id="avatar-file" type="file" name="avatar"
               accept=".png,.jpg,.jpeg,.gif,.webp">
        <p class="acct-avatar-edit__file">PNG / JPG / GIF
  / WebP，不超过 4 MB</p>
        <button type="submit" class="btn
  btn--sm">上传头像</button>
      </form>
      {% if current_user.avatar %}
      <form method="post" action="/profile/avatar/delete"
            onsubmit="return confirm('确定移除当前头像吗？');">
        {{ avatar_form.hidden_tag() }}
        <input type="hidden" name="tab" value="{{ tab }}">
        <button type="submit" class="btn btn--sm
  btn--ghost">移除头像</button>
      </form>
      {% endif %}
    </div>
  </details>
</aside>
{# ---------------- 右栏：功能卡 ---------------- #}
<div class="acct-main">
  {% if tab == 'security' %}
  <h1 class="acct-title">登录与安全性</h1>
  <p class="acct-lead">管理与登录账号、账号安全有关的设置。</p>
  <div class="acct-grid">
    <details class="acct-card"
             {% if pwd_form.old_password.errors or
pwd_form.new_password.errors or pwd_form.confirm.errors
%}open{% endif %}>
    <summary>
      <span class="acct-card__title">密码</span>
      <span class="acct-card__value">修改登录密码</span>
      <span class="acct-card__icon">{{ icon("lock", 15)
}}</span>
    </summary>
    <div class="acct-card__body">
      <form method="post">
        {{ pwd_form.hidden_tag() }}
        <input type="hidden" name="action"
value="password">
        <input type="hidden" name="tab"
value="security">
        <div class="field">
          <label class="field__label" for="{{
pwd_form.old_password.id }}">当前密码</label>
          {{ pwd_form.old_password(id=pwd_form.old_passw
ord.id,

autocomplete="current-password") }}
          {% for e in pwd_form.old_password.errors %}
          <p class="field__error">{{ e }}</p>
          {% endfor %}
        </div>
        <div class="field">
          <label class="field__label" for="{{
pwd_form.new_password.id }}">新密码</label>
          {{ pwd_form.new_password(id=pwd_form.new_passw
ord.id,

autocomplete="new-password") }}
          {% for e in pwd_form.new_password.errors %}
          <p class="field__error">{{ e }}</p>
          {% endfor %}
        </div>
        <div class="field">
          <label class="field__label" for="{{
pwd_form.confirm.id }}">确认新密码</label>
          {{ pwd_form.confirm(id=pwd_form.confirm.id,
                              autocomplete="new-password
") }}
          {% for e in pwd_form.confirm.errors %}
          <p class="field__error">{{ e }}</p>
          {% endfor %}
        </div>
        <p class="actions">{{ pwd_form.submit_pwd(class_
="btn") }}</p>
      </form>
    </div>
  </details>
  <details class="acct-card">
    <summary>
      <span class="acct-card__title">登录账号</span>
      <span class="acct-card__value">{{
current_user.username }}</span>
      <span class="acct-card__icon">{{ icon("user", 15)
}}</span>
    </summary>
    <div class="acct-card__body">
      <dl class="acct-kv">
        <dt>登录账号</dt><dd class="mono">{{
current_user.username }}</dd>
        <dt>职务</dt>
        <dd>{{ role_labels.get(current_user.role,
current_user.role) }}</dd>
        <dt>状态</dt>
        <dd>
          {% if current_user.active %}
          <span class="tag tag--ok">启用</span>
          {% else %}
          <span class="tag tag--warn">停用</span>
          {% endif %}
        </dd>
        <dt>创建时间</dt>
        <dd class="mono">
          {{ current_user.created_at.strftime("%Y-%m-%d
%H:%M")
             if current_user.created_at else "—" }}
        </dd>
      </dl>
    </div>
  </details>
</div>
{% elif tab == 'users' %}
<h1 class="acct-title">账号</h1>
<p class="acct-lead">管理本系统的账号、职务与启用状态。新注册的账号默认为「操作员」。</p
>
{% include "auth/_users.html" %}
{% elif tab == 'logs' %}
<h1 class="acct-title">操作日志</h1>
<p class="acct-lead">记录各账号在本系统中的关键操作。</p>
{% include "auth/_logs.html" %}
{% else %}
<h1 class="acct-title">个人信息</h1>
<p class="acct-lead">管理你的昵称、性别以及可以联系到你的邮箱与手机号。</p>
<form method="post">
  {{ info_form.hidden_tag() }}
  <input type="hidden" name="action" value="info">
  <input type="hidden" name="tab" value="info">
  <div class="acct-grid">
    <details class="acct-card" {% if
info_form.display_name.errors %}open{% endif %}>
      <summary>
        <span class="acct-card__title">昵称</span>
        <span class="acct-card__value">
          {{ current_user.display_name or "未设置" }}
        </span>
        <span class="acct-card__icon">{{ icon("user",
15) }}</span>
      </summary>
      <div class="acct-card__body">
        <div class="field">
          <label class="field__label" for="{{
info_form.display_name.id }}">
            昵称
          </label>
          {{ info_form.display_name(id=info_form.display
_name.id,

placeholder="留空则显示账户名称") }}
          {% for e in info_form.display_name.errors %}
          <p class="field__error">{{ e }}</p>
          {% endfor %}
        </div>
        <p class="actions">{{ info_form.submit_info(clas
s_="btn") }}</p>
      </div>
    </details>
    <details class="acct-card" {% if
info_form.gender.errors %}open{% endif %}>
      <summary>
        <span class="acct-card__title">性别</span>
        <span class="acct-card__value">{{
current_user.gender or "未设置" }}</span>
        <span class="acct-card__icon">{{ icon("user",
15) }}</span>
      </summary>
      <div class="acct-card__body">
        <div class="field">
          <label class="field__label" for="{{
info_form.gender.id }}">性别</label>
          {{ info_form.gender(id=info_form.gender.id) }}
          {% for e in info_form.gender.errors %}
          <p class="field__error">{{ e }}</p>
          {% endfor %}
        </div>
        <p class="actions">{{ info_form.submit_info(clas
s_="btn") }}</p>
```

```
        </div>
      </details>
      <details class="acct-card" {% if
  info_form.email.errors %}open{% endif %}>
        <summary>
          <span class="acct-card__title">邮箱</span>
          <span class="acct-card__value">{{
  current_user.email or "未设置" }}</span>
          <span class="acct-card__icon">{{ icon("mail",
  15) }}</span>
        </summary>
        <div class="acct-card__body">
          <div class="field">
            <label class="field__label" for="{{
  info_form.email.id }}">邮箱</label>
            {{ info_form.email(id=info_form.email.id) }}
            {% for e in info_form.email.errors %}
            <p class="field__error">{{ e }}</p>
            {% endfor %}
          </div>
          <p class="actions">{{ info_form.submit_info(clas
  s_="btn") }}</p>
        </div>
      </details>
      <details class="acct-card" {% if
  info_form.phone.errors %}open{% endif %}>
        <summary>
          <span class="acct-card__title">手机号</span>
          <span class="acct-card__value">{{
  current_user.phone or "未设置" }}</span>
          <span class="acct-card__icon">{{ icon("phone",
  15) }}</span>
        </summary>
        <div class="acct-card__body">
          <div class="field">
            <label class="field__label" for="{{
  info_form.phone.id }}">手机号</label>
            {{ info_form.phone(id=info_form.phone.id) }}
            {% for e in info_form.phone.errors %}
            <p class="field__error">{{ e }}</p>
            {% endfor %}
          </div>
          <p class="actions">{{ info_form.submit_info(clas
  s_="btn") }}</p>
        </div>
      </details>
    </div>
  </form>
  {% endif %}
</div>
```

```
</div>
{% endblock %}
{% block scripts %}
<script>
// 同时只展开一张卡，避免点开多张后页面被撑得很长
(function () {
  var cards = document.querySelectorAll('.acct-card');
  Array.prototype.forEach.call(cards, function (card) {
    card.addEventListener('toggle', function () {
      if (!card.open) return;
      Array.prototype.forEach.call(cards, function (other) {
        if (other !== card && other.open) other.open =
    false;
      });
    });
  });
})();
</script>
{% endblock %}
# ==== app/templates/auth/_users.html ====
{# 管理员的「账号」分类。由 auth/profile.html 内联引入，不单独成页。 #}
<div class="acct-panel">
  <div class="acct-panel__head">
    <p class="acct-panel__count">共 {{ users|length }}
    个账号</p>
    <a class="btn btn--sm" href="{{ url_for('auth.add_user')
    }}">新增账号</a>
  </div>
  <table>
    <thead>
      <tr>
        <th scope="col">ID</th>
        <th scope="col">账户名称</th>
        <th scope="col">昵称</th>
        <th scope="col">职务</th>
        <th scope="col">状态</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for u in users %}
      <tr>
        <td class="num">{{ u.id }}</td>
        <td class="mono">
          {{ u.username }}
          {% if u.id == current_user.id %}<span
    class="muted">（当前登录）</span>{% endif %}
        </td>
        <td style="text-align:left;">
          {% if u.display_name %}{{ u.display_name }}
          {% else %}<span class="muted">—</span>{% endif %}
        </td>
        <td>
          <span class="tag {{ 'tag--ok' if u.role ==
    'admin' else 'tag--muted' }}">
            {{ role_labels.get(u.role, u.role) }}
          </span>
        </td>
        <td>
          {% if u.active %}<span class="tag
    tag--ok">启用</span>
          {% else %}<span class="tag tag--muted">停用</span>{%
    endif %}
        </td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="{{
    url_for('auth.edit_user', user_id=u.id) }}">编辑</a>
            <form method="post"
                  action="{{ url_for('auth.delete_user',
    user_id=u.id) }}"
                  onsubmit="return confirm('确定删除账号「{{
    u.username }}」吗？\n该操作无法恢复。');">
              {{ form.hidden_tag() }}
              <button type="submit" class="link-btn
    link-btn--danger"
                      {% if u.id == current_user.id
    %}disabled
                      title="不能删除当前登录的账号"{% endif
    %}>删除</button>
            </form>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
# ==== app/templates/auth/_logs.html ====
{# 管理员的「操作日志」分类。由 auth/profile.html 内联引入，不单独成页。 #}
<div class="acct-panel">
  <div class="acct-panel__head">
    <p class="acct-panel__count">
      共 {{ total }} 条记录，当前第 {{ page }} / {{ pages }} 页
    </p>
    {% if pages > 1 %}
    <nav class="row-actions" aria-label="日志分页">
      {% if page > 1 %}
      <a class="link-btn"
         href="{{ url_for('auth.profile', tab='logs',
    page=page - 1) }}">上一页</a>
      {% endif %}
      {% if page < pages %}
      <a class="link-btn"
         href="{{ url_for('auth.profile', tab='logs',
    page=page + 1) }}">下一页</a>
      {% endif %}
    </nav>
    {% endif %}
  </div>
  {% if not records %}
  <p class="muted" style="padding:18px 20px;">暂无操作记录。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">时间</th>
        <th scope="col">操作人</th>
        <th scope="col">操作</th>
        <th scope="col">对象</th>
        <th scope="col">说明</th>
      </tr>
    </thead>
    <tbody>
      {% for r in records %}
      <tr>
        <td class="mono nowrap">{{ r.created_at.strftime("%Y
    -%m-%d %H:%M:%S") if r.created_at else "—" }}</td>
        <td>{{ users_map.get(r.user_id, "—") }}</td>
        <td style="text-align:left;">{{ r.action }}</td>
        <td>
          {% if r.target_type %}{{ r.target_type }}{% if
    r.target_id %}#{{ r.target_id }}{% endif %}
          {% else %}—{% endif %}
        </td>
        <td style="text-align:left;">{{ r.detail or "—"
    }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
# ==== app/templates/project/index.html ====
{% extends "base.html" %}
{% block title %}项目与矿山档案{% endblock %}
{% block content %}
<div class="card">
  <h2>项目与矿山档案</h2>
  <p class="actions" style="margin-bottom:12px;">
  <a class="btn btn--sm" href="/project/add">新建项目</a>
</p>
{% if not projects %}
<p class="muted">
  暂无项目。点击上方「新建项目」开始建立评价项目，
  或运行 <span class="mono">python seed_demo.py</span>
  载入论文案例。
</p>
{% else %}
<table>
  <caption class="muted">共 {{ projects|length }}
  个项目</caption>
  <thead>
    <tr>
      <th scope="col">项目名称</th>
      <th scope="col">编号</th>
      <th scope="col">状态</th>
      <th scope="col">图斑数</th>
      <th scope="col">操作</th>
    </tr>
  </thead>
  <tbody>
    {% for p in projects %}
    <tr>
      <td style="text-align:left;">{{ p.name }}</td>
      <td class="mono">{{ p.code or "—" }}</td>
      <td><span class="tag tag--muted">{{ p.status
  }}</span></td>
      <td class="num">
        {% if p.patches.count() %}
          {{ p.patches.count() }}
        {% else %}
          <span class="tag tag--warn">0</span>
        {% endif %}
      </td>
      <td class="nowrap">
        <span class="row-actions">
          <a class="link-btn" href="/project/{{ p.id
  }}/patches">{% if p.patches.count() %}矿山图斑{% else %}+
  新建图斑{% endif %}</a>
          <a class="link-btn" href="/project/{{ p.id
  }}/edit">编辑</a>
          <form method="post" action="/project/{{ p.id
  }}/delete"
                onsubmit="return confirm('确定删除项目「{{
  p.name }}」吗？\n该项目下的图斑、微地质单元与评价数据将一并删除，且无法恢复。');">
            {{ form.hidden_tag() }}
            <button type="submit" class="link-btn
  link-btn--danger">删除</button>
          </form>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/project/edit.html ====
{% extends "base.html" %}
{% block title %}{{ "编辑项目" if project else "新建项目" }}{%
    endblock %}
{% block content %}
<div class="card" style="max-width:640px;">
  <h2>{{ "编辑项目" if project else "新建项目" }}</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="name">项目名称</label>
      <input id="name" type="text" name="name"
    style="width:100%;" required
             value="{{ project.name if project else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="code">项目编号</label>
      <input id="code" type="text" name="code"
    style="width:240px;"
             value="{{ project.code or '' if project else
    '' }}">
    </div>
    {% if project %}
    <div class="field">
      <label class="field__label" for="status">状态</label>
      <select id="status" name="status">
        {% for s in ["进行中", "已完成", "已归档"] %}
        <option value="{{ s }}" {% if project.status == s
    %}selected{% endif %}>{{ s }}</option>
        {% endfor %}
      </select>
    </div>
    {% endif %}
    <div class="field">
      <label class="field__label" for="description">项目说明</la
    bel>
      <textarea id="description" name="description">{{
    project.description or '' if project else ''
    }}</textarea>
    </div>
    <p class="actions" style="margin-top:18px;">
```

```
      <button type="submit" class="btn">保存</button>
      <a class="btn btn--ghost" href="/project/">返回</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/project/patches.html ====
{% extends "base.html" %}
{% block title %}矿山图斑 · {{ project.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>矿山图斑 · {{ project.name }}</h2>
  <p class="actions" style="margin-bottom:12px;">
    <a class="btn btn--sm" href="/project/{{ project.id
    }}/patch/add">新建图斑</a>
    <a class="btn btn--sm btn--ghost"
    href="/project/">返回项目列表</a>
  </p>
  {% if not project.patches.count() %}
  <p class="muted">该项目下暂无图斑。点击上方「新建图斑」建立第一个矿山图斑。</p>
  {% else %}
  <table>
    <caption class="muted">共 {{ project.patches.count() }}
    个图斑</caption>
    <thead>
      <tr>
        <th scope="col">图斑名称</th>
        <th scope="col">行政区</th>
        <th scope="col">面积（公顷）</th>
        <th scope="col">开采矿种</th>
        <th scope="col">微地质单元</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in project.patches %}
      <tr>
        <td style="text-align:left;">{{ p.name }}</td>
        <td>{{ p.admin_region or "—" }}</td>
        <td class="num">{{ p.area_ha if p.area_ha is not
    none else "—" }}</td>
        <td>{{ p.mineral_type or "—" }}</td>
        <td class="num">{{ p.units.count() }}</td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/unit/{{ p.id
    }}">微地质单元</a>
            <a class="link-btn" href="/constraint/{{ p.id
    }}">限制性判定</a>
            <a class="link-btn" href="/project/patch/{{
```

```
    p.id }}/edit">编辑</a>
            <form method="post" action="/project/patch/{{
    p.id }}/delete"
                  onsubmit="return confirm('确定删除图斑「{{
    p.name }}」吗？\n该图斑的微地质单元、指标数据与评价结果将一并删除，且无法恢复。');">
              {{ form.hidden_tag() }}
              <button type="submit" class="link-btn
    link-btn--danger">删除</button>
            </form>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/project/patch_edit.html ====
{% extends "base.html" %}
{% block title %}{{ "编辑图斑" if patch else "新建图斑" }}{%
    endblock %}
{% block content %}
<div class="card" style="max-width:720px;">
  <h2>{{ "编辑图斑" if patch else "新建图斑" }} · {{ project.name
    }}</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="patch-name">图斑名称</lab
    el>
      <input id="patch-name" type="text" name="name"
    style="width:100%;" required
             value="{{ patch.name if patch else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="admin-region">所在行政区</
    label>
      <input id="admin-region" type="text"
    name="admin_region" style="width:100%;"
             value="{{ patch.admin_region or '' if patch
    else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="area-ha">图斑面积（公顷）</la
    bel>
      <input id="area-ha" type="number" step="any"
    name="area_ha"
             value="{{ patch.area_ha if patch and
    patch.area_ha is not none else '' }}">
```

```
</div>
<div class="field">
  <label class="field__label" for="mineral-type">开采矿种</l
abel>
  <input id="mineral-type" type="text"
name="mineral_type" style="width:240px;"
         value="{{ patch.mineral_type or '' if patch
else '' }}">
</div>
<div class="field">
  <label class="field__label" for="mining-method">开采方式</
label>
  <input id="mining-method" type="text"
name="mining_method" style="width:240px;"
         value="{{ patch.mining_method or '' if patch
else '' }}">
</div>
<div class="field">
  <label class="field__label" for="stop-years">停产年限</lab
el>
  <input id="stop-years" type="number" step="any"
name="stop_years"
         value="{{ patch.stop_years if patch and
patch.stop_years is not none else '' }}">
</div>
<div class="field">
  <label class="checkbox" for="responsible-missing">
    <input id="responsible-missing" type="checkbox"
name="responsible_entity_missing" value="1"
           {% if not patch or patch.responsible_entity_m
issing %}checked{% endif %}>
    <span>责任主体缺失</span>
  </label>
</div>
<h3>参照生态系统（指标标准化基准）</h3>
<p class="muted">
  论文 4.2.2 节取值：植被覆盖度 75%、Shannon-Wiener 多样性指数 0.7。
  该基准与图斑绑定管理，可按不同参照生态系统分别设定。
</p>
<div class="field">
  <label class="field__label" for="coverage-benchmark">植
被覆盖度基准（%）</label>
  <input id="coverage-benchmark" type="number"
step="any" name="coverage_benchmark"
         value="{{ reference.coverage_benchmark if
reference else 75 }}">
</div>
<div class="field">
  <label class="field__label" for="shannon-benchmark">Sh
annon-Wiener 多样性指数基准</label>
```

```
      <input id="shannon-benchmark" type="number"
    step="any" name="shannon_benchmark"
             value="{{ reference.shannon_benchmark if
    reference else 0.7 }}">
    </div>
    <div class="field">
      <label class="field__label" for="dominant-species">建群种
    </label>
      <input id="dominant-species" type="text"
    name="dominant_species" style="width:100%;"
             value="{{ reference.dominant_species or '' if
    reference else '' }}">
    </div>
    <p class="actions" style="margin-top:18px;">
      <button type="submit" class="btn">保存</button>
      <a class="btn btn--ghost" href="/project/{{
    project.id }}/patches">返回</a>
    </p>
  </form>
</div>
{% if patch %}
<div class="card" style="max-width:720px;">
  <h2>遥感影像挂接</h2>
  <p class="muted">
    挂接遥感底图或无人机正射影像，用于辅助微地质单元划分与植被覆盖度判读。
    支持 PNG、JPG、GIF、WebP、BMP、TIFF，单个文件不超过 32 MB。
  </p>
  {% if patch.image_path %}
  <figure style="margin:0 0 16px;">
    <img src="{{ url_for('static', filename=patch.image_path
    ) }}"
         alt="图斑「{{ patch.name }}」已挂接的遥感影像"
         style="max-width:100%;border:1px solid
    var(--line);border-radius:6px;">
    <figcaption class="muted" style="margin-top:6px;">
      {{ patch.image_note or "遥感影像" }}
      {% if patch.image_uploaded_at %}
      · 上传于 {{ patch.image_uploaded_at.strftime("%Y-%m-%d
    %H:%M") }}
      {% endif %}
    </figcaption>
  </figure>
  <form method="post" action="/project/patch/{{ patch.id
    }}/image/delete"
        onsubmit="return confirm('确定移除该遥感影像吗？');">
    {{ form.hidden_tag() }}
    <button type="submit" class="btn btn--sm
    btn--danger">移除影像</button>
  </form>
  <hr style="margin:16px 0;border:none;border-top:1px solid
```

```
    var(--line);">
  {% endif %}
  <form method="post" action="/project/patch/{{ patch.id
    }}/image"
        enctype="multipart/form-data">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="image-file">
        {{ "替换影像文件" if patch.image_path else "选择影像文件" }}
      </label>
      <input id="image-file" type="file" name="image"
             accept=".png,.jpg,.jpeg,.gif,.webp,.bmp,.tif,.t
    iff" required>
    </div>
    <div class="field">
      <label class="field__label" for="image-note">影像说明（如来源、
    时相）</label>
      <input id="image-note" type="text" name="image_note"
    style="width:100%;"
             placeholder="如：2025年6月无人机正射影像"
             value="{{ patch.image_note or '' }}">
    </div>
    <p class="actions">
      <button type="submit" class="btn">{{ "替换影像" if
    patch.image_path else "上传并挂接" }}</button>
    </p>
  </form>
</div>
{% endif %}
{% endblock %}
# ==== app/templates/unit/list.html ====
{% extends "base.html" %}
{% from "_macros.html" import patch_guide %}
{% block title %}微地质单元{% endblock %}
{% block content %}
<div class="card">
  <h2>微地质单元</h2>
  {{ patch_guide(projects_without_patch) }}
  {% if not rows %}
  <p class="muted">
    暂无矿山图斑。请先到「项目与矿山档案」建立图斑。
  </p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">矿山图斑</th>
        <th scope="col">已划分单元</th>
        <th scope="col">占比合计</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for r in rows %}
      <tr>
        <td>{{ r.patch.project.name if r.patch.project else
    "—" }}</td>
        <td style="text-align:left;">{{ r.patch.name }}</td>
        <td class="num">{{ r.count }} 个</td>
        <td class="num">
          {% if r.count %}
            {{ "%.2f%%"|format(r.ratio_sum * 100) }}
            {% if r.ratio_sum < 0.999 or r.ratio_sum >
    1.001 %}
            <span class="warn">（不等于 100%）</span>
            {% endif %}
          {% else %}—{% endif %}
        </td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/unit/{{ r.patch.id
    }}">划分与维护</a>
            <a class="link-btn" href="/indicator/">录入指标数据</a
    >
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/unit/index.html ====
{% extends "base.html" %}
{% block title %}微地质单元 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>微地质单元 · {{ patch.name }}</h2>
  {% if units %}
  <table>
    <caption class="muted">
      共 {{ units|length }} 个单元；
      面积合计 <span class="num">{{ "%.4f"|format(total_area)
    }}</span>；
      占比合计 <span class="num">{{ "%.2f%%"|format(ratio_sum *
    100) }}</span>
      {% if ratio_sum and (ratio_sum < 0.999 or ratio_sum >
    1.001) %}
    <span class="warn">（占比合计不等于 100%，请检查面积录入）</span>
    {% endif %}
  </caption>
  <thead>
    <tr>
      <th scope="col">微地质单元</th>
      <th scope="col">面积</th>
      <th scope="col">面积占比</th>
      <th scope="col">说明</th>
      <th scope="col">操作</th>
    </tr>
  </thead>
  <tbody>
    {% for u in units %}
    <tr>
      <td>{{ u.unit_type }}</td>
      <td class="num">{{ u.area if u.area is not none
  else "—" }}</td>
      <td class="num">
        {% if u.ratio is not none %}{{
  "%.2f%%"|format(u.ratio * 100) }}{% else %}—{% endif %}
      </td>
      <td style="text-align:left;">{{ u.description or
  "—" }}</td>
      <td class="nowrap">
        <span class="row-actions">
          <a class="link-btn" href="/unit/edit/{{ u.id
  }}">编辑</a>
          <form method="post" action="/unit/delete/{{
  u.id }}"
                onsubmit="return confirm('确定删除微地质单元「{{
  u.unit_type }}」吗？\n该单元已录入的指标数据将一并删除，且无法恢复。');">
            {{ form.hidden_tag() }}
            <button type="submit" class="link-btn
  link-btn--danger">删除</button>
          </form>
        </span>
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% else %}
<p class="muted">尚未划分微地质单元。请在下方添加。</p>
{% endif %}
<h3>添加微地质单元</h3>
<form method="post" action="/unit/{{ patch.id }}/add">
  {{ form.hidden_tag() }}
  <div class="actions">
    <label class="field__label" for="add-unit-type"
    style="margin:0;">类型</label>
      <select id="add-unit-type" name="unit_type">
        {% for t in unit_types %}<option value="{{ t }}">{{
    t }}</option>{% endfor %}
      </select>
      <label class="field__label" for="add-unit-area"
    style="margin:0;">面积</label>
      <input id="add-unit-area" type="number" step="any"
    name="area">
      <label class="field__label" for="add-unit-desc"
    style="margin:0;">说明</label>
      <input id="add-unit-desc" type="text"
    name="description" style="width:220px;">
      <button type="submit" class="btn btn--sm">添加</button>
    </div>
  </form>
  <p class="actions" style="margin-top:20px;">
    <a class="btn btn--sm btn--ghost" href="/project/{{
    patch.project_id }}/patches">返回图斑列表</a>
    <a class="btn btn--sm btn--ghost"
    href="/indicator/">去录入指标数据</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/unit/edit.html ====
{% extends "base.html" %}
{% block title %}编辑微地质单元{% endblock %}
{% block content %}
<div class="card">
  <h2>编辑微地质单元 · {{ patch.name }}</h2>
  <p class="muted">修改面积后，系统会自动重算本图斑各单元的面积占比。</p>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="unit-type">微地质单元类型</l
    abel>
      <select id="unit-type" name="unit_type">
        {% for t in unit_types %}
        <option value="{{ t }}" {% if unit.unit_type == t
    %}selected{% endif %}>{{ t }}</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="unit-area">面积</label>
      <input id="unit-area" type="number" step="any"
    name="area"
             value="{{ unit.area if unit.area is not none
    else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="unit-desc">说明</label>
      <input id="unit-desc" type="text" name="description"
    style="width:420px;"
             value="{{ unit.description or '' }}">
    </div>
    <p class="actions" style="margin-top:16px;">
      <button type="submit" class="btn">保存</button>
      <a class="btn btn--ghost" href="/unit/{{
    unit.patch_id }}">返回</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/standard/index.html ====
{% extends "base.html" %}
{% block title %}政策与标准知识库{% endblock %}
{% block content %}
<div class="card">
  <h2>政策与标准知识库</h2>  <p class="actions"
    style="margin-bottom:14px;">
    <a class="btn btn--sm" href="/standard/add">添加标准</a>
    <a class="btn btn--sm btn--ghost"
    href="/standard/compare">省际口径比对</a>
  </p>
  <h3>地方标准</h3>
  {% if local %}
  <table>
    <thead>
      <tr>
        <th scope="col">省份</th>
        <th scope="col">名称</th>
        <th scope="col">编号</th>
        <th scope="col">规则数</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for s in local %}
      <tr>
        <td>{{ s.province }}</td>
        <td style="text-align:left;">{{ s.name }}</td>
        <td class="mono">{{ s.doc_number or "—" }}</td>
        <td class="num">{{ s.rules|length }}</td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/standard/{{ s.id
    }}/rules">规则</a>
            <a class="link-btn" href="/standard/{{ s.id
    }}/edit">编辑</a>
```

```
          <form method="post" action="/standard/{{ s.id }}/delete"
                onsubmit="return confirm('确定删除「{{ s.province }} {{ s.name }}」吗?\n该标准下的 {{ s.rules|length }} 条规则将一并删除，且无法恢复。');">
            {{ form.hidden_tag() }}
            <button type="submit" class="link-btn link-btn--danger">删除</button>
          </form>
        </span>
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% else %}
<p class="muted">暂无地方标准。运行 <span class="mono">python seed_demo.py</span> 可载入论文表 1 的 11 项标准与政策文件。</p>
{% endif %}
<h3>政策文件</h3>
{% if policy %}
<table>
  <thead>
    <tr>
      <th scope="col">省份</th>
      <th scope="col">名称</th>
      <th scope="col">文号</th>
      <th scope="col">规则数</th>
      <th scope="col">操作</th>
    </tr>
  </thead>
  <tbody>
    {% for s in policy %}
    <tr>
      <td>{{ s.province }}</td>
      <td style="text-align:left;">{{ s.name }}</td>
      <td class="mono">{{ s.doc_number or "—" }}</td>
      <td class="num">{{ s.rules|length }}</td>
      <td class="nowrap">
        <span class="row-actions">
          <a class="link-btn" href="/standard/{{ s.id }}/rules">规则</a>
          <a class="link-btn" href="/standard/{{ s.id }}/edit">编辑</a>
          <form method="post" action="/standard/{{ s.id }}/delete"
                onsubmit="return confirm('确定删除「{{ s.province }} {{ s.name }}」吗?\n该文件下的 {{ s.rules|length }} 条规则将一并删除，且无法恢复。');">
            {{ form.hidden_tag() }}
```

```
              <button type="submit" class="link-btn
    link-btn--danger">删除</button>
            </form>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% else %}
  <p class="muted">暂无政策文件。</p>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/standard/edit.html ====
{% extends "base.html" %}
{% block title %}{{ "编辑标准" if standard else "添加标准" }}{%
    endblock %}
{% block content %}
<div class="card" style="max-width:800px;">
  <h2>{{ "编辑标准" if standard else "添加标准" }}</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="province">省份</label>
      <input id="province" type="text" name="province"
    style="width:160px;" required
             value="{{ standard.province if standard else
    '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="std-name">名称</label>
      <input id="std-name" type="text" name="name"
    style="width:100%;" required
             value="{{ standard.name if standard else ''
    }}">
    </div>
    <div class="field">
      <label class="field__label" for="doc-number">标准编号 /
    文号</label>
      <input id="doc-number" type="text" name="doc_number"
    style="width:320px;"
             value="{{ standard.doc_number or '' if
    standard else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="category">类别</label>
      <select id="category" name="category">
        {% for c in ["地方标准", "政策文件"] %}
        <option value="{{ c }}" {% if standard and
    standard.category == c %}selected{% endif %}>{{ c
    }}</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="publish-date">发布时间</l
    abel>
      <input id="publish-date" type="text"
    name="publish_date" style="width:200px;"
             placeholder="如 2023-03-30"
             value="{{ standard.publish_date or '' if
    standard else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="implement-date">实施时间<
    /label>
      <input id="implement-date" type="text"
    name="implement_date" style="width:200px;"
             placeholder="如 2023-04-30（未明确可留空）"
             value="{{ standard.implement_date or '' if
    standard else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="summary">特点 /
    判定标准</label>
      <textarea id="summary" name="summary"
    style="min-height:96px;">{{ standard.summary or '' if
    standard else '' }}</textarea>
    </div>
    <p class="actions" style="margin-top:18px;">
      <button type="submit" class="btn">保存</button>
      {% if standard %}
      <form method="post" action="/standard/{{ standard.id
    }}/delete" style="display:inline;"
            onsubmit="return confirm('确定删除「{{
    standard.province }} {{ standard.name
    }}」吗？\n该标准下的规则将一并删除，且无法恢复。');">
        {{ form.hidden_tag() }}
        <button type="submit" class="btn
    btn--danger">删除该标准</button>
      </form>
      {% endif %}
      <a class="btn btn--ghost" href="/standard/">返回</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/standard/rules.html ====
{% extends "base.html" %}
{% block title %}结构化规则 · {{ standard.province }}{% endblock
    %}
{% block content %}
<div class="card">
  <h2>结构化规则 · {{ standard.province }} {{ standard.name
    }}</h2>
  {% if rules %}
  <table>
    <caption class="muted">共 {{ rules|length }}
    条规则</caption>
    <thead>
      <tr>
        <th scope="col">判定项</th>
        <th scope="col">条件</th>
        <th scope="col">条件类型</th>
        <th scope="col">适用分区</th>
        <th scope="col">备注</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for r in rules %}
      <tr>
        <td style="text-align:left;">{{ r.item }}</td>
        <td class="mono nowrap">{{ r.operator }} {{
    r.threshold or "" }}</td>
        <td>{{ r.condition_type }}</td>
        <td>{{ r.region or "—" }}</td>
        <td style="text-align:left;">{{ r.note or "—"
    }}</td>
        <td>
          <form method="post" action="/standard/rule/{{
    r.id }}/delete"
                onsubmit="return confirm('确定删除规则「{{ r.item
    }} {{ r.operator }} {{ r.threshold or '' }}」吗?');">
            {{ form.hidden_tag() }}
            <button type="submit" class="btn btn--sm
    btn--danger">删除</button>
          </form>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% else %}
  <p class="muted">该标准暂无规则。请在下方添加第一条规则。</p>
  {% endif %}
  <h3>添加规则</h3>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="actions">
      <div>
        <label class="field__label" for="r-item">判定项</label>
        <input id="r-item" type="text" name="item"
    placeholder="如：植被覆盖度" required>
      </div>
      <div>
        <label class="field__label" for="r-op">比较符</label>
        <select id="r-op" name="operator">
          {% for o in [">=", ">", "<=", "<", "=="]
    %}<option value="{{ o }}">{{ o }}</option>{% endfor %}
        </select>
      </div>
      <div>
        <label class="field__label" for="r-thr">阈值</label>
        <input id="r-thr" type="text" name="threshold"
    placeholder="如：50" style="width:110px;">
      </div>
      <div>
        <label class="field__label" for="r-cond">条件类型</label
    >
        <select id="r-cond" name="condition_type">
          <option value="必要条件">必要条件</option>
          <option value="充分条件">充分条件</option>
        </select>
      </div>
      <div>
        <label class="field__label" for="r-region">适用分区</lab
    el>
        <input id="r-region" type="text" name="region"
    placeholder="如宁夏分区" style="width:130px;">
      </div>
      <div>
        <label class="field__label" for="r-note">备注</label>
        <input id="r-note" type="text" name="note"
    style="width:220px;">
      </div>
      <div>
        <button type="submit" class="btn">添加规则</button>
      </div>
    </div>
  </form>
  <p class="actions" style="margin-top:20px;">
    <a class="btn btn--sm btn--ghost"
    href="/standard/">返回标准列表</a>
    <a class="btn btn--sm btn--ghost"
    href="/standard/compare">省际口径比对</a>
  </p>
</div>
{% endblock %}
```

```
# ==== app/templates/standard/compare.html ====
{% extends "base.html" %}
{% block title %}省际口径比对{% endblock %}
{% block content %}
<div class="card">
  <h2>省际口径比对</h2>
  {% if not groups %}
  <p class="muted">
    暂无规则数据。请先在「标准知识库」中为标准录入规则，
    或运行 <span class="mono">python seed_demo.py</span> 载入论文表
    1 数据。
  </p>
  {% else %}
  {% for item, rs in groups.items() %}
  <h3>{{ item }}</h3>
  <table>
    <thead>
      <tr>
        <th scope="col">省份</th>
        <th scope="col">标准 / 文件</th>
        <th scope="col">条件</th>
        <th scope="col">条件类型</th>
        <th scope="col">备注</th>
      </tr>
    </thead>
    <tbody>
      {% for r in rs %}
      <tr>
        <td>{{ r.standard.province }}</td>
        <td style="text-align:left;">{{ r.standard.name
    }}</td>
        <td class="mono nowrap">{{ r.operator }} {{
    r.threshold or "" }}</td>
        <td>{{ r.condition_type }}</td>
        <td style="text-align:left;">{{ r.note or "—"
    }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endfor %}
  {% endif %}
  <p class="actions" style="margin-top:16px;">
    <a class="btn btn--ghost" href="/standard/">返回标准列表</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/constraint/index.html ====
{% extends "base.html" %}
{% from "_macros.html" import patch_guide %}
{% block title %}限制性指标判定{% endblock %}
{% block content %}
<div class="card">
  <h2>限制性指标判定</h2>
  {{ patch_guide(projects_without_patch) }}
  {% if not patches %}
  <p class="muted">暂无矿山图斑。请先在「项目与矿山档案」中建立图斑。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">矿山图斑</th>
        <th scope="col">判定结果</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in patches %}
      {% set r = latest.get(p.id) %}
      <tr>
        <td>{{ p.project.name if p.project else "—" }}</td>
        <td style="text-align:left;">{{ p.name }}</td>
        <td style="text-align:left;">
          {% if r %}
            {% if r.passed %}
            <span class="tag tag--ok">通过</span>
            {% else %}
            <span class="tag tag--warn">不通过</span>
            {% endif %}
            <span class="muted">{{ r.reason }}</span>
          {% else %}
          <span class="tag tag--muted">未判定</span>
          {% endif %}
        </td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/constraint/{{ p.id
    }}">执行判定</a>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/constraint/check.html ====
{% extends "base.html" %}
{% block title %}限制性条件判定 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:760px;">
  <h2>限制性条件判定 · {{ patch.name }}</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    {% for field, name, val in field_states %}
    <fieldset style="border:1px solid
    var(--line);border-radius:6px;padding:10px
    14px;margin:0 0 10px;">
      <legend style="padding:0 6px;font-weight:600;">{{
    name }}</legend>
      <div class="actions">
        <label class="checkbox">
          <input type="radio" name="{{ field }}" value="1"
                 {% if val is none or val %}checked{% endif
    %}>
          <span>通过</span>
        </label>
        <label class="checkbox">
          <input type="radio" name="{{ field }}" value="0"
                 {% if val is not none and not val
    %}checked{% endif %}>
          <span>不通过</span>
        </label>
      </div>
    </fieldset>
    {% endfor %}
    <p class="actions" style="margin-top:16px;">
      <button type="submit" class="btn">保存判定</button>
      <a class="btn btn--ghost" href="/constraint/">返回</a>
    </p>
  </form>
  {% if record %}
  <div style="margin-top:18px;padding:14px;border-radius:6px
    ;border-left:4px solid;
              {% if record.passed %}background:var(--ok-soft
    );border-left-color:var(--ok);
              {% else %}background:var(--danger-soft);border
    -left-color:var(--danger);{% endif %}">
    <p style="margin:0 0 4px;font-weight:600;">
      判定结论：
      {% if record.passed %}
      <span class="ok">通过，可进入定量评价</span>
      {% else %}
      <span class="warn">不通过 —— 建议生态重建或人工辅助</span>
      {% endif %}
    </p>
    <p class="muted" style="margin:0;">{{ record.reason
    }}</p>
  </div>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/ahp/index.html ====
{% extends "base.html" %}
{% block title %}AHP 权重计算{% endblock %}
{% block content %}
<div class="card" style="max-width:720px;">
  <h2>AHP 权重计算 · 选择指标层级</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="{{ form.unit_type.id
    }}">微地质单元类型</label>
      {{ form.unit_type(id=form.unit_type.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.level.id
    }}">指标层级</label>
      {{ form.level(id=form.level.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.l1.id }}">
        一级指标（层级选「二级指标」时需选择）
      </label>
      {{ form.l1(id=form.l1.id) }}
    </div>
    <p class="actions" style="margin-top:16px;">
      {{ form.submit(class_="btn") }}
      <a class="btn btn--ghost" href="/ahp/indicators">指标定义维
    护</a>
      <a class="btn btn--ghost" href="/ahp/experts">多专家矩阵</a
    >
    </p>
  </form>
</div>
<div class="card">
  <h2>各微地质单元当前生效权重</h2>
  <table>
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">当前权重来源</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for r in weight_rows %}
      <tr>
```

```
        <td style="text-align:left;">{{ r.unit_type }}</td>
        <td style="text-align:left;">
          {% if r.custom %}
          <span class="tag tag--warn">{{ r.label }}</span>
          {% else %}
          <span class="tag tag--muted">{{ r.label }}</span>
          {% endif %}
        </td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/ahp/matrix?unit_type=
    {{ r.unit_type }}&level=1">一级指标权重</a>
            <a class="link-btn" href="/ahp/matrix?unit_type=
    {{ r.unit_type }}&level=1&calc=1">查看一致性</a>
            {% if r.custom %}
            <form method="post" action="/ahp/weights/reset"
                  onsubmit="return confirm('确定把「{{
    r.unit_type }}」恢复为论文表 3 的默认权重吗？');">
              <input type="hidden" name="csrf_token"
    value="{{ csrf_token() }}">
              <input type="hidden" name="unit_type"
    value="{{ r.unit_type }}">
              <button type="submit" class="link-btn
    link-btn--danger">恢复默认</button>
            </form>
            {% endif %}
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endblock %}
# ==== app/templates/ahp/matrix.html ====
{% extends "base.html" %}
{% block title %}判断矩阵录入{% endblock %}
{% block content %}
<div class="card">
  <h2>
    判断矩阵录入
    · {{ unit_type }}
    · {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{%
    endif %}
  </h2>
  <p class="muted">
    采用 1–9 标度：1 同等重要、3 稍微重要、5 明显重要、7 强烈重要、9 极端重要，
    2、4、6、8 为相邻判断的中间值。只需录入上三角，对角线为 1，下三角自动取倒数。
    矩阵已按论文表 3 的权重预填。
  </p>
```

```
<form method="post">
  {{ form.hidden_tag() }}
  <table style="width:auto;">
    <thead>
      <tr>
        <th></th>
        {% for name in names %}<th scope="col">{{ name
  }}</th>{% endfor %}
      </tr>
    </thead>
    <tbody>
      {% for i in range(names|length) %}
      <tr>
        <th scope="row">{{ names[i] }}</th>
        {% for j in range(names|length) %}
          {% if i == j %}
          <td class="muted num">1</td>
          {% elif i < j %}
          <td>
            <label class="muted" style="position:absolute;
  left:-9999px;"
                   for="m-{{ i }}-{{ j }}">{{ names[i] }}
  相对 {{ names[j] }} 的重要性</label>
            <input id="m-{{ i }}-{{ j }}" type="number"
  step="0.01" min="0.01"
                   name="m_{{ i }}_{{ j }}" value="{{
  default[i][j] }}">
          </td>
          {% else %}
          <td class="muted num">{{ default[i][j] }}</td>
          {% endif %}
        {% endfor %}
      </tr>
      {% endfor %}
    </tbody>
  </table>
  <p class="actions">
    <button type="submit" class="btn" name="action"
  value="calc">计算权重</button>
    {% if result and result.consistent %}
    <button type="submit" class="btn" name="action"
  value="save">保存并生效</button>
    {% endif %}
    <a class="btn btn--ghost" href="/ahp/">重新选择</a>
  </p>
</form>
<p class="muted" style="margin-top:10px;">
  当前「{{ unit_type }}」生效权重：<strong>{{ source_label
  }}</strong>。
  点击「保存并生效」后，本单元类型的适宜性评价将采用此处算出的权重；
  如需回到论文取值，可在 <a href="/ahp/">权重首页</a> 恢复默认。
</p>
{% if result %}
<hr style="margin:22px 0;border:none;border-top:1px solid var(--line);">
<h3>计算结果</h3>
<table style="width:auto;">
  <thead>
    <tr>
      <th scope="col">指标</th>
      {% for name in names %}<th scope="col">{{ name }}</th>{% endfor %}
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">权重</th>
      {% for w in result.weights %}<td class="mono">{{ w }}</td>{% endfor %}
    </tr>
  </tbody>
</table>
<div class="stat-row">
  <div>
    <div class="stat__label">最大特征值 λmax</div>
    <div class="stat__value mono">{{ result.lambda_max }}</div>
  </div>
  <div>
    <div class="stat__label">一致性指标 CI</div>
    <div class="stat__value mono">{{ "%.4f"|format(result.ci) }}</div>
  </div>
  <div>
    <div class="stat__label">一致性比例 CR</div>
    <div class="stat__value mono">{{ "%.4f"|format(result.cr) }}</div>
  </div>
  <div>
    <div class="stat__label">一致性检验（要求 CR &lt; 0.1）</div>
    <div class="stat__value">
      {% if result.consistent %}<span class="ok">通过</span>
      {% else %}<span class="warn">不通过</span>{% endif %}
    </div>
  </div>
</div>
{% if not result.consistent and result.most_inconsistent %}
<p style="margin-top:14px;padding:12px;border-radius:6px;background:var(--danger-soft);
```

```
            border-left:4px solid var(--danger);">
    <strong class="warn">建议优先调整：</strong>
    第 {{ result.most_inconsistent.i + 1 }} 行、第 {{
    result.most_inconsistent.j + 1 }} 列的比较
    （当前 <span class="mono num">{{ result.most_inconsistent.v
    alue|round(3) }}</span>,
    理想约 <span class="mono num">{{ result.most_inconsistent.e
    xpected|round(3) }}</span>)
  </p>
  {% endif %}
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/ahp/indicators.html ====
{% extends "base.html" %}
{% block title %}指标定义维护{% endblock %}
{% block content %}
<div class="card">
  <h2>指标体系维护</h2>
  <p class="actions" style="margin-bottom:14px;">
    <a class="btn btn--sm btn--ghost"
    href="/ahp/">返回权重计算</a>
    <a class="btn btn--sm btn--ghost"
    href="/ahp/experts">多专家矩阵</a>
    <form method="post" action="/ahp/indicators/reset"
    style="display:inline;"
          onsubmit="return confirm('将按论文取值重置全部指标定义，自定义修改会丢失。
    确定继续？');">
      {{ form.hidden_tag() }}
      <button type="submit" class="btn btn--sm
    btn--danger">按论文取值重置</button>
    </form>
  </p>
  {% for level, title in [(1, "一级指标"), (2, "二级指标"), (3,
    "三级指标")] %}
  {% set level_items = all_ind|selectattr("level",
    "equalto", level)|list %}
  {% if level_items %}
  <h3>{{ title }}({{ level_items|length }} 项)</h3>
  <table>
    <thead>
      <tr>
        <th scope="col">指标</th>
        <th scope="col">上级指标</th>
        <th scope="col">单位</th>
        <th scope="col">标准化类型</th>
        <th scope="col">理想取值</th>
        <th scope="col">启用单元</th>
        <th scope="col">限制性</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for i in level_items %}
      <tr>
        <td style="text-align:left;">
          <span class="mono muted">{{ i.code
    }}</span><br>{{ i.name }}
        </td>
        <td>{{ (all_ind|selectattr("code", "equalto",
    i.parent_code)|list|first).name if i.parent_code else
    "—" }}</td>
        <td>{{ i.unit or "—" }}</td>
        <td>{{ type_label(i.ideal_type) }}</td>
        <td class="mono">{{ i.ideal_value or "—" }}</td>
        <td style="text-align:left;">{{ i.enabled_units or
    "—" }}</td>
        <td>
          {% if i.is_constraint %}<span class="tag
    tag--warn">是</span>
          {% else %}<span class="tag tag--muted">否</span>{%
    endif %}
        </td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/ahp/indicators/{{
    i.code }}/edit">编辑</a>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
  {% endfor %}
</div>
{% endblock %}
# ==== app/templates/ahp/indicator_edit.html ====
{% extends "base.html" %}
{% block title %}编辑指标 · {{ ind.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:680px;">
  <h2>编辑指标 · {{ ind.name }}</h2>
  <p class="muted">
    指标编码 <span class="mono">{{ ind.code }}</span>、层级 {{
    ind.level }} 级,
    由论文指标体系确定,不可修改。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
```

```
<div class="field">
  <label class="field__label" for="name">指标名称</label>
  <input id="name" type="text" name="name"
style="width:100%;" required
         value="{{ ind.name }}">
</div>
<div class="field">
  <label class="field__label" for="unit">计量单位</label>
  <input id="unit" type="text" name="unit"
style="width:180px;"
         value="{{ ind.unit or '' }}">
</div>
<div class="field">
  <label class="field__label" for="ideal-type">标准化类型</label>
  <select id="ideal-type" name="ideal_type">
    {% for code, label in [("benefit","越大越优"),("cost","越小越优"),("mid","适中型"),("discrete","离散赋值")] %}
    <option value="{{ code }}" {% if ind.ideal_type ==
code %}selected{% endif %}>{{ label }}</option>
    {% endfor %}
  </select>
</div>
<div class="field">
  <label class="field__label" for="ideal-value">理想取值</label>
  <input id="ideal-value" type="text"
name="ideal_value" style="width:100%;"
         value="{{ ind.ideal_value or '' }}">
  <p class="muted" style="margin:4px 0 0;">
    如土石比 0.8、土层厚度 20、pH 5~9、植被覆盖度 75。
  </p>
</div>
<fieldset style="border:1px solid
var(--line);border-radius:6px;padding:10px
14px;margin:0 0 12px;">
  <legend style="padding:0 6px;font-weight:600;">启用该指标的微地质单元</legend>
  {% for t in unit_types %}
  <label class="checkbox" style="margin-right:18px;">
    <input type="checkbox" name="enabled_units"
value="{{ t }}"
           {% if t in ind.units_list() %}checked{%
endif %}>
    <span>{{ t }}</span>
  </label>
  {% endfor %}
  <p class="muted" style="margin:8px 0 0;">
    论文设定：岩质边坡不启用土壤指标，平台不启用地形指标。
  </p>
```

```
    </fieldset>
    <div class="field">
      <label class="checkbox" for="is-constraint">
        <input id="is-constraint" type="checkbox"
    name="is_constraint" value="1"
               {% if ind.is_constraint %}checked{% endif %}>
        <span>同时作为限制性指标（参与一票否决判定）</span>
      </label>
    </div>
    <div class="field">
      <label class="field__label" for="description">取值依据说明</
    label>
      <textarea id="description" name="description"
    style="min-height:88px;">{{ ind.description or '' 
    }}</textarea>
    </div>
    <p class="actions" style="margin-top:16px;">
      <button type="submit" class="btn">保存</button>
      <a class="btn btn--ghost" href="/ahp/indicators">返回指标列
    表</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/ahp/experts.html ====
{% extends "base.html" %}
{% block title %}多专家判断矩阵{% endblock %}
{% block content %}
<div class="card">
  <h2>多专家判断矩阵 · 录入与汇总</h2>
  <form method="get" class="actions" style="margin-bottom:14
    px;">
    <div>
      <label class="field__label" for="unit-type">微地质单元类型</l
    abel>
      <select id="unit-type" name="unit_type"
    onchange="this.form.submit()">
        {% for t in ["岩质边坡", "碎石边坡", "平台"] %}
        <option value="{{ t }}" {% if unit_type == t
    %}selected{% endif %}>{{ t }}</option>
        {% endfor %}
      </select>
    </div>
    <div>
      <label class="field__label" for="level">指标层级</label>
      <select id="level" name="level"
    onchange="this.form.submit()">
        <option value="1" {% if level == 1 %}selected{%
    endif %}>一级指标</option>
        <option value="2" {% if level == 2 %}selected{%
  endif %}>二级指标</option>
    </select>
  </div>
</form>
<p class="muted">
  当前范围：<strong>{{ unit_type }}</strong> ·
  {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{% endif
  %}。
  若选二级指标，请先到
  <a href="/ahp/">权重计算</a> 选择一级指标后进入本页。
</p>
{% if rows %}
<table>
  <caption class="muted">已录入 {{ rows|length }}
  位专家的矩阵</caption>
  <thead>
    <tr>
      <th scope="col">专家</th>
      <th scope="col">矩阵规模</th>
      <th scope="col">该矩阵 CR</th>
      <th scope="col">一致性</th>
      <th scope="col">录入时间</th>
      <th scope="col">操作</th>
    </tr>
  </thead>
  <tbody>
    {% for r in rows %}
    <tr>
      <td>{{ r.expert_name }}</td>
      <td class="num">{{ n }} × {{ n }}</td>
      <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is
  not none else "—" }}</td>
      <td>
        {% if r.cr is not none and r.cr < 0.1 %}<span
  class="tag tag--ok">通过</span>
        {% else %}<span class="tag tag--warn">不通过</span>{%
  endif %}
      </td>
      <td class="mono nowrap">{{ r.created_at.strftime("%Y
  -%m-%d %H:%M") if r.created_at else "—" }}</td>
      <td>
        <form method="post" action="/ahp/experts/{{ r.id
  }}/delete"
              onsubmit="return confirm('确定删除专家「{{
  r.expert_name }}」的矩阵吗？');">
          {{ form.hidden_tag() }}
          <button type="submit" class="btn btn--sm
  btn--danger">删除</button>
        </form>
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>
<p class="actions" style="margin-bottom:10px;">
  <a class="btn" href="/ahp/experts/aggregate?unit_type={{
  unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{%
  endif %}">
    汇总计算（几何平均）
  </a>
</p>
{% else %}
<p class="muted">尚未录入任何专家矩阵。请在下方录入第一位专家的判断矩阵。</p>
{% endif %}
<h3>录入一位专家的判断矩阵</h3>
<form method="post">
  {{ form.hidden_tag() }}
  <input type="hidden" name="action" value="add">
  <input type="hidden" name="unit_type" value="{{
  unit_type }}">
  <input type="hidden" name="level" value="{{ level }}">
  {% if l1 %}<input type="hidden" name="l1" value="{{ l1
  }}">{% endif %}
  <div class="field">
    <label class="field__label" for="expert-name">专家姓名</la
  bel>
    <input id="expert-name" type="text" name="expert_name"
           style="width:220px;" placeholder="如：张工">
  </div>
  <table style="width:auto;">
    <thead>
      <tr>
        <th></th>
        {% for name in names %}<th scope="col">{{ name
  }}</th>{% endfor %}
      </tr>
    </thead>
    <tbody>
      {% for i in range(n) %}
      <tr>
        <th scope="row">{{ names[i] }}</th>
        {% for j in range(n) %}
          {% if i == j %}
          <td class="muted num">1</td>
          {% elif i < j %}
          <td>
            <label class="muted" style="position:absolute;
  left:-9999px;"
                   for="e-{{ i }}-{{ j }}">{{ names[i] }}
  相对 {{ names[j] }} 的重要性</label>
```

```
              <input id="e-{{ i }}-{{ j }}" type="number"
    step="0.01" min="0.01"
                     name="m_{{ i }}_{{ j }}" value="{{
    default[i][j] }}">
            </td>
            {% else %}
            <td class="muted num">{{ default[i][j] }}</td>
            {% endif %}
          {% endfor %}
        </tr>
        {% endfor %}
      </tbody>
    </table>
    <p class="actions" style="margin-top:12px;">
      <button type="submit" class="btn">录入该专家矩阵</button>
      <a class="btn btn--ghost" href="/ahp/indicators">指标维护<
    /a>
      <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/ahp/experts_aggregate.html ====
{% extends "base.html" %}
{% block title %}多专家矩阵汇总{% endblock %}
{% block content %}
<div class="card">
  <h2>多专家矩阵汇总 · {{ unit_type }} · {% if level == 1 %}一级指标{%
    else %}二级指标（{{ l1 }}）{% endif %}</h2>
  {% if not result %}
  <p class="muted">
    该范围内尚未录入专家矩阵。
    <a href="/ahp/experts?unit_type={{ unit_type
    }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif
    %}">去录入</a>
  </p>
  {% else %}
  <div class="stat-row">
    <div>
      <div class="stat__label">参与汇总的专家</div>
      <div class="stat__value">{{ result.count }} 位</div>
    </div>
    <div>
      <div class="stat__label">λmax</div>
      <div class="stat__value mono">{{ result.lambda_max
    }}</div>
    </div>
    <div>
      <div class="stat__label">CI</div>
      <div class="stat__value mono">{{
```

```
  "%.4f"|format(result.ci) }}</div>
  </div>
  <div>
    <div class="stat__label">CR（要求 &lt; 0.1）</div>
    <div class="stat__value mono">{{
  "%.4f"|format(result.cr) }}</div>
  </div>
  <div>
    <div class="stat__label">一致性检验</div>
    <div class="stat__value">
      {% if result.consistent %}<span class="ok">通过</span>
      {% else %}<span class="warn">不通过</span>{% endif %}
    </div>
  </div>
</div>
<h3>汇总后权重</h3>
<table style="width:auto;">
  <thead>
    <tr>
      <th scope="col">指标</th>
      {% for name in names %}<th scope="col">{{ name
  }}</th>{% endfor %}
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">权重</th>
      {% for w in result.weights %}<td class="mono">{{ w
  }}</td>{% endfor %}
    </tr>
  </tbody>
</table>
<h3>各专家矩阵的一致性</h3>
<table style="width:auto;">
  <thead>
    <tr><th scope="col">专家</th><th scope="col">该矩阵
  CR</th><th scope="col">判定</th></tr>
  </thead>
  <tbody>
    {% for r in rows %}
    <tr>
      <td>{{ r.expert_name }}</td>
      <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is
  not none else "—" }}</td>
      <td>
        {% if r.cr is not none and r.cr < 0.1 %}<span
  class="ok">通过</span>
        {% else %}<span class="warn">不通过</span>{% endif %}
      </td>
    </tr>
```

```
      {% endfor %}
    </tbody>
  </table>
  {% if not result.consistent and result.most_inconsistent
    %}
  <p style="padding:12px;border-radius:6px;background:var(--
    danger-soft);
            border-left:4px solid var(--danger);">
    <strong class="warn">汇总矩阵一致性不通过，建议优先调整：</strong>
    第 {{ result.most_inconsistent.i + 1 }} 行、第 {{
    result.most_inconsistent.j + 1 }} 列的比较
    （当前 <span class="mono">{{ result.most_inconsistent.value
    |round(3) }}</span>，
    理想约 <span class="mono">{{ result.most_inconsistent.expec
    ted|round(3) }}</span>）。
    该偏差通常来自个别专家打分，可先核对一致性较差的专家矩阵。
  </p>
  {% endif %}
  {% endif %}
  <p class="actions" style="margin-top:16px;">
    <form method="post"
          action="/ahp/experts?unit_type={{ unit_type
    }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif %}"
          style="display:inline;"
          onsubmit="return confirm('保存后，「{{ unit_type
    }}」的适宜性评价将改用这组汇总权重计算。确定继续吗？');">
      {{ form.hidden_tag() }}
      <input type="hidden" name="action" value="save_agg">
      <button type="submit" class="btn" {% if not
    result.consistent %}disabled{% endif %}>
        保存为生效权重
      </button>
    </form>
    <a class="btn btn--ghost" href="/ahp/experts?unit_type={
    { unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1
    }}{% endif %}">返回录入</a>
    <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/indicator/index.html ====
{% extends "base.html" %}
{% from "_macros.html" import patch_guide %}
{% block title %}指标数据采集{% endblock %}
{% block content %}
<div class="card">
  <h2>指标数据采集与管理</h2>
  <p class="actions" style="margin-bottom:12px;">
    <a class="btn btn--sm" href="/indicator/import">批量导入（CSV
    / Excel）</a>
```

```
</p>
{{ patch_guide(projects_without_patch) }}
{% if not patches %}
<p class="muted">
  暂无矿山图斑。请先在「项目与矿山档案」中建立图斑，
  或运行 <span class="mono">python seed_demo.py</span>
  载入论文案例。
</p>
{% else %}
{% for p in patches %}
<div class="patch-panel">
  <div class="patch-panel__head">
    <h3 class="patch-panel__name">{{ p.name }}</h3>
    <span class="patch-panel__meta">
      所属项目：{{ p.project.name if p.project else "—" }}
    </span>
    <span class="patch-panel__meta">
      微地质单元：{{ p.units.count() }} 个
    </span>
    {% if p.units.count() %}
    <span class="patch-panel__meta patch-panel__tail">
      <a class="link-btn" href="/indicator/import">批量导入本图斑
  数据</a>
    </span>
    {% endif %}
  </div>
  {% if not p.units.count() %}
  <div class="notice notice--warn">
    <p class="notice__title">该图斑尚未划分微地质单元，暂不能录入指标数据。</p>
    <p style="margin:0;">
      指标数据是按微地质单元（岩质边坡 / 碎石边坡 / 平台）分别组织的，
      请先完成划分，再回到本页录入。
    </p>
    <p style="margin:10px 0 0;">
      <a class="btn btn--sm" href="/unit/{{ p.id
  }}">去划分微地质单元</a>
    </p>
  </div>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">面积占比</th>
        <th scope="col">数据状态</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for u in p.units %}
```

```
        <tr>
          <td>{{ u.unit_type }}</td>
          <td class="num">
            {% if u.ratio is not none %}{{
    "%.2f%%"|format(u.ratio * 100) }}{% else %}—{% endif %}
          </td>
          <td>
            {% if u.indicator_data %}
            <span class="tag tag--ok">已录入</span>
            {% else %}
            <span class="tag tag--muted">未录入</span>
            {% endif %}
          </td>
          <td class="nowrap">
            <span class="row-actions">
              <a class="link-btn" href="/indicator/{{ u.id
    }}">录入 / 编辑</a>
            </span>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% endif %}
  </div>
  {% endfor %}
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/indicator/edit.html ====
{% extends "base.html" %}
{% block title %}指标数据录入 · {{ unit.unit_type }}{% endblock %}
{% block content %}
<div class="card" style="max-width:820px;">
  <h2>指标数据录入 · {{ unit.unit_type }}</h2>
  <p class="muted">
    输入框留空表示该指标「不适用」，与「实测值为 0」含义不同，计算时会被跳过。
    岩质边坡不设土壤指标、平台不设地形指标，系统已按单元类型隐藏不相关字段。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    {% for group_name, fields, has_hm in groups %}
    <h3>{{ group_name }}</h3>
    {% for fname, cname, unit_ in fields %}
    <div class="field">
      <label class="field__label" for="f-{{ fname }}">
        {{ cname }}{% if unit_ %}({{ unit_ }}){% endif %}
      </label>
      {% if fname == "aspect" %}
      <select id="f-{{ fname }}" name="aspect">
      <option value="">— 不适用 —</option>
      {% for a in aspect_types %}
      <option value="{{ a }}" {% if current.get("aspect")
  == a %}selected{% endif %}>{{ a }}</option>
      {% endfor %}
    </select>
    {% else %}
    <input id="f-{{ fname }}" type="number" step="any"
  name="{{ fname }}"
           value="{{ current.get(fname) if
  current.get(fname) is not none else '' }}">
    {% endif %}
  </div>
  {% endfor %}
  {% endfor %}
  {% for group_name, fields, has_hm in groups %}
    {% if has_hm %}
    <h3>重金属含量（mg/kg）</h3>
    <p class="muted">未检测的金属可留空，系统按论文注释视为未受污染处理。</p>
    <table style="width:auto;">
      <thead>
        <tr>
          {% for m in heavy_metals %}<th scope="col">{{ m
  }}</th>{% endfor %}
        </tr>
      </thead>
      <tbody>
        <tr>
          {% for m in heavy_metals %}
          <td>
            <label class="muted" style="position:absolute;
  left:-9999px;"
                   for="hm-{{ m }}">{{ m }} 含量</label>
            <input id="hm-{{ m }}" type="number"
  step="any" name="hm_{{ m }}"
                   value="{{ current.get('heavy',
  {}).get(m) if current.get('heavy', {}).get(m) is not
  none else '' }}">
          </td>
          {% endfor %}
        </tr>
      </tbody>
    </table>
    {% endif %}
  {% endfor %}
  <p class="actions" style="margin-top:18px;">
    <button type="submit" class="btn">保存</button>
    <a class="btn btn--ghost" href="/indicator/">返回</a>
  </p>
</form>
```

```
</div>
{% endblock %}
# ==== app/templates/indicator/import.html ====
{% extends "base.html" %}
{% block title %}批量导入指标数据{% endblock %}
{% block content %}
<div class="card" style="max-width:860px;">
  <h2>批量导入指标数据</h2>
  <p class="muted">
    支持 CSV 与 Excel（.xlsx/.xls）。文件每行对应一个微地质单元，
    系统按「单元类型」匹配图斑内已划分的单元后写入数据。
    先下载模板按格式填写，可减少出错。
  </p>
  <p class="actions" style="margin-bottom:14px;">
    <a class="btn btn--sm btn--ghost"
    href="/indicator/import/template">下载导入模板（CSV）</a>
    <a class="btn btn--sm btn--ghost"
    href="/indicator/">返回指标数据</a>
  </p>
  <form method="post" enctype="multipart/form-data">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="patch-id">目标矿山图斑</lab
    el>
      <select id="patch-id" name="patch_id" required>
        <option value="">— 请选择 —</option>
        {% for p in patches %}
        <option value="{{ p.id }}">{{ p.name }}（{{
    p.units.count() }} 个单元）</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="datafile">数据文件</label
    >
      <input id="datafile" type="file" name="datafile"
             accept=".csv,.xlsx,.xls" required>
    </div>
    <p class="actions" style="margin-top:16px;">
      <button type="submit" class="btn">开始导入</button>
    </p>
  </form>
  <h3>模板列说明</h3>
  <p class="muted">
    共 {{ headers|length }} 列。留空表示该指标「不适用」。
    坡向须填「阳坡 / 阴阳坡 / 阴坡」之一。未检测的重金属留空即可
    （按论文注释视为未受污染）。
  </p>
  <p class="mono muted" style="line-height:1.9;">{{
    headers|join(" , ") }}</p>
  <h3>导入规则</h3>
  <ul class="muted" style="padding-left:20px;line-height:1.9
    ;">
    <li>「单元类型」必须与图斑内已划分的单元一致，否则该行会被跳过并在结果中提示。</li>
    <li>同一单元类型若在文件中出现多行，以最后一行为准。</li>
    <li>已有数据的单元会被覆盖更新；没有数据的单元将新建记录。</li>
    <li>数值列请填纯数字，不要带单位（如填 <span class="mono">10</span> 而非
    <span class="mono">10cm</span>）。</li>
  </ul>
</div>
{% if report %}
<div class="card" style="max-width:860px;">
  <h2>导入结果</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">目标图斑</div>
      <div style="font-size:15px;">{{ report.patch }}</div>
    </div>
    <div>
      <div class="stat__label">成功导入</div>
      <div class="stat__value ok">{{ report.saved }}
    个单元</div>
    </div>
    <div>
      <div class="stat__label">问题行</div>
      <div class="stat__value {% if report.errors %}warn{%
    endif %}">{{ report.errors|length }}</div>
    </div>
  </div>
  {% if report.errors %}
  <h3>未导入的行</h3>
  <ul class="muted" style="padding-left:20px;line-height:1.9
    ;">
    {% for e in report.errors %}<li>{{ e }}</li>{% endfor %}
  </ul>
  {% else %}
  <p class="ok" style="margin-top:12px;">全部数据行均已成功导入。</p>
  {% endif %}
  <p class="actions" style="margin-top:16px;">
    <a class="btn btn--ghost" href="/indicator/">查看指标数据</a>
  </p>
</div>
{% endif %}
{% endblock %}
# ==== app/templates/evaluate/index.html ====
{% extends "base.html" %}
{% from "_macros.html" import patch_guide %}
{% block title %}适宜性定量评价{% endblock %}
{% block content %}
<div class="card">
```

```
<h2>适宜性定量评价</h2>
{{ patch_guide(projects_without_patch) }}
{% if not patches %}
<p class="muted">暂无矿山图斑。请先到「项目与矿山档案」建立图斑。</p>
{% else %}
<table>
  <thead>
    <tr>
      <th scope="col">所属项目</th>
      <th scope="col">矿山图斑</th>
      <th scope="col">前置条件</th>
      <th scope="col">最近评价结果</th>
      <th scope="col">操作</th>
    </tr>
  </thead>
  <tbody>
    {% for p in patches %}
    {% set pr = progress.get(p.id) %}
    <tr>
      <td>{{ p.project.name if p.project else "—" }}</td>
      <td style="text-align:left;">{{ p.name }}</td>
      <td style="text-align:left;">
        {% if pr and pr.ready %}
        <span class="tag tag--ok">已具备</span>
        {% elif pr %}
        <span class="tag tag--warn">缺：{{
  pr.blocking[0].label }}</span>
        <span class="muted">{{ pr.blocking[0].reason
  }}</span>
        {% endif %}
      </td>
      <td>
        {% if latest.get(p.id) %}
        F = <span class="mono num">{{
  latest[p.id].final_score }}</span>,
        <span class="tag tag--ok">{{ latest[p.id].grade
  }}</span>
        {% else %}
        <span class="tag tag--muted">未评价</span>
        {% endif %}
      </td>
      <td class="nowrap">
        <span class="row-actions">
          <a class="link-btn" href="/evaluate/{{ p.id
  }}">执行评价</a>
          {% if latest.get(p.id) %}
          <a class="link-btn" href="/evaluate/result/{{
  latest[p.id].id }}">查看结果</a>
          {% endif %}
        </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/evaluate/run.html ====
{% extends "base.html" %}
{% import "_macros.html" as ui %}
{% block title %}执行评价 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:880px;">
  <h2>执行适宜性评价 · {{ patch.name }}</h2>
  {{ ui.workflow_steps(progress) }}
  {{ ui.workflow_blocking(progress) }}
  {% if progress.ready %}
  <form method="post">
    {{ form.hidden_tag() }}
    <p class="actions">
      <button type="submit" class="btn">开始评价</button>
      <a class="btn btn--ghost" href="/evaluate/">返回</a>
    </p>
  </form>
  {% else %}
  <p class="actions">
    <button type="button" class="btn" disabled>开始评价</button>
    <a class="btn btn--ghost" href="/evaluate/">返回</a>
  </p>
  {% endif %}
</div>
<div class="card" style="max-width:880px;">
  <h2>当前流程状态</h2>
  <table>
    <thead>
      <tr><th scope="col">步骤</th><th scope="col">状态</th><th
    scope="col">说明</th></tr>
    </thead>
    <tbody>
      <tr>
        <td>限制性条件判定</td>
        <td>
          {% if progress.constraint.passed %}<span
    class="tag tag--ok">已通过</span>
          {% elif progress.constraint.done %}<span
    class="tag tag--warn">未通过</span>
          {% else %}<span class="tag tag--muted">待完成</span>{
    % endif %}
        </td>
        <td style="text-align:left;">{{
    progress.constraint.detail }}</td>
      </tr>
      <tr>
        <td>划分微地质单元</td>
        <td>
          {% if progress.units.done %}<span class="tag
    tag--ok">已完成</span>
          {% else %}<span class="tag tag--muted">待完成</span>{
    % endif %}
        </td>
        <td style="text-align:left;">{{
    progress.units.detail }}</td>
      </tr>
      <tr>
        <td>录入指标数据</td>
        <td>
          {% if progress.data.ready %}<span class="tag
    tag--ok">已完成</span>
          {% else %}<span class="tag tag--muted">待完成</span>{
    % endif %}
        </td>
        <td style="text-align:left;">{{
    progress.data.detail }}</td>
      </tr>
    </tbody>
  </table>
</div>
{% if weight_sources %}
<div class="card" style="max-width:880px;">
  <h2>本次将采用的指标权重</h2>  <table style="max-width:720px;">
    <thead>
      <tr><th scope="col">微地质单元</th><th
    scope="col">权重来源</th></tr>
    </thead>
    <tbody>
      {% for ut, label in weight_sources.items() %}
      <tr>
        <td style="text-align:left;">{{ ut }}</td>
        <td style="text-align:left;">
          {% if "自定义" in label %}
          <span class="tag tag--warn">{{ label }}</span>
          {% else %}
          <span class="tag tag--muted">{{ label }}</span>
          {% endif %}
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
```

```
</div>
{% endif %}
{% endblock %}
# ==== app/templates/evaluate/result.html ====
{% extends "base.html" %}
{% block title %}评价结果{% endblock %}
{% block content %}
<div class="card">
  <h2>适宜性评价结果 · {{ patch.name if patch else "" }}</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">最终评价因子 F</div>
      <div class="stat__value mono">{{ r.final_score
    }}</div>
    </div>
    <div>
      <div class="stat__label">适宜性等级</div>
      <div class="stat__value ok">{{ r.grade }}</div>
    </div>
    <div style="flex:1;min-width:240px;">
      <div class="stat__label">推荐措施</div>
      <div style="font-size:15px;">{{ r.measures }}</div>
    </div>
  </div>
  {% if unit_scores %}
  <h3>各微地质单元得分</h3>
  <table style="max-width:860px;">
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">面积占比</th>
        <th scope="col">单元评价因子 F'</th>
        <th scope="col">加权贡献</th>
        <th scope="col">权重来源</th>
      </tr>
    </thead>
    <tbody>
      {% for u in unit_scores %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td class="num">{{ "%.2f%%"|format(u.ratio * 100)
    }}</td>
        <td class="mono">{{ u.unit_score }}</td>
        <td class="mono">{{ "%.4f"|format(u.ratio *
    u.unit_score) }}</td>
        <td class="muted">{{ u.weight_source or "论文表 3
    默认权重" }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
  <p class="actions" style="margin-top:18px;">
    <a class="btn btn--ghost" href="/report/{{ r.patch_id
    }}">查看得分明细</a>
    <a class="btn btn--ghost" href="/evaluate/">返回评价列表</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/report/index.html ====
{% extends "base.html" %}
{% from "_macros.html" import patch_guide %}
{% block title %}报告输出{% endblock %}
{% block content %}
<div class="card">
  <h2>报告与结果输出</h2>
  {{ patch_guide(projects_without_patch) }}
  {% if not patches %}
  <p class="muted">暂无矿山图斑。请先到「项目与矿山档案」建立图斑。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">矿山图斑</th>
        <th scope="col">可生成报告</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in patches %}
      {% set pr = progress.get(p.id) %}
      <tr>
        <td>{{ p.project.name if p.project else "—" }}</td>
        <td style="text-align:left;">{{ p.name }}</td>
        <td style="text-align:left;">
          {% if pr and pr.ready %}
          <span class="tag tag--ok">可以</span>
          {% elif pr %}
          <span class="tag tag--warn">暂不可</span>
          <span class="muted">{{ pr.blocking[0].reason
    }}</span>
          {% endif %}
        </td>
        <td class="nowrap">
          {% if pr and pr.ready %}
          <span class="row-actions">
            <a class="link-btn" href="/report/{{ p.id
    }}">查看得分明细</a>
            <a class="link-btn" href="/report/{{ p.id
    }}/pdf">导出 PDF</a>
            <a class="link-btn" href="/report/{{ p.id
    }}/word">导出 Word</a>
          </span>
          {% else %}
          <span class="muted">—</span>
          {% endif %}
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/report/detail.html ====
{% extends "base.html" %}
{% block title %}得分明细 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>得分明细 · {{ patch.name }}</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">最终评价因子 F</div>
      <div class="stat__value mono">{{
    "%.4f"|format(result.final_score) }}</div>
    </div>
    <div>
      <div class="stat__label">适宜性等级</div>
      <div class="stat__value ok">{{ result.grade }}</div>
    </div>
    <div style="flex:1;min-width:240px;">
      <div class="stat__label">推荐措施</div>
      <div style="font-size:15px;">{{
    result.grade_info.measure }}</div>
    </div>
  </div>
  <p class="muted" style="margin-top:12px;">
    本页与导出报告均取自
    <strong>{% if record %}{{ record.created_at.strftime("%Y
    -%m-%d %H:%M:%S") }}{% endif %}
    记录的那次评价结果</strong>（评价记录编号 {{ record.id if record else
    "—" }}），
    不会因事后修改指标数据而变化。
  </p>
  <p class="actions" style="margin-top:16px;">
    <a class="btn" href="/report/{{ patch.id }}/pdf">导出 PDF
    报告</a>
    <a class="btn btn--ghost" href="/report/{{ patch.id
    }}/word">导出 Word 报告</a>
    <a class="btn btn--ghost" href="/report/">返回</a>
  </p>
</div>
<div class="card">
  <h2>结果图表</h2>
  <h3>各微地质单元得分对比</h3>
  <div>{{ chart_units|safe }}</div>
  <h3>各一级指标平均得分</h3>
  <p class="muted">反映各一级指标对评价结果的实际影响，便于识别关键控制项。</p>
  <div>{{ chart_weight|safe }}</div>
</div>
{% for u in result.unit_results %}
<div class="card">
  <h2>{{ u.unit_type }} · 占比 {{ "%.2f%%"|format(u.ratio *
    100) }} · 单元得分 {{ "%.4f"|format(u.unit_score) }}</h2>
  {% if u.weight_source %}
  <p class="muted" style="margin-top:-6px;">权重来源：{{
    u.weight_source }}</p>
  {% endif %}
  <table>
    <thead>
      <tr>
        <th scope="col">一级指标</th>
        <th scope="col">二级指标</th>
        <th scope="col">实测值</th>
        <th scope="col">标准化值</th>
        <th scope="col">权重</th>
        <th scope="col">贡献</th>
      </tr>
    </thead>
    <tbody>
      {% for d in u.details %}
      <tr>
        <td>{{ d.l1 }}</td>
        <td style="text-align:left;">{{ d.l2 }}</td>
        <td class="mono" style="text-align:left;">{{ "—" if
    d.raw is none else d.raw }}</td>
        <td class="mono">{{ "—" if d.std_value is none else
    "%.4f"|format(d.std_value) }}</td>
        <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
        <td class="mono">{{ "—" if d.contribution is none
    else "%.4f"|format(d.contribution) }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  <p class="muted" style="margin:0;">
    一级指标得分：
    {% for k, v in u.l1_scores.items() %}{{ k }} = <span
    class="num">{{ "%.4f"|format(v) }}</span>{% if not
    loop.last %};{% endif %}{% endfor %}
  </p>
</div>
{% endfor %}
{% endblock %}
# ==== app/templates/report/print.html ====
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<title>自然恢复适宜性评价报告 · {{ patch.name }}</title>
<style>
  @page { size: A4; margin: 18mm 16mm; }
  * { box-sizing: border-box; }
  body {
    margin: 0; color: #23262b; background: #fff;
    font-family: "Microsoft YaHei", "SimSun", sans-serif;
    font-size: 10.5pt; line-height: 1.65;
  }
  h1 { font-size: 17pt; text-align: center; margin: 0 0
    6px; }
  h2 { font-size: 13pt; color: #1f3a5f; margin: 20px 0 8px;
       border-bottom: 1px solid #dfe3e8; padding-bottom:
    4px; }
  h3 { font-size: 11.5pt; color: #1f3a5f; margin: 16px 0
    6px; }
  .sub { text-align: center; color: #6b7280; font-size:
    9.5pt; margin-bottom: 18px; }
  table { width: 100%; border-collapse: collapse; margin:
    8px 0 14px;
          font-variant-numeric: tabular-nums; }
  th, td { border: 1px solid #c9ccd1; padding: 4px 8px;
    text-align: center; }
  th { background: #eef2f7; font-weight: 600; }
  .left { text-align: left; }
  .mono { font-family: Consolas, "Courier New", monospace; }
  .kv { width: auto; }
  .kv th { width: 30%; text-align: left; }
  .kv td { text-align: left; }
  ul { margin: 6px 0 12px; padding-left: 20px; }
  .note { color: #6b7280; font-size: 9.5pt; }
  .chart { margin: 10px 0 16px; }
  .sig { margin-top: 28px; font-size: 10pt; }
  .avoid-break { break-inside: avoid; }
  svg { max-width: 100%; }
</style>
</head>
<body>
  <h1>历史遗留废弃矿山自然恢复适宜性评价报告</h1>
  <p class="sub">
```

```
    {{ patch.name }}
    {% if patch.admin_region %} · {{ patch.admin_region
    }}{% endif %}
    <br>历史遗留废弃矿山自然恢复适宜性评价系统 V1.0 ·
    生成时间 {{ generated_at }}
  </p>
  <h2>一、图斑基本信息</h2>
  <table class="kv">
    <tr><th>图斑名称</th><td>{{ patch.name }}</td></tr>
    <tr><th>所在行政区</th><td>{{ patch.admin_region or "—"
    }}</td></tr>
    <tr><th>图斑面积（公顷）</th><td>{{ patch.area_ha if
    patch.area_ha is not none else "—" }}</td></tr>
    <tr><th>开采矿种</th><td>{{ patch.mineral_type or "—"
    }}</td></tr>
    <tr><th>开采方式</th><td>{{ patch.mining_method or "—"
    }}</td></tr>
    <tr><th>责任主体缺失</th><td>{{ "是" if patch.responsible_entit
    y_missing else "否" }}</td></tr>
    {% if reference %}
    <tr><th>参照生态系统</th>
        <td>植被覆盖度基准 {{ reference.coverage_benchmark }}%,
            多样性指数基准 {{ reference.shannon_benchmark
    }}</td></tr>
    {% endif %}
  </table>
  <h2>二、评价结论</h2>
  <table class="kv">
    <tr><th>最终评价因子 F</th><td class="mono"><strong>{{
    "%.4f"|format(result.final_score) }}</strong></td></tr>
    <tr><th>适宜性等级</th><td><strong>{{ result.grade
    }}</strong></td></tr>
    <tr><th>判定依据</th><td>{{ result.grade_info.meaning
    }}</td></tr>
    <tr><th>推荐措施</th><td>{{ result.grade_info.measure
    }}</td></tr>
  </table>
  <h2>三、各微地质单元得分</h2>
  <table>
    <thead>
      <tr><th>微地质单元</th><th>面积占比</th><th>单元评价因子
    F'</th><th>加权贡献</th></tr>
    </thead>
    <tbody>
      {% for u in result.unit_results %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td class="mono">{{ "%.2f%%"|format(u.ratio * 100)
    }}</td>
        <td class="mono">{{ "%.4f"|format(u.unit_score)
  }}</td>
      <td class="mono">{{ "%.4f"|format(u.weighted)
  }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
<div class="chart">{{ chart_units|safe }}</div>
<h2>四、一级指标权重分布</h2>
<div class="chart">{{ chart_weight|safe }}</div>
<h2>五、得分明细</h2>
{% for u in result.unit_results %}
<h3>{{ u.unit_type }}（占比 {{ "%.2f%%"|format(u.ratio *
  100) }}，单元得分 {{ "%.4f"|format(u.unit_score) }}）</h3>
<table>
  <thead>
    <tr>
      <th>一级指标</th><th>二级指标</th><th>实测值</th>
      <th>标准化值</th><th>权重</th><th>贡献</th>
    </tr>
  </thead>
  <tbody>
    {% for d in u.details %}
    <tr>
      <td>{{ d.l1 }}</td>
      <td class="left">{{ d.l2 }}</td>
      <td class="mono left">{{ "—" if d.raw is none else
  d.raw }}</td>
      <td class="mono">{{ "—" if d.std_value is none else
  "%.4f"|format(d.std_value) }}</td>
      <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
      <td class="mono">{{ "—" if d.contribution is none
  else "%.4f"|format(d.contribution) }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
<p class="note">
  一级指标得分：
  {% for k, v in u.l1_scores.items() %}{{ k }} = {{
  "%.4f"|format(v) }}{% if not loop.last %}；{% endif %}{%
  endfor %}
</p>
{% endfor %}
<h2>六、评价依据说明</h2>
<ul class="note">
  <li>指标标准化按论文式(8) 执行，按指标性质分为越大越优、越小越优、
      适中型与离散赋值四类。</li>
  <li>单元评价因子按论文式(9) 由二级指标权重加权求得；
      整矿评价因子由各微地质单元按面积占比加权合成。</li>
    <li>适宜性等级按论文表 4 划分：F&lt;0.3 不适宜、0.3≤F≤0.7 基本适宜、F≥0.7
    适宜。</li>
    <li>指标权重取自论文表 3，按岩质边坡、碎石边坡、平台三类微地质单元分别确定。</li>
  </ul>
  <p class="sig">
    评价人（签字）：______________________   日期：_____________
  </p>
</body>
</html>
# ==== app/templates/search.html ====
{% extends "base.html" %}
{% from "_macros.html" import icon %}
{% block title %}搜索 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<div class="card">
  <h2>搜索结果</h2>
  {% if not q %}
  <p class="muted">请输入关键词。可检索项目名称与编号、图斑名称与行政区/矿种、
    标准名称与文号。</p>
  {% else %}
  <p class="muted">
    关键词「<strong>{{ q }}</strong>」共找到
    <span class="num">{{ total }}</span> 条结果。
  </p>
  {% if not total %}
  <p class="muted">没有匹配的内容。换个词试试，或检查是否有错别字。</p>
  {% endif %}
  {% endif %}
</div>
{% if projects %}
<div class="card">
  <h2>评价项目（{{ projects|length }}）</h2>
  <table>
    <thead>
      <tr>
        <th scope="col">项目名称</th>
        <th scope="col">编号</th>
        <th scope="col">状态</th>
        <th scope="col">图斑数</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in projects %}
      <tr>
        <td style="text-align:left;">{{ p.name }}</td>
        <td class="mono">{{ p.code or "—" }}</td>
        <td><span class="tag tag--muted">{{ p.status
    }}</span></td>
        <td class="num">{{ p.patches.count() }}</td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/project/{{ p.id
    }}/patches">矿山图斑</a>
            <a class="link-btn" href="/project/{{ p.id
    }}/edit">编辑</a>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endif %}
{% if patches %}
<div class="card">
  <h2>矿山图斑（{{ patches|length }}）</h2>
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">图斑名称</th>
        <th scope="col">行政区</th>
        <th scope="col">矿种</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for m in patches %}
      <tr>
        <td>{{ m.project.name if m.project else "—" }}</td>
        <td style="text-align:left;">{{ m.name }}</td>
        <td>{{ m.admin_region or "—" }}</td>
        <td>{{ m.mineral_type or "—" }}</td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/constraint/{{ m.id
    }}">限制性判定</a>
            <a class="link-btn" href="/unit/{{ m.id
    }}">微地质单元</a>
            <a class="link-btn" href="/evaluate/{{ m.id
    }}">执行评价</a>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endif %}
{% if standards %}
<div class="card">
  <h2>标准与政策文件（{{ standards|length }}）</h2>
  <table>
    <thead>
      <tr>
        <th scope="col">类别</th>
        <th scope="col">省份</th>
        <th scope="col">名称</th>
        <th scope="col">文号</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for s in standards %}
      <tr>
        <td>{{ s.category }}</td>
        <td>{{ s.province }}</td>
        <td style="text-align:left;">{{ s.name }}</td>
        <td class="mono">{{ s.doc_number or "—" }}</td>
        <td class="nowrap">
          <span class="row-actions">
            <a class="link-btn" href="/standard/{{ s.id
    }}/rules">规则</a>
          </span>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endif %}
{% endblock %}
# ==== seed_demo.py ====
# -*- coding: utf-8 -*-
"""内置玉田县案例：把论文第 5 节案例数据写入数据库。
用于功能验证与操作演示。数据逐字取自论文表 5 与第 5 节。
运行：python seed_demo.py                  幂等，只补写缺失的演示数据，
                                              不会删除用户自建的项目与图斑
      python seed_demo.py --reset         连同标准库与指标定义一并重建
"""
import json
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
from app.models import (db, User, Project, MinePatch,
    ReferenceEcosystem,
                        MicroUnit, IndicatorData, Standard,
    StandardRule,
                        ConstraintCheck, EvaluationResult,
    ScoreDetail)
app = create_app()
# 内置演示案例的项目编号，用于区分「演示数据」与「用户自建数据」
DEMO_CODE = "DEMO-001"
# 论文表1：11项地方标准与政策文件
STANDARDS = [
    dict(category="地方标准", province="辽宁",
    name="矿山废弃地植被自然恢复技术规程",
        doc_number="DB21/T 3702—2023",
    publish_date="2023-03-30",
        implement_date="2023-04-30",
        summary="首个正式发布的矿山自然恢复地方标准；提出必要条件和充分条件",
        rules=[("植被覆盖度", ">=", "50", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None),
               ("与参照系差距", "<", "20", "充分条件", "%")]),
    dict(category="地方标准", province="湖北",
    name="废弃露天矿山生态环境自然恢复标准",
        doc_number="DB42/T 2421—2025",
    publish_date="2025-08-20",
        implement_date="2025-10-20",
        summary="结合矿山生态需求进行自然恢复适宜性分级判断",
        rules=[("自然恢复适宜性分级", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="内蒙古",
    name="历史遗留废弃矿山自然恢复判定规范",
        doc_number="DB15/T 4280—2026",
    publish_date="2026-01-30",
        implement_date="2026-03-01",
        summary="调查内容包含参照生态系统调查",
        rules=[("地质灾害发育程度", "<=", "弱发育", "必要条件", None),
               ("地形地貌与周边协调", "==", "是", "必要条件", None),
               ("有利于植被恢复的表层土壤", "==", "是", "必要条件", None),
               ("已形成稳定植被覆盖", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="江西",
    name="废弃矿山自然恢复技术规范",
        doc_number="DB36/T 2221—2026",
    publish_date="2026-02-02",
        implement_date="2026-08-01",
        summary="对参照生态系统进行了要求；提出自然恢复适宜性指数",
        rules=[("参照生态系统调查", "==", "是", "必要条件", None),
               ("自然恢复适宜性指数", ">=", "阈值", "充分条件", None)]),
    dict(category="地方标准", province="宁夏", name="矿山自然恢复认定规程",
        doc_number=None, publish_date="2026-06-18",
    implement_date=None,
        summary="分区差异化：三个一级生态修复分区及22个二级分区",
        rules=[("无地质灾害风险", "==", "是", "必要条件", None),
               ("植被覆盖度趋势", "==", "稳定或正增长", "必要条件", None),
               ("与参照系差距", "<", "20", "充分条件", "%")]),
    dict(category="政策文件", province="安徽",
        name="安徽省自然资源厅关于规范废弃露天矿山采取自然恢复方式进行治理的通知",
        doc_number="皖自然资修函[2020]34号",
    publish_date="2020-03-03", implement_date=None,
        summary="自然恢复条件：责任主体为政府、无地质灾害、无水土污染、不在三线三边、水土条件良好",
        rules=[("责任主体", "==", "政府", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="山东",
        name="山东省自然资源厅关于进一步规范历史遗留废弃露天开采矿山采取自然恢复方式进行治理的通知",
        doc_number="鲁自然资字[2022]104号",
    publish_date="2022-08-02", implement_date=None,
        summary="条件与安徽省相同，另增加自然恢复认定标准",
        rules=[("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="湖南",
        name="关于印发《湖南省历史遗留矿山自然恢复指导意见（试行）》的通知",
        doc_number="湘自资办发[2023]58号",
    publish_date="2023-07-20", implement_date=None,
        summary="自然恢复认定标准：无地质灾害或无威胁对象、无水土污染、设备拆除、符合规划、植被覆盖度
    >75%、连续3年正向增长",
        rules=[("植被覆盖度", ">", "75", "必要条件", "%"),
               ("植被覆盖率趋势", "==", "连续3年正向增长", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("符合国土空间规划", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="新疆",
        name="自治区历史遗留废弃矿山自然恢复方式销号图斑工作指南(试行)",
        doc_number="新自然资源办函〔2024〕219号",
    publish_date="2024-12-20", implement_date=None,
        summary="荒漠区沙地/裸土地/裸岩石砾地满足无地质灾害、地形协调、无水土污染即可判定",
        rules=[("植被覆盖度", ">=", "周边30%", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("地形平缓地貌稳定", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="广西",
        name="广西壮族自治区历史遗留矿山自然恢复技术指南",
        doc_number="桂自然资办〔2024〕250号",
    publish_date="2024-12-30", implement_date=None,
        summary="除高陡边坡和裸露岩体外，水土水热条件良好、植被自然恢复趋势明显",
        rules=[("无地质灾害", "==", "是", "必要条件", None),
               ("植被自然恢复趋势", "==", "明显", "必要条件", None),
               ("生产设备拆除完毕", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="西藏",
        name="西藏自治区历史遗留废弃矿山生态自然恢复认定指导意见（试行）",
        doc_number="藏自然资发〔2025〕19号",
    publish_date="2025-09-24", implement_date=None,
        summary="含修复成本高、生态修复效益不显著等适用条件",
        rules=[("责任主体", "==", "政府", "必要条件", None),
               ("地形地貌完整与周边协调", "==", "是", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None)]),
]
CASE = {
    "name": "玉田县典型历史遗留废弃矿山案例",
    "patch_name": "玉田县孤树镇白云岩采石场",
    "area_ha": 5.6,
    "mineral_type": "白云岩",
    "units": [
        {
            "unit_type": "岩质边坡", "ratio": 0.1386,
            "data": {"coverage": 68.88, "shannon": 0.00,
                    "slope": 63.33, "aspect": "阴阳坡"},
        },
        {
            "unit_type": "碎石边坡", "ratio": 0.6295,
            "data": {"soil_ratio": 0.34, "thickness": 10,
    "ph": 9.24,
                    "organic_matter": 4.89,
                    "heavy_metals": {"Hg": 0.05, "As":
    5.17, "Ni": 12.60,
                                    "Cu": 11.70, "Cd":
    0.04, "Pb": 6.50},
                    "coverage": 87.92, "shannon": 0.60,
                    "slope": 28.75, "aspect": "阳坡"},
        },
        {
            "unit_type": "平台", "ratio": 0.2319,
            "data": {"soil_ratio": 0.56, "thickness": 30,
    "ph": 8.72,
                    "organic_matter": 10.92,
                    "heavy_metals": {"Hg": 0.02, "As":
    3.13, "Ni": 12.33,
                                    "Cu": 11.14, "Cd":
    0.09, "Pb": 9.98},
                    "coverage": 58.42, "shannon": 0.72},
        },
    ],
}
def clean_case(reset=False):
    """清理数据。
    默认只清理内置演示案例（code = DEMO-001）自身的数据，**不触碰
    用户自己建立的项目与图斑**。加 --reset 时才额外清空标准库与指标定义，
    以便把演示环境恢复到初始状态。
    注意：不要在此处执行 Project.query.delete()，否则用户新建的项目
    会在每次启动时被删除。
    """
    from app.models import EvaluationResult, ScoreDetail,
    ConstraintCheck
    demo_projects = Project.query.filter_by(code=DEMO_CODE)
    all()
    demo_ids = [p.id for p in demo_projects]
    patch_ids = []
    if demo_ids:
        patch_ids = [p.id for p in MinePatch.query.filter(
            MinePatch.project_id.in_(demo_ids)).all()]
    if patch_ids:
        unit_ids = [u.id for u in MicroUnit.query.filter(
            MicroUnit.patch_id.in_(patch_ids)).all()]
        if unit_ids:
            IndicatorData.query.filter(
                IndicatorData.unit_id.in_(unit_ids)).delete(
                synchronize_session=False)
        result_ids = [r.id for r in EvaluationResult.query.f
    ilter(
            EvaluationResult.patch_id.in_(patch_ids)).all()]
        if result_ids:
            ScoreDetail.query.filter(
                ScoreDetail.result_id.in_(result_ids)).delet
    e(
                synchronize_session=False)
        EvaluationResult.query.filter(
            EvaluationResult.patch_id.in_(patch_ids)).delete
    (
            synchronize_session=False)
        ConstraintCheck.query.filter(
            ConstraintCheck.patch_id.in_(patch_ids)).delete(
            synchronize_session=False)
        ReferenceEcosystem.query.filter(
            ReferenceEcosystem.patch_id.in_(patch_ids)).dele
    te(
            synchronize_session=False)
        MicroUnit.query.filter(MicroUnit.patch_id.in_(patch_
    ids)).delete(
            synchronize_session=False)
        MinePatch.query.filter(MinePatch.id.in_(patch_ids)).
    delete(
            synchronize_session=False)
    if demo_ids:
        Project.query.filter(Project.id.in_(demo_ids)).delet
    e(
            synchronize_session=False)
    if reset:
        StandardRule.query.delete()
        Standard.query.delete()
    # 兜底：清理指向已不存在评价记录的孤儿得分明细，
    # 避免长期运行后 score_details 表堆积无效数据。
    _purge_orphan_details()
    db.session.commit()
    return len(demo_ids)
def demo_patch():
    """取内置演示案例的图斑。
    库中可能存在用户自建项目或验证算例（编号 VERIFY-），
    故不能简单用 MinePatch.query.first()，否则取到的对象不确定。
    没有演示案例时才退回第一个图斑。
    """
    proj = Project.query.filter_by(code=DEMO_CODE).first()
    if proj is not None:
        m = MinePatch.query.filter_by(project_id=proj.id).fi
    rst()
        if m is not None:
            return m
    return MinePatch.query.first()
def _purge_orphan_details():
    """清理指向已不存在评价记录的孤儿得分明细。
    长期运行后 score_details 表可能堆积无效明细，启动时顺带清一次。
    """
    live_ids = [r.id for r in EvaluationResult.query.all()]
    if live_ids:
        ScoreDetail.query.filter(~ScoreDetail.result_id.in_(
    live_ids)).delete(
            synchronize_session=False)
    else:
        ScoreDetail.query.delete()
    db.session.commit()
def seed(reset=False):
    with app.app_context():
        db.create_all()      # 全新环境下先建表，保证本脚本可独立运行
        _purge_orphan_details()
        admin = User.query.filter_by(username="admin").first
    ()
        if not admin:
            from werkzeug.security import
    generate_password_hash
            admin = User(username="admin",
                         password_hash=generate_password_has
    h("admin123"),
                         role="admin")
            db.session.add(admin)
            db.session.commit()
        # 幂等：演示案例已存在就不重复写入，更不清空用户自己的项目
        existing = Project.query.filter_by(code=DEMO_CODE).f
    irst()
        if existing and not reset:
            print("演示案例已存在，跳过写入（用户自建项目不受影响）")
            _report()
            _seed_reference_data(reset=False)
            return
        clean_case(reset=reset)
        project = Project(name=CASE["name"], code=DEMO_CODE,
                          description="论文第 5 节典型案例",
    created_by=admin.id)
        db.session.add(project)
        db.session.flush()
        patch = MinePatch(project_id=project.id,
name=CASE["patch_name"],
                      admin_region="河北省唐山市玉田县",
                      area_ha=CASE["area_ha"],
                      mineral_type=CASE["mineral_type"],
                      mining_method="露天开采",
stop_years=None,
                      responsible_entity_missing=True)
    db.session.add(patch)
    db.session.flush()
    ref = ReferenceEcosystem(patch_id=patch.id,
                             coverage_benchmark=75.0,
shannon_benchmark=0.7)
    db.session.add(ref)
    for u in CASE["units"]:
        unit = MicroUnit(patch_id=patch.id,
unit_type=u["unit_type"],
                         ratio=u["ratio"], area=None)
        db.session.add(unit)
        db.session.flush()
        d = u["data"]
        ind = IndicatorData(
            unit_id=unit.id,
            soil_ratio=d.get("soil_ratio"),
            thickness=d.get("thickness"),
            ph=d.get("ph"),
            organic_matter=d.get("organic_matter"),
            heavy_metals=json.dumps(d["heavy_metals"],
ensure_ascii=False)
                          if d.get("heavy_metals") else
None,
            coverage=d.get("coverage"),
            shannon=d.get("shannon"),
            slope=d.get("slope"),
            aspect=d.get("aspect"),
        )
        db.session.add(ind)
    # 限制性条件判定记录：演示案例按论文第5节描述属「无地质灾害隐患点」、
    # 具备自然恢复条件，故五项全部通过。缺此记录会使后续评价被流程门禁拦住。
    db.session.add(ConstraintCheck(
        patch_id=patch.id,
        policy_pass=True, location_pass=True,
pollution_pass=True,
        geo_pass=True, farmland_pass=True,
        passed=True, reason="全部通过"))
    db.session.commit()
    print("已写入案例：", CASE["name"])
    print("  图斑：", CASE["patch_name"])
    print("  微地质单元：岩质边坡 13.86% / 碎石边坡 62.95% / 平台 
23.19%")
    print("  管理员账号：admin / admin123")
```

```
        _seed_reference_data(reset=reset)
        _report()
def _seed_reference_data(reset=False):
    """写入标准库与指标定义。
    参考资料只在库为空（或显式 --reset）时写入，避免覆盖用户自行维护
    的标准条目与指标定义。
    """
    if reset or Standard.query.count() == 0:
        seed_standards()
    from app.services.indicator_service import
    seed_indicators
    n = seed_indicators()
    print("指标定义：%d 项" % n)
def _report():
    total_p = Project.query.count()
    total_patch = MinePatch.query.count()
    print()
    print("当前库内：项目 %d 个、图斑 %d 个" % (total_p, total_patch))
    if total_p > total_patch:
        print("  提示：项目是归档单位，评价针对的是图斑。")
        print("        新项目需先建立图斑，才会出现在后续各模块中。")
def seed_standards():
    """写入论文表 1 的 11 项标准与政策文件及其结构化规则。"""
    StandardRule.query.delete()
    Standard.query.delete()
    db.session.commit()
    for s in STANDARDS:
        std = Standard(category=s["category"],
    province=s["province"],
                       name=s["name"],
    doc_number=s["doc_number"],
                       publish_date=s["publish_date"],
                       implement_date=s["implement_date"],
                       summary=s["summary"])
        db.session.add(std)
        db.session.flush()
        for item, op, thr, ctype, note in s["rules"]:
            db.session.add(StandardRule(standard_id=std.id,
    item=item,
                                        operator=op,
    threshold=thr,

    condition_type=ctype, note=note))
    db.session.commit()
    print("已写入标准/政策文件：%d 项，规则 %d 条"
          % (len(STANDARDS), sum(len(s["rules"]) for s in
    STANDARDS)))
if __name__ == "__main__":
    # --reset：把演示环境恢复到初始状态（清空标准库与指标定义后重建）。
    # 无论是否 --reset，都不会删除用户自建的项目与图斑。
    seed(reset="--reset" in sys.argv[1:])
# ==== 测试/test_ahp.py ====
# -*- coding: utf-8 -*-
"""地基自检：验证 AHP 引擎与应用可启动。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import numpy as np
import os
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app.services import ahp_service as ahp
LINE = "=" * 66
# ----------------------------------------------------------
    -----------------
# 1) 完全一致矩阵 → 权重应精确还原，CR=0
# ----------------------------------------------------------
    -----------------
print(LINE)
print("测试 1  完全一致判断矩阵的权重还原与一致性")
print(LINE)
# 由权重构造完全一致矩阵：a_ij = w_i / w_j
w_true = np.array([0.44, 0.31, 0.25])
A = np.outer(w_true, 1.0 / w_true)
r = ahp.weights_and_consistency(A)
print("  判断矩阵 A:")
for row in A:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print()
print("  计算权重 :", "  ".join(f"{v:.4f}" for v in
    r["weights"]))
print("  真实权重 :", "  ".join(f"{v:.4f}" for v in w_true))
print("  λmax = %.6f   CI = %.3e   CR = %.3e   一致: %s"
      % (r["lambda_max"], r["ci"], r["cr"],
    r["consistent"]))
assert np.allclose(r["weights"], w_true, atol=1e-6),
    "权重未精确还原"
assert r["consistent"], "完全一致矩阵应通过检验"
# ----------------------------------------------------------
    -----------------
# 2) 上三角输入 → 完整矩阵 → 一致性定位
# ----------------------------------------------------------
    -----------------
print()
print(LINE)
print("测试 2  上三角录入与最不一致项定位")
print(LINE)
```

```
# 一个不太一致的 3 阶矩阵：上三角 [2, 5, 3]（a12=2, a13=5, a23=3）
M = ahp.matrix_from_upper([2, 5, 3])
r2 = ahp.weights_and_consistency(M)
bad = ahp.find_most_inconsistent(M)
print("  由 [a12=2, a13=5, a23=3] 构造矩阵：")
for row in M:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print("  权重：", "  ".join(f"{v:.4f}" for v in r2["weights"]),
      "  CR = %.3f  一致: %s" % (r2["cr"], r2["consistent"]))
print("  最不一致比较项: 第(%d,%d) 项，当前 %.3f，理想 %.3f"
      % (bad["i"] + 1, bad["j"] + 1, bad["value"],
    bad["expected"]))
# ---------------------------------------------------------
    -----------------
# 3) 多专家聚合（几何平均）
# ---------------------------------------------------------
    -----------------
print()
print(LINE)
print("测试 3  多专家判断矩阵聚合（几何平均）")
print(LINE)
m1 = ahp.matrix_from_upper([2, 4, 3])
m2 = ahp.matrix_from_upper([3, 5, 2])
agg = ahp.aggregate_matrices([m1, m2])
print("  专家1 上三角 [2,4,3]，专家2 上三角 [3,5,2]")
print("  聚合后上三角: a12=%.4f a13=%.4f a23=%.4f"
      % (agg[0, 1], agg[0, 2], agg[1, 2]))
assert abs(agg[0, 1] - np.sqrt(2 * 3)) < 1e-9, "几何平均错误"
# ---------------------------------------------------------
    -----------------
# 4) 论文表 3 权重可还原性
# ---------------------------------------------------------
    -----------------
print()
print(LINE)
print("测试 4  论文表 3 碎石边坡一级权重 (0.44, 0.31, 0.25)")
print(LINE)
j = ahp.judge_from_weights([0.44, 0.31, 0.25])
print("  λmax = %.6f  CR = %.3e  一致: %s" %
    (j["lambda_max"], j["cr"], j["consistent"]))
assert j["consistent"], "论文权重应能作为完全一致矩阵主特征向量"
# ---------------------------------------------------------
    -----------------
# 5) 应用可启动 + 建表
# ---------------------------------------------------------
    -----------------
print()
print(LINE)
print("测试 5  Flask 应用启动与建表")
print(LINE)
```

```
from app import create_app
from app.models import db
app = create_app()
with app.app_context():
    db.create_all()
    tables = sorted(db.metadata.tables.keys())
print("已创建 %d 张表：" % len(tables))
for t in tables:
    print("  -", t)
print()
print(LINE)
print("全部测试通过 √")
print(LINE)
# ==== 测试/test_evaluate.py ====
# -*- coding: utf-8 -*-
"""评价引擎自检：用论文玉田县案例跑通完整计算链。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import os
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app.services import evaluation_service as ev
from app.services import standardization as std
LINE = "=" * 70
# ---------------------------------------------------------------------------
# 论文第 5 节案例数据（表 5，逐字抄录）
# ---------------------------------------------------------------------------
# 三类单元面积占比：岩质边坡 13.86%、碎石边坡 62.95%、平台 23.19%
# 注：岩质边坡无土壤指标、平台无地形指标，对应"不适用"。
case = [
    {
        "unit_type": "岩质边坡",
        "ratio": 0.1386,
        "data": {
            "coverage": 68.88, "shannon": 0.00,
            "slope": 63.33, "aspect": "阴阳坡",
        },
    },
    {
        "unit_type": "碎石边坡",
        "ratio": 0.6295,
        "data": {
            "soil_ratio": 0.34, "thickness": 10, "ph": 9.24, "organic_matter": 4.89,
            "heavy_metals": {"Hg": 0.05, "As": 5.17, "Ni":
    12.60,
                             "Cu": 11.70, "Cd": 0.04, "Pb":
    6.50},
            "coverage": 87.92, "shannon": 0.60,
            "slope": 28.75, "aspect": "阳坡",
        },
    },
    {
        "unit_type": "平台",
        "ratio": 0.2319,
        "data": {
            "soil_ratio": 0.56, "thickness": 30, "ph":
    8.72, "organic_matter": 10.92,
            "heavy_metals": {"Hg": 0.02, "As": 3.13, "Ni":
    12.33,
                             "Cu": 11.14, "Cd": 0.09, "Pb":
    9.98},
            "coverage": 58.42, "shannon": 0.72,
        },
    },
]
# ---------------------------------------------------------
    -----------------
# 逐单元输出标准化与得分
# ---------------------------------------------------------
    -----------------
print(LINE)
print("玉田县案例 · 评价引擎完整计算")
print(LINE)
for u in case:
    r = ev.score_unit(u["unit_type"], u["data"])
    print()
    print(" 【%s】 (面积占比 %.2f%%)" % (u["unit_type"],
    u["ratio"] * 100))
    print(" " + "-" * 60)
    for l1, score in r["l1_scores"].items():
        print("   %s 得分 = %.4f" % (l1, score))
    print("   单元评价因子 F' = %.4f" % r["unit_score"])
print()
print(LINE)
print("面积加权合成(式(9))")
print(LINE)
res = ev.score_mine(case)
total = res["final_score"]
print()
for ur in res["unit_results"]:
    print(" %s %.2f%% × %.4f = %.4f"
          % (ur["unit_type"], ur["ratio"] * 100,
    ur["unit_score"], ur["weighted"]))
```

```
print("  " + "-" * 60)
print("  最终评价因子 F = %.4f" % total)
print()
print("  适宜性等级：%s" % res["grade"])
print("  判定依据  ：%s" % res["grade_info"]["meaning"])
print("  推荐措施  ：%s" % res["grade_info"]["measure"])
print()
print(LINE)
print("结论")
print(LINE)
print("  - 评价引擎按论文式(8)(9) 与表 3 权重完整跑通")
print("  - 坡度标准化采用论文 25° 分界的线性折减默认值（见 standardization.py）")
print("  - 最终 F ≈ %.2f，判定为「%s」，与论文「最终适宜自然恢复」的结论一致"
      % (total, res["grade"]))
print("  - 论文表 5 所载最终得分为 0.76；差异集中在坡度、坡向的标准化口径，")
print("    该口径在 standardization.std_slope / aspect_score
    中集中定义、可配置")
print(LINE)
# ==== 测试/test_web_ahp.py ====
# -*- coding: utf-8 -*-
"""AHP 网页界面自检：走通选择→录入→计算 全流程。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import os
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
# 测试配置：关闭 CSRF 以简化测试客户端（真实运行仍开启 CSRF）
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///:memory:"
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 建立表并创建测试用户，登录后再访问业务页面
with app.app_context():
    from app.models import db, User
    from werkzeug.security import generate_password_hash
    db.create_all()
    if not User.query.filter_by(username="admin").first():
        db.session.add(User(username="admin",
                            password_hash=generate_password_
    hash("admin123"),
                            role="admin"))
        db.session.commit()
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  首页与 AHP 入口")
print(LINE)
r = client.get("/")
print("  首页状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
r = client.get("/ahp/")
print("  AHP 入口状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
assert "微地质单元类型" in r.get_data(as_text=True)
print()
print(LINE)
print("测试 2  选择碎石边坡一级指标 → 跳转矩阵页")
print(LINE)
r = client.post("/ahp/", data={
    "unit_type": "碎石边坡", "level": "1", "submit": "生成判断矩阵",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转后页面含「判断矩阵录入」：", "判断矩阵录入" in body)
print("  含三个一级指标（土壤/植被/地形）：", all(n in body for n in
    ("土壤指标", "植被指标", "地形指标")))
assert "判断矩阵录入" in body
assert all(n in body for n in ("土壤指标", "植被指标", "地形指标"))
print()
print(LINE)
print("测试 3  提交论文表 3 一致矩阵 → 还原权重")
print(LINE)
# 碎石边坡一级权重 (0.44, 0.31, 0.25) 对应的完全一致矩阵上三角：
# a12 = 0.44/0.31 = 1.4194, a13 = 0.44/0.25 = 1.76, a23 =
    0.31/0.25 = 1.24
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  结果页含「计算结果」：", "计算结果" in body)
print("  含权重 0.44:", "0.44" in body, " 含 0.31:", "0.31" in
    body, " 含 0.25:", "0.25" in body)
print("  含一致性检验通过：", "通过" in body)
assert "计算结果" in body
assert "0.44" in body and "0.31" in body and "0.25" in body
print()
print(LINE)
print("测试 4  不一致矩阵应提示不通过并定位")
print(LINE)
# 明显不一致：a12=9, a13=1, a23=9
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "9", "m_0_2": "1", "m_1_2": "9",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  含「不通过」:", "不通过" in body)
print("  含「建议优先调整」:", "建议优先调整" in body)
assert "不通过" in body
assert "建议优先调整" in body
print()
print(LINE)
print("网页界面全部测试通过 √")
print(LINE)
# ==== 测试/test_web_modules.py ====
# -*- coding: utf-8 -*-
"""模块自检：登录、项目档案、微地质单元、标准库、限制性判定。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
print(LINE)
print("测试 1  未登录应被重定向到登录页")
print(LINE)
r = client.get("/project/")
print("  /project/ 状态码:", r.status_code, " 重定向到:",
    r.headers.get("Location"))
assert r.status_code == 302 and "login" in
    r.headers.get("Location", "")
print()
print(LINE)
print("测试 2  登录")
print(LINE)
r = client.post("/login", data={"username": "admin",
    "password": "admin123"},
                follow_redirects=True)
body = r.get_data(as_text=True)
```

```
# 工作台：页首横幅 + 评价流程分区 + 数据概览
print("  登录后含「评价流程」：", "评价流程" in body)
print("  含账户 admin:", "admin" in body)
print("  含页首横幅：", 'class="hero"' in body)
print("  含数据概览：", 'class="stat-card"' in body)
assert "评价流程" in body and 'class="hero"' in body
print()
print(LINE)
print("测试 3  项目与矿山档案")
print(LINE)
r = client.get("/project/")
body = r.get_data(as_text=True)
print("  项目列表状态码：", r.status_code)
print("  含案例项目：", "玉田县典型历史遗留废弃矿山案例" in body)
assert "玉田县典型历史遗留废弃矿山案例" in body
# 新建项目
client.post("/project/add", data={"name": "测试项目A", "code":
    "T-001"})
r = client.get("/project/")
print("  新建项目后含「测试项目A」：", "测试项目A" in r.get_data(as_text=True
    ))
assert "测试项目A" in r.get_data(as_text=True)
# 清理测试数据，避免污染演示库
with app.app_context():
    from app.models import db, Project
    for p in Project.query.filter_by(name="测试项目A").all():
        db.session.delete(p)
    db.session.commit()
print("  已清理测试项目")
print()
print(LINE)
print("测试 4  微地质单元（面积占比自动计算）")
print(LINE)
from app.models import MinePatch
with app.app_context():
    patch = MinePatch.query.filter_by(name="玉田县孤树镇白云岩采石场").f
    irst()
    pid = patch.id
r = client.get("/unit/%d" % pid)
body = r.get_data(as_text=True)
print("  单元列表含三类单元：", all(t in body for t in ("岩质边坡",
    "碎石边坡", "平台")))
print("  含占比合计：", "占比合计" in body)
assert "岩质边坡" in body and "占比合计" in body
print()
print(LINE)
print("测试 5  标准知识库与省际口径比对")
print(LINE)
r = client.get("/standard/")
body = r.get_data(as_text=True)
```

```
print("  含辽宁/湖南/新疆标准：", all(p in body for p in ("辽宁", "湖南",
    "新疆")))
assert "辽宁" in body
r = client.get("/standard/compare")
body = r.get_data(as_text=True)
print("  口径比对页含「植被覆盖度」分组：", "植被覆盖度" in body)
print("  含多省份并列：", "湖南" in body and "新疆" in body)
assert "植被覆盖度" in body
print()
print(LINE)
print("测试 6  限制性指标判定（一票否决）")
print(LINE)
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  含五项条件：", all(x in body for x in ("政策约束", "区位约束",
    "土壤污染", "地质灾害", "耕地属性")))
assert all(x in body for x in ("政策约束", "区位约束", "土壤污染",
    "地质灾害", "耕地属性"))
# 全部通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  全通过后显示「通过，可进入定量评价」：", "可进入定量评价" in body)
assert "可进入定量评价" in body
# 一项不通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "0",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  一项不通过后显示「不通过」：", "不通过" in body)
print("  含失败项「区位约束」：", "区位约束" in body)
assert "不通过" in body
# 恢复为全部通过，避免把演示数据留在「不通过」状态影响后续步骤
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
print("  已恢复为全部通过：", "可进入定量评价" in r.get_data(as_text=True))
print()
print(LINE)
print("全部测试通过 √")
print(LINE)
# ==== 测试/test_web_evaluate.py ====
# -*- coding: utf-8 -*-
```

```
"""端到端自检：录数据 → 执行评价 → 查看结果。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
# 与真实配置共用同一个 app.db，但关闭 CSRF 便于测试客户端
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  指标数据列表")
print(LINE)
r = client.get("/indicator/")
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含案例图斑：", "玉田县孤树镇白云岩采石场" in body)
print("  含三个单元：", all(n in body for n in ("岩质边坡", "碎石边坡",
    "平台")))
assert r.status_code == 200
assert "岩质边坡" in body and "碎石边坡" in body and "平台" in body
print()
print(LINE)
print("测试 2  指标数据录入页（按单元类型显示字段）")
print(LINE)
from app.models import MicroUnit, MinePatch
with app.app_context():
    unit_rock = MicroUnit.query.filter_by(unit_type="岩质边坡")
    first()
    unit_gravel = MicroUnit.query.filter_by(unit_type="碎石边坡"
    ).first()
r = client.get("/indicator/%d" % unit_rock.id)
b_rock = r.get_data(as_text=True)
print("  岩质边坡页：无「土石比」字段：", "土石比" not in b_rock, "（应 True）")
print("  岩质边坡页：含「植被覆盖度」：", "植被覆盖度" in b_rock)
```

```
assert "土石比" not in b_rock and "植被覆盖度" in b_rock
r = client.get("/indicator/%d" % unit_gravel.id)
b_gravel = r.get_data(as_text=True)
print("  碎石边坡页：含「土石比」字段：", "土石比" in b_gravel)
print("  碎石边坡页：含 8 种重金属：", all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn")))
assert "土石比" in b_gravel and all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn"))
print()
print(LINE)
print("测试 3  执行评价")
print(LINE)
with app.app_context():
    import seed_demo
    patch = seed_demo.demo_patch()
r = client.post("/evaluate/%d" % patch.id, data={},
    follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转至结果页：", "适宜性评价结果" in body)
print("  含最终评价因子 F：", "最终评价因子 F" in body)
print("  含等级「适宜」：", "适宜" in body)
print("  含推荐措施「自然恢复」：", "自然恢复" in body)
assert "最终评价因子 F" in body and "自然恢复" in body
print()
print(LINE)
print("测试 4  结果持久化")
print(LINE)
from app.models import EvaluationResult
with app.app_context():
    res = EvaluationResult.query.order_by(EvaluationResult.i
    d.desc()).first()
    print("  最终得分 F =", res.final_score)
    print("  等级 =", res.grade)
    assert res.grade == "适宜"
    assert 0.7 <= res.final_score < 1.0
print()
print(LINE)
print("端到端全部测试通过 √")
print(LINE)
# ==== 测试/test_web_report.py ====
# -*- coding: utf-8 -*-
"""报告输出自检：得分明细页 + Word 导出。"""
import os
import sys
import io
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
```

```
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
from app.models import MinePatch, EvaluationResult
with app.app_context():
    import seed_demo
    patch = seed_demo.demo_patch()
    pid = patch.id
# 报告取自已保存的评价记录，因此先按网页路径真实执行一次评价
from app.views.evaluate import load_patch_input,
    save_evaluation
with app.app_context():
    units = load_patch_input(pid)
    rec = save_evaluation(pid, units)
    rid = rec.id
    print("前置：已执行评价并保存记录 id=%d，F=%.4f（%s）"
          % (rid, rec.final_score, rec.grade))
    from app.models import ScoreDetail
    n_detail = ScoreDetail.query.filter_by(result_id=rid).co
    unt()
    print("      逐层得分明细已落库：%d 条" % n_detail)
print()
print(LINE)
print("测试 1  报告列表")
print(LINE)
r = client.get("/report/")
print("  状态码：", r.status_code)
body = r.get_data(as_text=True)
print("  含「所属项目」列：", "所属项目" in body)
assert r.status_code == 200
print()
print(LINE)
print("测试 2  得分明细页")
print(LINE)
r = client.get("/report/%d" % pid)
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含「得分明细」:", "得分明细" in body)
print("  含「最终评价因子 F」:", "最终评价因子 F" in body)
print("  含二级指标明细（植被覆盖度/坡度/坡向）:", all(n in body for n in
    ("植被覆盖度", "坡度", "坡向")))
print("  含权重/标准化值列:", "标准化值" in body and "贡献" in body)
assert r.status_code == 200
assert "植被覆盖度" in body and "标准化值" in body
print()
print(LINE)
print("测试 3  Word 报告导出")
print(LINE)
r = client.get("/report/%d/word" % pid)
print("  状态码:", r.status_code)
print("  Content-Type:", r.headers.get("Content-Type",
    "")[:60])
data = r.data
print("  文件大小:", len(data), "字节")
print("  是否为有效 docx（PK 头）:", data[:2] == b"PK")
assert r.status_code == 200 and data[:2] == b"PK"
# 用 python-docx 打开验证内容
import docx
d = docx.Document(io.BytesIO(data))
text = "\n".join(p.text for p in d.paragraphs)
print("  含报告标题:", "自然恢复适宜性评价报告" in text)
print("  含评价结论章节:", "评价结论" in text)
print("  含得分明细章节:", "得分明细" in text)
print("  段落数:", len(d.paragraphs), " 表格数:", len(d.tables))
assert "自然恢复适宜性评价报告" in text and len(d.tables) >= 3
print()
print(LINE)
print("报告输出全部测试通过 √")
print(LINE)
# ==== 测试/test_ui_fixes.py ====
# -*- coding: utf-8 -*-
"""界面修复自检：「记住我」与各模块删除功能。"""
import os
import re
import sys
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
sys.stdout.reconfigure(encoding="utf-8")
LINE = "=" * 70
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
    REMEMBER_COOKIE_DURATION = __import__("datetime").timedelta(days=30)
app = create_app(TestConfig)
client = app.test_client()
# ------------------------------------------------------------------------------
print(LINE)
print("一、「记住我」")
print(LINE)
html = client.get("/login").get_data(as_text=True)
# 1) 复选框必须有可点击关联的 label（这是原先失效的根因）
#    只判断「复选框与文字是否落在同一个 <label> 内」，不绑定具体类名 ——
#    否则换个样式类就会误报，而真正的回归点其实与类名无关。
labels = re.findall(r"<label[^>]*>(.*?)</label>", html,
    re.S)
wrapped = any(('name="remember"' in b and "记住我" in b) for b
    in labels)
print("  %s 「记住我」文字与复选框包在同一 label 内" % ("[符合]" if wrapped
    else "[不符合]"))
assert wrapped, "复选框未与文字关联，点击文字不会勾选"
has_for = 'for="remember"' in html
print("  %s label 带 for 属性指向复选框" % ("[符合]" if has_for else
    "[不符合]"))
# 2) 勾选后应下发带过期时间的 remember_token
c = app.test_client()
r = c.post("/login", data={"username": "admin", "password":
    "admin123",
                            "remember": "y"},
    follow_redirects=False)
cookies = r.headers.getlist("Set-Cookie")
rt = [h for h in cookies if h.startswith("remember_token=")]
print("  %s 勾选后下发 remember_token" % ("[符合]" if rt else
    "[不符合]"))
assert rt, "未下发 remember_token"
exp = re.search(r"Expires=([^;]+)", rt[0])
print("  %s 该 cookie 带过期时间：%s"
      % ("[符合]" if exp else "[不符合]", exp.group(1) if exp
    else "无（会话 cookie）"))
assert exp, "remember_token 无过期时间，关浏览器即失效"
# 3) 不勾选时不应下发 remember_token
c2 = app.test_client()
r2 = c2.post("/login", data={"username": "admin",
    "password": "admin123"},
             follow_redirects=False)
rt2 = [h for h in r2.headers.getlist("Set-Cookie") if
    h.startswith("remember_token=")]
```

```
print("  %s 未勾选时不下发 remember_token" % ("[符合]" if not rt2
    else "[不符合]"))
assert not rt2
# 4) 携带 remember_token 的新会话应自动登录
c3 = app.test_client()
c3.set_cookie("remember_token",
               rt[0].split(";")[0].split("=", 1)[1],
    domain="localhost")
r3 = c3.get("/", follow_redirects=False)
print("  %s 仅凭 remember_token 访问首页可自动登录（状态码 %d）"
      % ("[符合]" if r3.status_code == 200 else "[不符合]",
    r3.status_code))
assert r3.status_code == 200
# ------------------------------------------------------------
    ------------------
print()
print(LINE)
print("二、删除功能")
print(LINE)
client.post("/login", data={"username": "admin",
    "password": "admin123"})
# 项目：列表页须有删除按钮
html = client.get("/project/").get_data(as_text=True)
has_del = "/delete" in html and "btn--danger" in html
print("  %s 项目列表含删除按钮" % ("[符合]" if has_del else "[不符合]"))
assert has_del, "项目列表缺少删除按钮"
print("  %s 删除前有二次确认" % ("[符合]" if "confirm(" in html else
    "[不符合]"))
assert "confirm(" in html
# 图斑列表：地址用的是项目主键，不是图斑主键
with app.app_context():
    from app.models import MinePatch, Project
    import seed_demo
    patch = seed_demo.demo_patch()
    patch_id = patch.id
    project_id = patch.project_id
html = client.get("/project/%d/patches" %
    project_id).get_data(as_text=True)
print("  %s 图斑列表含删除按钮" % ("[符合]" if "/project/patch/" in
    html and "btn--danger" in html else "[不符合]"))
assert "/project/patch/" in html and "btn--danger" in html
# 微地质单元
html = client.get("/unit/%d" % patch_id).get_data(as_text=Tr
    ue)
print("  %s 微单元列表含删除按钮" % ("[符合]" if "/unit/delete/" in
    html else "[不符合]"))
assert "/unit/delete/" in html
print("  %s 微单元编辑为独立页面（表单不再嵌在表格行内）"
      % ("[符合]" if "/unit/edit/" in html else "[不符合]"))
# 标准
```

```
html = client.get("/standard/").get_data(as_text=True)
print("  %s 标准列表含删除按钮" % ("[符合]" if "btn--danger" in html
    else "[不符合]"))
assert "btn--danger" in html
# 实际执行一次删除：新建项目 → 删除 → 确认消失
client.post("/project/add", data={"name": "待删除测试项目",
    "code": "DEL-1"})
with app.app_context():
    from app.models import db, Project
    tmp = Project.query.filter_by(name="待删除测试项目").first()
    tmp_id = tmp.id
html = client.get("/project/").get_data(as_text=True)
print("  %s 新建项目成功" % ("[符合]" if "待删除测试项目" in html else
    "[不符合]"))
r = client.post("/project/%d/delete" % tmp_id,
    follow_redirects=True)
html = r.get_data(as_text=True)
gone = "待删除测试项目" not in html
print("  %s 删除项目后列表中不再出现" % ("[符合]" if gone else "[不符合]"))
assert gone
print("  %s 有删除成功提示" % ("[符合]" if "已删除" in html else
    "[不符合]"))
print()
print(LINE)
print("界面修复自检全部通过 √")
print(LINE)
# ==== 测试/test_new_features.py ====
# -*- coding: utf-8 -*-
"""补齐模块自检：操作日志与用户维护、遥感影像、指标与多专家矩阵、
批量导入、图表与 PDF 导出。
对应计划书功能清单与系统实现的差异核对结论。
"""
import io
import json
import os
import re
import sys
sys.stdout.reconfigure(encoding="utf-8")
import fitz
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, User, Project, MinePatch,
    MicroUnit, IndicatorData,
                        Indicator, ExpertMatrix,
    OperationLog)
```

```
from app.utils.schema import ensure_schema
from app.utils import browser, charts
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
    UPLOAD_FOLDER = os.path.join(BASE, "app", "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
app = create_app(TestConfig)
with app.app_context():
    db.create_all()
    ensure_schema(db)
    ExpertMatrix.query.delete()
    db.session.commit()
    # 固定取内置演示案例的图斑，避免受库中用户自建项目 / 验证算例的影响
    import seed_demo
    patch_id = seed_demo.demo_patch().id
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
def ok(flag, label, extra=""):
    print("  %s %-44s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    assert flag, label
# ---------------------------------------------------------
    -----------------
print(LINE)
print("操作日志与用户维护")
print(LINE)
before = None
with app.app_context():
    before = OperationLog.query.count()
c.post("/project/add", data={"name": "日志核对项目", "code":
    "LOG-T"})
with app.app_context():
    after = OperationLog.query.count()
    last = OperationLog.query.order_by(OperationLog.id.desc(
    )).first()
ok(after == before + 1, "写操作被写入操作日志", "%d -> %d" % (before,
    after))
ok(last.action == "新建项目", "日志记录了可读动作名", last.action)
ok("project" in (last.target_type or ""), "日志记录了对象类型",
    last.target_type)
r = c.get("/profile?tab=logs")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "操作日志" in body, "操作日志分类可访问")
ok("新建项目" in body, "日志页显示操作记录")
# 账号编辑 / 删除（职务只有管理员与操作员两种）
with app.app_context():
    u = User(username="tmp_user", role="operator",
             password_hash=User.query.first().password_hash)
    db.session.add(u); db.session.commit()
    uid = u.id
r = c.get("/users/%d/edit" % uid)
ok(r.status_code == 200, "账号编辑页可访问")
# 保护校验要趁系统里只有一名管理员时做：
# 先降级唯一的 admin 应被拒绝，再把临时账号提为管理员看修改是否生效
with app.app_context():
    admin_id = User.query.filter_by(role="admin").first().id
c.post("/users/%d/edit" % admin_id,
       data={"role": "operator", "active": "1"})
with app.app_context():
    ok(db.session.get(User, admin_id).role == "admin",
    "拒绝把最后一个管理员降级")
c.post("/users/%d/edit" % uid, data={"role": "admin",
    "active": "1"})
with app.app_context():
    ok(db.session.get(User, uid).role == "admin",
    "职务已修改并保存")
c.post("/users/%d/delete" % uid, follow_redirects=True)
with app.app_context():
    ok(db.session.get(User, uid) is None, "账号删除生效")
# 清理日志核对用的项目
with app.app_context():
    for p in Project.query.filter_by(name="日志核对项目").all():
        db.session.delete(p)
    db.session.commit()
# ---------------------------------------------------------
    -----------------
print()
print(LINE)
print("遥感影像挂接")
print(LINE)
doc = fitz.open()
pg = doc.new_page(width=320, height=200)
pg.insert_text((20, 60), "RS image", fontname="china-s",
    fontsize=16)
png = doc[0].get_pixmap(dpi=100).tobytes("png")
doc.close()
r = c.post("/project/patch/%d/image" % patch_id,
           data={"image": (io.BytesIO(png), "影像.png"),
                 "image_note": "2025年6月无人机正射影像"},
           content_type="multipart/form-data",
    follow_redirects=True)
body = r.get_data(as_text=True)
ok("已挂接" in body and "<img" in body, "影像上传并显示")
with app.app_context():
    rel = db.session.get(MinePatch, patch_id).image_path
ok(bool(rel), "数据库记录影像路径", rel)
abs_path = os.path.join(TestConfig.UPLOAD_FOLDER,
    rel.replace("uploads/", "", 1))
ok(os.path.isfile(abs_path), "影像文件落盘")
ok(c.get("/static/" + rel).status_code == 200, "影像可经 URL
    访问")
r = c.post("/project/patch/%d/image" % patch_id,
           data={"image": (io.BytesIO(b"x"), "bad.txt")},
           content_type="multipart/form-data",
    follow_redirects=True)
ok("不支持" in r.get_data(as_text=True), "拒绝非图片格式")
c.post("/project/patch/%d/image/delete" % patch_id,
    follow_redirects=True)
with app.app_context():
    ok(db.session.get(MinePatch, patch_id).image_path is
    None, "影像移除后字段清空")
ok(not os.path.isfile(abs_path), "影像文件已删除")
# ---------------------------------------------------------
    -----------------
print()
print(LINE)
print("指标定义维护与多专家矩阵")
print(LINE)
r = c.get("/ahp/indicators")
body = r.get_data(as_text=True)
with app.app_context():
    n_ind = Indicator.query.count()
ok(r.status_code == 200 and n_ind > 0, "指标定义已入库并可查看", "%d
    项" % n_ind)
ok(all(k in body for k in ("一级指标", "二级指标", "三级指标")),
   "指标树按层级展示")
ok("岩质边坡" in body and "平台" in body, "显示各指标启用的微地质单元")
c.post("/ahp/indicators/coverage/edit",
       data={"name": "植被覆盖度", "unit": "%", "ideal_type":
    "benefit",
             "ideal_value": "75", "enabled_units": ["岩质边坡"],
             "description": "自检修改"})
with app.app_context():
    ok(Indicator.query.filter_by(code="coverage").first().de
    scription == "自检修改",
       "指标定义可编辑")
c.post("/ahp/indicators/reset")
with app.app_context():
    ok("自检修改" not in (Indicator.query.filter_by(code="covera
    ge").first().description or ""),
       "可按论文取值重置")
base = {"m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24"}
for name in ("张工", "李工", "王工"):
    d = dict(base); d.update({"action": "add", "unit_type":
    "碎石边坡",
                               "level": "1", "expert_name":
    name})
    c.post("/ahp/experts", data=d)
with app.app_context():
    ok(ExpertMatrix.query.filter_by(unit_type="碎石边坡",
    level=1).count() == 3,
       "录入 3 位专家的判断矩阵")
r = c.get("/ahp/experts/aggregate?unit_type=碎石边坡&level=1")
body = r.get_data(as_text=True)
ok("汇总后权重" in body and "各专家矩阵的一致性" in body,
   "多专家矩阵几何平均汇总可用")
ws = re.search(r"<th scope=\"row\">权重</th>\s*(.*?)</tr>",
    body, re.S)
vals = re.findall(r">([0-9.]+)</td>", ws.group(1)) if ws
    else []
ok(len(vals) == 3 and abs(sum(map(float, vals)) - 1.0) <
    0.01,
   "汇总权重归一", "、".join(vals))
ok(abs(float(vals[0]) - 0.44) < 0.02, "汇总权重接近论文表 3 取值",
    vals[0] if vals else "")
# -----------------------------------------------------------
    ----------------
print()
print(LINE)
print("批量导入")
print(LINE)
r = c.get("/indicator/import/template")
tpl = r.data.decode("utf-8-sig")
ok(r.status_code == 200 and "单元类型" in tpl, "导入模板可下载")
ok("土石比" in tpl and "植被覆盖度" in tpl, "模板包含指标列")
csv_text = (
    "单元类型,土石比,土层厚度,pH,有机质,Hg,As,Ni,Cu,Cd,Pb,植被覆盖度,Shannon,坡度
    ,坡向\n"
    "碎石边坡,0.34,10,9.24,4.89,0.05,5.17,12.60,11.70,0.04,6.50,
    87.92,0.60,28.75,阳坡\n"
    "平台,0.56,30,8.72,10.92,0.02,3.13,12.33,11.14,0.09,9.98,5
    8.42,0.72,,,\n"
    "不存在单元,1,1,7,1,,,,,,,,,,\n"
)
r = c.post("/indicator/import",
           data={"patch_id": str(patch_id),
                 "datafile": (io.BytesIO(csv_text.encode("ut
    f-8")), "d.csv")},
           content_type="multipart/form-data",
    follow_redirects=True)
body = r.get_data(as_text=True)
ok("导入结果" in body, "导入结果页展示")
ok("不存在单元" in body, "错误行被报告")
with app.app_context():
    u = MicroUnit.query.filter_by(patch_id=patch_id,
    unit_type="碎石边坡").first()
    ind = IndicatorData.query.filter_by(unit_id=u.id).first(
    )
    ok(abs(ind.soil_ratio - 0.34) < 1e-6 and
    abs(ind.coverage - 87.92) < 1e-6,
       "导入数据已写入数据库")
r = c.post("/indicator/import",
           data={"patch_id": str(patch_id),
                 "datafile": (io.BytesIO(b"x"), "d.pdf")},
           content_type="multipart/form-data",
    follow_redirects=True)
ok("仅支持" in r.get_data(as_text=True), "拒绝不支持的导入格式")
# ---------------------------------------------------------
    -----------------
print()
print(LINE)
print("图表与 PDF 导出")
print(LINE)
svg = charts.hbars([("岩质边坡", 0.54, "0.5401"), ("平台", 0.84,
    "0.8376")])
ok("<svg" in svg and "0.8376" in svg, "条形图生成且带数值标签")
svg2 = charts.donut([("植被指标", 0.6, "0.6000"), ("土壤指标", 0.4,
    "0.4000")])
ok("<svg" in svg2 and "占比" in svg2, "环形图生成且带图例")
# 报告是评价结论的呈现，需先真实执行一次评价才能打开
with app.app_context():
    from app.views.evaluate import load_patch_input,
    save_evaluation
    save_evaluation(patch_id, load_patch_input(patch_id))
r = c.get("/report/%d" % patch_id)
body = r.get_data(as_text=True)
ok(body.count("<svg") >= 2, "得分明细页含图表", "%d 个 SVG" %
    body.count("<svg"))
exe = browser.browser_path()
if exe is None:
    print("  [跳过] 未找到浏览器，PDF 导出无法验证")
else:
    r = c.get("/report/%d/pdf" % patch_id)
    data = r.data
    ok(data[:4] == b"%PDF", "PDF 报告导出成功", "%d 字节" %
    len(data))
    d = fitz.open(stream=data, filetype="pdf")
    txt = "".join(d[i].get_text() for i in
    range(d.page_count))
    ok("自然恢复适宜性评价报告" in txt, "PDF 含报告标题")
    ok("评价结论" in txt and "得分明细" in txt, "PDF 含结论与明细章节")
    ok(d.page_count >= 2, "PDF 页数合理", "%d 页" % d.page_count)
    d.close()
print()
print(LINE)
print("补齐模块自检全部通过 √")
print(LINE)
# ==== 测试/test_workflow.py ====
# -*- coding: utf-8 -*-
"""流程门禁自检：前置步骤未完成时，后续模块必须拦得住。
论文 4.1 节确定的评价流程存在先后关系，缺任何一环都不应产出评价结论。
本测试逐个构造「缺前置」的场景，验证系统不会放行。
"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, Project, MinePatch, MicroUnit, IndicatorData,
                        ConstraintCheck, EvaluationResult)
from app.services import workflow as wf
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password": "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]", label, extra))
    if not flag:
        _fails.append(label)
def new_patch(tag):
    with app.app_context():
        p = Project(name="门禁-" + tag, code="WF-" + tag)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="门禁图斑-" +
    tag)
        db.session.add(patch); db.session.flush()
        pid = patch.id
        db.session.commit()
    return pid
def add_units(pid):
    with app.app_context():
        db.session.add(MicroUnit(patch_id=pid,
    unit_type="碎石边坡",
                                 area=1.0, ratio=1.0))
        db.session.commit()
def add_constraint(pid, passed=True):
    with app.app_context():
        db.session.add(ConstraintCheck(
            patch_id=pid, policy_pass=True,
    location_pass=passed,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=passed, reason="全部通过" if passed else
    "未通过：区位约束"))
        db.session.commit()
def add_data(pid):
    with app.app_context():
        u = MicroUnit.query.filter_by(patch_id=pid).first()
        db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.34, thickness=10,
                                     ph=9.24,
    organic_matter=4.89, coverage=87.92,
                                     shannon=0.60,
    slope=28.75, aspect="阳坡"))
        db.session.commit()
def cleanup(pid, tag):
    with app.app_context():
        from app.models import ScoreDetail
        # 先删得分明细，再删评价记录，避免留下孤儿明细
        rids = [r.id for r in EvaluationResult.query.filter_
    by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.i
    n_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delet
    e()
        ConstraintCheck.query.filter_by(patch_id=pid).delete
    ()
        uids = [u.id for u in MicroUnit.query.filter_by(patc
    h_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id
    .in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
        m = db.session.get(MinePatch, pid)
        if m:
            db.session.delete(m)
        pr = Project.query.filter_by(name="门禁-" +
    tag).first()
        if pr:
            db.session.delete(pr)
        db.session.commit()
def results_of(pid):
    with app.app_context():
        return EvaluationResult.query.filter_by(patch_id=pid
    ).count()
def post_eval(pid):
    return c.post("/evaluate/%d" % pid, data={},
    follow_redirects=False)
def get_report(pid):
    return c.get("/report/%d" % pid, follow_redirects=False)
# ---------------------------------------------------------
    -----------------
print(LINE)
print("一、前置条件缺失时，评价必须被拦住")
print(LINE)
print()
print("场景 1：连微地质单元都没划分")
pid = new_patch("1")
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未划分单元 → 不产生评价结果",
    "HTTP %d，结果数 %d" % (r.status_code, n))
with app.app_context():
    pr = wf.patch_progress(pid)
steps = [b["step"] for b in pr["blocking"]]
ok(not pr["ready"] and "units" in steps,
    "拦原因包含「划分微地质单元」", "、".join(steps))
ok(steps == ["constraint", "units"],
    "拦截顺序符合论文 4.1 节：限制性判定在前，划分单元在后",
    "、".join(steps))
cleanup(pid, "1")
print()
print("场景 2：有单元，但未做限制性条件判定")
pid = new_patch("2"); add_units(pid); add_data(pid)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未做限制性判定 → 不产生评价结果",
    "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "constraint" for b in pr["blocking"]),
   "拦原因指向「执行限制性条件判定」")
cleanup(pid, "2")
print()
print("场景 3：限制性条件判定【未通过】")
pid = new_patch("3"); add_units(pid); add_data(pid);
    add_constraint(pid, passed=False)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "判定未通过 → 不进入定量评价",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(not pr["constraint"]["passed"], "流程状态显示判定未通过")
blocked = [b for b in pr["blocking"] if b["step"] ==
    "constraint"]
# 提示只写「缺什么」：label 说明未通过，reason 直接引用判定记录里没过的项
ok(bool(blocked) and "未通过" in blocked[0]["label"],
   "拦提示指明限制性条件判定未通过", blocked[0]["label"] if blocked else
    "")
ok(bool(blocked) and "区位约束" in (blocked[0]["reason"] or ""),
   "并把具体没通过的条件带出来", blocked[0]["reason"] if blocked else "")
cleanup(pid, "3")
print()
print("场景 4：判定通过，但指标数据全空")
pid = new_patch("4"); add_units(pid); add_constraint(pid,
    passed=True)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "数据未录入 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "data" for b in pr["blocking"]),
   "拦原因指向「录入指标数据」")
ok(pr["data"]["missing"] == ["碎石边坡"],
   "指出具体缺哪个单元的数据", "、".join(pr["data"]["missing"]))
cleanup(pid, "4")
print()
print("场景 5：单元面积占比合计不为 100%")
pid = new_patch("5"); add_constraint(pid, passed=True)
with app.app_context():
    u = MicroUnit(patch_id=pid, unit_type="碎石边坡", area=1.0,
    ratio=0.5)
    db.session.add(u); db.session.commit()
    db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.3, thickness=10,
                                 ph=8.0,
    organic_matter=5.0, coverage=80.0,
                                   shannon=0.6, slope=20.0,
    aspect="阳坡"))
    db.session.commit()
r = post_eval(pid)
ok(results_of(pid) == 0, "占比不足 100% → 不产生评价结果")
cleanup(pid, "5")
print()
print(LINE)
print("二、前置条件齐备时才放行")
print(LINE)
pid = new_patch("OK"); add_units(pid); add_constraint(pid,
    passed=True); add_data(pid)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(pr["ready"], "单元 + 判定通过 + 数据齐备 → 判定为可执行")
r = post_eval(pid)
n = results_of(pid)
ok(n == 1, "此时执行评价成功产生结果", "结果数 %d" % n)
print("  重定向到:", r.headers.get("Location"))
print()
print(LINE)
print("三、报告不得先于评价产生")
print(LINE)
pid2 = new_patch("R")   # 什么都不做
r = get_report(pid2)
ok(r.status_code == 302 and "/report/" ==
    (r.headers.get("Location") or ""),
   "前置缺失时 /report/<id> 被退回列表", "HTTP %d" % r.status_code)
for suffix in ("/pdf", "/word"):
    rr = c.get("/report/%d%s" % (pid2, suffix),
    follow_redirects=False)
    ok(rr.status_code == 302, "导出 %s 同样被拦住" % suffix, "HTTP
    %d" % rr.status_code)
cleanup(pid2, "R")
cleanup(pid, "OK")
print()
print(LINE)
print("四、前置齐备但从未执行评价时，报告仍不得产生")
print(LINE)
# 报告是评价结论的呈现：即使前置步骤都做完了，只要没真正执行过定量评价，
# 也不应凭空推算出结论。
pid3 = new_patch("N")
add_units(pid3); add_constraint(pid3, passed=True);
    add_data(pid3)
with app.app_context():
    pr = wf.report_progress(pid3)
ok(not pr["ready"], "无评价记录 → 报告流程判定为未就绪")
ok(bool(pr["blocking"]) and pr["blocking"][0]["step"] ==
    "evaluate",
   "拦原因指向「执行适宜性评价」")
```

```
r = get_report(pid3)
ok(r.status_code == 302 and results_of(pid3) == 0,
   "直接访问 /report/<id> 被退回，且不产生任何评价结果",
   "HTTP %d" % r.status_code)
# 真正执行评价之后，报告才放行，且取值与评价记录一致
r = post_eval(pid3)
ok(results_of(pid3) == 1, "先执行评价 → 产生评价记录")
with app.app_context():
    pr = wf.report_progress(pid3)
    rec = pr["result"]
ok(pr["ready"] and rec is not None, "有评价记录后，报告流程就绪")
rr = get_report(pid3)
body = rr.get_data(as_text=True)
ok(rr.status_code == 200, "此时可打开得分明细页", "HTTP %d" %
    rr.status_code)
ok(("%.4f" % rec.final_score) in body,
   "报告数值取自评价记录，而非重新计算",
   "F=%.4f" % rec.final_score)
cleanup(pid3, "N")
print()
print(LINE)
print("五、内置演示数据的清理不得波及用户自建数据")
print(LINE)
# 回归保护：seed_demo.clean_case() 曾执行 Project.query.delete(),
# 导致用户新建的项目在每次启动时被连带删除。
import seed_demo as sd
with app.app_context():
    keep_p = Project(name="用户自建项目-守护", code="KEEP-001")
    db.session.add(keep_p); db.session.flush()
    keep_patch = MinePatch(project_id=keep_p.id,
    name="用户自建图斑-守护")
    db.session.add(keep_patch); db.session.flush()
    keep_pid = keep_p.id
    keep_patch_id = keep_patch.id
    db.session.commit()
with app.app_context():
    sd.clean_case(reset=False)
with app.app_context():
    still_p = db.session.get(Project, keep_pid)
    still_patch = db.session.get(MinePatch, keep_patch_id)
ok(still_p is not None, "清理演示数据后，用户自建项目仍存在")
ok(still_patch is not None, "清理演示数据后，用户自建图斑仍存在")
# 演示数据清掉后应能重新写入，保证演示环境可恢复
sd.seed()
with app.app_context():
    demo_back = Project.query.filter_by(code=sd.DEMO_CODE).f
    irst()
ok(demo_back is not None, "演示案例可重新写入（幂等恢复）")
with app.app_context():
    m = db.session.get(MinePatch, keep_patch_id)
    if m:
        db.session.delete(m)
    p = db.session.get(Project, keep_pid)
    if p:
        db.session.delete(p)
    db.session.commit()
print()
print(LINE)
print("六、同一图斑不得重复划分同类型微地质单元")
print(LINE)
# 论文 4.1 节把图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。
# 允许重复会让面积占比、按类型组织的指标数据与批量导入对不上号。
pid4 = new_patch("D")
c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
    "area": "1.0"},
       follow_redirects=True)
with app.app_context():
    n1 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n1 == 1, "首次添加「平台」成功", "单元数 %d" % n1)
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
    "area": "2.0"},
           follow_redirects=True)
with app.app_context():
    n2 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n2 == 1, "重复添加同类型被拒绝", "单元数 %d" % n2)
ok("同一类型只能划分一个" in r.get_data(as_text=True), "给出重复提示")
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "火山口",
    "area": "1.0"},
           follow_redirects=True)
with app.app_context():
    n3 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n3 == 1, "非法单元类型被拒绝", "单元数 %d" % n3)
c.post("/unit/%d/add" % pid4, data={"unit_type": "碎石边坡",
    "area": "3.0"},
       follow_redirects=True)
with app.app_context():
    uid = MicroUnit.query.filter_by(patch_id=pid4,
                                    unit_type="碎石边坡").first(
    ).id
r = c.post("/unit/edit/%d" % uid,
           data={"unit_type": "平台", "area": "3.0"},
    follow_redirects=True)
with app.app_context():
    types = sorted(u.unit_type for u in
                   MicroUnit.query.filter_by(patch_id=pid4).
    all())
ok(types == ["平台", "碎石边坡"], "改类型时同样拦截重复", "、".join(types))
cleanup(pid4, "D")
print()
print(LINE)
```

```
if _fails:
    print("存在未通过项：")
    for f in _fails:
        print("  -", f)
    sys.exit(1)
print("流程门禁自检全部通过 √")
print(LINE)
# ==== 测试/test_weights.py ====
# -*- coding: utf-8 -*-
"""权重生效链路自检：AHP 算出的权重必须真正参与评价计算。
此前 weight_schemes / weight_items 两张表从未被写入，评价引擎也只用
代码里的论文表 3 默认权重，相当于权重模块算完即弃。本测试守住这条链路：
    默认 → 论文表 3
    保存 AHP 权重 → 评价改用新权重，结果随之变化
    恢复默认 → 结果回到论文取值
"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult,
    WeightScheme,
                        WeightItem)
from app.services import workflow as wf
from app.services import weight_service as ws
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    if not flag:
        _fails.append(label)
TAG = "W"
UNIT_TYPE = "碎石边坡"
def build_case():
    """建一个只含单一微地质单元的图斑，便于观察权重变化对得分的影响。"""
    with app.app_context():
        p = Project(name="权重测试-%s" % TAG, code="WEIGHT-%s" % TAG)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="权重测试图斑-%s" % TAG)
        db.session.add(patch); db.session.flush()
        db.session.add(MicroUnit(patch_id=patch.id, unit_type=UNIT_TYPE,
                                 area=1.0, ratio=1.0))
        db.session.flush()
        u = MicroUnit.query.filter_by(patch_id=patch.id).first()
        db.session.add(IndicatorData(
            unit_id=u.id, soil_ratio=0.34, thickness=10, ph=9.24,
            organic_matter=4.89, coverage=87.92, shannon=0.60,
            slope=28.75, aspect="阳坡"))
        db.session.add(ConstraintCheck(
            patch_id=patch.id, policy_pass=True, location_pass=True,
            pollution_pass=True, geo_pass=True, farmland_pass=True,
            passed=True, reason="全部通过"))
        db.session.commit()
        return patch.id
def cleanup(pid):
    with app.app_context():
        from app.models import ScoreDetail
        rids = [r.id for r in EvaluationResult.query.filter_by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.in_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delete()
        ConstraintCheck.query.filter_by(patch_id=pid).delete()
        uids = [u.id for u in MicroUnit.query.filter_by(patch_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id
    .in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
        m = db.session.get(MinePatch, pid)
        if m:
            db.session.delete(m)
        p = Project.query.filter_by(code="WEIGHT-%s" %
    TAG).first()
        if p:
            db.session.delete(p)
        db.session.commit()
def run_eval(pid):
    c.post("/evaluate/%d" % pid, data={},
    follow_redirects=False)
    with app.app_context():
        return EvaluationResult.query.filter_by(patch_id=pid
    ) \
            .order_by(EvaluationResult.id.desc()).first().fi
    nal_score
print(LINE)
print("一、默认应使用论文表 3 的权重")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
    w, label, scheme = ws.effective_weights(UNIT_TYPE)
ok(scheme is None and "论文表" in label, "未配置时权重来源为论文表 3",
    label)
ok(abs(w["l1"]["土壤指标"] - 0.44) < 1e-9, "一级权重取自论文表 3",
   "土壤指标 %.2f" % w["l1"]["土壤指标"])
pid = build_case()
base_score = run_eval(pid)
print("  默认权重下的评价值 F = %.4f" % base_score)
print()
print(LINE)
print("二、保存 AHP 算出的权重后，评价应改用新权重")
print(LINE)
with app.app_context():
    # 模拟 AHP 计算结果：把土壤指标权重提高、植被与地形降低（仍和为 1）
    ws.save_weights(UNIT_TYPE, [("soil", 0.70), ("veg",
    0.20),
                                ("terrain", 0.10)])
    n_items = WeightItem.query.count()
ok(n_items >= 3, "weight_items 表已写入权重条目", "%d 条" % n_items)
with app.app_context():
    w2, label2, scheme2 = ws.effective_weights(UNIT_TYPE)
ok(scheme2 is not None and "自定义" in label2, "权重来源变为自定义方案",
    label2)
ok(abs(w2["l1"]["土壤指标"] - 0.70) < 1e-9, "新权重已生效",
   "土壤指标 %.2f" % w2["l1"]["土壤指标"])
ok(abs(w2["l1"]["植被指标"] - 0.20) < 1e-9, "同批保存的其他项也已生效")
ok(abs(w2["l2"]["土壤指标"]["土石比"] - 0.25) < 1e-9,
   "未覆盖的二级权重仍保留论文取值",
   "土石比 %.2f" % w2["l2"]["土壤指标"]["土石比"])
new_score = run_eval(pid)
print("  新权重下的评价值 F = %.4f" % new_score)
ok(abs(new_score - base_score) > 1e-4,
   "权重变化确实改变了评价结果",
   "%.4f → %.4f" % (base_score, new_score))
print()
print(LINE)
print("三、结果应记录所用权重来源")
print(LINE)
import json
with app.app_context():
    rec = EvaluationResult.query.filter_by(patch_id=pid) \
        .order_by(EvaluationResult.id.desc()).first()
    stored = json.loads(rec.unit_scores)
ok(any(u.get("weight_source") for u in stored),
   "评价记录中标注了权重来源",
   stored[0].get("weight_source") or "")
print()
print(LINE)
print("四、恢复默认后应回到论文取值")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
    w3, label3, scheme3 = ws.effective_weights(UNIT_TYPE)
ok(scheme3 is None and "论文表" in label3, "已恢复为论文表 3 权重",
    label3)
back_score = run_eval(pid)
print("  恢复后评价值 F = %.4f（默认时 %.4f）" % (back_score,
    base_score))
ok(abs(back_score - base_score) < 1e-9, "评价结果与默认时一致")
print()
print(LINE)
print("五、权重首页概览")
print(LINE)
with app.app_context():
    rows = ws.summary()
ok(len(rows) == 3, "三种微地质单元均有权重来源说明", "%d 行" % len(rows))
r = c.get("/ahp/")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "当前生效权重" in body, "权重首页正常展示")
print()
print(LINE)
print("六、一致性不通过的矩阵不得保存生效")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
n = 3
```

```
# 明显矛盾的矩阵：a12=9, a13=1/9, a23=9 —— CR 必然超限
mats = [(0, 1, "9"), (0, 2, "0.1111"), (1, 2, "9")]
data = {"csrf_token": "", "action": "save"}
for i, j, v in mats:
    data["m_%d_%d" % (i, j)] = v
r = c.post("/ahp/matrix?unit_type=%s&level=1" % UNIT_TYPE,
    data=data,
           follow_redirects=True)
with app.app_context():
    _, label4, scheme4 = ws.effective_weights(UNIT_TYPE)
ok(scheme4 is None, "CR 超限的矩阵未被保存生效", label4)
ok("一致性" in r.get_data(as_text=True), "页面给出了未通过一致性的提示")
cleanup(pid)
print()
print(LINE)
if _fails:
    print("存在未通过项：")
    for f in _fails:
        print("  -", f)
    sys.exit(1)
print("权重生效链路自检全部通过 √")
print(LINE)
# ==== app/services/__init__.py ====
# -*- coding: utf-8 -*-
# ==== app/utils/__init__.py ====
# -*- coding: utf-8 -*-
# ==== app/views/__init__.py ====
# -*- coding: utf-8 -*-
```